Đồ họa máy tính · Trường Đại học Quốc tế, ĐHQG-HCM

Splat Editor

Sách hướng dẫn cấu trúc chương trình

Giải thích từng thư mục, từng file, từng hàm, kèm mã nguồn thật và toán học

Cách đọc cuốn sách. Sách đi theo *cấu trúc thư mục* của mã nguồn: mỗi chương là một thư mục, mỗi mục là một file, mỗi tiểu mục là một lớp hoặc một hàm. Với mỗi hàm bạn có ba thứ: **mô tả** (chú thích của tác giả trong mã), **giải thích** (phần diễn giải thêm) và **mã nguồn nguyên văn** kèm số dòng thật trong file. Cơ sở toán nằm ngay trong chương của file tương ứng, ở các mục *Cơ sở toán*; các chỗ có ghi § trỏ tới mục đó, và Chương 7 có bảng tra nhanh công thức–file.

Quy ước. Vector cột, tọa độ tay phải, trục Y hướng lên, camera nhìn dọc $-z$. Ma trận 4×4 dùng tọa độ thuần nhất $(x, y, z, 1)$.

Contents

Tổng quan: bản đồ chương trình và điểm vào 3

Tổng quan: điểm vào và app.py 3

0.1 `main.py` . . . 3
0.2 `splat_editor/__main__.py` . . . 3
0.3 `splat_editor/app.py` . . . 4

1 Thư mục core: logic thuần 13

1.1 `splat_editor/core/math3d.py` . . . 13
1.2 `splat_editor/core/transform.py` . . . 17
1.3 `splat_editor/core/splats.py` . . . 19
1.4 `splat_editor/core/layer.py` . . . 20
1.5 `splat_editor/core/scene.py` . . . 25
1.6 `splat_editor/core/sh.py` . . . 30
1.7 `splat_editor/core/lod.py` . . . 34
1.8 `splat_editor/core/lighting.py` . . . 38
1.9 `splat_editor/core/physics.py` . . . 45
1.10 `splat_editor/core/avatar_pose.py` . . . 52
1.11 `splat_editor/core/history.py` . . . 56
1.12 `splat_editor/core/rect.py` . . . 59

2 Thư mục editor: thao tác chỉnh sửa 60

2.1 `splat_editor/editor/editor.py` . . . 60
2.2 `splat_editor/editor/brush.py` . . . 65

3 Thư mục io: đọc ghi và giải mã 69

3.1 `splat_editor/io/ply.py` . . . 69
3.2 `splat_editor/io/codec.py` . . . 71

4 Thư mục render: OpenGL 75

4.1 `splat_editor/render/shader.py` . . . 75
4.2 `splat_editor/render/camera.py` . . . 77

4.3 splat_editor/render/sorter.py . . . 81
4.4 splat_editor/render/splat_renderer.py . . . 84
4.5 splat_editor/render/viewport.py . . . 93
4.6 splat_editor/render/grid_renderer.py . . . 96
4.7 splat_editor/render/light_renderer.py . . . 97
4.8 splat_editor/render/avatar_renderer.py . . . 99
4.9 shaders/ . . . 103

5 Thư mục ui: giao diện 104
5.1 splat_editor/ui/window.py . . . 104
5.2 splat_editor/ui/gui.py . . . 106
5.3 splat_editor/ui/theme.py . . . 107
5.4 splat_editor/ui/actions.py . . . 108
5.5 splat_editor/ui/input.py . . . 110
5.6 splat_editor/ui/layout.py . . . 113
5.7 splat_editor/ui/menu_bar.py . . . 114
5.8 splat_editor/ui/side_panel.py . . . 115
5.9 splat_editor/ui/sections.py . . . 116
5.10 splat_editor/ui/world_sections.py . . . 120
5.11 splat_editor/ui/status_bar.py . . . 122
5.12 splat_editor/ui/viewport_overlay.py . . . 124
5.13 splat_editor/ui/shortcuts_window.py . . . 126
5.14 splat_editor/ui/widgets.py . . . 127
5.15 splat_editor/ui/file_dialog.py . . . 128

6 Thư mục render/shaders: vertex và fragment shader 130
6.1 splat_prep.vert . . . 131
6.2 splat.vert . . . 136
6.3 splat.frag . . . 137
6.4 shadow.vert . . . 137
6.5 shadow.frag . . . 138
6.6 grid.vert . . . 138
6.7 grid.frag . . . 139
6.8 light_gizmo.vert . . . 140
6.9 light_gizmo.frag . . . 141
6.10 firefly.vert . . . 141
6.11 firefly.frag . . . 142
6.12 avatar.vert . . . 142
6.13 avatar.frag . . . 143

7 Bảng tra nhanh: công thức và file 144

Tổng quan

Cấu trúc thư mục

```
SplatOpenGL/
+- main.py ................. diem vao
+- splat_editor/
|  +- __main__.py, app.py .. khoi dong, vong lap chinh (Tong quan)
|  +- core/    (Chuong 1)   toan thuan NumPy, KHONG co OpenGL
|  +- editor/  (Chuong 2)   chon, xoa, undo, co chon
|  +- io/      (Chuong 3)   doc/ghi PLY, giai ma/ma hoa
|  +- render/  (Chuong 4)   OpenGL: camera, bo ve gaussian, luong sap xep
|  |   +- shaders/ (Chuong 6)   ma GLSL .vert / .frag
|  +- ui/      (Chuong 5)   GLFW + Dear ImGui
+- report/ ................. bao cao LaTeX goc
+- guideline/report_src/ .... sach huong dan nay
```

Năm tầng và quan hệ phụ thuộc

Chương	Thư mục	Phụ thuộc vào
1	`core`	chỉ `io` (đọc/ghi PLY)
2	`editor`	`core`
3	`io`	`core.splats` (kiểu dữ liệu đích)
4	`render`	`core`, `shaders/`, OpenGL
5	`ui`	`editor`, `render`, `core`, GLFW, ImGui
6	`render/shaders`	được nạp bởi `render/shader.py`

Quy tắc: `core`, `editor` và `io` không biết OpenGL hay giao diện; chỉ `render` chạm OpenGL và chỉ `ui` chạm ImGui. Nhờ vậy phần logic có thể chạy và kiểm thử không cần cửa sổ.

Đường đi của dữ liệu

PLY → `io` (`PlyFile`, `SplatCodec`) → `core` (`Layer`, `Scene`, `Splats`) → `editor` (`Editor`) → `render` (`Viewport`, `SplatRenderer`) → **shader** → màn hình. Song song là camera, cọ, vật lý, ánh sáng, hoạt họa. Mỗi khung hình đi qua `Application.run` (mục dưới).

Điểm vào và vòng lặp chính

0.1. `main.py`

Vai trò. Điểm vào của chương trình: chỉ import và gọi `splat_editor.app.main`.

```
"""Splat Editor - xem / chỉnh sửa file Gaussian Splat (.ply) bằng Python + OpenGL.

Cách chạy:
    python main.py              # mở cửa sổ trống, bấm Ctrl+O để chọn file
    python main.py scene.ply    # mở luôn file (thêm nhiều file: python main.py a.ply b.ply)
"""

from splat_editor.app import main

if __name__ == '__main__':
    main()
```

0.2. `splat_editor/__main__.py`

Vai trò. Cho phép chạy bằng `python -m splat_editor`; cùng làm việc với `main.py`.

```
"""Cho phép chạy bằng:  python -m splat_editor [file.ply]"""

"""
# Trong terminal:
python -m splat_editor                # mở cửa sổ trống
python -m splat_editor scene.ply      # mở luôn file
"""

from .app import main

main()
```

0.3. splat_editor/app.py

509 dòng

Vai trò. `Application` ghép mọi thành phần (cửa sổ, ImGui, Editor, Viewport, Physics, Lighting, các bảng giao diện) và chạy vòng lặp chính. Đây là nơi duy nhất biết **tất cả** các tầng; các tầng còn lại không import `app`. Mỗi khung hình: `poll_events` → `input.update` → `update_world` → dựng ImGui → `viewport.sync` → `viewport.render` → `gui.draw` → `swap_buffers`.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Application: ghép cửa sổ, giao diện, Editor và khung nhìn 3D lại với nhau, chạy vòng lặp chính."""

"""
# Cách dùng thông thường (main.py làm đúng như vậy)
from splat_editor.app import main
main()                                  # đọc đường dẫn file từ sys.argv rồi chạy vòng lặp chính

# Hoặc dùng trực tiếp
app = Application(1500, 900)
app.open_file("scene.ply")              # mở file (thay cảnh hiện tại) và đưa camera về nhìn toàn cảnh
app.add_files(["a.ply", "b.ply"])       # thêm mỗi file thành một lớp
app.actions.trigger("save")             # chạy một hành động theo tên (giống bấm menu / phím tắt)
app.run()                               # vòng lặp: poll_events -> input.update -> vẽ GUI -> viewport -> swap_buffers

# Mỗi khung hình:
#   input.update(dt)           điều khiển camera / cọ
#   gui.begin_frame() ... gui.end_frame()   dựng giao diện ImGui
#   viewport.sync(editor)      gửi dữ liệu mới lên GPU nếu Editor đã đổi
#   viewport.render(...)       vẽ cảnh 3D vào vùng giữa
#   gui.draw()                 vẽ ImGui đè lên
"""

```

Khai báo mức module (dòng 23–45)

```
import math
import sys
import time

import glfw
import numpy as np

from .core.avatar_pose import AvatarPose, limb_matrices
from .core.lighting import MAX_LIGHTS, Lighting
from .core.physics import Physics
from .editor.editor import Editor
from .render.viewport import Viewport
from .ui.actions import Action, ActionRegistry
from .ui.file_dialog import FileDialog
from .ui.gui import Gui
from .ui.input import InputController
from .ui.layout import Layout
from .ui.menu_bar import MainMenuBar
from .ui.shortcuts_window import ShortcutsWindow
from .ui.side_panel import SidePanel
from .ui.status_bar import StatusBar
from .ui.viewport_overlay import ViewportOverlay
from .ui.window import Window
```

`class FpsCounter`

Dòng 48–59

Các phương thức: `__init__`, `tick`.

```
class FpsCounter:
```

`FpsCounter.__init__(self, interval=0.5)`

Dòng 49–53

```
    def __init__(self, interval=0.5):
        self.interval = interval
        self.value = 0.0
        self._frames = 0
        self._start = time.perf_counter()
```

`FpsCounter.tick(self, now)`

Dòng 55–59

```
    def tick(self, now):
        self._frames += 1
        if now - self._start >= self.interval:
            self.value = self._frames / (now - self._start)
            self._frames, self._start = 0, now
```

`class Application`

Dòng 62–500

Các phương thức: `__init__`, `_create_actions`, `open_file`, `add_files`, `open_dialog`, `add_dialog`, `save_dialog`, `frame_scene`, `_update_bounds`, `toggle_lighting`, `toggle_shadows`, `add_sun`, `add_light`, `pick_light`, `delete_key`, `set_mode`, `_spawn_player`, `toggle_mode`, `walk`, `cycle_view`, `_leave_third_person`, `_update_avatar`, `update_world`, `toggle_grid`, `toggle_sh`, `set_flip`, `exit_brush`, `shortcuts_window_toggle`, `paint`, `drag_speed`, `_draw_gui`, `run`.

```
class Application:
    TITLE = "Splat Editor"
```

`Application.__init__(self, width=1500, height=900)`

Dòng 65–88

Giải thích. Tạo mọi thành phần theo thứ tự phụ thuộc: cửa sổ → ImGui → Editor, Viewport, Physics, Lighting → `ActionRegistry` → các bảng giao diện.

```
    def __init__(self, width=1500, height=900):
        self.window = Window(self.TITLE, width, height)
        self.gui = Gui(self.window)
        self.editor = Editor()
        self.viewport = Viewport()
        self.layout = Layout()
        self.fps = FpsCounter()
        self.physics = Physics()
        self.lighting = Lighting()
        self.viewport.lighting = self.lighting
        self.mode = "creative"       # "creative": bay tự do; "survival": đi bộ có trọng lực + va chạm
        self.pose = AvatarPose()     # tư thế nhân vật người chơi (đi, cúi, ngồi, cử chỉ) — thấy khi bấm F5
        self.body_yaw = 0.0          # hướng thân nhân vật (rad), đầu quay theo camera trong giới hạn
        self._last_feet = None
        self._world_versions = None
        self._spawn_pending = False
        self.actions = self._create_actions()

        self.menu_bar = MainMenuBar(self.actions)
        self.side_panel = SidePanel(self)
        self.status_bar = StatusBar(self)
        self.shortcuts_window = ShortcutsWindow(self.actions)
        self.overlay = ViewportOverlay(self)
        self.input = InputController(self)
```

`Application._create_actions(self)`

Dòng 91–172

Giải thích. Khai báo toàn bộ hành động (mở, lưu, hoàn tác, chọn, cọ, xem, ánh sáng, chế độ…) cùng phím tắt ở một nơi.

```
    def _create_actions(self):
        editor, viewport = self.editor, self.viewport
        has_layer = lambda: editor.scene.active_layer is not None  # noqa: E731
        has_selection = lambda: editor.scene.selected_count() > 0  # noqa: E731
        brush = editor.brush
        actions = ActionRegistry()
        add = actions.add

        # File
        add("open", Action("Open...", self.open_dialog, "File", "Ctrl+O", glfw.KEY_O, ctrl=True,
                           description="Open a file (replaces the scene)", deferred=True))
        add("add", Action("Add to Scene...", self.add_dialog, "File", "Ctrl+I", glfw.KEY_I, ctrl=True,
                          description="Add files to the scene as new layers", deferred=True))
        add("save", Action("Save As...", self.save_dialog, "File", "Ctrl+S", glfw.KEY_S, ctrl=True,
                           description="Save visible layers merged into one .ply", deferred=True,
                           enabled=editor.can_save))
        add("quit", Action("Exit", self.window.close, "File", "Ctrl+Q", glfw.KEY_Q, ctrl=True))

        # Edit
        add("undo", Action("Undo", editor.undo, "Edit", "Ctrl+Z", glfw.KEY_Z, ctrl=True,
                           description="Undo the last delete", enabled=lambda: editor.history.can_undo))
        add("select_all", Action("Select All", editor.select_all, "Edit", "Ctrl+A", glfw.KEY_A, ctrl=True))
        add("select_none", Action("Select None", editor.select_none, "Edit", "Ctrl+D", glfw.KEY_D, ctrl=True))
```

```
        add("invert_selection", Action("Invert Selection", editor.invert_selection, "Edit"))
        add("delete_selected", Action("Delete Selected", editor.delete_selected, "Edit", "Delete",
                                      enabled=has_selection))
        add("delete", Action("Delete", self.delete_key, "Edit", "Delete", glfw.KEY_DELETE,
                             description="Delete the picked light, else selected gaussians "
                                         "(or the active layer if none)"))
        add("remove_layer", Action("Remove Layer", editor.remove_layer, "Edit", deferred=True,
                                   enabled=has_layer))

        # View
        add("frame_scene", Action("Frame Scene", self.frame_scene, "View", "F", glfw.KEY_F,
                                  description="Frame the whole scene"))
        add("frame_layer", Action("Frame Active Layer", lambda: self.frame_scene(editor.scene.active_layer),
                                  "View", enabled=has_layer))
        add("show_grid", Action("Show Grid", self.toggle_grid, "View", "G", glfw.KEY_G,
                                description="Show / hide the floor grid", checked=lambda: viewport.show_grid))
        add("view_sh", Action("View-Dependent Color", self.toggle_sh, "View",
                              description="Use spherical harmonics (f_rest_*) so color changes with view angle",
                              checked=lambda: viewport.splat_renderer.use_sh))
        add("flip_y", Action("Flip Y Axis (COLMAP)", lambda: self.set_flip(not editor.scene.flip), "View",
                             checked=lambda: editor.scene.flip))
        # World: ánh sáng + vật lý
        add("toggle_mode", Action("Walk Mode", self.toggle_mode, "World", "M", glfw.KEY_M,
                                  description="Switch Fly <-> Walk (gravity, collision)",
                                  checked=lambda: self.mode == "survival"))
        survival = lambda: self.mode == "survival"  # noqa: E731
        add("third_person", Action("Third Person (F5)", self.cycle_view, "World", "F5", glfw.KEY_F5,
                                   description="See yourself: first person -> behind -> in front (Walk mode)",
                                   enabled=survival, checked=lambda: self.viewport.camera.view_mode != 0))
        add("sit", Action("Sit Down", self.pose.toggle_sit, "World", "C", glfw.KEY_C,
                          description="Sit on the ground / stand up (moving also stands up)", enabled=survival,
                          checked=lambda: self.pose.sitting))
        for name, label, key, letter in (("wave", "Wave", glfw.KEY_V, "V"),
                                         ("zombie", "Zombie Arms", glfw.KEY_X, "X"),
                                         ("cheer", "Cheer", glfw.KEY_Y, "Y")):
            add(name, Action(label, lambda n=name: self.pose.toggle_gesture(n), "World", letter, key,
                             description=f"Gesture: {label.lower()} (press again to stop)", enabled=survival,
                             checked=lambda n=name: self.pose.gesture == n))
        add("lighting", Action("Lighting", self.toggle_lighting, "World",
                               description="Ambient + sun + light blocks with Phong shading",
                               checked=lambda: self.lighting.enabled))
        add("shadows", Action("Sun Shadows", self.toggle_shadows, "World",
                              description="Shadow mapping for the sun", checked=lambda: self.lighting.shadows))
        add("add_sun", Action("Add Sun", self.add_sun, "World", description="Add a sun sphere (day-night cycle)"))
        add("add_light", Action("Add Light Block", self.add_light, "World", "L", glfw.KEY_L,
                                description="Drop a light block in front of the camera"))
        add("shortcuts", Action("Keyboard Shortcuts", self.shortcuts_window_toggle, "Help", "F1", glfw.KEY_F1,
                                description="Show this window"))

        # Brush
        add("brush", Action("Brush Tool", brush.toggle, "Brush", "B", glfw.KEY_B,
                            description="Toggle the selection brush", checked=lambda: brush.active))
        add("brush_smaller", Action("Smaller Brush", lambda: brush.resize(1 / 1.2), "Brush", "[",
                                    glfw.KEY_LEFT_BRACKET, description="Smaller brush"))
        add("brush_larger", Action("Larger Brush", lambda: brush.resize(1.2), "Brush", "]",
                                   glfw.KEY_RIGHT_BRACKET, description="Larger brush"))
        add("brush_off", Action("Exit Brush", self.exit_brush, "Brush", "Esc", glfw.KEY_ESCAPE,
                                description="Turn the brush off"))
        return actions
```

Application.open_file(self, path)

Dòng 175–177

```
    def open_file(self, path):
        if self.editor.open_file(path):
            self.frame_scene()
```

Application.add_files(self, paths)

Dòng 179–183

```
    def add_files(self, paths):
        for path in paths:
            first = self.editor.scene.is_empty
            if self.editor.add_file(path) is not None and first:
                self.frame_scene()
```

Application.open_dialog(self)

Dòng 185–189

```
    def open_dialog(self):
```

```
        path = FileDialog.open_file()
        self.input.reset()
        if path:
            self.open_file(path)
```

`Application.add_dialog(self)`

Dòng 191–194

```
    def add_dialog(self):
        paths = FileDialog.open_files()
        self.input.reset()
        self.add_files(paths)
```

`Application.save_dialog(self)`

Dòng 196–200

```
    def save_dialog(self):
        path = FileDialog.save_file(self.editor.suggested_save_name())
        self.input.reset()
        if path:
            self.editor.save_file(path)
```

`Application.frame_scene(self, layer=None)`

Dòng 202–208

Mô tả trong mã nguồn. Đưa camera về nhìn toàn cảnh (hoặc chỉ một lớp).

```
    def frame_scene(self, layer=None):
        """Đưa camera về nhìn toàn cảnh (hoặc chỉ một lớp)."""
        scene = self.editor.scene
        positions = None if scene.is_empty else scene.world_positions(layer)
        self.viewport.frame(positions)
        if layer is None and positions is not None:
            self._update_bounds(positions, reset_scale=True)
```

`Application._update_bounds(self, positions=None, reset_scale=False)`

Dòng 211–224

Mô tả trong mã nguồn. Kích thước cảnh -> tầm đèn, bản đồ bóng và chiều cao người chơi mặc định.

```
    def _update_bounds(self, positions=None, reset_scale=False):
        """Kích thước cảnh -> tầm đèn, bản đồ bóng và chiều cao người chơi mặc định."""
        scene = self.editor.scene
        if positions is None:
            if scene.is_empty:
                return
            positions = scene.world_positions(limit=200_000)
        if len(positions) == 0:
            return
        center = np.median(positions, axis=0)
        radius = float(np.percentile(np.linalg.norm(positions - center, axis=1), 95)) or 1.0
        self.lighting.set_bounds(center, radius)
        if reset_scale:
            self.physics.set_scale(radius)
```

`Application.toggle_lighting(self)`

Dòng 226–227

```
    def toggle_lighting(self):
        self.lighting.enabled = not self.lighting.enabled
```

`Application.toggle_shadows(self)`

Dòng 229–230

```
    def toggle_shadows(self):
        self.lighting.shadows = not self.lighting.shadows
```

`Application.add_sun(self)`

Dòng 232–236

```
    def add_sun(self):
        if self.editor.scene.is_empty:
            return
        self.lighting.add_sun()
        self.editor.set_status("Sun added: drag the time of day or animate it in the Lighting panel")
```

`Application.add_light(self)`

Dòng 238–250

```
    def add_light(self):
        if self.editor.scene.is_empty:
            return
        camera = self.viewport.camera
        if self.mode == "survival":
            position = camera.position() + camera.forward() * self.physics.height * 2.0
        else:
            position = camera.target.copy()
        light = self.lighting.add_light(position, self.physics.height * 4.0)
        if light is None:
            self.editor.set_status(f"At most {MAX_LIGHTS} light blocks")
        else:
            self.editor.set_status(f"{light.name} added: it falls onto the surface below")
```

`Application.pick_light(self, x, y)`

Dòng 252–280

Mô tả trong mã nguồn. Click trong cảnh 3D: chọn khối đèn gần con trỏ nhất (và không bị cảnh che); click ra ngoài: bỏ chọn.

```
    def pick_light(self, x, y):
        """Click trong cảnh 3D: chọn khối đèn gần con trỏ nhất (và không bị cảnh che); click ra ngoài: bỏ chọn."""
        lighting, rect = self.lighting, self.layout.viewport
        lighting.picked = -1
        if not lighting.lights or rect.is_empty:
            return False
        view = self.viewport.camera.view_matrix()
        projection = self.viewport.camera.projection_matrix(rect.aspect)
        cam_pos = -view[:3, :3].T @ view[:3, 3]
        half_tan = math.tan(math.radians(self.viewport.camera.fov) / 2)
        best, best_dist = -1, 1e9
        for i, light in enumerate(lighting.lights):
            clip = projection @ (view @ np.append(light.position, 1.0))
            if clip[3] <= 1e-9:
                continue                                    # sau lưng camera
            sx = rect.left + (clip[0] / clip[3] * 0.5 + 0.5) * rect.width
            sy = rect.top + (1.0 - (clip[1] / clip[3] * 0.5 + 0.5)) * rect.height
            distance = float(np.linalg.norm(light.position - cam_pos))
            radius = max(self.physics.height * 0.12, distance * 0.012)
            radius_px = radius / (distance * half_tan) * rect.height * 0.5
            pixels = math.hypot(sx - x, sy - y)
            if pixels <= max(radius_px * 1.6, 14.0) and pixels < best_dist:
                if self.physics.visibility(cam_pos, [light.position], skip_end=1.5) > 0:
                    best, best_dist = i, pixels
        lighting.picked = best                    # -1 khi click ra ngoài: Delete quay lại xoá gaussian
        if best >= 0:
            lighting.selected = best
            self.editor.set_status(f"{lighting.lights[best].name} selected: press Delete to remove it")
        return best >= 0
```

`Application.delete_key(self)`

Dòng 282–290

Mô tả trong mã nguồn. Delete: xoá khối đèn đang chọn trong cảnh; không có thì xoá gaussian như trước.

```
    def delete_key(self):
        """Delete: xoá khối đèn đang chọn trong cảnh; không có thì xoá gaussian như trước."""
        lighting = self.lighting
        if 0 <= lighting.picked < len(lighting.lights):
            name = lighting.lights[lighting.picked].name
            lighting.remove_light(lighting.picked)
            self.editor.set_status(f"{name} deleted")
        else:
            self.editor.delete()
```

`Application.set_mode(self, mode)`

Dòng 292–306

```
    def set_mode(self, mode):
        if mode == self.mode:
            return
        if mode == "survival":
            if self.editor.scene.is_empty:
                return
            self.viewport.camera.velocity[:] = 0
            self._spawn_pending = True       # đặt người chơi khi bản đồ va chạm dựng xong
            self.editor.set_status("Walk: WASD walk, Space jump, Ctrl sprint, Shift sneak, drag to look, "
                                   "F5 see yourself, C sit, V/X/Y gestures")
            self._last_feet = None
        else:
            self.editor.set_status("Fly: WASD fly, Space up, Shift down")
            self._leave_third_person()
        self.mode = mode
```

`Application._spawn_player(self)`

Dòng 308–328

Mô tả trong mã nguồn. Đặt người chơi ở nơi có mặt đất bên dưới: chỗ camera đang đứng, rồi tới điểm camera đang nhìn, cuối cùng là giữa cảnh (camera bay ở xa trong khoảng trống thì rơi ra ngoài thế giới).

```
    def _spawn_player(self):
        """Đặt người chơi ở nơi có mặt đất bên dưới: chỗ camera đang đứng, rồi tới điểm camera đang nhìn,
        cuối cùng là giữa cảnh (camera bay ở xa trong khoảng trống thì rơi ra ngoài thế giới)."""
        camera, physics = self.viewport.camera, self.physics
        eye = None
        if physics.flat_ground:
            pos = camera.position()
            eye = np.array([pos[0], physics.eye_height, pos[2]])
        for point in (() if eye is not None else (camera.position(), camera.target)):
            ground = physics.drop_height(point[0], point[2], point[1], physics.height * 0.3)
            if ground is not None:     # bắt đầu cao hơn mặt đất một chút, kẻo chân nằm dưới bề mặt
                eye = np.array([point[0], max(point[1], ground + physics.eye_height * 1.1), point[2]])
                break
        if eye is None and not physics.flat_ground:
            center = self.lighting.center
            top = physics.world.max_y + physics.height
            ground = physics.drop_height(center[0], center[2], top, physics.height * 0.3)
            eye = np.array([center[0], (top if ground is None else ground) + physics.eye_height * 1.5, center[2]])
        physics.player.place(eye, physics.eye_height)
        camera.set_eye(eye)
        self._spawn_pending = False
```

`Application.toggle_mode(self)`

Dòng 330–331

```
    def toggle_mode(self):
        self.set_mode("creative" if self.mode == "survival" else "survival")
```

`Application.walk(self, forward, right, jump, sprint, sneak, dt)`

Dòng 333–357

Mô tả trong mã nguồn. Một bước đi bộ của chế độ sinh tồn; camera đi theo mắt người chơi.

Giải thích. Một bước đi bộ của chế độ sinh tồn: lấy vector vào từ phím, gọi `Physics.walk`, rồi đặt camera theo mắt người chơi.

```
    def walk(self, forward, right, jump, sprint, sneak, dt):
        """Một bước đi bộ của chế độ sinh tồn; camera đi theo mắt người chơi."""
        if self._spawn_pending:
            if not self.physics.ready:
                return
            self._spawn_player()
        camera, pose = self.viewport.camera, self.pose
        if forward or right or jump:
            pose.stand()                       # đi / nhảy thì đứng dậy
        if pose.sitting:
            forward = right = 0
            jump = False
        self.physics.walk(forward, right, camera.yaw, jump, sprint, sneak, dt)
        if self.physics.ready:
            player, height = self.physics.player, self.physics.height
            feet = player.feet.copy()
```

```
            walked = 0.0
            if self._last_feet is not None:
                walked = float(np.hypot(feet[0] - self._last_feet[0], feet[2] - self._last_feet[2]))
                if walked > height * 2.0:      # dịch chuyển (hồi sinh), không phải bước đi
                    walked = 0.0
            self._last_feet = feet
            pose.update(dt, walked, sneak, height)
            camera.set_eye(player.eye(self.physics.eye_height * pose.stance, dt))
            self._update_avatar(feet, walked, dt)
```

`Application.cycle_view(self)`

Dòng 360–374

Mô tả trong mã nguồn. F5: góc nhìn thứ nhất -> thứ ba (sau lưng) -> thứ ba (trước mặt) -> thứ nhất.

```
    def cycle_view(self):
        """F5: góc nhìn thứ nhất -> thứ ba (sau lưng) -> thứ ba (trước mặt) -> thứ nhất."""
        camera = self.viewport.camera
        if self.mode != "survival":
            self.editor.set_status("F5 (see yourself) works in Walk mode: press M")
            return
        camera.view_mode = (camera.view_mode + 1) % 3
        if camera.view_mode == 0:
            self._leave_third_person()
        else:
            camera.third_distance = 0.0        # tính lại ở khung hình kế tiếp
            self._last_feet = None
            if self.body_yaw is None or camera.view_mode == 1:
                self.body_yaw = camera.yaw
        self.editor.set_status(("First person", "Third person (behind)", "Third person (front)")[camera.view_mode])
```

`Application._leave_third_person(self)`

Dòng 376–380

```
    def _leave_third_person(self):
        camera = self.viewport.camera
        camera.view_mode = 0
        camera.third_distance = 0.0
        self.viewport.avatar = None
```

`Application._update_avatar(self, feet, walked, dt)`

Dòng 382–418

Mô tả trong mã nguồn. Hướng thân, ma trận các khớp và khoảng lùi camera (rút ngắn khi sau lưng có tường).

```
    def _update_avatar(self, feet, walked, dt):
        """Hướng thân, ma trận các khớp và khoảng lùi camera (rút ngắn khi sau lưng có tường)."""
        camera, physics, pose = self.viewport.camera, self.physics, self.pose
        if not camera.view_mode:
            self.viewport.avatar = None
            return
        # thân quay theo hướng nhìn khi đi; đứng yên thì chỉ quay khi đầu lệch quá 50 độ (như Minecraft)
        diff = (camera.yaw - self.body_yaw + math.pi) % (2 * math.pi) - math.pi
        limit = math.radians(50.0)
        if walked > 1e-6 or pose.gesture == "zombie":
            self.body_yaw += diff * min(dt * 12.0, 1.0)
        elif abs(diff) > limit:
            self.body_yaw += diff - math.copysign(limit, diff)
        head_yaw = (camera.yaw - self.body_yaw + math.pi) % (2 * math.pi) - math.pi
        head_pitch = float(np.clip(camera.pitch, -1.2, 1.2))
        matrices = limb_matrices(feet, self.body_yaw, head_yaw, head_pitch, pose.joints(), physics.height / 32.0)
        brightness = (1.0, 1.0, 1.0)
        if self.lighting.enabled:
            brightness = tuple(float(np.clip(0.55 + 0.6 * c, 0.35, 1.0)) for c in self.lighting.ambient_color())
        self.viewport.avatar = (matrices, brightness)

        wanted = physics.height * (2.4 if camera.view_mode == 1 else 2.0)
        eye, forward = camera.position(), camera.forward()
        direction = -forward if camera.view_mode == 1 else forward
        distance = wanted
        if physics.has_world and physics.world.segment_blocked(eye, eye + direction * wanted, skip=0.0):
            low, high = 0.0, 1.0                  # tìm đoạn dài nhất chưa chạm tường
            for _ in range(7):
                mid = (low + high) / 2
                if physics.world.segment_blocked(eye, eye + direction * wanted * mid, skip=0.0):
                    high = mid
                else:
                    low = mid
            distance = max(low * wanted - physics.height * 0.1, physics.height * 0.15)
        current = camera.third_distance
        camera.third_distance = distance if current <= 0.0 or distance < current else \
            current + (distance - current) * min(dt * 6.0, 1.0)
```

`Application.update_world(self, dt)`

Dòng 420–439

Mô tả trong mã nguồn. Mỗi khung hình: bản đồ va chạm (luồng nền), thời gian trong ngày, khối đèn rơi.

```
    def update_world(self, dt):
        """Mỗi khung hình: bản đồ va chạm (luồng nền), thời gian trong ngày, khối đèn rơi."""
        editor = self.editor
        versions = (editor.geometry_version, editor.transform_version, editor.deleted_version)
        if versions != self._world_versions:
            self._world_versions = versions
            self._update_bounds()
            self.physics.invalidate()
            for light in self.lighting.lights:
                light.resting = False
        lights_fall = any(l.falls and not l.resting for l in self.lighting.lights)
        # cần bản đồ để cảnh che mặt trời và các khối đèn
        sun_visible = self.lighting.enabled and (self.lighting.sun_on or bool(self.lighting.lights))
        self.physics.ensure_world(editor.scene, self.mode == "survival" or lights_fall or sun_visible)
        self.viewport.occluder = self.physics
        self.lighting.update(dt, self.physics)
        while self.lighting.events:
            editor.set_status(self.lighting.events.pop(0))
        self.viewport.clock += dt
        self.viewport.unit = self.physics.height
```

`Application.toggle_grid(self)`

Dòng 441–442

```
    def toggle_grid(self):
        self.viewport.show_grid = not self.viewport.show_grid
```

`Application.toggle_sh(self)`

Dòng 444–445

```
    def toggle_sh(self):
        self.viewport.splat_renderer.use_sh = not self.viewport.splat_renderer.use_sh
```

`Application.set_flip(self, flip)`

Dòng 447–450

```
    def set_flip(self, flip):
        self.editor.scene.flip = flip
        self.editor.notify_transform_changed()
        self.frame_scene()
```

`Application.exit_brush(self)`

Dòng 452–454

```
    def exit_brush(self):
        self.editor.brush.active = False
        self.editor.brush.end_stroke()
```

`Application.shortcuts_window_toggle(self)`

Dòng 456–457

```
    def shortcuts_window_toggle(self):
        self.shortcuts_window.toggle()
```

`Application.paint(self, x, y)`

Dòng 459–462

```
    def paint(self, x, y):
        rect = self.layout.viewport
        if not rect.is_empty:
            self.editor.paint(x, y, self.viewport.view_projection(rect), rect)
```

`Application.drag_speed(self)`

Dòng 464–466

Mô tả trong mã nguồn. Tốc độ kéo thanh trượt vị trí, tỉ lệ theo khoảng cách camera.

Giải thích. Công thức cốt lõi của hàm này:

$$\text{speed} = \max(0.005\,d,\ 10^{-4})$$

```
    def drag_speed(self):
        """Tốc độ kéo thanh trượt vị trí, tỉ lệ theo khoảng cách camera."""
        return max(self.viewport.camera.distance * 0.005, 1e-4)
```

`Application._draw_gui(self)`

Dòng 469–476

```
    def _draw_gui(self):
        menu_height = self.menu_bar.draw()
        width, height = self.window.size
        self.layout.update(width, height, menu_height, self.status_bar.height())
        self.side_panel.draw(self.layout.side_panel)
        self.status_bar.draw(self.layout.status_bar)
        self.shortcuts_window.draw()
        self.overlay.draw(self.layout.viewport)
```

`Application.run(self)`

Dòng 478–500

Giải thích. Vòng lặp chính. Đọc ngược từ trên xuống dưới chính là thứ tự một khung hình: sự kiện → cập nhật camera/thế giới → dựng ImGui → đồng bộ GPU → vẽ cảnh → vẽ ImGui → hiển thị.

```
    def run(self):
        last_time = time.perf_counter()
        while not self.window.should_close:
            Window.poll_events()
            now = time.perf_counter()
            dt = now - last_time
            self.input.update(dt)
            self.update_world(dt)
            last_time = now

            self.gui.begin_frame()
            self._draw_gui()
            self.gui.end_frame()

            self.viewport.sync(self.editor)
            self.viewport.render(self.layout.viewport, self.window.size, self.window.framebuffer_size)
            self.gui.draw()
            self.window.swap_buffers()

            self.actions.run_pending()
            self.fps.tick(now)
        self.gui.shutdown()
        self.window.destroy()
```

`main(argv=None)`

Dòng 503–509

```
def main(argv=None):
    argv = sys.argv[1:] if argv is None else argv
    app = Application()
    if argv:
        app.open_file(argv[0])
        app.add_files(argv[1:])
    app.run()
```

Chương 1. Thư mục core: logic thuần

Thư mục `core` chứa **toàn bộ logic thuần**: toán học, dữ liệu gaussian, lớp (layer), cảnh (scene), ánh sáng, vật lý, LOD, hoạt họa, lịch sử hoàn tác. Quy tắc thiết kế quan trọng nhất của thư mục này là **không import OpenGL, GLFW hay ImGui**; mọi thứ chạy bằng NumPy nên có thể kiểm thử mà không cần cửa sổ. Các thư mục khác phụ thuộc vào `core`, còn `core` chỉ phụ thuộc `io` (để đọc/ghi PLY) và không phụ thuộc ngược lại ai.

Dữ liệu chảy trong `core` theo chuỗi: `Splats` (mảng gaussian đã giải mã) → `Layer` (một file PLY cộng phép biến đổi và vùng chọn) → `Scene` (nhiều layer). Các module còn lại (`lod`, `lighting`, `physics`, `avatar_pose`) đọc dữ liệu đó để phục vụ hiển thị và tương tác.

1.1. `splat_editor/core/math3d.py`

135 dòng

Vai trò. Thư viện toán 3D nhỏ, chỉ dùng `numpy`. Gồm ba nhóm: ma trận camera (`perspective`, `orthographic`, `look_at`), phép quay (`euler_to_matrix`, `matrix_to_euler`, `axis_angle_matrix`) và quaternion (`matrix_to_quat`, `quat_mul`, `quat_to_matrix`). Toàn bộ chương trình dùng chung các hàm này nên quy ước (vector cột, tay phải, Y lên, nhìn dọc $-z$) được thống nhất ở một nơi. Công thức chi tiết xem §1.1, §1.1, §1.4.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Toán 3D: ma trận chiếu / nhìn, góc Euler, quaternion."""

"""
proj = perspective(fov_y_deg=45, aspect=16 / 9, near=0.01, far=1000)
view = look_at(eye=[0, 1, 5], target=[0, 0, 0], up=[0, 1, 0])

R = euler_to_matrix([0, 90, 0])        # góc Euler (độ) quanh X, Y, Z -> ma trận 3x3
angles = matrix_to_euler(R)            # ngược lại
R2 = axis_angle_matrix([0, 1, 0], 30)  # xoay 30 độ quanh trục Y

q = matrix_to_quat(R)                  # quaternion (w, x, y, z)
q2 = quat_mul(q, q)                    # nhân hai quaternion (xoay liên tiếp)
R3 = quat_to_matrix(q2)
"""

```

Cơ sở toán: Quaternion

Quaternion đơn vị $q = (w, x, y, z)$ biểu diễn phép quay góc θ quanh trục n: $w = \cos\frac{\theta}{2}$, $(x, y, z) = \sin\frac{\theta}{2}\, n$. Dùng quaternion vì không bị gimbal, nhân gọn và dễ chuẩn hóa.

Quaternion sang ma trận (`quat_to_matrix`, giống hệt trong `splat_prep.vert`).

$$R = \begin{bmatrix} 1-2(y^2+z^2) & 2(xy-wz) & 2(xz+wy) \\ 2(xy+wz) & 1-2(x^2+z^2) & 2(yz-wx) \\ 2(xz-wy) & 2(yz+wx) & 1-2(x^2+y^2) \end{bmatrix}$$

Trong GLSL, `mat3(...)` điền theo **cột**, nên các số liệt kê trong shader là cột của cùng ma trận này.

Ma trận sang quaternion (`matrix_to_quat`, phương pháp Shepperd). Tránh chia cho số gần 0 bằng cách chọn nhánh theo phần tử lớn nhất.

- trace $= R_{00} + R_{11} + R_{22} > 0$: $s = 2\sqrt{\text{trace} + 1}$, $w = s/4$, $x = (R_{21} - R_{12})/s$, $y = (R_{02} - R_{20})/s$, $z = (R_{10} - R_{01})/s$.
- Ngược lại, chọn nhánh theo R_{00}, R_{11} hoặc R_{22} lớn nhất: $s = 2\sqrt{1 + R_{ii} - R_{jj} - R_{kk}}$, phần tử $i = s/4$, các phần tử còn lại $= (R_{ij} \pm R_{ji})/s$.

Nhân quaternion (`quat_mul`, tích Hamilton).

$$\begin{aligned} w &= a_w b_w - a_x b_x - a_y b_y - a_z b_z, & x &= a_w b_x + a_x b_w + a_y b_z - a_z b_y, \\ y &= a_w b_y - a_x b_z + a_y b_w + a_z b_x, & z &= a_w b_z + a_x b_y - a_y b_x + a_z b_w. \end{aligned}$$

$a \otimes b$ là quay b trước rồi a (giống $R_a R_b$).

Lượng tử hóa quaternion lên GPU (`splat_renderer.pack`, `splat_prep.vert`). Mỗi thành phần thuộc $[-1, 1]$ được lưu 8 bit: $u = \text{clip}(\text{round}(q \cdot 127) + 128, 1, 255)$. Shader giải bằng $q = \text{normalize}((u - 128)/127)$, chuẩn hóa lại để bù sai số.

Cơ sở toán: Camera và phép chiếu

Các ma trận camera dưới đây nằm trong `core/math3d.py`; phần camera quỹ đạo `OrbitCamera` (`render/camera.py`) được trình bày ở mục 4.2.

Ma trận nhìn (`look_at(e, target, up)`).

$$f = \frac{\text{target} - e}{\|\cdot\|}, \quad r = \frac{f \times \text{up}}{\|\cdot\|}, \quad u = r \times f, \quad V = \begin{bmatrix} r^\top & -r\cdot e \\ u^\top & -u\cdot e \\ -f^\top & f\cdot e \\ 0\,0\,0 & 1 \end{bmatrix}.$$

Cột 3 là $-(Re)$: đưa mắt về gốc rồi xoay hệ trục camera về $(x, y, z) = (\text{phải}, \text{lên}, -\text{trước})$. Khối 3×3 trái trên gọi là W (trực giao).

Phép chiếu phối cảnh (`perspective(fov, a, n, f)`). Với $k = 1/\tan(\text{fov}/2)$:

$$P = \begin{bmatrix} k/a & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

- clip $= Pv$, $c_w = -v_z$ (độ sâu dương). NDC $= \text{clip}_{xyz}/c_w \in [-1, 1]$ (**chia phối cảnh**, vật xa nhỏ đi).
- Hàng 3: $z_{ndc} = \dfrac{(f+n)v_z + 2fn}{(n-f)(-v_z)}$ ánh xạ $v_z = -n \mapsto -1$ và $v_z = -f \mapsto +1$. Quan hệ này phi tuyến, độ phân giải tập trung gần camera.
- Camera dùng `near = distance*0.001`, `far = near*1e6` (`camera.projection_matrix`).

Phép chiếu trực giao (`orthographic(h, n, f)`). $x' = x/h$, $y' = y/h$, $z' = -\dfrac{2z}{f-n} - \dfrac{f+n}{f-n}$. Hộp $[-h, h]^2 \times [n, f]$ ánh xạ vào $[-1, 1]^3$. Dùng cho **bản đồ bóng** mặt trời (tia song song, không chia phối cảnh). Hàm này bị định nghĩa trùng hai lần trong file, bản sau ghi đè bản trước, cùng công thức.

Ví dụ số (`report/project3d_table.tex`). $p_w = (0.5, 1, 0.5) \to v = Vp = (0.109, 0.293, -2.536) \to \text{clip} = Pv = (0.197, 0.709, 2.341, 2.536)$, $c_w = 2.536 \to \text{NDC} = (0.078, 0.279, 0.923) \to$ pixel $(215.5, 108.1)$ theo mục 2.2.

Khai báo mức module (dòng 16–18)

```
16 import math
17
18 import numpy as np
```

`perspective(fov_y_deg, aspect, near, far)`

Dòng 23–31

Giải thích. Dựng ma trận chiếu phối cảnh OpenGL. `f=1/tan(fov/2)` là tỉ lệ thu phóng; hàng cuối $(0, 0, -1, 0)$ làm cho $c_w = -v_z$ nên phép chia phối cảnh sau đó cho vật xa nhỏ đi. Hai phần tử `m[2,2]`, `m[2,3]` ánh xạ $[-near, -far]$ sang $[-1, 1]$ (xem §1.1).

Công thức.

$$k = \frac{1}{\tan(\text{fov}/2)}, \qquad P = \begin{bmatrix} k/a & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & \dfrac{f+n}{n-f} & \dfrac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Với điểm trong không gian camera v: clip $= Pv$, $c_w = -v_z$ và

$$\text{NDC} = \frac{\text{clip}_{xyz}}{c_w}, \qquad z_{ndc} = \frac{(f+n)\,v_z + 2fn}{(n-f)\,(-v_z)} \;\Rightarrow\; v_z = -n \mapsto -1, \quad v_z = -f \mapsto +1.$$

```
23 def perspective(fov_y_deg, aspect, near, far):
24     f = 1.0 / math.tan(math.radians(fov_y_deg) / 2)
25     m = np.zeros((4, 4), np.float32)
```

```
    m[0, 0] = f / aspect
    m[1, 1] = f
    m[2, 2] = (far + near) / (near - far)
    m[2, 3] = 2 * far * near / (near - far)
    m[3, 2] = -1
    return m
```

`orthographic(half_size, near, far)`

Dòng 34–40

Mô tả trong mã nguồn. Phép chiếu trực giao đối xứng (dùng cho bản đồ bóng của mặt trời): hộp [-h,h]^2 x [near,far].

Giải thích. Phép chiếu trực giao dùng cho bản đồ bóng của mặt trời. **Lưu ý:** hàm này bị định nghĩa trùng hai lần trong file; bản định nghĩa sau ghi đè bản trước và hai bản giống hệt công thức, nên không gây lỗi nhưng có thể xóa một bản cho gọn.

Công thức.

$$x' = \frac{x}{h}, \quad y' = \frac{y}{h}, \quad z' = -\frac{2z}{f-n} - \frac{f+n}{f-n} \qquad \left([-h,h]^2 \times [n,f] \to [-1,1]^3\right)$$

```
def orthographic(half_size, near, far):
    """Phép chiếu trực giao đối xứng (dùng cho bản đồ bóng của mặt trời): hộp [-h,h]^2 x [near,far]."""
    m = np.identity(4, np.float32)
    m[0, 0] = m[1, 1] = 1.0 / half_size
    m[2, 2] = -2.0 / (far - near)
    m[2, 3] = -(far + near) / (far - near)
    return m
```

`orthographic(half_size, near, far)`

Dòng 43–49

Mô tả trong mã nguồn. Phép chiếu trực giao đối xứng (bản đồ bóng của mặt trời): hộp [-h,h]^2 x [near,far].

Giải thích. Phép chiếu trực giao dùng cho bản đồ bóng của mặt trời. **Lưu ý:** hàm này bị định nghĩa trùng hai lần trong file; bản định nghĩa sau ghi đè bản trước và hai bản giống hệt công thức, nên không gây lỗi nhưng có thể xóa một bản cho gọn.

Công thức.

$$x' = \frac{x}{h}, \quad y' = \frac{y}{h}, \quad z' = -\frac{2z}{f-n} - \frac{f+n}{f-n} \qquad \left([-h,h]^2 \times [n,f] \to [-1,1]^3\right)$$

```
def orthographic(half_size, near, far):
    """Phép chiếu trực giao đối xứng (bản đồ bóng của mặt trời): hộp [-h,h]^2 x [near,far]."""
    m = np.identity(4, np.float32)
    m[0, 0] = m[1, 1] = 1.0 / half_size
    m[2, 2] = -2.0 / (far - near)
    m[2, 3] = -(far + near) / (far - near)
    return m
```

`look_at(eye, target, up)`

Dòng 52–63

Giải thích. Dựng ma trận nhìn (view). Ba vector $r, u, -f$ là hàng của khối quay; cột tịnh tiến $-R\,e$ đưa mắt về gốc. Dòng `m[:3, 3] = -m[:3, :3] @ eye` chính là bước đó.

Công thức.

$$f = \frac{t-e}{\|t-e\|}, \quad r = \frac{f \times u_{up}}{\|f \times u_{up}\|}, \quad u = r \times f, \qquad V = \begin{bmatrix} r^\top & -r\cdot e \\ u^\top & -u\cdot e \\ -f^\top & f\cdot e \\ 0\,0\,0 & 1 \end{bmatrix} = \begin{bmatrix} W & -We \\ 0 & 1 \end{bmatrix}$$

```
def look_at(eye, target, up):
    forward = target - eye
    forward /= np.linalg.norm(forward)
    right = np.cross(forward, up)
    right /= np.linalg.norm(right)
    true_up = np.cross(right, forward)
    m = np.identity(4, np.float32)
    m[0, :3] = right
    m[1, :3] = true_up
    m[2, :3] = -forward
    m[:3, 3] = -m[:3, :3] @ eye
    return m
```

`euler_to_matrix(degrees)`

Dòng 68–75

Mô tả trong mã nguồn. Góc Euler (độ) quanh X, Y, Z -> ma trận xoay R = Rz * Ry * Rx.

Giải thích. Ghép ba ma trận quay đơn theo thứ tự $R = R_z R_y R_x$ (áp dụng X trước). Nhận đầu vào là **độ**, đổi sang radian bằng `np.radians`.

Công thức.

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_x & -s_x \\ 0 & s_x & c_x \end{bmatrix},\ R_y = \begin{bmatrix} c_y & 0 & s_y \\ 0 & 1 & 0 \\ -s_y & 0 & c_y \end{bmatrix},\ R_z = \begin{bmatrix} c_z & -s_z & 0 \\ s_z & c_z & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad R = R_z R_y R_x$$

```
68 def euler_to_matrix(degrees):
69     """Góc Euler (độ) quanh X, Y, Z -> ma trận xoay R = Rz * Ry * Rx."""
70     x, y, z = np.radians(degrees)
71     cx, sx, cy, sy, cz, sz = math.cos(x), math.sin(x), math.cos(y), math.sin(y), math.cos(z), math.sin(z)
72     rx = np.array([[1, 0, 0], [0, cx, -sx], [0, sx, cx]])
73     ry = np.array([[cy, 0, sy], [0, 1, 0], [-sy, 0, cy]])
74     rz = np.array([[cz, -sz, 0], [sz, cz, 0], [0, 0, 1]])
75     return rz @ ry @ rx
```

`matrix_to_euler(m)`

Dòng 78–87

Mô tả trong mã nguồn. Ngược lại của euler_to_matrix: ma trận R = Rz * Ry * Rx -> góc (độ) quanh X, Y, Z.

Giải thích. Phép ngược của `euler_to_matrix`. Nhánh `else` xử lý khóa gimbal ($y = \pm 90°$), khi đó chỉ còn xác định được tổng/hiệu của x và z nên gộp hết vào z.

Công thức.

$$y = \arcsin(-R_{20}), \quad x = \operatorname{atan2}(R_{21}, R_{22}), \quad z = \operatorname{atan2}(R_{10}, R_{00})$$

Khóa gimbal $|R_{20}| \geq 0.999999$: $x = 0$, $z = \operatorname{atan2}(-R_{01}, R_{11})$.

```
78 def matrix_to_euler(m):
79     """Ngược lại của euler_to_matrix: ma trận R = Rz * Ry * Rx -> góc (độ) quanh X, Y, Z."""
80     y = math.asin(min(max(-m[2, 0], -1.0), 1.0))
81     if abs(m[2, 0]) < 0.999999:
82         x = math.atan2(m[2, 1], m[2, 2])
83         z = math.atan2(m[1, 0], m[0, 0])
84     else:                                  # khoá gimbal (Y = ±90°): gộp hết vào Z
85         x = 0.0
86         z = math.atan2(-m[0, 1], m[1, 1])
87     return [math.degrees(x), math.degrees(y), math.degrees(z)]
```

`axis_angle_matrix(axis, degrees)`

Dòng 90–97

Mô tả trong mã nguồn. Ma trận xoay ‘degrees‘ độ quanh trục ‘axis‘ (công thức Rodrigues).

Giải thích. Công thức Rodrigues; trục được chuẩn hóa trước. Dùng cho thao tác xoay theo trục bất kỳ (ví dụ `rotate_about`).

Công thức.

$$R = I + \sin a\,[k]_\times + (1 - \cos a)\,[k]_\times^2, \qquad [k]_\times = \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}$$

```
90 def axis_angle_matrix(axis, degrees):
91     """Ma trận xoay `degrees` độ quanh trục `axis` (công thức Rodrigues)."""
92     x, y, z = np.asarray(axis, float) / np.linalg.norm(axis)
93     a = math.radians(degrees)
94     c, s, t = math.cos(a), math.sin(a), 1 - math.cos(a)
95     return np.array([[t * x * x + c, t * x * y - s * z, t * x * z + s * y],
96                      [t * x * y + s * z, t * y * y + c, t * y * z - s * x],
97                      [t * x * z - s * y, t * y * z + s * x, t * z * z + c]])
```

`matrix_to_quat(m)`

Dòng 100–115

Mô tả trong mã nguồn. Ma trận xoay 3x3 -> quaternion (w, x, y, z).

Giải thích. Phương pháp Shepperd: chọn nhánh theo phần tử lớn nhất trong (trace, R_{00}, R_{11}, R_{22}) để mẫu số s không bao giờ gần 0.

Công thức.

$$\text{tr} = R_{00} + R_{11} + R_{22} > 0: \quad s = 2\sqrt{\text{tr} + 1},\ q = \left(\frac{s}{4},\ \frac{R_{21} - R_{12}}{s},\ \frac{R_{02} - R_{20}}{s},\ \frac{R_{10} - R_{01}}{s}\right)$$

Nhánh còn lại (phần tử đường chéo lớn nhất R_{ii}): $s = 2\sqrt{1 + R_{ii} - R_{jj} - R_{kk}}$, $q_i = s/4$.

```
def matrix_to_quat(m):
    """Ma trận xoay 3x3 -> quaternion (w, x, y, z)."""
    trace = m[0, 0] + m[1, 1] + m[2, 2]
    if trace > 0:
        s = 2 * math.sqrt(trace + 1)
        q = [0.25 * s, (m[2, 1] - m[1, 2]) / s, (m[0, 2] - m[2, 0]) / s, (m[1, 0] - m[0, 1]) / s]
    elif m[0, 0] > m[1, 1] and m[0, 0] > m[2, 2]:
        s = 2 * math.sqrt(1 + m[0, 0] - m[1, 1] - m[2, 2])
        q = [(m[2, 1] - m[1, 2]) / s, 0.25 * s, (m[0, 1] + m[1, 0]) / s, (m[0, 2] + m[2, 0]) / s]
    elif m[1, 1] > m[2, 2]:
        s = 2 * math.sqrt(1 + m[1, 1] - m[0, 0] - m[2, 2])
        q = [(m[0, 2] - m[2, 0]) / s, (m[0, 1] + m[1, 0]) / s, 0.25 * s, (m[1, 2] + m[2, 1]) / s]
    else:
        s = 2 * math.sqrt(1 + m[2, 2] - m[0, 0] - m[1, 1])
        q = [(m[1, 0] - m[0, 1]) / s, (m[0, 2] + m[2, 0]) / s, (m[1, 2] + m[2, 1]) / s, 0.25 * s]
    return np.array(q)
```

`quat_mul(a, b)`

Dòng 118–125

Mô tả trong mã nguồn. Nhân quaternion a (4,) với từng quaternion trong b (N,4).

Giải thích. Tích Hamilton vector hóa: `a` là một quaternion $(4,)$ nhân với hàng loạt quaternion `b` $(N, 4)$. Dùng khi xoay cả layer: `q_rot` nhân với mọi quaternion gaussian.

Công thức.

$$a \otimes b = \begin{bmatrix} a_w b_w - a_x b_x - a_y b_y - a_z b_z \\ a_w b_x + a_x b_w + a_y b_z - a_z b_y \\ a_w b_y - a_x b_z + a_y b_w + a_z b_x \\ a_w b_z + a_x b_y - a_y b_x + a_z b_w \end{bmatrix}, \qquad R(a \otimes b) = R(a)\,R(b)$$

```
def quat_mul(a, b):
    """Nhân quaternion a (4,) với từng quaternion trong b (N,4)."""
    aw, ax, ay, az = a
    bw, bx, by, bz = b[:, 0], b[:, 1], b[:, 2], b[:, 3]
    return np.stack([aw * bw - ax * bx - ay * by - az * bz,
                     aw * bx + ax * bw + ay * bz - az * by,
                     aw * by - ax * bz + ay * bw + az * bx,
                     aw * bz + ax * by - ay * bx + az * bw], axis=1)
```

`quat_to_matrix(q)`

Dòng 128–135

Mô tả trong mã nguồn. (N,4) quaternion (w,x,y,z) -> (N,3,3) ma trận xoay.

Giải thích. Đổi hàng loạt quaternion $(N, 4)$ thành $(N, 3, 3)$. Cùng công thức với đoạn GLSL trong `splat_prep.vert` (chỉ khác là shader điền theo cột).

Công thức.

$$R = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}, \qquad q = \left(\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\hat{n}\right)$$

```
def quat_to_matrix(q):
    """(N,4) quaternion (w,x,y,z) -> (N,3,3) ma trận xoay."""
    w, x, y, z = q[:, 0], q[:, 1], q[:, 2], q[:, 3]
    return np.stack([
        np.stack([1 - 2 * (y * y + z * z), 2 * (x * y - w * z), 2 * (x * z + w * y)], axis=1),
        np.stack([2 * (x * y + w * z), 1 - 2 * (x * x + z * z), 2 * (y * z - w * x)], axis=1),
        np.stack([2 * (x * z - w * y), 2 * (y * z + w * x), 1 - 2 * (x * x + y * y)], axis=1),
    ], axis=1)
```

1.2. `splat_editor/core/transform.py`

43 dòng

Vai trò. Phép biến đổi của một layer: vị trí, góc Euler (độ) và tỉ lệ đều. Đây là dữ liệu thuần (`dataclass`); **dữ liệu gaussian không bị sửa** khi người dùng kéo/xoay/phóng layer, chỉ phép biến đổi này đổi, nhờ đó thao tác tức thời với hàng chục triệu gaussian.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Phép biến đổi của một lớp: vị trí, góc xoay (độ), tỉ lệ đều."""

"""
t = Transform()
t.position = [1.0, 0.0, 0.0]
t.rotation = [0.0, 45.0, 0.0]          # độ, quanh X Y Z
t.scale = 1.5                           # tỉ lệ đều

R = t.rotation_matrix()                 # ma trận 3x3
p = t.translation()                     # numpy (3,)
t.rotate_about(delta_matrix, pivot=[0, 0, 0])   # xoay thêm quanh một điểm (trong toạ độ thế giới)
t.reset()                               # về vị trí 0, góc 0, tỉ lệ 1
"""

```

Khai báo mức module (dòng 15–19)

```
from dataclasses import dataclass, field

import numpy as np

from .math3d import euler_to_matrix, matrix_to_euler
```

`class Transform`

Dòng 22–43

Giải thích. Ba trường: `position` (3 số), `rotation` (độ quanh X, Y, Z) và `scale` (một số, tỉ lệ đều). Dùng `default_factory` để mỗi `Transform` có danh sách riêng.

Các phương thức: `reset`, `rotation_matrix`, `translation`, `rotate_about`.

```
@dataclass
class Transform:
    position: list = field(default_factory=lambda: [0.0, 0.0, 0.0])
    rotation: list = field(default_factory=lambda: [0.0, 0.0, 0.0])   # độ, quanh X Y Z
    scale: float = 1.0
```

`Transform.reset(self)`

Dòng 28–31

```
    def reset(self):
        self.position = [0.0, 0.0, 0.0]
        self.rotation = [0.0, 0.0, 0.0]
        self.scale = 1.0
```

`Transform.rotation_matrix(self)`

Dòng 33–34

```
    def rotation_matrix(self):
        return euler_to_matrix(self.rotation)
```

`Transform.translation(self)`

Dòng 36–37

```
    def translation(self):
        return np.array(self.position, float)
```

`Transform.rotate_about(self, delta, pivot)`

Dòng 39–43

Mô tả trong mã nguồn. Xoay thêm ‘delta‘ (ma trận 3x3, trong toạ độ thế giới) quanh điểm ‘pivot‘.

Giải thích. Xoay thêm một ma trận `delta` quanh điểm `pivot` trong tọa độ thế giới: $R' = \Delta R$, $t' = \Delta(t - c) + c$. Góc Euler mới lấy lại bằng `matrix_to_euler`.

Công thức.

$$R' = \Delta R, \qquad t' = \Delta\,(t - c) + c$$

```
    def rotate_about(self, delta, pivot):
        """Xoay thêm `delta` (ma trận 3x3, trong toạ độ thế giới) quanh điểm `pivot`."""
        pivot = np.asarray(pivot, float)
        self.rotation = matrix_to_euler(delta @ self.rotation_matrix())
        self.position = list(delta @ (self.translation() - pivot) + pivot)
```

1.3. `splat_editor/core/splats.py`

77 dòng

Vai trò. `Splats` là mảng cấu trúc-của-mảng (structure of arrays) giữ toàn bộ gaussian **đã giải mã** để vẽ: tâm, màu bậc 0, độ đục, scale, quaternion và hệ số SH bậc cao. Cách tổ chức theo cột giúp NumPy xử lý vector hóa và giúp gửi lên GPU theo từng đoạn. Điểm đáng chú ý là trường `sh_frames`: xoay SH được **hoãn** (lazy) để không phải giữ thêm một bản sao SH rất lớn.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Tập gaussian đã giải mã sẵn để render."""

"""
splats = SplatCodec.decode(data)        # từ cột PLY ra Splats (xem io/codec.py)
splats.count                            # số gaussian
splats.positions.shape                  # (N, 3)
splats.sh_count                         # K = 0, 3, 8 hoặc 15

kept = splats.subset(~deleted_mask)     # bản mới chỉ gồm các gaussian được giữ
merged = Splats.concat([a, b, c])       # gộp nhiều Splats thành một
sh_world = splats.world_sh(index=None)  # hệ số SH đã xoay về toạ độ thế giới (float32)
"""

```

Khai báo mức module (dòng 14–18)

```
from dataclasses import dataclass

import numpy as np

from . import sh
```

`class Splats`

Dòng 21–76

Giải thích. Sáu mảng song song: `positions` $(N, 3)$, `colors` $(N, 3)$, `alphas` $(N,)$, `scales` $(N, 3)$, `rotations` $(N, 4)$, `sh_rest` $(N, K, 3)$. `ARRAYS` liệt kê tên các mảng để `subset`/`concat` xử lý đồng loạt.

Các phương thức: `count`, `sh_count`, `world_sh`, `subset`, `concat`.

```
@dataclass
class Splats:
    positions: np.ndarray   # (N,3) tâm
    colors: np.ndarray      # (N,3) màu bậc 0 = 0.5 + C0 * f_dc (chưa cắt về 0..1)
    alphas: np.ndarray      # (N,)  độ đục 0..1
    scales: np.ndarray      # (N,3) bán kính theo 3 trục (đã exp)
    rotations: np.ndarray   # (N,4) quaternion (w,x,y,z) đã chuẩn hoá
    sh_rest: np.ndarray     # (N,K,3) hệ số SH bậc 1..3 (K = 0, 3, 8, 15), xem core/sh.py; thường là float16
    # ((đầu, cuối, R), ...): sh_rest[đầu:cuối] vẫn ở toạ độ file, phải xoay theo R mới ra toạ độ
    # thế giới. Xoay lười như vậy để khỏi giữ thêm một bản sao SH (bản sao lớn nhất: ~1.4 GB với
    # 15 triệu gaussian); cần SH thế giới thì gọi world_sh. (Renderer không dùng: nó giữ SH ở toạ
    # độ file và xoay hướng nhìn trong shader.)
    sh_frames: tuple = ()

    ARRAYS = ('positions', 'colors', 'alphas', 'scales', 'rotations', 'sh_rest')
```

`Splats.count(self)`

Dòng 37–39

```
    @property
    def count(self):
        return len(self.positions)
```

`Splats.sh_count(self)`

Dòng 41–43

```
41    @property
42    def sh_count(self):
43        return self.sh_rest.shape[1]
```

`Splats.world_sh(self, index=None, k=None)`

Dòng 45–54

Mô tả trong mã nguồn. Hệ số SH (float32, toạ độ thế giới) của các gaussian ‘index‘ (mảng chỉ số; None: tất cả). k: chỉ lấy k hệ số đầu (mỗi bậc SH xoay độc lập nên cắt bớt bậc cao vẫn đúng).

Giải thích. Trả về hệ số SH đã xoay về tọa độ thế giới. Với mỗi khung `(start, end, rot)` trong `sh_frames`, chỉ những hàng nằm trong đoạn đó mới được xoay bằng `sh.rotate`.

```
45    def world_sh(self, index=None, k=None):
46        """Hệ số SH (float32, toạ độ thế giới) của các gaussian `index` (mảng chỉ số; None: tất cả).
47        k: chỉ lấy k hệ số đầu (mỗi bậc SH xoay độc lập nên cắt bớt bậc cao vẫn đúng)."""
48        rows = np.arange(self.count) if index is None else np.asarray(index)
49        out = self.sh_rest[rows, :k].astype(np.float32)
50        for start, end, rot in self.sh_frames:
51            inside = (rows >= start) & (rows < end)
52            if inside.any():
53                out[inside] = sh.rotate(out[inside], rot)
54        return out
```

`Splats.subset(self, mask)`

Dòng 56–62

Mô tả trong mã nguồn. Splats mới chỉ gồm các gaussian có mask True (chỉ dùng cho dữ liệu chưa xoay).

Giải thích. Lọc theo mặt nạ. `np.compress` nhanh hơn chỉ mục boolean khoảng 2 lần với mảng lớn (SH có thể chiếm cỡ 900 MB). Có `assert` không cho dùng khi đã có `sh_frames`.

```
56    def subset(self, mask):
57        """Splats mới chỉ gồm các gaussian có mask True (chỉ dùng cho dữ liệu chưa xoay)."""
58        assert not self.sh_frames
59        if isinstance(mask, np.ndarray) and mask.dtype == bool:
60            # np.compress nhanh hơn a[mask] khoảng 2 lần với mảng lớn (vd. sh_rest ~900 MB)
61            return Splats(*(np.compress(mask, getattr(self, name), axis=0) for name in self.ARRAYS))
62        return Splats(*(getattr(self, name)[mask] for name in self.ARRAYS))
```

`Splats.concat(cls, parts)`

Dòng 64–76

Mô tả trong mã nguồn. Gộp nhiều Splats thành một (giữ thứ tự). Lớp có ít hệ số SH hơn được thêm 0.

Giải thích. Gộp nhiều `Splats`. Layer có ít hệ số SH hơn được đệm 0 (`np.pad`); các khung xoay SH được dịch chỉ số bằng `offset`.

```
64    @classmethod
65    def concat(cls, parts):
66        """Gộp nhiều Splats thành một (giữ thứ tự). Lớp có ít hệ số SH hơn được thêm 0."""
67        if len(parts) == 1:
68            return parts[0]
69        k = max(p.sh_count for p in parts)
70        sh_parts = [np.pad(p.sh_rest, ((0, 0), (0, k - p.sh_count), (0, 0))) for p in parts]
71        frames, offset = [], 0
72        for p in parts:
73            frames += [(offset + start, offset + end, rot) for start, end, rot in p.sh_frames]
74            offset += p.count
75        arrays = [np.concatenate([getattr(p, name) for p in parts]) for name in cls.ARRAYS[:-1]]
76        return cls(*arrays, np.concatenate(sh_parts), tuple(frames))
```

1.4. `splat_editor/core/layer.py`

187 dòng

Vai trò. Một `Layer` tương ứng một file PLY đã mở, cộng phép biến đổi, vùng chọn và cờ xóa mềm. Hai ý tưởng cốt lõi: (1) **hai hệ tọa độ** (tọa độ file và tọa độ thế giới, nối nhau bằng ma trận M); (2) **xóa mềm** (chỉ bật cờ `deleted`) nên xóa và hoàn tác gần như tức thì. Khi lưu, dữ liệu được đổi ngược về tọa độ file để file xuất ra vẫn đúng quy ước của file gốc. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§1.4).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Một lớp (layer) = một file PLY đã mở, kèm phép biến đổi và vùng chọn riêng.

Quy ước toạ độ:
- "toạ độ file": toạ độ gốc trong file PLY.
- "toạ độ thế giới": toạ độ dùng để hiển thị (trục Y hướng lên, lưới nằm trên mặt y = 0).
  Nếu bật `flip`, toạ độ file được xoay 180 độ quanh trục Z (F = diag(-1,-1,1)) trước,
  vì phần lớn file 3DGS (từ COLMAP) có trục Y hướng xuống.
- Phép biến đổi của lớp (vị trí / góc xoay / tỉ lệ) tính trong toạ độ thế giới:
      p_thế_giới = s * R * F * p_file + t
- Khi lưu, ta đổi ngược về toạ độ file: p_lưu = F * p_thế_giới, nên file lưu ra vẫn
  cùng quy ước với file gốc (mở bằng phần mềm khác vẫn đúng chiều).
- Hệ số SH bậc cao (màu theo hướng nhìn) xoay cùng gaussian, cả khi vẽ lẫn khi lưu. Khi vẽ,
  phép xoay được ghi lại (Splats.sh_frames) và renderer xoay lúc gửi lên GPU, không giữ bản sao.
- Dữ liệu lớn (hàng chục triệu gaussian) được xử lý từng đoạn CHUNK hàng để RAM tạm nhỏ.

Xoá "mềm" (giống SuperSplat): xoá gaussian chỉ bật cờ `deleted`, mảng dữ liệu giữ nguyên, nên xoá /
hoàn tác gần như tức thì (không chép mảng, không dựng lại LOD, không gửi lại dữ liệu lên GPU).
Gaussian đã xoá bị bỏ qua khi vẽ, khi chọn và khi lưu file. `size` = số hàng của mảng (kể cả đã
xoá, dùng để đánh chỉ số); `count` = số gaussian còn lại.
"""

"""
layer = Layer.from_file("scene.ply")   # đọc PLY (memmap) + giải mã thành layer.splats
layer.count                             # số gaussian còn lại (chưa xoá)

layer.transform.position = [0, 1, 0]   # kéo lớp lên 1 đơn vị (không phải gửi lại dữ liệu lên GPU)
layer.transform.rotation = [0, 90, 0]  # xoay 90 độ quanh Y
layer.transform.scale = 2.0

layer.select_all()                      # chọn tất cả (trừ cái đã xoá)
layer.invert_selection()
indices = layer.delete_selected()      # xoá mềm; trả về chỉ số để hoàn tác (None nếu không chọn gì)
layer.restore(indices)                 # hoàn tác

world = layer.world_splats(flip=False) # gaussian ở toạ độ thế giới (đã áp biến đổi, bỏ cái đã xoá)
M = layer.world_matrix(flip=False)     # ma trận 4x4: p_thế_giới = M @ (p_file, 1)
"""

```

Cơ sở toán: Biến đổi layer

Phần này giải thích toán của `core/layer.py`, cùng với `core/transform.py` (dữ liệu `Transform`) và `core/math3d.py` (các hàm quay, xem mục 1.1).

Ma trận quay từ góc Euler (`euler_to_matrix`).

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_x & -s_x \\ 0 & s_x & c_x \end{bmatrix},\ R_y = \begin{bmatrix} c_y & 0 & s_y \\ 0 & 1 & 0 \\ -s_y & 0 & c_y \end{bmatrix},\ R_z = \begin{bmatrix} c_z & -s_z & 0 \\ s_z & c_z & 0 \\ 0 & 0 & 1 \end{bmatrix},\ R = R_z R_y R_x$$

Với vector cột, X được áp dụng trước, rồi Y, rồi Z.

Ngược lại (`matrix_to_euler`). Từ $R = R_z R_y R_x$ có $R_{20} = -\sin y$, nên $y = \operatorname{asin}(-R_{20})$ (kẹp $[-1, 1]$ chống lỗi làm tròn), $x = \operatorname{atan2}(R_{21}, R_{22})$, $z = \operatorname{atan2}(R_{10}, R_{00})$. Khi $|R_{20}| \geq 0.999999$ ($y = \pm 90°$, **khóa gimbal**) thì x, z không xác định riêng, đặt $x = 0$, $z = \operatorname{atan2}(-R_{01}, R_{11})$.

Công thức Rodrigues (`axis_angle_matrix`). Quay góc a quanh trục đơn vị (x, y, z), $c = \cos a$, $s = \sin a$, $t = 1 - c$:

$$R = \begin{bmatrix} tx^2 + c & txy - sz & txz + sy \\ txy + sz & ty^2 + c & tyz - sx \\ txz - sy & tyz + sx & tz^2 + c \end{bmatrix} = I + s[k]_\times + t[k]_\times^2.$$

Ma trận thế giới của layer (`Layer.world_transform / world_matrix`).

$$p_w = sA\,p_f + t, \quad A = R(\text{euler})\,F, \quad F = \operatorname{diag}(-1, -1, 1), \quad \mathbf{\Sigma}_w = s^2 A \mathbf{\Sigma}_f A^\top, \quad M = \begin{bmatrix} sA & t \\ 0 & 1 \end{bmatrix}.$$

- F là phép **xoay** 180° **quanh** Z (det $= +1$, không phải phản xạ). Nó đưa dữ liệu COLMAP (Y hướng xuống) về Y hướng lên.
- Chỉ M thay đổi (dữ liệu gaussian giữ nguyên), nên kéo/xoay/zoom layer là tức thời. M được gửi lên GPU thành 4 texel (`splat_renderer.set_transforms`, chuyển vị để ra 4 cột).

Xoay quanh điểm pivot (`Transform.rotate_about(Δ, c)`). $R' = \Delta R$, $t' = \Delta(t - c) + c$ (dời pivot về gốc, xoay, dời lại). Góc Euler mới lấy bằng `matrix_to_euler`(ΔR).

Xuất file (`Layer._export_block`). Đổi từ thế giới về file: $\text{rot}' = F \cdot \text{rot}$, $t' = Ft$. Khi đó:

- vị trí $p' = (s\,\text{rot}')p + t'$;
- pháp tuyến $n' = \text{rot}'\,n$ (xoay thuần, không co giãn);
- scale (log): $\text{scale}'_i = \text{scale}_i + \ln s$, vì $\exp(\text{scale} + \ln s) = s\exp(\text{scale})$;
- quaternion $q' = q_{\text{rot}} \otimes q$;
- SH: $\text{sh}' = D(\text{rot}')\,\text{sh}$ (mục 1.6).

`Layer.world_splats` (dùng cho vật lý, bounds). `positions = sp.positions @ (s*rot).T + t`; `rotations = quat_mul(q_rot, q)`; `scales *= s`.

Khai báo mức module (dòng 39–51)

```
39 import math
40 import os
41 import numpy as np
42
43 from ..io.codec import SplatCodec
44 from ..io.ply import PlyFile
45 from . import sh
46 from .math3d import matrix_to_quat, quat_mul
47 from .splats import Splats
48 from .transform import Transform
49
50 FLIP = np.diag([-1.0, -1.0, 1.0])
51 CHUNK = 1 << 20
```

`class Layer`

Dòng 54–187

Giải thích. Các trường quan trọng: `data` (dữ liệu PLY gốc, thường là memmap, để lưu lại đủ thuộc tính), `splats` (đã giải mã), `selection` và `deleted` (hai mặt nạ boolean), `transform`, `visible`.

Các phương thức: `__init__`, `from_file`, `size`, `count`, `live_rows`, `selected_count`, `select_all`, `clear_selection`, `invert_selection`, `delete_selected`, `restore`, `world_transform`, `world_matrix`, `world_splats`, `export_blocks`, `export_data`, `_export_block`.

```
54 class Layer:
```

`Layer.__init__(self, name, data, path=None)`

Dòng 55–64

```
55     def __init__(self, name, data, path=None):
56         self.name = name
57         self.path = path
58         self.data = data                                    # dữ liệu gốc (để lưu lại đủ mọi thuộc tính), thường là memmap
59         self.splats = SplatCodec.decode(data)
60         self.selection = np.zeros(self.splats.count, bool)   # gaussian đang được chọn (cọ), không bao giờ gồm cái đã xoá
61         self.deleted = np.zeros(self.splats.count, bool)     # gaussian đã xoá (xoá mềm)
62         self.deleted_count = 0
63         self.visible = True
64         self.transform = Transform()
```

`Layer.from_file(cls, path)`

Dòng 66–68

```
66     @classmethod
67     def from_file(cls, path):
68         return cls(os.path.basename(path), PlyFile.read(path), path)
```

`Layer.size(self)`

Dòng 70–73

Mô tả trong mã nguồn. Số hàng của mảng dữ liệu, kể cả gaussian đã xoá.

```
70     @property
71     def size(self):
72         """Số hàng của mảng dữ liệu, kể cả gaussian đã xoá."""
73         return self.splats.count
```

`Layer.count(self)`

Dòng 75–78

Mô tả trong mã nguồn. Số gaussian còn lại (chưa xoá).

```
75      @property
76      def count(self):
77          """Số gaussian còn lại (chưa xoá)."""
78          return self.size - self.deleted_count
```

`Layer.live_rows(self, rows=slice(None))`

Dòng 80–83

Mô tả trong mã nguồn. Chỉ số (trong toàn mảng) các gaussian chưa xoá thuộc đoạn ‘rows‘.

```
80      def live_rows(self, rows=slice(None)):
81          """Chỉ số (trong toàn mảng) các gaussian chưa xoá thuộc đoạn `rows`."""
82          start = rows.start or 0
83          return np.flatnonzero(~self.deleted[rows]) + start
```

`Layer.selected_count(self)`

Dòng 86–88

```
86      @property
87      def selected_count(self):
88          return int(np.count_nonzero(self.selection))
```

`Layer.select_all(self)`

Dòng 90–91

```
90      def select_all(self):
91          np.logical_not(self.deleted, out=self.selection)
```

`Layer.clear_selection(self)`

Dòng 93–94

```
93      def clear_selection(self):
94          self.selection[:] = False
```

`Layer.invert_selection(self)`

Dòng 96–97

```
96      def invert_selection(self):
97          self.selection = ~(self.selection | self.deleted)
```

`Layer.delete_selected(self)`

Dòng 99–108

Mô tả trong mã nguồn. Xoá (mềm) các gaussian đang chọn. Trả về chỉ số các gaussian đã xoá (để hoàn tác), hoặc None nếu không có gì.

Giải thích. Xóa mềm: bật cờ `deleted`, bỏ chọn, tăng `deleted_count`, trả về chỉ số để hoàn tác. Không chép mảng nào.

```
99      def delete_selected(self):
100         """Xoá (mềm) các gaussian đang chọn. Trả về chỉ số các gaussian đã xoá (để hoàn tác), hoặc
101         None nếu không có gì."""
102         indices = np.flatnonzero(self.selection)
103         if len(indices) == 0:
104             return None
105         self.deleted[indices] = True
106         self.selection[indices] = False
107         self.deleted_count += len(indices)
108         return indices
```

`Layer.restore(self, indices)`

Dòng 110–114

Mô tả trong mã nguồn. Hoàn tác delete_selected: các gaussian ‘indices‘ hiện lại và được chọn lại như lúc xoá.

Giải thích. Hoàn tác xóa: tắt cờ `deleted`, chọn lại đúng các gaussian đó và giảm `deleted_count`. Độ phức tạp $O(k)$ theo số gaussian khôi phục.

```
110     def restore(self, indices):
111         """Hoàn tác delete_selected: các gaussian `indices` hiện lại và được chọn lại như lúc xoá."""
112         self.deleted[indices] = False
113         self.selection[indices] = True
114         self.deleted_count -= len(indices)
```

`Layer.world_transform(self, flip)`

Dòng 117–120

Mô tả trong mã nguồn. (A, t, s): p_thế_giới = s * A @ p_file + t, với A = R * F là phép xoay thuần.

Giải thích. Trả về bộ ba (A, t, s) với $A = R \cdot F$ (F chỉ có khi bật `flip`). Mọi nơi khác (ma trận 4×4, xuất file, gaussian thế giới) đều bắt đầu từ hàm này.

Công thức.

$$A = R(\text{euler})\,F, \qquad F = \text{diag}(-1,-1,1), \qquad p_w = s\,A\,p_f + t$$

```
117     def world_transform(self, flip):
118         """(A, t, s): p_thế_giới = s * A @ p_file + t, với A = R * F là phép xoay thuần."""
119         rot = self.transform.rotation_matrix() @ (FLIP if flip else np.identity(3))
120         return rot, self.transform.translation(), float(self.transform.scale)
```

`Layer.world_matrix(self, flip)`

Dòng 122–129

Mô tả trong mã nguồn. Ma trận 4x4: p_thế_giới = M @ (p_file, 1). Renderer / LOD / cọ dùng ma trận này với dữ liệu toạ độ file, nên đổi phép biến đổi không phải tính lại hay gửi lại dữ liệu.

Giải thích. Ma trận 4×4 $M = [sA \mid t]$. Renderer, LOD và cọ dùng M với dữ liệu **ở tọa độ file**, nên đổi phép biến đổi không phải tính lại hay gửi lại dữ liệu.

Công thức.

$$M = \begin{bmatrix} sA & t \\ 0\,0\,0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} p_w \\ 1 \end{bmatrix} = M \begin{bmatrix} p_f \\ 1 \end{bmatrix}, \qquad \mathbf{\Sigma}_w = s^2 A\,\mathbf{\Sigma}_f\,A^\top$$

```
122     def world_matrix(self, flip):
123         """Ma trận 4x4: p_thế_giới = M @ (p_file, 1). Renderer / LOD / cọ dùng ma trận này với dữ liệu
124         toạ độ file, nên đổi phép biến đổi không phải tính lại hay gửi lại dữ liệu."""
125         rot, t, s = self.world_transform(flip)
126         m = np.identity(4)
127         m[:3, :3] = s * rot
128         m[:3, 3] = t
129         return m
```

`Layer.world_splats(self, flip)`

Dòng 131–147

Mô tả trong mã nguồn. Các gaussian còn lại ở toạ độ thế giới.

Giải thích. Tính gaussian ở tọa độ thế giới, từng đoạn `CHUNK` hàng. Vị trí: $p\,(sA)^\top + t$; quaternion: `quat_mul(q_rot, q)`; scale nhân s. SH không xoay ngay mà ghi vào `sh_frames`.

Công thức.

$$p_w = p_f\,(sA)^\top + t, \qquad q_w = q_A \otimes q_f, \qquad s_w = s \cdot s_f$$

```
131     def world_splats(self, flip):
132         """Các gaussian còn lại ở toạ độ thế giới."""
133         rot, t, s = self.world_transform(flip)
134         sp = self.splats if self.deleted_count == 0 else self.splats.subset(~self.deleted)
135         n = sp.count
136         positions = np.empty((n, 3), np.float32)
137         rotations = np.empty((n, 4), np.float32)
138         q_rot = matrix_to_quat(rot).astype(np.float32)
139         m, t = (s * rot).astype(np.float32), t.astype(np.float32)
140         for a in range(0, n, CHUNK):
```

```
            positions[a:a + CHUNK] = sp.positions[a:a + CHUNK] @ m.T + t
            rotations[a:a + CHUNK] = quat_mul(q_rot, sp.rotations[a:a + CHUNK])
        identity = np.allclose(rot, np.identity(3), atol=1e-9)
        return Splats(positions=positions, colors=sp.colors, alphas=sp.alphas,
                      scales=sp.scales if s == 1.0 else (sp.scales * np.float32(s)),
                      rotations=rotations, sh_rest=sp.sh_rest,
                      sh_frames=() if identity or sp.sh_count == 0 else ((0, n, rot),))
```

`Layer.export_blocks(self, flip, dtype)`

Dòng 149–154

Mô tả trong mã nguồn. Dữ liệu của lớp sau khi áp phép biến đổi, đổi về toạ độ file, theo kiểu ‘dtype‘, trả về từng khối CHUNK hàng (để ghi file mà không giữ cả lớp trong RAM).

```
    def export_blocks(self, flip, dtype):
        """Dữ liệu của lớp sau khi áp phép biến đổi, đổi về toạ độ file, theo kiểu `dtype`,
        trả về từng khối CHUNK hàng (để ghi file mà không giữ cả lớp trong RAM)."""
        for a in range(0, self.size, CHUNK):
            rows = slice(a, a + CHUNK) if self.deleted_count == 0 else self.live_rows(slice(a, a + CHUNK))
            yield self._export_block(rows, flip, dtype)
```

`Layer.export_data(self, flip, dtype)`

Dòng 156–159

Mô tả trong mã nguồn. Toàn bộ dữ liệu của lớp sau khi biến đổi (xem export_blocks).

```
    def export_data(self, flip, dtype):
        """Toàn bộ dữ liệu của lớp sau khi biến đổi (xem export_blocks)."""
        blocks = list(self.export_blocks(flip, dtype))
        return np.concatenate(blocks) if blocks else np.zeros(0, dtype)
```

`Layer._export_block(self, rows, flip, dtype)`

Dòng 161–187

Giải thích. Xuất một khối về tọa độ file. Từng bước: đổi `rot, t` về tọa độ file bằng F; xoay vị trí, pháp tuyến, cộng $\ln s$ vào scale (vì scale lưu dạng log), nhân quaternion và xoay SH. Chi tiết toán ở §1.4.

Công thức. Đổi ngược về tọa độ file: $A' = FA$, $t' = Ft$. Khi đó

$$p' = sA'p + t', \quad n' = A'n, \quad \text{scale}_i' = \text{scale}_i + \ln s, \quad q' = q_{A'} \otimes q, \quad \text{sh}' = D(A')\,\text{sh}$$

vì $\exp(\text{scale} + \ln s) = s\,e^{\text{scale}}$.

```
    def _export_block(self, rows, flip, dtype):
        rot, t, s = self.world_transform(flip)
        back = FLIP if flip else np.identity(3)
        rot, t = back @ rot, back @ t            # toạ độ thế giới -> toạ độ file
        src = np.asarray(self.data[rows])
        if src.dtype != dtype:
            src = SplatCodec.encode(src, self.splats.subset(rows), dtype)
        out = src.copy()
        names = dtype.names

        p = np.stack([src['x'], src['y'], src['z']], axis=1).astype(np.float64)
        out['x'], out['y'], out['z'] = (p @ (s * rot).T + t).T
        if {'nx', 'ny', 'nz'} <= set(names):
            n = np.stack([src['nx'], src['ny'], src['nz']], axis=1).astype(np.float64)
            out['nx'], out['ny'], out['nz'] = (n @ rot.T).T
        if 'scale_0' in names and s != 1.0:
            for i in range(3):
                out[f'scale_{i}'] = src[f'scale_{i}'] + math.log(s)
        if 'rot_0' in names:
            q = np.stack([src[f'rot_{i}'] for i in range(4)], axis=1).astype(np.float64)
            q = quat_mul(matrix_to_quat(rot), q)
            for i in range(4):
                out[f'rot_{i}'] = q[:, i]
        coeffs = sh.from_columns(src, names)
        if coeffs.shape[1]:
            sh.to_columns(sh.rotate(coeffs, rot), out)
        return out
```

1.5. `splat_editor/core/scene.py`

174 dòng

Vai trò. `Scene` quản lý danh sách layer và **gộp các layer đang hiện thành một mảng** để vẽ, chọn và lưu. Nó cũng quyết định kiểu dữ liệu khi lưu (`export_dtype`) và cung cấp các tập điểm thưa dùng cho camera (`world_positions`) và va chạm (`solid_positions`).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Cảnh gồm nhiều lớp. Các lớp đang hiện được gộp thành một mảng để vẽ / chọn / lưu."""

"""
scene = Scene()
scene.add(Layer.from_file("a.ply"))    # mỗi file là một lớp
scene.add(Layer.from_file("b.ply"))
scene.active_layer                      # lớp đang chọn trong bảng Layers
scene.visible_layers()                  # các lớp đang hiện

scene.flip = True                       # lật trục Y (cho dữ liệu COLMAP)
splats = scene.world_splats()           # gộp các lớp đang hiện ở toạ độ thế giới
scene.selected_count()                  # tổng số gaussian đang chọn

dtype = scene.export_dtype()           # kiểu cột dùng khi lưu
scene.remove(0)                         # xoá lớp thứ 0
scene.clear()
"""

```

Khai báo mức module (dòng 19–22)

```
import numpy as np

from ..io.codec import CANONICAL_FIELDS
from .splats import Splats
```

`class Scene`

Dòng 25–174

Giải thích. `active_index` là layer đang chọn trong bảng Layers; `flip` là cờ lật COLMAP dùng chung cho mọi layer.

Các phương thức: `__init__`, `active_layer`, `is_empty`, `add`, `remove`, `clear`, `visible_layers`, `visible_ranges`, `selection_mask`, `deleted_mask`, `apply_selection`, `apply_selection_indices`, `selected_count`, `count`, `render_splats`, `layer_matrices`, `world_positions`, `solid_positions`, `world_splats`, `export_dtype`, `export_blocks`, `export_data`.

```
class Scene:
```

`Scene.__init__(self)`

Dòng 26–29

```
    def __init__(self):
        self.layers = []
        self.active_index = -1     # lớp đang được chọn trong danh sách
        self.flip = True           # lật file COLMAP (trục Y hướng xuống)
```

`Scene.active_layer(self)`

Dòng 32–34

```
    @property
    def active_layer(self):
        return self.layers[self.active_index] if 0 <= self.active_index < len(self.layers) else None
```

`Scene.is_empty(self)`

Dòng 36–38

```
    @property
    def is_empty(self):
        return not self.layers
```

`Scene.add(self, layer)`

Dòng 40–43

```
    def add(self, layer):
        self.layers.append(layer)
        self.active_index = len(self.layers) - 1
        return layer
```

`Scene.remove(self, index)`

Dòng 45–48

```
45    def remove(self, index):
46        layer = self.layers.pop(index)
47        self.active_index = min(self.active_index, len(self.layers) - 1)
48        return layer
```

`Scene.clear(self)`

Dòng 50–52

```
50    def clear(self):
51        self.layers = []
52        self.active_index = -1
```

`Scene.visible_layers(self)`

Dòng 54–55

```
54    def visible_layers(self):
55        return [layer for layer in self.layers if layer.visible]
```

`Scene.visible_ranges(self)`

Dòng 57–64

Mô tả trong mã nguồn. (lớp, đầu, cuối): vị trí của mỗi lớp đang hiện trong mảng gộp của render_splats() (kể cả gaussian đã xoá).

Giải thích. Sinh ra bộ ba (layer, đầu, cuối): vị trí của từng layer đang hiện trong mảng gộp. Là xương sống để chuyển giữa ”chỉ số theo layer” và ”chỉ số theo mảng gộp”.

```
57    def visible_ranges(self):
58        """(lớp, đầu, cuối): vị trí của mỗi lớp đang hiện trong mảng gộp của render_splats()
59        (kể cả gaussian đã xoá)."""
60        start = 0
61        for layer in self.visible_layers():
62            end = start + layer.size
63            yield layer, start, end
64            start = end
```

`Scene.selection_mask(self)`

Dòng 67–70

Mô tả trong mã nguồn. Vùng chọn của các lớp đang hiện, gộp theo đúng thứ tự của render_splats().

```
67    def selection_mask(self):
68        """Vùng chọn của các lớp đang hiện, gộp theo đúng thứ tự của render_splats()."""
69        parts = [layer.selection for layer in self.visible_layers()]
70        return np.concatenate(parts) if parts else np.zeros(0, bool)
```

`Scene.deleted_mask(self)`

Dòng 72–75

Mô tả trong mã nguồn. Gaussian đã xoá (xoá mềm) của các lớp đang hiện, theo thứ tự của render_splats().

```
72    def deleted_mask(self):
73        """Gaussian đã xoá (xoá mềm) của các lớp đang hiện, theo thứ tự của render_splats()."""
74        parts = [layer.deleted for layer in self.visible_layers()]
75        return np.concatenate(parts) if parts else np.zeros(0, bool)
```

`Scene.apply_selection(self, mask, value)`

Dòng 77–80

Mô tả trong mã nguồn. Đặt vùng chọn = value cho các gaussian có mask True (mask theo mảng gộp; bỏ qua cái đã xoá).

```
77    def apply_selection(self, mask, value):
78        """Đặt vùng chọn = value cho các gaussian có mask True (mask theo mảng gộp; bỏ qua cái đã xoá)."""
79        for layer, start, end in self.visible_ranges():
80            layer.selection[mask[start:end] & ~layer.deleted] = value
```

`Scene.apply_selection_indices(self, indices, value)`

Dòng 82–92

Mô tả trong mã nguồn. Đặt vùng chọn = value cho các gaussian ‘indices‘ (chỉ số theo mảng gộp). Gaussian đã xoá bị bỏ qua. Trả về các chỉ số thật sự được đặt.

```
    def apply_selection_indices(self, indices, value):
        """Đặt vùng chọn = value cho các gaussian `indices` (chỉ số theo mảng gộp). Gaussian đã xoá
        bị bỏ qua. Trả về các chỉ số thật sự được đặt."""
        changed = []
        for layer, start, end in self.visible_ranges():
            inside = indices[(indices >= start) & (indices < end)] - start
            if layer.deleted_count:
                inside = inside[~layer.deleted[inside]]
            layer.selection[inside] = value
            changed.append(inside + start)
        return np.concatenate(changed) if changed else np.zeros(0, np.int64)
```

`Scene.selected_count(self)`

Dòng 94–95

```
    def selected_count(self):
        return sum(layer.selected_count for layer in self.visible_layers())
```

`Scene.count(self)`

Dòng 98–100

Mô tả trong mã nguồn. Số gaussian còn lại (chưa xoá) của các lớp đang hiện.

```
    def count(self):
        """Số gaussian còn lại (chưa xoá) của các lớp đang hiện."""
        return sum(layer.count for layer in self.visible_layers())
```

`Scene.render_splats(self)`

Dòng 102–107

Mô tả trong mã nguồn. Gộp các lớp đang hiện ở toạ độ file của từng lớp (không chép nếu chỉ có một lớp), kể cả gaussian đã xoá (renderer / cọ bỏ qua chúng theo deleted_mask()). Toạ độ thế giới = layer_matrices()[lớp] @ toạ độ file.

Giải thích. Gộp layer đang hiện **ở tọa độ file** (không chép nếu chỉ có một layer). Tọa độ thế giới = `layer_matrices()[lớp]` × tọa độ file.

```
    def render_splats(self):
        """Gộp các lớp đang hiện ở toạ độ file của từng lớp (không chép nếu chỉ có một lớp), kể cả
        gaussian đã xoá (renderer / cọ bỏ qua chúng theo deleted_mask()).
        Toạ độ thế giới = layer_matrices()[lớp] @ toạ độ file."""
        parts = [layer.splats for layer in self.visible_layers()]
        return Splats.concat(parts) if parts else None
```

`Scene.layer_matrices(self)`

Dòng 109–111

Mô tả trong mã nguồn. Ma trận thế giới 4x4 của mỗi lớp đang hiện (theo thứ tự trong render_splats).

```
    def layer_matrices(self):
        """Ma trận thế giới 4x4 của mỗi lớp đang hiện (theo thứ tự trong render_splats)."""
        return [layer.world_matrix(self.flip) for layer in self.visible_layers()]
```

`Scene.world_positions(self, layer=None, limit=2000000)`

Dòng 113–123

Mô tả trong mã nguồn. Tâm gaussian (toạ độ thế giới) của một lớp / các lớp đang hiện, lấy thưa tối đa ‘limit‘ điểm (đủ để đặt camera, khỏi biến đổi cả chục triệu điểm).

```
    def world_positions(self, layer=None, limit=2_000_000):
        """Tâm gaussian (toạ độ thế giới) của một lớp / các lớp đang hiện, lấy thưa tối đa `limit`
        điểm (đủ để đặt camera, khỏi biến đổi cả chục triệu điểm)."""
        layers = [layer] if layer is not None else self.visible_layers()
        total = sum(l.count for l in layers)
        step = max(total // limit, 1)
        parts = []
        for l in layers:
            m = l.world_matrix(self.flip)
            parts.append(l.splats.positions[::step] @ m[:3, :3].T.astype(np.float32) + m[:3, 3].astype(np.float32))
        return np.concatenate(parts) if parts else np.zeros((0, 3), np.float32)
```

`Scene.solid_positions(self, limit=3000000, min_alpha=0.35)`

Dòng 125–137

Mô tả trong mã nguồn. Tâm gaussian ”rắn” (toạ độ thế giới) dùng làm bề mặt va chạm: bỏ gaussian đã xoá và gaussian quá mờ (thường là nhiễu / vật bay lơ lửng). Lấy thưa tối đa ‘limit‘ điểm; trả về (điểm, bước lấy thưa).

Giải thích. Chọn các tâm ”rắn” làm bề mặt va chạm: bỏ gaussian đã xóa và gaussian quá mờ ($\alpha < 0.35$), lấy thưa tối đa 3 triệu điểm bằng bước `step`.

```
    def solid_positions(self, limit=3_000_000, min_alpha=0.35):
        """Tâm gaussian "rắn" (toạ độ thế giới) dùng làm bề mặt va chạm: bỏ gaussian đã xoá và gaussian
        quá mờ (thường là nhiễu / vật bay lơ lửng). Lấy thưa tối đa `limit` điểm; trả về (điểm, bước lấy thưa)."""
        layers = self.visible_layers()
        total = sum(l.size for l in layers)
        step = max(total // limit, 1)
        parts = []
        for l in layers:
            keep = (l.splats.alphas[::step] >= min_alpha) & ~l.deleted[::step]
            m = l.world_matrix(self.flip)
            pts = l.splats.positions[::step][keep]
            parts.append(pts @ m[:3, :3].T.astype(np.float32) + m[:3, 3].astype(np.float32))
        return (np.concatenate(parts) if parts else np.zeros((0, 3), np.float32)), step
```

`Scene.world_splats(self)`

Dòng 139–142

Mô tả trong mã nguồn. Gộp các gaussian còn lại của các lớp đang hiện thành một Splats (toạ độ thế giới).

```
    def world_splats(self):
        """Gộp các gaussian còn lại của các lớp đang hiện thành một Splats (toạ độ thế giới)."""
        parts = [layer.world_splats(self.flip) for layer in self.visible_layers()]
        return Splats.concat(parts) if parts else None
```

`Scene.export_dtype(self)`

Dòng 144–161

Mô tả trong mã nguồn. Kiểu dữ liệu PLY khi lưu các lớp đang hiện (None nếu không có lớp nào).

Nếu các file cùng cấu trúc thì giữ nguyên mọi thuộc tính (kể cả f_rest_*). Nếu khác nhau thì chuyển hết sang dạng 3DGS chuẩn (giữ f_rest_* khi tất cả đều có).

Giải thích. Chọn kiểu cột khi lưu: các file cùng cấu trúc thì giữ nguyên mọi thuộc tính; khác nhau thì chuyển sang dạng 3DGS chuẩn và chỉ giữ `f_rest_*` khi tất cả layer đều có.

```
    def export_dtype(self):
        """Kiểu dữ liệu PLY khi lưu các lớp đang hiện (None nếu không có lớp nào).

        Nếu các file cùng cấu trúc thì giữ nguyên mọi thuộc tính (kể cả f_rest_*).
        Nếu khác nhau thì chuyển hết sang dạng 3DGS chuẩn (giữ f_rest_* khi tất cả đều có).
        """
        layers = self.visible_layers()
        if not layers:
            return None
        dtypes = {layer.data.dtype for layer in layers}
        if len(dtypes) == 1:
            return layers[0].data.dtype
        fields = list(CANONICAL_FIELDS)
        rest = [set(n for n in layer.data.dtype.names if n.startswith('f_rest_')) for layer in layers]
        common = set.intersection(*rest)
        if all(r == common for r in rest):
            fields[6:6] = sorted(common, key=lambda n: int(n.split('_')[-1]))
        return np.dtype([(f, '<f4') for f in fields])
```

`Scene.export_blocks(self, dtype)`

Dòng 163–166

Mô tả trong mã nguồn. Dữ liệu PLY của các lớp đang hiện (đã áp phép biến đổi), từng khối một.

```
163     def export_blocks(self, dtype):
164         """Dữ liệu PLY của các lớp đang hiện (đã áp phép biến đổi), từng khối một."""
165         for layer in self.visible_layers():
166             yield from layer.export_blocks(self.flip, dtype)
```

`Scene.export_data(self)`

Dòng 168–174

Mô tả trong mã nguồn. Dữ liệu PLY của tất cả lớp đang hiện, gộp thành một mảng (None nếu không có lớp nào).

```
168     def export_data(self):
169         """Dữ liệu PLY của tất cả lớp đang hiện, gộp thành một mảng (None nếu không có lớp nào)."""
170         dtype = self.export_dtype()
171         if dtype is None:
172             return None
173         blocks = list(self.export_blocks(dtype))
174         return np.concatenate(blocks) if blocks else np.zeros(0, dtype)
```

1.6. `splat_editor/core/sh.py`

147 dòng

Vai trò. Spherical Harmonics (SH) bậc 1–3 cho màu thay đổi theo hướng nhìn. Module cung cấp: hàm cơ sở (`basis`), đánh giá màu (`evaluate`), **xoay hệ số** khi xoay gaussian (`rotation_matrix`, `rotate`) và đọc/ghi cột `f_rest_*` của PLY. Xoay hệ số được giải bằng bình phương tối thiểu trên 64 hướng Fibonacci thay vì công thức Wigner-D. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§1.6).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Spherical harmonic (SH) bậc 1..3: màu thay đổi theo hướng nhìn.

Màu của một gaussian khi nhìn theo hướng d (vector đơn vị từ camera tới tâm gaussian):
    màu(d) = 0.5 + C0 * f_dc + sum_k Y_k(d) * sh_k
với Y_k là các hàm cơ sở SH thực bậc 1..3 (3 + 5 + 7 = 15 hàm), theo đúng quy ước dấu của
3DGS gốc (giống shader splat.vert). Hệ số bậc cao lưu ở mảng (N, K, 3): K hệ số, mỗi hệ số
một vector RGB. Trong file PLY chúng nằm ở f_rest_0..f_rest_{3K-1}, xếp theo kênh màu:
f_rest_{c*K + k} = hệ số k của kênh c.

Khi xoay gaussian bằng ma trận R thì hàm màu cũng phải xoay theo: màu'(d) = màu(R^T d).
Mỗi bậc SH là một không gian đóng với phép xoay, nên hệ số mới là sh' = D(R) sh với D là
ma trận K x K (khối chéo theo từng bậc). Ta tìm D bằng bình phương tối thiểu trên một tập
hướng cố định — chính xác (sai số làm tròn) và luôn khớp với hàm cơ sở ở dưới.
"""

"""
# sh_rest: (N, K, 3) hệ số SH bậc 1..3, K = 0, 3, 8 hoặc 15
rgb_offset = evaluate(sh_rest, dirs)    # dirs (N,3) hướng đơn vị camera -> gaussian; ra (N,3) phần cộng thêm vào màu

R = euler_to_matrix([0, 90, 0])         # (từ core.math3d)
sh_rotated = rotate(sh_rest, R)         # xoay hệ số khi xoay gaussian, màu theo hướng nhìn vẫn đúng

# Đọc / ghi cột f_rest_* của PLY
k = column_count(data.dtype.names)      # số hệ số bậc cao có trong file
sh_rest = from_columns(data, data.dtype.names)
to_columns(sh_rest, out)                # ghi ngược vào bản ghi PLY
"""

```

Cơ sở toán: Spherical Harmonics, màu theo hướng nhìn

Phần này liên quan đến `core/sh.py` và hàm `shColor` trong shader `splat_prep.vert`.

Công thức màu.

$$c(d) = \mathrm{clip}\Big(0.5 + C_0 f_{dc} + \sum_{k=1}^{15} Y_k(d)\, h_k,\ 0,\ 1\Big),$$

với d là vector đơn vị từ camera tới gaussian (3 kênh RGB độc lập).

Hàm cơ sở thực (đúng quy ước dấu của 3DGS gốc). Với $d = (x, y, z)$, $xx = x^2$, …:

- Bậc 1 (3 hàm): $-C_1 y,\ C_1 z,\ -C_1 x$ với $C_1 = \sqrt{3/4\pi} = 0.4886025$.
- Bậc 2 (5 hàm): $1.0925\,xy,\ -1.0925\,yz,\ 0.3154\,(2zz - xx - yy),\ -1.0925\,xz,\ 0.5463\,(xx - yy)$. Các hằng số: $1.0925 = \frac{1}{2}\sqrt{15/\pi}$, $0.3154 = \frac{1}{4}\sqrt{5/\pi}$, $0.5463 = \frac{1}{4}\sqrt{15/\pi}$.
- Bậc 3 (7 hàm): $-0.5900\,y(3xx - yy),\ 2.8906\,xyz,\ -0.4570\,y(4zz - xx - yy),\ 0.3732\,z(2zz - 3xx - 3yy),\ -0.4570\,x(4zz - xx - yy),\ 1.4453\,z(xx - yy),\ -0.5900\,x(xx - 3yy)$.

Đây là các đa thức Legendre liên kết trên mặt cầu: $1 + 3 + 5 + 7 = 16 = (L+1)^2$ với $L = 3$. Số hệ số bậc cao $K \in \{0, 3, 8, 15\}$ (`COEFFS_FOR_DEGREE`). Trong file, `f_rest_{c*K+k}` là hệ số k của kênh c (xếp theo kênh). Shader chọn bậc theo `uShDegree`.

Xoay hệ số SH khi xoay gaussian (`sh.rotation_matrix, sh.rotate`). Khi gaussian quay R, hàm màu mới là $\text{màu}'(d) = \text{màu}(R^\top d)$. Mỗi bậc SH đóng kín dưới phép quay, nên tồn tại $D(R)$ ($K \times K$, khối chéo theo bậc) sao cho $\text{sh}' = D \cdot \text{sh}$. Thay vì dùng công thức Wigner-D, code **giải bình phương tối thiểu**: chọn 64 hướng d_i, cần $B(d_i)D = B(R^\top d_i)$, nên

$$D = \text{lstsq}\big(B(\text{dirs}),\ B(\text{dirs}\,R)\big)$$

(hàng $d_i^\top R = (R^\top d_i)^\top$). Kết quả chính xác vì B có hạng đầy đủ.

- **Hướng mẫu Fibonacci** (`_sample_dirs`): $z = 1 - 2(i + \frac{1}{2})/n$, $r = \sqrt{1 - z^2}$, $\varphi = (i + \frac{1}{2})\,\pi(3 - \sqrt{5})$ (góc vàng 2.39996 rad), cho điểm phân bố gần đều trên cầu.
- Nhân khối bằng BLAS: $(n \cdot 3, K)\,@\,D^\top$, xử lý từng `CHUNK` 2^{20}.

Trong shader (không xoay hệ số mỗi frame). $d_{\text{file}} = \text{normalize}(\text{linear}^\top(p_w - e))$: đưa hướng nhìn **ngược** về tọa độ file (dùng $A^\top$ vì quay trực giao; scale dương bị chuẩn hóa mất). Cách này rẻ hơn xoay hệ số mỗi khung.

Khai báo mức module (dòng 29–39)

```
29 import math
30
31 import numpy as np
32
33 COEFFS_FOR_DEGREE = {0: 0, 1: 3, 2: 8, 3: 15}   # số hệ số bậc cao (không tính f_dc)
34
35 C1 = 0.4886025119029199
36 C2 = (1.0925484305920792, -1.0925484305920792, 0.31539156525252005, -1.0925484305920792,
37       0.5462742152960396)
38 C3 = (-0.5900435899266435, 2.890611442640554, -0.4570457994644658, 0.3731763325901154,
39       -0.4570457994644658, 1.445305721320277, -0.5900435899266435)
```

`degree_for_coeffs(k)`

Dòng 42–47

Mô tả trong mã nguồn. Số hệ số bậc cao -> bậc SH (None nếu không hợp lệ).

```
42 def degree_for_coeffs(k):
43     """Số hệ số bậc cao -> bậc SH (None nếu không hợp lệ)."""
44     for degree, count in COEFFS_FOR_DEGREE.items():
45         if count == k:
46             return degree
47     return None
```

`basis(dirs, k=15)`

Dòng 50–61

Mô tả trong mã nguồn. (M,3) hướng đơn vị -> (M,k) giá trị K hàm cơ sở SH bậc cao đầu tiên.

Giải thích. Giá trị các hàm cơ sở SH thực tại các hướng đơn vị; đúng quy ước dấu của 3DGS gốc. Trả về (M, k) với $k \le 15$.

Công thức.

$$Y_1 = \big(-C_1 y,\ C_1 z,\ -C_1 x\big), \quad C_1 = \sqrt{\frac{3}{4\pi}}$$

$$Y_2 = \big(1.0925\,xy,\ -1.0925\,yz,\ 0.3154\,(2z^2 - x^2 - y^2),\ -1.0925\,xz,\ 0.5463\,(x^2 - y^2)\big)$$

$$Y_3 = \big(-0.59\,y(3x^2 - y^2),\ 2.8906\,xyz,\ -0.457\,y(4z^2 - x^2 - y^2),\ 0.3732\,z(2z^2 - 3x^2 - 3y^2),\ -0.457\,x(4z^2 - x^2 - y^2),\ 1.4453\,z(x^2 - y^2),\ -0.5$$

```
def basis(dirs, k=15):
    """(M,3) hướng đơn vị -> (M,k) giá trị K hàm cơ sở SH bậc cao đầu tiên."""
    x, y, z = dirs[:, 0], dirs[:, 1], dirs[:, 2]
    xx, yy, zz = x * x, y * y, z * z
    columns = [
        -C1 * y, C1 * z, -C1 * x,
        C2[0] * x * y, C2[1] * y * z, C2[2] * (2 * zz - xx - yy), C2[3] * x * z, C2[4] * (xx - yy),
        C3[0] * y * (3 * xx - yy), C3[1] * x * y * z, C3[2] * y * (4 * zz - xx - yy),
        C3[3] * z * (2 * zz - 3 * xx - 3 * yy), C3[4] * x * (4 * zz - xx - yy),
        C3[5] * z * (xx - yy), C3[6] * x * (xx - 3 * yy),
    ]
    return np.stack(columns[:k], axis=1)
```

`evaluate(sh_rest, dirs)`

Dòng 64–66

Mô tả trong mã nguồn. Phần màu do SH bậc cao đóng góp: (N,K,3) hệ số, (N,3) hướng -> (N,3).

Giải thích. Phần màu do SH bậc cao: `einsum('nk,nkc->nc')` nhân hàm cơ sở với hệ số rồi cộng theo k.

Công thức.

$$c(d) = \mathrm{clip}\Big(0.5 + C_0 f_{dc} + \sum_{k=1}^{K} Y_k(d)\, h_k,\ 0,\ 1\Big), \qquad C_0 = \frac{1}{2\sqrt{\pi}}$$

```
def evaluate(sh_rest, dirs):
    """Phần màu do SH bậc cao đóng góp: (N,K,3) hệ số, (N,3) hướng -> (N,3)."""
    return np.einsum('nk,nkc->nc', basis(dirs, sh_rest.shape[1]), sh_rest)
```

`_sample_dirs(n=64)`

Dòng 69–75

Mô tả trong mã nguồn. n hướng phân bố đều trên mặt cầu (xoắn ốc Fibonacci).

Giải thích. 64 hướng phân bố đều trên mặt cầu bằng xoắn ốc Fibonacci (góc vàng $\pi(3-\sqrt{5})$).

Công thức.

$$z_i = 1 - \frac{2(i+\frac{1}{2})}{n}, \quad r_i = \sqrt{1-z_i^2}, \quad \varphi_i = i\,\pi(3-\sqrt{5}), \quad d_i = (r_i\cos\varphi_i,\ r_i\sin\varphi_i,\ z_i)$$

```
def _sample_dirs(n=64):
    """n hướng phân bố đều trên mặt cầu (xoắn ốc Fibonacci)."""
    i = np.arange(n) + 0.5
    z = 1 - 2 * i / n
    r = np.sqrt(1 - z * z)
    phi = i * math.pi * (3 - math.sqrt(5))
    return np.stack([r * np.cos(phi), r * np.sin(phi), z], axis=1)
```

Khai báo mức module (dòng 78–78)

```
_DIRS = _sample_dirs()
```

`rotation_matrix(rot, k)`

Dòng 81–85

Mô tả trong mã nguồn. Ma trận D (k,k) để xoay hệ số SH theo phép xoay 3x3 ‘rot‘: sh’ = D @ sh.

Giải thích. Một dòng làm cả việc: giải $B(d)\,D = B(R^{\top}d)$ bằng `np.linalg.lstsq` trên 64 hướng, ra ma trận D cỡ $K \times K$.

Công thức. Màu sau khi xoay gaussian: $c'(d) = c(R^{\top}d)$. Cần $B(d)\,D = B(R^{\top}d)$ với mọi d, nên trên 64 hướng d_i:

$$D = \arg\min_{D} \Big\| B(\mathrm{dirs})\,D - B(\mathrm{dirs}\,R) \Big\|_F^2, \qquad \mathrm{sh}' = D\,\mathrm{sh}$$

```
def rotation_matrix(rot, k):
    """Ma trận D (k,k) để xoay hệ số SH theo phép xoay 3x3 `rot`: sh' = D @ sh."""
    rot = np.asarray(rot, float)
    # cần: basis(d) @ sh' = basis(rot^T d) @ sh với mọi d
    return np.linalg.lstsq(basis(_DIRS, k), basis(_DIRS @ rot, k), rcond=None)[0]
```

Khai báo mức module (dòng 88–88)

```
CHUNK = 1 << 20   # xử lý từng đoạn 1 triệu gaussian để mảng tạm nhỏ
```

`rotate(sh_rest, rot)`

Dòng 91–104

Mô tả trong mã nguồn. Xoay hệ số SH (N,K,3) theo phép xoay 3x3 ‘rot‘. Trả về mảng mới cùng kiểu dữ liệu.

Giải thích. Xoay hàng loạt hệ số SH. Nhân ma trận 2 chiều (BLAS) từng đoạn `CHUNK`; nhanh hơn `einsum` nhiều lần khi có hàng triệu gaussian.

```
def rotate(sh_rest, rot):
    """Xoay hệ số SH (N,K,3) theo phép xoay 3x3 `rot`. Trả về mảng mới cùng kiểu dữ liệu."""
    k = sh_rest.shape[1]
    if k == 0 or np.allclose(rot, np.identity(3), atol=1e-9):
        return sh_rest.copy()
    work = np.float64 if sh_rest.dtype == np.float64 else np.float32
    D = rotation_matrix(rot, k).astype(work)
    out = np.empty_like(sh_rest)
    for a in range(0, len(sh_rest), CHUNK):
        part = sh_rest[a:a + CHUNK].astype(work, copy=False)
        # nhân ma trận 2D (BLAS) nhanh hơn einsum nhiều lần với hàng triệu gaussian
        flat = part.transpose(0, 2, 1).reshape(-1, k) @ D.T          # (n*3, K)
        out[a:a + CHUNK] = flat.reshape(-1, 3, k).transpose(0, 2, 1)
    return out
```

`column_count(names)`

Dòng 107–113

Mô tả trong mã nguồn. Số hệ số SH bậc cao K đọc được từ các cột f_rest_* (0 nếu không có / không hợp lệ).

```
def column_count(names):
    """Số hệ số SH bậc cao K đọc được từ các cột f_rest_* (0 nếu không có / không hợp lệ)."""
    count = sum(1 for n in names if n.startswith('f_rest_'))
    k = count // 3
    if k == 0 or count % 3 or degree_for_coeffs(k) is None or any(f'f_rest_{i}' not in names for i in range(count)):
        return 0
    return k
```

`from_columns(data, names, dtype=np.float32)`

Dòng 116–126

Mô tả trong mã nguồn. Lấy hệ số SH bậc cao (N,K,3) từ các cột f_rest_* của PLY (K = 0 nếu không có / không hợp lệ). Đọc từng đoạn hàng (dữ liệu có thể là memmap trên đĩa) để không tạo mảng tạm khổng lồ.

Giải thích. Đọc hệ số từ cột `f_rest_*`. Dữ liệu có thể là memmap nên đọc từng đoạn (`read_block`) để không tạo mảng tạm khổng lồ.

```
def from_columns(data, names, dtype=np.float32):
    """Lấy hệ số SH bậc cao (N,K,3) từ các cột f_rest_* của PLY (K = 0 nếu không có / không hợp lệ).
    Đọc từng đoạn hàng (dữ liệu có thể là memmap trên đĩa) để không tạo mảng tạm khổng lồ."""
    from ..io.ply import read_block
    k = column_count(names)
    out = np.empty((len(data), k, 3), dtype)
    if k:
        for a in range(0, len(data), CHUNK):
            block, matrix = read_block(data, a, a + CHUNK)
            out[a:a + len(block)] = block_coeffs(block, matrix, k)
    return out
```

`block_coeffs(block, matrix, k)`

Dòng 129–139

Mô tả trong mã nguồn. (n,K,3) hệ số SH của một khối hàng PLY (matrix: khối dạng ma trận float32, hoặc None).

```
def block_coeffs(block, matrix, k):
    """(n,K,3) hệ số SH của một khối hàng PLY (matrix: khối dạng ma trận float32, hoặc None)."""
    if matrix is not None:
        columns = [block.dtype.names.index(f'f_rest_{i}') for i in range(3 * k)]
        first = columns[0]
        # file 3DGS chuẩn: f_rest_* là các cột liền nhau -> cắt slice (view), không gather
```

```
        flat = (matrix[:, first:first + 3 * k] if columns == list(range(first, first + 3 * k))
                else np.ascontiguousarray(matrix[:, columns]))
    else:
        flat = np.stack([block[f'f_rest_{i}'] for i in range(3 * k)], axis=1)
    return flat.reshape(len(block), 3, k).transpose(0, 2, 1)
```

`to_columns(sh_rest, out)`

Dòng 142–147

Mô tả trong mã nguồn. Ghi hệ số SH (N,K,3) vào các cột f_rest_* của mảng PLY ‘out‘.

```
def to_columns(sh_rest, out):
    """Ghi hệ số SH (N,K,3) vào các cột f_rest_* của mảng PLY `out`."""
    k = sh_rest.shape[1]
    for c in range(3):
        for i in range(k):
            out[f'f_rest_{c * k + i}'] = sh_rest[:, i, c]
```

1.7. `splat_editor/core/lod.py`

208 dòng

Vai trò. Mức chi tiết (LOD) để vẽ cảnh hàng chục triệu gaussian: chia thành khối theo mã Morton, xếp gaussian trong khối theo độ quan trọng, mỗi khung hình bỏ khối ngoài khung nhìn và chia ngân sách gaussian theo khoảng cách. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§1.7).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Mức chi tiết (LOD): chỉ vẽ tối đa `budget` gaussian mỗi khung hình, dù cảnh có hàng chục triệu.

Ý tưởng:
1. Chia gaussian thành các khối (chunk) nhỏ gần nhau trong không gian: sắp xếp theo mã Morton
   (đường cong Z) rồi cắt thành từng đoạn CHUNK_SIZE gaussian. Mỗi khối có hình cầu bao.
2. Trong mỗi khối, xếp gaussian theo độ "quan trọng" giảm dần (độ đục x diện tích), nên lấy
   k gaussian đầu của khối = k gaussian quan trọng nhất (cái to, đục giữ lại; cái nhỏ, mờ bỏ đi).
3. Mỗi lần camera đổi:
   - bỏ khối nằm ngoài khung nhìn (frustum culling theo hình cầu bao);
   - khối gần camera lấy nhiều gaussian, khối xa lấy ít: khối c lấy min(n_c, lambda * n_c / d_c),
     với d_c là khoảng cách tới camera và lambda tìm bằng chia đôi sao cho tổng = budget.
4. Renderer lưu gaussian trên GPU theo đúng thứ tự `order` ("vị trí"), nên gaussian gần nhau
   trong không gian cũng gần nhau trong bộ nhớ -> GPU đọc liền mạch, ít trượt cache.
5. Cọ chọn (editor/brush.py) dùng hộp bao các khối để chỉ kiểm tra gaussian gần nét cọ.
"""

"""
# positions (N,3), scales (N,3), alphas (N,) lấy từ Splats; ranges: mỗi lớp một đoạn (đầu, cuối)
lod = LodIndex(positions, scales, alphas, ranges=[(0, n1), (n1, n1 + n2)])

# Mỗi khi camera đổi: chọn tối đa 2 triệu gaussian cần vẽ
slots = lod.select(model_view, projection, budget=2_000_000, transforms=layer_matrices)
indices = lod.order[slots]              # chỉ số gaussian gốc của các vị trí được chọn

# Hàm tiện ích (đa luồng)
codes = morton_codes(positions)        # mã Morton: gaussian gần nhau -> mã gần nhau
perm = radix_argsort(codes)            # sắp xếp nhanh hơn np.argsort cho khoá số nguyên
"""

```

Cơ sở toán: LOD, Morton, Frustum

Các công thức dưới đây được cài trong `core/lod.py`; phần chọn gaussian theo độ sâu được `render/sorter.py` gọi (xem mục 4.3).

Mã Morton 3D, đường cong Z (`morton_codes`). Hộp bao theo phân vị 0.1%–99.9% (bỏ nhiễu): $q = \text{clip}((p - lo) \cdot 1023/(hi - lo), 0, 1023)$ (10 bit/trục). Xen kẽ bit: `spread(v)` đặt bit i của v vào vị trí $3i$ (bảng tra); code = $\text{spread}(q_x) \mid \text{spread}(q_y) \ll 1 \mid \text{spread}(q_z) \ll 2$ (30 bit). Điểm gần nhau trong không gian có mã gần nhau, tốt cho cache.

Radix argsort (`radix_argsort`). Hai lượt ổn định 16 bit (thấp rồi cao) cho thứ tự đúng theo khóa 32 bit (LSD radix).

Khối và độ quan trọng (`_build_group`). Cắt theo Morton thành khối 1024. Trong khối xếp theo importance = $\alpha(s_0 s_1 + s_1 s_2 + s_0 s_2)$ giảm dần (tỉ lệ với diện tích bề mặt ellipsoid × độ đục). Mẹo: khóa = 65535 − (16 bit cao của float32 dương), vì float dương sắp theo thứ tự bit nên không cần log.

Cầu bao mỗi khối. radius $= \frac{1}{2}\|hi - lo\| + 3\max(s)$ (nửa đường chéo hộp tâm $+3\sigma$ lớn nhất). Khi có ma trận layer: center$' = sA\,\text{center} + t$, radius$' = \text{radius} \cdot \|\text{cột 0 của } M\| = \text{radius} \cdot s$.

Frustum culling (`_frustum_planes, select`). Với $VP = PV$, 6 mặt phẳng (Gribb–Hartmann): hàng 3 ± hàng 0/1/2 cho (a, b, c, d), chuẩn hóa bằng $\|(a, b, c)\|$. Khoảng cách có dấu dist $= a\,c_x + b\,c_y + c\,c_z + d$; khối nhìn thấy nếu dist $> -$radius với 5 mặt (bỏ mặt xa).

Phân bổ ngân sách (`select`). Khối c lấy take$_c = \min(n_c, \lceil \lambda\, n_c/d_c \rceil)$ với $d_c = \max(\text{depth} - \text{radius}, \epsilon)$. Tìm λ bằng **chia đôi 40 vòng** sao cho $\sum$ take $=$ budget. Khối gần (d nhỏ) lấy nhiều gaussian, khối xa lấy ít; ưu tiên top importance vì đã được xếp trước. Độ sâu khối: depth $= -(\text{center} \cdot mv_2 + mv_{23})$.

Khai báo mức module (dòng 30–38)

```
import os
from concurrent.futures import ThreadPoolExecutor

import numpy as np

CHUNK_SIZE = 1024


_WORKERS = min(max((os.cpu_count() or 2) - 1, 1), 6)
```

`_parallel(fn, n, chunk=1 << 20)`

Dòng 41–44

Mô tả trong mã nguồn. Chạy fn(start, end) trên các đoạn [0, n) ở nhiều luồng (numpy nhả GIL).

```
def _parallel(fn, n, chunk=1 << 20):
    """Chạy fn(start, end) trên các đoạn [0, n) ở nhiều luồng (numpy nhả GIL)."""
    with ThreadPoolExecutor(_WORKERS) as pool:
        list(pool.map(lambda a: fn(a, min(a + chunk, n)), range(0, n, chunk)))
```

`gather(array, index)`

Dòng 47–55

Mô tả trong mã nguồn. array[index] theo trục 0, chia nhiều luồng: truy cập ngẫu nhiên bị nghẽn bởi độ trễ bộ nhớ nên nhiều luồng cùng đọc nhanh hơn ~3 lần.

```
def gather(array, index):
    """array[index] theo trục 0, chia nhiều luồng: truy cập ngẫu nhiên bị nghẽn bởi độ trễ bộ
    nhớ nên nhiều luồng cùng đọc nhanh hơn ~3 lần."""
    out = np.empty((len(index),) + array.shape[1:], array.dtype)

    def part(a, b):
        out[a:b] = array[index[a:b]]
    _parallel(part, len(index))
    return out
```

`_spread_table()`

Dòng 58–64

Mô tả trong mã nguồn. Bảng tra: số 10 bit -> các bit đó cách nhau 2 bit 0 (để xen kẽ thành mã Morton 3D).

```
def _spread_table():
    """Bảng tra: số 10 bit -> các bit đó cách nhau 2 bit 0 (để xen kẽ thành mã Morton 3D)."""
    v = np.arange(1024, dtype=np.uint32)
    out = np.zeros(1024, np.uint32)
    for bit in range(10):
        out |= ((v >> bit) & 1) << (3 * bit)
    return out
```

Khai báo mức module (dòng 67–67)

```
_SPREAD = _spread_table()
```

`morton_codes(positions)`

Dòng 70–84

Mô tả trong mã nguồn. (N,3) -> (N,) mã Morton 30 bit (10 bit / trục, đủ để chia khối). Dùng phân vị 0.1%..99.9% làm hộp bao để điểm nhiễu ở xa không làm cả cảnh dồn vào vài ô.

Giải thích. Mã Morton 30 bit (10 bit mỗi trục) trên hộp bao theo phân vị 0.1%–99.9% để bỏ nhiễu. Điểm gần nhau trong không gian có mã gần nhau.

Công thức.

$$q = \mathrm{clip}\Big(\frac{p - lo}{hi - lo} \cdot 1023,\ 0,\ 1023\Big), \qquad \mathrm{code} = \mathrm{spread}(q_x)\ \mid\ \mathrm{spread}(q_y)\ll 1\ \mid\ \mathrm{spread}(q_z)\ll 2$$

$\mathrm{spread}(v)$ đặt bit i của v vào vị trí $3i$ (mã 30 bit).

```
def morton_codes(positions):
    """(N,3) -> (N,) mã Morton 30 bit (10 bit / trục, đủ để chia khối). Dùng phân vị
    0.1%..99.9% làm hộp bao để điểm nhiễu ở xa không làm cả cảnh dồn vào vài ô."""
    sample = positions[::max(len(positions) // 200_000, 1)]
    lo = np.percentile(sample, 0.1, axis=0).astype(np.float32)
    hi = np.percentile(sample, 99.9, axis=0).astype(np.float32)
    scale = 1023.0 / np.maximum(hi - lo, 1e-9).astype(np.float32)
    code = np.zeros(len(positions), np.uint32)

    def part(a, b):
        for axis in range(3):
            q = np.clip((positions[a:b, axis] - lo[axis]) * scale[axis], 0, 1023).astype(np.uint16)
            code[a:b] |= _SPREAD[q] << np.uint32(axis)
    _parallel(part, len(positions))
    return code
```

`radix_argsort(key)`

Dòng 87–91

Mô tả trong mã nguồn. argsort ổn định cho khoá uint32: 2 lượt sắp theo 16 bit (numpy dùng radix sort cho số 16 bit, O(N), nhanh hơn nhiều so với argsort số 32 bit).

Giải thích. Sắp xếp ổn định khóa uint32 bằng hai lượt 16 bit (thấp rồi cao). NumPy dùng radix sort cho kiểu 16 bit nên nhanh hơn `argsort` thường.

```
def radix_argsort(key):
    """argsort ổn định cho khoá uint32: 2 lượt sắp theo 16 bit (numpy dùng radix sort cho
    số 16 bit, O(N), nhanh hơn nhiều so với argsort số 32 bit)."""
    order = np.argsort((key & 0xFFFF).astype(np.uint16), kind='stable')
    return order[np.argsort((key[order] >> 16).astype(np.uint16), kind='stable')]
```

`_build_group(positions, scales, alphas, chunk_size)`

Dòng 94–132

Mô tả trong mã nguồn. Chỉ mục cho một nhóm gaussian (một lớp): (order, positions, sizes, box_lo, box_hi, radii).

Giải thích. Dựng chỉ mục cho một layer: sắp theo Morton, cắt thành khối 1024, trong khối sắp theo độ quan trọng $\alpha(s_0s_1 + s_1s_2 + s_0s_2)$ giảm dần, tính hộp bao và cầu bao ($\frac{1}{2}\|hi - lo\| + 3\max s$).

Công thức.

$$\mathrm{importance} = \alpha\,(s_0s_1 + s_1s_2 + s_0s_2), \qquad \mathrm{radius} = \tfrac{1}{2}\|hi - lo\| + 3\max(s)$$

Với layer có ma trận M: $\mathrm{center}' = sA\,\mathrm{center} + t$, $\mathrm{radius}' = s \cdot \mathrm{radius}$.

```
def _build_group(positions, scales, alphas, chunk_size):
    """Chỉ mục cho một nhóm gaussian (một lớp): (order, positions, sizes, box_lo, box_hi, radii)."""
    n = len(positions)
    order = radix_argsort(morton_codes(positions)).astype(np.uint32)

    # trong mỗi khối (chunk_size gaussian liên tiếp theo Morton) xếp theo độ quan trọng giảm dần:
    # sắp ổn định theo độ quan trọng rồi theo số thứ tự khối (2 lượt radix sort 16 bit).
    # Khoá độ quan trọng = 16 bit cao của số float32 dương (giữ nguyên thứ tự, khỏi tính log);
    # tính theo thứ tự gốc (đọc tuần tự) rồi mới gather khoá 2 byte theo thứ tự Morton.
    chunks = -(-n // chunk_size)
    key = np.empty(n, np.uint16)
    max_scale = np.empty(n, np.float32)

    def part(a, b):
        s = scales[a:b]
        importance = alphas[a:b] * (s[:, 0] * s[:, 1] + s[:, 1] * s[:, 2] + s[:, 0] * s[:, 2])
        key[a:b] = np.uint16(65535) - (np.maximum(importance, 0).astype(np.float32).view(np.uint32) >> 16)
        max_scale[a:b] = s.max(axis=1)
```

```
    _parallel(part, n)
    within = np.argsort(gather(key, order), kind='stable')
    chunk_id = (within // chunk_size).astype(np.uint32)
    within = within[radix_argsort(chunk_id) if chunks > 65536 else
                    np.argsort(chunk_id.astype(np.uint16), kind='stable')]
    order = gather(order, within)            # theo khối, trong khối theo độ quan trọng
    sorted_positions = gather(positions, order)
    sizes = np.minimum(chunk_size, n - np.arange(chunks, dtype=np.int64) * chunk_size)

    # hộp bao các tâm của mỗi khối; đệm cho đủ số khối tròn bằng phần tử đầu khối cuối
    # (min / max không đổi), tính từng trục trên mảng liền mạch cho nhanh
    pad = chunks * chunk_size - n
    lo, hi = np.empty((chunks, 3)), np.empty((chunks, 3))
    for axis in range(3):
        v = sorted_positions[:, axis]
        v = np.concatenate([v, np.full(pad, v[n - sizes[-1]], v.dtype)]).reshape(chunks, chunk_size)
        lo[:, axis], hi[:, axis] = v.min(axis=1), v.max(axis=1)
    # hình cầu bao: cộng thêm bán kính gaussian lớn nhất trong khối (3 sigma) để không cắt nhầm
    max_scale = np.concatenate([gather(max_scale, order), np.zeros(pad, np.float32)])
    radii = 0.5 * np.linalg.norm(hi - lo, axis=1) + 3.0 * max_scale.reshape(chunks, chunk_size).max(axis=1)
    return order, sorted_positions, sizes, lo, hi, radii
```

`class LodIndex`

Dòng 135–208

Mô tả trong mã nguồn. Chỉ mục khối của N gaussian, chia theo nhóm ‘ranges‘ (mỗi lớp một nhóm, khối không bao giờ trộn hai lớp). Toạ độ là toạ độ riêng của từng nhóm (toạ độ file của lớp); phép biến đổi của nhóm (ma trận 4x4, đổi được bất cứ lúc nào) được áp khi chọn / chiếu, nên kéo, xoay, phóng to một lớp không phải dựng lại chỉ mục.

Giải thích. Giữ `order` (hoán vị Morton), các khối với hộp bao và bán kính, chia theo nhóm (mỗi layer một nhóm để khối không vượt qua ranh giới layer).

Các phương thức: `__init__`, `world_bounds`, `_frustum_planes`, `select`.

```
class LodIndex:
    """Chỉ mục khối của N gaussian, chia theo nhóm `ranges` (mỗi lớp một nhóm, khối không bao giờ
    trộn hai lớp). Toạ độ là toạ độ riêng của từng nhóm (toạ độ file của lớp); phép biến đổi của
    nhóm (ma trận 4x4, đổi được bất cứ lúc nào) được áp khi chọn / chiếu, nên kéo, xoay, phóng to
    một lớp không phải dựng lại chỉ mục."""
```

`LodIndex.__init__(self, positions, scales, alphas, chunk_size=CHUNK_SIZE, ranges=None)`

Dòng 141–157

```
    def __init__(self, positions, scales, alphas, chunk_size=CHUNK_SIZE, ranges=None):
        n = len(positions)
        self.count = n
        ranges = [(0, n)] if ranges is None else [(a, b) for a, b in ranges if b > a]
        parts = [_build_group(positions[a:b], scales[a:b], alphas[a:b], chunk_size) for a, b in ranges]
        self.group_count = len(ranges)
        self.order = np.concatenate([p[0] + np.uint32(a) for p, (a, _) in zip(parts, ranges)]) if parts            else np.zeros(0, np.uint32)
        self.positions = np.concatenate([p[1] for p in parts]) if parts else np.zeros((0, 3), np.float32)
        self.sizes = np.concatenate([p[2] for p in parts]) if parts else np.zeros(0, np.int64)
        self.starts = np.cumsum(self.sizes) - self.sizes
        self.box_lo = np.concatenate([p[3] for p in parts]) if parts else np.zeros((0, 3))
        self.box_hi = np.concatenate([p[4] for p in parts]) if parts else np.zeros((0, 3))
        self.centers = (self.box_lo + self.box_hi) * 0.5
        self.radii = np.concatenate([p[5] for p in parts]) if parts else np.zeros(0)
        self.chunk_group = np.repeat(np.arange(len(parts)), [len(p[2]) for p in parts])
        # vị trí (trong order) đầu / cuối của mỗi nhóm: nhóm g chiếm [group_slots[g], group_slots[g+1])
        self.group_slots = np.concatenate([[0], np.cumsum([b - a for a, b in ranges])]).astype(np.int64)
```

`LodIndex.world_bounds(self, transforms=None)`

Dòng 159–166

Mô tả trong mã nguồn. (tâm, bán kính) hình cầu bao của các khối trong toạ độ thế giới. transforms: ma trận 4x4 (s * R | t) của từng nhóm; None: đơn vị.

```
    def world_bounds(self, transforms=None):
        """(tâm, bán kính) hình cầu bao của các khối trong toạ độ thế giới.
        transforms: ma trận 4x4 (s * R | t) của từng nhóm; None: đơn vị."""
        if transforms is None:
            return self.centers, self.radii
        m = np.asarray(transforms, np.float64)[self.chunk_group]           # (C,4,4)
        centers = np.einsum('cij,cj->ci', m[:, :3, :3], self.centers) + m[:, :3, 3]
        return centers, self.radii * np.linalg.norm(m[:, :3, 0], axis=1)
```

`LodIndex._frustum_planes(view_proj)`

Dòng 168–173

Mô tả trong mã nguồn. 6 mặt phẳng (a,b,c,d) của khung nhìn, pháp tuyến hướng vào trong, đã chuẩn hoá.

Giải thích. Tách 6 mặt phẳng khung nhìn từ $VP = PV$ theo Gribb–Hartmann: hàng 3 ± hàng 0/1/2, chuẩn hóa bằng độ dài pháp tuyến.

Công thức. Với $VP = PV$ có hàng $r_0, \dots, r_3$, sáu mặt phẳng (a, b, c, d) (Gribb–Hartmann):

$$r_3 \pm r_0, \quad r_3 \pm r_1, \quad r_3 \pm r_2, \qquad (a,b,c,d) \leftarrow \frac{(a,b,c,d)}{\|(a,b,c)\|}$$

```
168     @staticmethod
169     def _frustum_planes(view_proj):
170         """6 mặt phẳng (a,b,c,d) của khung nhìn, pháp tuyến hướng vào trong, đã chuẩn hoá."""
171         m = np.asarray(view_proj, np.float64)
172         planes = np.array([m[3] + m[0], m[3] - m[0], m[3] + m[1], m[3] - m[1], m[3] + m[2], m[3] - m[2]])
173         return planes / np.linalg.norm(planes[:, :3], axis=1, keepdims=True)
```

`LodIndex.select(self, model_view, projection, budget, transforms=None)`

Dòng 175–208

Mô tả trong mã nguồn. Vị trí trong ‘self.order‘ (tăng dần) của các gaussian cần vẽ, chưa sắp theo độ sâu. Chỉ số gaussian gốc = self.order[kết quả].

Giải thích. Hàm quan trọng nhất của LOD. Bỏ khối ngoài khung nhìn (khoảng cách có dấu > −bán kính), tìm λ bằng chia đôi 40 vòng sao cho tổng gaussian lấy bằng `budget`; khối gần lấy nhiều, khối xa lấy ít.

Công thức. Khối nhìn thấy nếu $a\,c_x + b\,c_y + c\,c_z + d > -\text{radius}$ với 5 mặt. Số gaussian lấy từ khối c:

$$\text{take}_c = \min\left(n_c,\ \left\lceil \frac{\lambda\, n_c}{d_c} \right\rceil\right), \quad d_c = \max(\text{depth}_c - \text{radius}_c,\ \epsilon), \quad \sum_c \text{take}_c = \text{budget}$$

λ tìm bằng chia đôi 40 vòng.

```
175     def select(self, model_view, projection, budget, transforms=None):
176         """Vị trí trong `self.order` (tăng dần) của các gaussian cần vẽ, chưa sắp theo độ sâu.
177         Chỉ số gaussian gốc = self.order[kết quả]."""
178         centers, radii = self.world_bounds(transforms)
179         planes = self._frustum_planes(projection @ model_view)
180         dist = centers @ planes[:, :3].T + planes[:, 3]               # (chunks, 6)
181         visible = np.flatnonzero((dist[:, :5] > -radii[:, None]).all(axis=1))   # bỏ qua mặt xa
182         if len(visible) == 0:
183             return np.zeros(0, np.int64)
184         sizes = self.sizes[visible]
185         if budget is None or sizes.sum() <= budget:
186             take = sizes
187         else:
188             mv = np.asarray(model_view, np.float64)
189             depth = -(centers[visible] @ mv[2, :3] + mv[2, 3])
190             # khối chứa camera (hoặc rất gần) luôn đủ chi tiết
191             d = np.maximum(depth - radii[visible], 1e-3 * max(float(radii.max()), 1e-9))
192             weight = sizes / d
193             lo, hi = 0.0, float(d.max())
194             for _ in range(40):                 # tìm lambda: sum(min(n_c, lambda * w_c)) = budget
195                 lam = 0.5 * (lo + hi)
196                 if np.minimum(sizes, np.ceil(lam * weight)).sum() > budget:
197                     hi = lam
198                 else:
199                     lo = lam
200             take = np.minimum(sizes, np.ceil(lo * weight)).astype(np.int64)
201             keep = take > 0
202             visible, take = visible[keep], take[keep]
203 
204         # gom các đoạn [start_c, start_c + take_c) thành một mảng
205         total = int(take.sum())
206         ends = np.cumsum(take)
207         offsets = np.repeat(self.starts[visible] - (ends - take), take)
208         return np.arange(total, dtype=np.int64) + offsets
```

1.8. `splat_editor/core/lighting.py`

276 dòng

Vai trò. Trạng thái ánh sáng: ambient, một mặt trời, tối đa 8 khối đèn điểm, chu kỳ ngày–đêm dạng máy trạng thái và bản đồ bóng. Không có OpenGL: `Lighting.params()` đổi trạng thái thành một `dict` để renderer gửi vào shader. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§6.1).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Ánh sáng và chu kỳ ngày đêm (T4 + T5).

- Ánh sáng môi trường (ambient) + một mặt trời (ánh sáng định hướng) + tối đa 8 "khối đèn" (đèn điểm).
- Phong shading: diffuse + specular, tính trong shader cho từng gaussian (xem splat_prep.vert).
- Bóng đổ của mặt trời bằng shadow mapping (bản đồ độ sâu nhìn từ mặt trời, vẽ ở SplatRenderer).
- Mặt trời chuyển động theo thời gian; chu kỳ ngày đêm là một máy trạng thái (NIGHT -> DAWN -> DAY -> DUSK)
  điều khiển màu bầu trời và tự bật / tắt các khối đèn.
- Khối đèn chịu trọng lực: thả xuống thì rơi, nảy nhẹ rồi nằm yên trên bề mặt của file ply.

Không có OpenGL ở đây: Lighting chỉ giữ trạng thái và đổi ra tham số (params) cho renderer.
"""

"""
lighting = Lighting()
lighting.set_bounds(center, radius)         # kích thước cảnh (để đặt tầm đèn, bản đồ bóng)
lighting.add_sun()                          # bật mặt trời (và bật hệ thống ánh sáng)
light = lighting.add_light(position)        # thêm một khối đèn, trả về None nếu đã đủ 8
lighting.update(dt, physics)                # mỗi khung hình: thời gian, máy trạng thái, khối đèn rơi
params = lighting.params()                  # dict cho SplatRenderer.set_lighting / LightRenderer
"""

```

Cơ sở toán: Mặt trời, ngày–đêm và bản đồ bóng

Phần này gồm quỹ đạo mặt trời (`Lighting.sun_direction`), máy trạng thái `DayCycle` và thiết lập bản đồ bóng (`Lighting.shadow_setup`). Phần dùng bản đồ bóng trong shader (`shadowFactor`, `shadow.vert`) xem mục 6.1 (`splat_prep.vert`) và `shadow.vert`. Mô hình Phong và pháp tuyến gaussian cũng nằm ở mục 6.1.

Mặt trời (`Lighting.sun_direction`).

$$\begin{aligned}
\text{angle} &= \frac{\text{hour}-6}{12}\pi \quad (\text{giờ } 6\to 0,\ 12\to\frac{\pi}{2},\ 18\to\pi)\\
\text{elevation} &= \sin(\text{angle})\cdot 65^\circ \quad (\text{âm ban đêm})\\
\text{azimuth} &= \text{azimuth}_0 + (\text{angle}-\frac{\pi}{2})\\
\text{dir} &= (\cos el\cos az,\ \sin el,\ \cos el\sin az) \quad (\text{cầu}\to\text{Descartes, đơn vị})
\end{aligned}$$

Chu kỳ: `hour += dt*24/day_length (mod 24)`, `day_length = 36 s`, tức 1.5 s/giờ.

- daylight $=$ smoothstep$(-0.08, 0.3, \text{dir}_y)$.
- Màu mặt trời: mix$((1,.5,.25),(1,.96,.88),\text{smoothstep}(0,.35,s))\cdot I\cdot\text{smoothstep}(-.02,.2,s)$ (cam lúc thấp, trắng lúc cao).
- Ambient: base $\cdot$ mix$((.12,.16,.32),1,\text{daylight})$ (xanh tối ban đêm).
- Bầu trời: mix(đêm, ngày, daylight) rồi trộn thêm cam với trọng số $0.55\exp(-(s/0.18)^2)$, một bump Gauss quanh đường chân trời (hoàng hôn).
- mix$(a,b,t) = a+(b-a)t$.

Máy trạng thái ngày–đêm (`DayCycle`). Mốc bắt đầu: DAWN 5h, DAY 7h, DUSK 17h, NIGHT 19h. `phase_for(h)`: $h\in[\text{start},\text{end})$, hoặc nếu start $>$ end (qua nửa đêm) thì $h\ge\text{start}\vee h<\text{end}$. Sự kiện `on_enter` bật/tắt đèn `auto_night`.

Bản đồ bóng (`Lighting.shadow_setup`, `shadow.vert`, `shadowFactor`).

- $r = 1.1\cdot$ radius cảnh; mắt đèn eye $=$ center $+ d\cdot 2r$; up $=(0,1,0)$ (hoặc $(0,0,1)$ nếu $|d_y|\ge 0.99$ để tránh $f\parallel$ up).
- $VP = \text{Ortho}(r, 0.01r, 4r)\cdot\text{LookAt}(\text{eye},\text{center})$; độ phân giải 2048^2.
- texel $= 2r/2048$; bias $= 3\,\text{texel}/(3.99r)$ (độ lệch độ sâu quy về NDC $[0,1]$; miền z dài $3.99r$).
- Điểm tra: $lp = VP\cdot(p + n\cdot 2\,\text{texel})$ (đẩy dọc pháp tuyến chống tự bóng); $s = lp_{xyz}/lp_w\cdot 0.5+0.5$ (NDC sang $[0,1]$ để lấy mẫu texture); nằm ngoài $[0,1]$ thì coi là sáng.
- **PCF** 3×3**:** $S=\frac{1}{9}\sum_{i,j=-1}^{1}\left[s_z-\text{bias}\le D(s_{xy}+(i,j)/2048)\right]$.
- Chỉ gaussian có $\alpha\ge 0.3$ và chưa xóa mới được vẽ vào depth map (dạng điểm).

Khai báo mức module (dòng 22–30)

```
import math
from dataclasses import dataclass
from enum import Enum

import numpy as np

from .math3d import look_at, orthographic

```

```
30 MAX_LIGHTS = 8
```

`class DayPhase(Enum)`

Dòng 33–37

```
33 class DayPhase(Enum):
34     NIGHT = "Night"
35     DAWN = "Dawn"
36     DAY = "Day"
37     DUSK = "Dusk"
```

`class DayCycle`

Dòng 40–65

Mô tả trong mã nguồn. Máy trạng thái ngày đêm. Mỗi trạng thái bắt đầu ở một giờ cố định; chuyển trạng thái khi giờ chạm mốc của trạng thái kế tiếp (hoặc nhảy thẳng tới trạng thái đúng nếu kéo thanh trượt giờ).

Giải thích. Máy trạng thái 4 pha: NIGHT → DAWN (5h) → DAY (7h) → DUSK (17h) → NIGHT (19h). Khi chuyển pha gọi sự kiện `on_enter` (bật/tắt đèn `auto_night`).

Công thức.

$$\text{phase}(h) = \begin{cases} \text{DAWN} & 5 \le h < 7 \\ \text{DAY} & 7 \le h < 17 \\ \text{DUSK} & 17 \le h < 19 \\ \text{NIGHT} & h \ge 19 \text{ hoặc } h < 5 \end{cases}$$

Các phương thức: `__init__`, `phase_for`, `update`.

```
40 class DayCycle:
41     """Máy trạng thái ngày đêm. Mỗi trạng thái bắt đầu ở một giờ cố định; chuyển trạng thái khi giờ
42     chạm mốc của trạng thái kế tiếp (hoặc nhảy thẳng tới trạng thái đúng nếu kéo thanh trượt giờ)."""
43
44     START = {DayPhase.DAWN: 5.0, DayPhase.DAY: 7.0, DayPhase.DUSK: 17.0, DayPhase.NIGHT: 19.0}
45     NEXT = {DayPhase.NIGHT: DayPhase.DAWN, DayPhase.DAWN: DayPhase.DAY,
46             DayPhase.DAY: DayPhase.DUSK, DayPhase.DUSK: DayPhase.NIGHT}
```

`DayCycle.__init__(self, hour=10.0)`

Dòng 48–49

```
48     def __init__(self, hour=10.0):
49         self.state = self.phase_for(hour)
```

`DayCycle.phase_for(cls, hour)`

Dòng 51–57

```
51     @classmethod
52     def phase_for(cls, hour):
53         for phase, start in cls.START.items():
54             end = cls.START[cls.NEXT[phase]]
55             if (start <= hour < end) if start < end else (hour >= start or hour < end):
56                 return phase
57         return DayPhase.DAY
```

`DayCycle.update(self, hour)`

Dòng 59–65

Mô tả trong mã nguồn. Trả về trạng thái mới nếu vừa chuyển, ngược lại None (sự kiện on_enter).

```
59     def update(self, hour):
60         """Trả về trạng thái mới nếu vừa chuyển, ngược lại None (sự kiện on_enter)."""
61         phase = self.phase_for(hour)
62         if phase is self.state:
63             return None
64         self.state = phase
65         return phase
```

class PointLight

Dòng 68–79

```
@dataclass
class PointLight:
    position: np.ndarray
    range: float
    color: tuple = (1.0, 0.82, 0.55)
    intensity: float = 3.0
    on: bool = True
    auto_night: bool = False     # tự bật lúc hoàng hôn, tắt lúc bình minh (máy trạng thái ngày đêm)
    falls: bool = False          # đèn đứng yên tại chỗ đặt, không chịu trọng lực
    vel_y: float = 0.0
    resting: bool = False
    name: str = "Light"
```

_smooth(edge0, edge1, x)

Dòng 82–84

```
def _smooth(edge0, edge1, x):
    t = min(max((x - edge0) / (edge1 - edge0), 0.0), 1.0)
    return t * t * (3.0 - 2.0 * t)
```

_mix(a, b, t)

Dòng 87–88

```
def _mix(a, b, t):
    return tuple(x + (y - x) * t for x, y in zip(a, b))
```

class Lighting

Dòng 91–276

Các phương thức: `__init__`, `set_bounds`, `add_sun`, `remove_sun`, `add_light`, `remove_light`, `active_light`, `sun_direction`, `daylight`, `sun_color`, `sun_color_for_gizmo`, `ambient_color`, `sky_color`, `update`, `_on_enter`, `_update_lights`, `shadow_setup`, `point_lights`, `params`.

```
class Lighting:
    SHADOW_SIZE = 2048
```

Lighting.__init__(self)

Dòng 94–116

```
    def __init__(self):
        self.enabled = False
        self.ambient = 0.3
        self.specular = 0.25
        self.shininess = 24.0
        self.shadows = True

        self.sun_on = False
        self.sun_intensity = 0.9
        self.sun_azimuth = 35.0          # độ
        self.hour = 10.0                 # giờ trong ngày, 0..24
        self.animate = True              # mặt trời tự di chuyển theo mặc định
        self.day_length = 36.0           # giây thật cho một ngày: 1.5 giây = 1 giờ
        self.sky_follows = True
        self.cycle = DayCycle(self.hour)

        self.lights = []
        self.selected = -1
        self.picked = -1                 # đèn vừa click chọn trong cảnh 3D: phím Delete xoá đèn này
        self.fireflies = True
        self.center = np.zeros(3)
        self.radius = 1.0
        self.events = []                 # thông báo ngắn cho thanh trạng thái
```

Lighting.set_bounds(self, center, radius)

Dòng 119–121

```
    def set_bounds(self, center, radius):
        self.center = np.asarray(center, float)
        self.radius = max(float(radius), 1e-6)
```

`Lighting.add_sun(self)`

Dòng 123–124

```
123     def add_sun(self):
124         self.enabled = self.sun_on = True
```

`Lighting.remove_sun(self)`

Dòng 126–127

```
126     def remove_sun(self):
127         self.sun_on = False
```

`Lighting.add_light(self, position, light_range=None)`

Dòng 129–137

```
129     def add_light(self, position, light_range=None):
130         if len(self.lights) >= MAX_LIGHTS:
131             return None
132         light = PointLight(np.array(position, float), light_range or self.radius * 0.5,
133                            name=f"Light {len(self.lights) + 1}")
134         self.lights.append(light)
135         self.selected = len(self.lights) - 1
136         self.enabled = True
137         return light
```

`Lighting.remove_light(self, index)`

Dòng 139–143

```
139     def remove_light(self, index):
140         if 0 <= index < len(self.lights):
141             del self.lights[index]
142             self.selected = min(self.selected, len(self.lights) - 1)
143             self.picked = -1
```

`Lighting.active_light(self)`

Dòng 145–147

```
145     @property
146     def active_light(self):
147         return self.lights[self.selected] if 0 <= self.selected < len(self.lights) else None
```

`Lighting.sun_direction(self)`

Dòng 150–156

Mô tả trong mã nguồn. Vector đơn vị từ cảnh hướng về mặt trời. Giờ 6: mọc, 12: trên cao nhất, 18: lặn.

Giải thích. Tọa độ cầu → Descartes: góc giờ $\frac{hour-6}{12}\pi$, độ cao $\sin(angle)\cdot 65°$. Giờ 6 mọc, 12 ở đỉnh, 18 lặn.

Công thức.

$$\theta = \frac{h-6}{12}\pi, \quad el = 65° \sin\theta, \quad az = az_0 + \left(\theta - \frac{\pi}{2}\right), \quad \hat{d} = \begin{bmatrix} \cos el \cos az \\ \sin el \\ \cos el \sin az \end{bmatrix}$$

```
150     def sun_direction(self):
151         """Vector đơn vị từ cảnh hướng về mặt trời. Giờ 6: mọc, 12: trên cao nhất, 18: lặn."""
152         angle = (self.hour - 6.0) / 12.0 * math.pi
153         elevation = math.sin(angle) * math.radians(65.0)        # âm vào ban đêm
154         azimuth = math.radians(self.sun_azimuth) + (angle - math.pi / 2)
155         return np.array([math.cos(elevation) * math.cos(azimuth), math.sin(elevation),
156                          math.cos(elevation) * math.sin(azimuth)])
```

`Lighting.daylight(self)`

Dòng 158–160

Mô tả trong mã nguồn. 0 (đêm) .. 1 (ban ngày), mượt quanh lúc mặt trời qua đường chân trời.

```
158     def daylight(self):
159         """0 (đêm) .. 1 (ban ngày), mượt quanh lúc mặt trời qua đường chân trời."""
160         return _smooth(-0.08, 0.3, self.sun_direction()[1])
```

`Lighting.sun_color(self)`

Dòng 162–166

```
    def sun_color(self):
        s = self.sun_direction()[1]
        warm = _mix((1.0, 0.5, 0.25), (1.0, 0.96, 0.88), _smooth(0.0, 0.35, s))
        strength = self.sun_intensity * _smooth(-0.02, 0.2, s)
        return tuple(c * strength for c in warm)
```

`Lighting.sun_color_for_gizmo(self)`

Dòng 168–170

Mô tả trong mã nguồn. Màu quả cầu mặt trời (luôn sáng rõ, đỏ cam khi gần đường chân trời).

```
    def sun_color_for_gizmo(self):
        """Màu quả cầu mặt trời (luôn sáng rõ, đỏ cam khi gần đường chân trời)."""
        return _mix((1.0, 0.45, 0.2), (1.0, 0.95, 0.75), _smooth(0.0, 0.35, self.sun_direction()[1]))
```

`Lighting.ambient_color(self)`

Dòng 172–177

```
    def ambient_color(self):
        base = self.ambient
        if not self.sun_on:
            return (base, base, base)
        tint = _mix((0.12, 0.16, 0.32), (1.0, 1.0, 1.0), self.daylight())
        return tuple(base * c for c in tint)
```

`Lighting.sky_color(self)`

Dòng 179–183

Giải thích. Màu trời: trộn đêm/ngày theo `daylight`, rồi thêm màu cam hoàng hôn với trọng số Gauss quanh đường chân trời.

Công thức.

$$\text{daylight} = \text{smoothstep}(-0.08, 0.3, \hat{d}_y), \qquad c_{sky} = \text{mix}(c_{night}, c_{day}, \text{daylight}) + 0.55\, e^{-(s/0.18)^2}\, c_{dusk}$$

$\text{smoothstep}(a,b,x) = t^2(3-2t)$, $t = \text{clip}\big(\frac{x-a}{b-a}, 0, 1\big)$.

```
    def sky_color(self):
        s = self.sun_direction()[1]
        sky = _mix((0.012, 0.018, 0.05), (0.42, 0.62, 0.92), self.daylight())
        twilight = math.exp(-(s / 0.18) ** 2)
        return _mix(sky, (0.85, 0.45, 0.32), 0.55 * twilight)
```

`Lighting.update(self, dt, physics=None)`

Dòng 186–194

Mô tả trong mã nguồn. Tiến thời gian, chạy máy trạng thái ngày đêm và vật lý của khối đèn.

Giải thích. Mỗi khung hình: tăng giờ ($24/day_length$ giờ mỗi giây), chạy máy trạng thái và vật lý khối đèn.

Công thức.

$$h \leftarrow (h + \Delta t \cdot 24/T_{day}) \bmod 24, \qquad T_{day} = 36 \text{ s} \Rightarrow 1.5 \text{ s/giờ}$$

```
    def update(self, dt, physics=None):
        """Tiến thời gian, chạy máy trạng thái ngày đêm và vật lý của khối đèn."""
        if self.animate and self.sun_on:
            self.hour = (self.hour + dt * 24.0 / max(self.day_length, 1.0)) % 24.0
        phase = self.cycle.update(self.hour)
        if phase is not None and self.sun_on:
            self._on_enter(phase)
        if physics is not None:
            self._update_lights(dt, physics)
```

`Lighting._on_enter(self, phase)`

Dòng 196–207

```
    def _on_enter(self, phase):
        if phase in (DayPhase.DUSK, DayPhase.NIGHT):
            changed = [l for l in self.lights if l.auto_night and not l.on]
            for light in changed:
                light.on = True
        elif phase in (DayPhase.DAWN, DayPhase.DAY):
            changed = [l for l in self.lights if l.auto_night and l.on]
            for light in changed:
                light.on = False
        else:
            changed = []
        self.events.append(f"{phase.value}" + (f": {len(changed)} auto light(s) switched" if changed else ""))
```

`Lighting._update_lights(self, dt, physics)`

Dòng 209–230

Giải thích. Khối đèn rơi bằng tích phân Euler; chạm đất mà tốc độ lớn hơn $0.6H$ thì nảy với hệ số phục hồi 0.35, ngược lại nằm yên.

Công thức.

$$v_y \leftarrow v_y - g\,\Delta t, \quad y \leftarrow y + v_y \Delta t, \qquad \text{chạm đất: } |v| > 0.6H \Rightarrow v \leftarrow -0.35\,v$$

```
    def _update_lights(self, dt, physics):
        dt = min(dt, 0.05)
        g = physics.gravity
        rest = physics.height * 0.12                # khối đèn nằm cách mặt đất một chút
        for light in self.lights:
            if not light.falls or light.resting:
                continue
            x, y, z = light.position
            light.vel_y -= g * dt
            new_y = y + light.vel_y * dt
            ground = physics.drop_height(x, z, y, radius=physics.height * 0.05) if physics.ready else None
            floor = ground if ground is not None else (physics.world.min_y if physics.ready else None)
            if floor is None:
                light.vel_y = 0.0                   # chưa có bản đồ va chạm: đứng yên chờ
                continue
            if new_y <= floor + rest:
                new_y = floor + rest
                if ground is not None and abs(light.vel_y) > 0.6 * physics.height:
                    light.vel_y = -light.vel_y * 0.35   # nảy lên
                else:
                    light.vel_y, light.resting = 0.0, True
            light.position[1] = new_y
```

`Lighting.shadow_setup(self)`

Dòng 233–242

Mô tả trong mã nguồn. (ma trận ánh sáng, kích thước texel theo thế giới, độ lệch độ sâu) cho bản đồ bóng trực giao.

Giải thích. Dựng ma trận ánh sáng cho bản đồ bóng trực giao: bán kính $r = 1.1\times$bán kính cảnh, mắt đèn = tâm $+ d \cdot 2r$; trả về $(VP,\ texel,\ bias)$.

Công thức.

$$r = 1.1\,R_{scene}, \quad e = c + \hat{d}\cdot 2r, \quad VP = \mathrm{Ortho}(r,\ 0.01r,\ 4r)\cdot \mathrm{LookAt}(e, c),$$

$$\text{texel} = \frac{2r}{2048}, \qquad \text{bias} = \frac{3\,\text{texel}}{3.99\,r}$$

```
    def shadow_setup(self):
        """(ma trận ánh sáng, kích thước texel theo thế giới, độ lệch độ sâu) cho bản đồ bóng trực giao."""
        d = self.sun_direction()
        r = self.radius * 1.1
        eye = self.center + d * 2.0 * r
        up = np.array([0.0, 1.0, 0.0]) if abs(d[1]) < 0.99 else np.array([0.0, 0.0, 1.0])
        view = look_at(eye, self.center, up).astype(np.float64)
        proj = orthographic(r, 0.01 * r, 4.0 * r).astype(np.float64)
        texel = 2.0 * r / self.SHADOW_SIZE
        return (proj @ view).astype(np.float32), texel, 3.0 * texel / (3.99 * r)
```

`Lighting.point_lights(self)`

Dòng 245–246

```
    def point_lights(self):
        return [l for l in self.lights if l.on]
```

`Lighting.params(self)`

Dòng 248–276

Mô tả trong mã nguồn. Trạng thái ánh sáng dạng dict cho renderer; ‘signature‘ đổi khi cần tính lại màu gaussian.

Giải thích. Đóng gói trạng thái thành `dict` cho renderer. Trường `signature` đổi khi phải tính lại màu của mọi gaussian (pass 1).

```
    def params(self):
        """Trạng thái ánh sáng dạng dict cho renderer; `signature` đổi khi cần tính lại màu gaussian."""
        if not self.enabled:
            return {'enabled': False, 'signature': (False,)}
        points = self.point_lights()
        sun = self.sun_on and self.sun_direction()[1] > -0.1
        sun_dir = self.sun_direction() if self.sun_on else np.array([0.0, 1.0, 0.0])
        shadows = bool(self.shadows and sun and self.sun_color()[0] > 1e-3)
        params = {
            'enabled': True,
            'ambient': self.ambient_color(),
            'sun_on': bool(sun),
            'sun_dir': sun_dir,
            'sun_color': self.sun_color() if sun else (0.0, 0.0, 0.0),
            'specular': self.specular,
            'shininess': self.shininess,
            'point_pos': np.array([l.position for l in points], np.float32).reshape(-1, 3),
            'point_color': np.array([np.array(l.color) * l.intensity for l in points], np.float32).reshape(-1, 3),
            'point_range': np.array([l.range for l in points], np.float32),
            'shadows': shadows,
        }
        if shadows:
            params['shadow_vp'], params['shadow_texel'], params['shadow_bias'] = self.shadow_setup()
        key = lambda a: tuple(np.round(np.asarray(a, np.float64).ravel(), 4))  # noqa: E731
        params['signature'] = (True, key(params['ambient']), params['sun_on'], key(sun_dir),
                               key(params['sun_color']), round(self.specular, 3), round(self.shininess, 1),
                               shadows, key(params['point_pos']), key(params['point_color']),
                               key(params['point_range']), round(self.radius, 4))
        return params
```

1.9. `splat_editor/core/physics.py`

320 dòng

Vai trò. Vật lý cho chế độ sinh tồn: voxel hóa tâm gaussian thành bản đồ va chạm, người chơi (trọng lực, nhảy, leo bậc, trượt tường) và vật rơi. Truy vấn cột dùng khóa 63 bit và `np.searchsorted` nên rất rẻ. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§1.9).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Vật lý cho file ply (T5): va chạm voxel dựng từ tâm gaussian, người chơi kiểu Minecraft sinh tồn
(trọng lực, nhảy, leo bậc, trượt dọc tường) và vật rơi (khối đèn).

Gaussian không có mặt phẳng / thể tích rõ ràng nên ta rời rạc hoá thành lưới voxel: ô nào có đủ
gaussian "rắn" (đủ đục, chưa xoá) thì là đặc. Mỗi ô đặc được mã hoá thành một số nguyên 64 bit
(ix, iz, iy) rồi xếp tăng dần; truy vấn "ô đặc cao nhất tại cột (ix, iz) không cao hơn iy" chỉ là một lần
np.searchsorted, nên vài chục truy vấn mỗi khung hình gần như không tốn gì.

Trục Y hướng lên (sau khi lật COLMAP), trọng lực kéo về -Y. Mọi kích thước tính theo
`Physics.height` (chiều cao người chơi, đơn vị cảnh) vì cảnh 3DGS không có đơn vị cố định.
"""

"""
physics = Physics()
physics.set_scale(radius)                  # đặt chiều cao người chơi theo kích thước cảnh
physics.ensure_world(scene, needed=True)    # mỗi khung hình: dựng bản đồ va chạm ở luồng nền khi cần
physics.player.place(eye, physics.eye_height)   # thả người chơi từ vị trí mắt
physics.walk(forward, right, yaw, jump, sprint, sneak, dt)   # đi bộ: cập nhật vị trí chân
eye = physics.player.eye(physics.eye_height, dt)
y = physics.drop_height(x, z, y_from)       # độ cao mặt đất dưới (x, z) hoặc None
"""

```

Cơ sở toán: Vật lý và va chạm

Voxel hóa (`CollisionWorld.build`). cell $= \lfloor (p - \text{origin})/v \rfloor$, origin $= \min - v$; $v = \text{height}/8$; kẹp $[0, 2^{21} - 1]$. **Khóa 63 bit:** key $= (c_x \ll 42) \mid (c_z \ll 21) \mid c_y$ (mỗi trục 21 bit). c_y ở bit thấp nên các ô cùng cột (c_x, c_z) nằm liền nhau

trong mảng đã sắp. Ô đặc nếu có ít nhất `min_count` gaussian (3/2/1 tùy mức lấy mẫu). `floor_y` là phân vị 1% của y (sàn an toàn).

Truy vấn cột (`_highest`). query $=$ key(i_x, i_z, i_y); idx $=$ searchsorted(keys, query, 'right') $-\,1$, tức khóa lớn nhất $\le$ query; hợp lệ nếu found $\gg 21 =$ query $\gg 21$ (cùng cột). Trả về i_y cao nhất $\le y$. Độ cao mặt trên ô: origin$_y + (i_y + 1)v$. Độ phức tạp $O(\log N)$ mỗi truy vấn, đã vector hóa.

Tia kiểm tra tầm nhìn (`segment_blocked`). Lấy mẫu dọc đoạn với bước $v/2$: $n = \lfloor L/(v/2) \rfloor + 1$, $t = \text{linspace}(0, 1, n)$; điểm $=$ start $+$ (end $-$ start)t; tra khóa; trúng ô đặc thì bị chặn.

Người chơi. Kích thước tỉ lệ với chiều cao H (2.74): mắt $0.92H$, bán kính $0.14H$, bước leo $0.35H$, nhảy $0.75H$, $g = 12H$, tốc độ đi $2.5H$ ($\times 1.7$ chạy, $\times 0.4$ rình).

- **Va chạm cột trụ:** 9 điểm vòng `_RING` $=$ tâm $+$ 8 điểm $(\cos\frac{k\pi}{4}, \sin\frac{k\pi}{4}) \cdot$ bán kính.
- **Trượt tường:** thử $(x + dx, z + dz)$; nếu bị chặn thử chỉ x; rồi chỉ z.
- **Nhảy:** $v_y = \sqrt{2g\,h_{\text{nhảy}}}$ (từ $v^2 = 2gh$).
- **Euler:** $v_y \leftarrow v_y - g\Delta t$, $y \leftarrow y + v_y \Delta t$ (Δt kẹp ≤ 0.05); chạm đất khi $v_y \le 0 \ \wedge\ y_{\text{mới}} \le$ ground.
- **Làm mượt mắt:** eye$_y$ $+=$ (target $-$ eye$_y$) $\min(18\Delta t, 1)$ (lọc mũ, che bậc thang).
- **Đèn rơi** (`Lighting._update_lights`, trong file `lighting.py`): cùng Euler; khi chạm đất nếu $|v| > 0.6H$ thì **nảy**: $v = -0.35v$ (hệ số phục hồi 0.35), ngược lại nằm yên.

Khai báo mức module (dòng 23–31)

```
import math
import threading
import time

import numpy as np

_BITS = 21
_MASK = (1 << _BITS) - 1
_RING = np.array([(0.0, 0.0)] + [(math.cos(a * math.pi / 4), math.sin(a * math.pi / 4)) for a in range(8)])
```

`class CollisionWorld`

Dòng 34–112

Mô tả trong mã nguồn. Tập các voxel đặc, dựng một lần từ vị trí gaussian (toạ độ thế giới).

Các phương thức: `__init__`, `ready`, `build`, `_highest`, `column_top`, `segment_blocked`, `blocked`.

```
class CollisionWorld:
    """Tập các voxel đặc, dựng một lần từ vị trí gaussian (toạ độ thế giới)."""
```

`CollisionWorld.__init__(self)`

Dòng 37–44

```
    def __init__(self):
        self.keys = np.zeros(0, np.int64)
        self.origin = np.zeros(3)
        self.voxel = 1.0
        self.min_y = 0.0
        self.max_y = 0.0
        self.solid_count = 0
        self.floor_y = 0.0        # sàn an toàn: không bao giờ rơi xuống dưới mức này
```

`CollisionWorld.ready(self)`

Dòng 46–48

```
    @property
    def ready(self):
        return len(self.keys) > 0
```

`CollisionWorld.build(self, positions, voxel, min_count=1)`

Dòng 50–69

Mô tả trong mã nguồn. positions (N,3) float; voxel: cạnh ô; min_count: số gaussian tối thiểu để ô là đặc.

Giải thích. Voxel hóa: `cell = floor((p-origin)/v)`, khóa 63 bit $(c_x \ll 42)|(c_z \ll 21)|c_y$, đếm gaussian mỗi ô và giữ những ô có ít nhất `min_count`. Mảng khóa được sắp để dùng `searchsorted`.

Công thức.

$$v = \frac{H}{8}, \quad \text{cell} = \left\lfloor \frac{p - \text{origin}}{v} \right\rfloor, \quad \text{key} = (c_x \ll 42) \mid (c_z \ll 21) \mid c_y$$

Ô đặc nếu có ≥ min_count gaussian rắn.

```
50    def build(self, positions, voxel, min_count=1):
51        """positions (N,3) float; voxel: cạnh ô; min_count: số gaussian tối thiểu để ô là đặc."""
52        self.voxel = float(voxel)
53        if len(positions) == 0:
54            return self
55        lo = positions.min(axis=0).astype(np.float64)
56        self.origin = lo - self.voxel
57        self.min_y, self.max_y = float(lo[1]), float(positions[:, 1].max())
58        self.floor_y = float(np.percentile(positions[:, 1], 1.0))   # bỏ qua gaussian lạc xa bên dưới
59        cells = np.floor((positions - self.origin) / self.voxel).astype(np.int64)
60        np.clip(cells, 0, _MASK, out=cells)
61        keys = (cells[:, 0] << (2 * _BITS)) | (cells[:, 2] << _BITS) | cells[:, 1]
62        if min_count > 1:
63            keys, counts = np.unique(keys, return_counts=True)
64            keys = keys[counts >= min_count]
65        else:
66            keys = np.unique(keys)
67        self.keys = keys
68        self.solid_count = len(keys)
69        return self
```

`CollisionWorld._highest(self, x, z, y)`

Dòng 71–82

Mô tả trong mã nguồn. Chỉ số iy của ô đặc cao nhất tại cột (x, z) mà iy <= ô chứa độ cao y; -1 nếu không có.

Giải thích. Tìm ô đặc cao nhất trong cột không vượt quá độ cao cho trước, bằng một lần `searchsorted` và kiểm tra cùng cột (`found>>21 == query>>21`). $O(\log N)$, vector hóa.

Công thức.

$$\text{idx} = \text{searchsorted(keys, } q\text{, 'right')} - 1, \qquad \text{hợp lệ nếu } \text{keys[idx]} \gg 21 = q \gg 21, \qquad y_{top} = \text{origin}_y + (i_y + 1)\, v$$

```
71    def _highest(self, x, z, y):
72        """Chỉ số iy của ô đặc cao nhất tại cột (x, z) mà iy <= ô chứa độ cao y; -1 nếu không có."""
73        x, z, y = np.broadcast_arrays(np.asarray(x, float), np.asarray(z, float), np.asarray(y, float))
74        cell = np.floor((np.stack([x, y, z], axis=-1) - self.origin) / self.voxel).astype(np.int64)
75        ix, iy, iz = cell[..., 0], cell[..., 1], cell[..., 2]
76        valid = (ix >= 0) & (ix <= _MASK) & (iz >= 0) & (iz <= _MASK) & (iy >= 0)
77        iy = np.minimum(iy, _MASK)
78        query = (np.clip(ix, 0, _MASK) << (2 * _BITS)) | (np.clip(iz, 0, _MASK) << _BITS) | iy
79        idx = np.searchsorted(self.keys, query, side='right') - 1
80        found = self.keys[np.maximum(idx, 0)]
81        same = valid & (idx >= 0) & ((found >> _BITS) == (query >> _BITS))
82        return np.where(same, found & _MASK, -1)
```

`CollisionWorld.column_top(self, x, z, y_max)`

Dòng 84–87

Mô tả trong mã nguồn. Độ cao mặt trên của ô đặc cao nhất không vượt quá y_max tại mỗi cột (-inf: không có).

```
84    def column_top(self, x, z, y_max):
85        """Độ cao mặt trên của ô đặc cao nhất không vượt quá y_max tại mỗi cột (-inf: không có)."""
86        top = self._highest(x, z, y_max)
87        return np.where(top >= 0, self.origin[1] + (top + 1) * self.voxel, -np.inf)
```

`CollisionWorld.segment_blocked(self, start, end, skip=1.0, skip_end=0.0)`

Dòng 89–106

Mô tả trong mã nguồn. Đoạn thẳng start→end có đi xuyên qua ô đặc nào không (bỏ qua 'skip' ô đầu và 'skip_end' ô cuối: đầu = ngay tại mắt, cuối = vật nằm sát / trong mặt đất như khối đèn đang nằm trên sàn).

Giải thích. Tia kiểm tra tầm nhìn: lấy mẫu dọc đoạn với bước $\upsilon/2$ rồi tra voxel. Dùng cho camera ngôi thứ ba (lùi khi sau lưng có tường) và chọn đèn.

Công thức.

$$n = \left\lfloor \frac{L}{\upsilon/2} \right\rfloor + 1, \qquad p_k = \text{start} + \frac{k}{n-1}(\text{end} - \text{start}), \quad k = 0, \dots, n-1$$

```
    def segment_blocked(self, start, end, skip=1.0, skip_end=0.0):
        """Đoạn thẳng start→end có đi xuyên qua ô đặc nào không (bỏ qua `skip` ô đầu và `skip_end` ô cuối:
        đầu = ngay tại mắt, cuối = vật nằm sát / trong mặt đất như khối đèn đang nằm trên sàn)."""
        start, end = np.asarray(start, float), np.asarray(end, float)
        length = float(np.linalg.norm(end - start))
        if length < 1e-9 or not self.ready:
            return False
        n = int(length / (self.voxel * 0.5)) + 1
        t = np.linspace(0.0, 1.0, n)[int(skip * 2):max(n - int(skip_end * 2), 0)]
        if len(t) == 0:
            return False
        points = start + (end - start) * t[:, None]
        cell = np.floor((points - self.origin) / self.voxel).astype(np.int64)
        valid = np.all((cell >= 0) & (cell <= _MASK), axis=1)
        cell = np.clip(cell, 0, _MASK)
        query = (cell[:, 0] << (2 * _BITS)) | (cell[:, 2] << _BITS) | cell[:, 1]
        idx = np.minimum(np.searchsorted(self.keys, query), len(self.keys) - 1)
        return bool(np.any(valid & (self.keys[idx] == query)))
```

`CollisionWorld.blocked(self, x, z, y0, y1)`

Dòng 108–112

Mô tả trong mã nguồn. Có ô đặc nào nằm trong khoảng độ cao [y0, y1] tại mỗi cột không.

```
    def blocked(self, x, z, y0, y1):
        """Có ô đặc nào nằm trong khoảng độ cao [y0, y1] tại mỗi cột không."""
        top = self._highest(x, z, y1)
        low = np.floor((y0 - self.origin[1]) / self.voxel)
        return (top >= 0) & (top >= low)
```

`class Player`

Dòng 115–141

Mô tả trong mã nguồn. Người chơi: một cột đứng có bán kính, chân ở ‘feet‘. Mắt ở trên chân.

Các phương thức: `__init__`, `place`, `eye`.

```
class Player:
    """Người chơi: một cột đứng có bán kính, chân ở `feet`. Mắt ở trên chân."""
```

`Player.__init__(self)`

Dòng 118–124

```
    def __init__(self):
        self.feet = np.zeros(3)
        self.vel_y = 0.0
        self.on_ground = False
        self.spawn = np.zeros(3)
        self.eye_y = 0.0           # độ cao mắt đã làm mượt (che bậc thang lởm chởm của bề mặt gaussian)
        self.respawns = 0
```

`Player.place(self, eye, eye_height)`

Dòng 126–132

```
    def place(self, eye, eye_height):
        self.feet = np.array(eye, float) - np.array([0.0, eye_height, 0.0])
        self.spawn = self.feet.copy()
        self.vel_y = 0.0
        self.on_ground = False
        self.eye_y = float(eye[1])
        self.respawns = 0
```

`Player.eye(self, eye_height, dt=None)`

Dòng 134–141

Mô tả trong mã nguồn. Vị trí mắt; dt != None: làm mượt độ cao khi đang đứng trên mặt đất.

```
134     def eye(self, eye_height, dt=None):
135         """Vị trí mắt; dt != None: làm mượt độ cao khi đang đứng trên mặt đất."""
136         target = self.feet[1] + eye_height
137         if dt is None or not self.on_ground or abs(target - self.eye_y) > eye_height * 0.6:
138             self.eye_y = target
139         else:
140             self.eye_y += (target - self.eye_y) * min(dt * 18.0, 1.0)
141         return np.array([self.feet[0], self.eye_y, self.feet[2]])
```

`class Physics`

Dòng 144–320

Các phương thức: `__init__`, `eye_height`, `gravity`, `walk_speed`, `set_scale`, `set_height`, `invalidate`, `building`, `ready`, `has_world`, `ensure_world`, `drop_height`, `visibility`, `walk`.

```
144 class Physics:
145     EYE_RATIO = 0.92       # mắt cao bằng 92% chiều cao người
146     VOXELS_PER_HEIGHT = 8  # cạnh voxel = chiều cao / 8
147     DEFAULT_HEIGHT = 2.74  # chiều cao người chơi mặc định (đơn vị cảnh)
```

`Physics.__init__(self)`

Dòng 149–162

```
149     def __init__(self):
150         self.world = None
151         self.player = Player()
152         self.height = self.DEFAULT_HEIGHT
153         self.radius_ratio = 0.14
154         self.step_ratio = 0.35
155         self.jump_ratio = 0.75
156         self.gravity_scale = 1.0
157         self.flat_ground = False  # mặc định: va chạm thật với cảnh (đứng được trên thùng xe...), có sàn an toàn nên không rơi mãi
158         self.status = "Collision map not built"
159         self._dirty = True
160         self._dirty_since = 0.0
161         self._building = False
162         self._token = 0
```

`Physics.eye_height(self)`

Dòng 165–167

```
165     @property
166     def eye_height(self):
167         return self.height * self.EYE_RATIO
```

`Physics.gravity(self)`

Dòng 169–171

```
169     @property
170     def gravity(self):
171         return 12.0 * self.height * self.gravity_scale
```

`Physics.walk_speed(self)`

Dòng 173–175

```
173     @property
174     def walk_speed(self):
175         return 2.5 * self.height
```

`Physics.set_scale(self, radius)`

Dòng 177–179

Mô tả trong mã nguồn. Giữ chiều cao mặc định cố định (2.74); chỉnh lại được trong giao diện.

```
177     def set_scale(self, radius):
178         """Giữ chiều cao mặc định cố định (2.74); chỉnh lại được trong giao diện."""
179         self.set_height(self.DEFAULT_HEIGHT)
```

`Physics.set_height(self, height)`

Dòng 181–184

```
181     def set_height(self, height):
182         if abs(height - self.height) > 1e-9:
183             self.height = float(height)
184             self.invalidate()
```

`Physics.invalidate(self)`

Dòng 187–190

Mô tả trong mã nguồn. Cảnh đổi (mở / xoá / kéo lớp): dựng lại bản đồ va chạm khi cần (chờ yên 0.4 giây).

```
187     def invalidate(self):
188         """Cảnh đổi (mở / xoá / kéo lớp): dựng lại bản đồ va chạm khi cần (chờ yên 0.4 giây)."""
189         self._dirty = True
190         self._dirty_since = time.perf_counter()
```

`Physics.building(self)`

Dòng 192–194

```
192     @property
193     def building(self):
194         return self._building
```

`Physics.ready(self)`

Dòng 196–198

```
196     @property
197     def ready(self):
198         return self.flat_ground or self.has_world
```

`Physics.has_world(self)`

Dòng 200–202

```
200     @property
201     def has_world(self):
202         return self.world is not None and self.world.ready
```

`Physics.ensure_world(self, scene, needed)`

Dòng 204–228

Mô tả trong mã nguồn. Gọi mỗi khung hình. Dựng bản đồ ở luồng nền nếu cần mà đang cũ.

Giải thích. Mỗi khung hình: nếu bản đồ va chạm cũ và cảnh đã yên 0.4 giây thì dựng lại **ở luồng nền**, tránh giật hình.

```
204     def ensure_world(self, scene, needed):
205         """Gọi mỗi khung hình. Dựng bản đồ ở luồng nền nếu cần mà đang cũ."""
206         if not needed or not self._dirty or self._building or scene.is_empty:
207             return
208         if time.perf_counter() - self._dirty_since < 0.4:
209             return
210         positions, step = scene.solid_positions()
211         voxel = self.height / self.VOXELS_PER_HEIGHT
```

```
        min_count = 3 if step == 1 else 2 if step == 2 else 1
        self._dirty = False
        self._building = True
        self._token += 1
        token = self._token
        self.status = "Building collision map..."

        def work():
            try:
                world = CollisionWorld().build(positions, voxel, min_count)
                if token == self._token:
                    self.world = world
                    self.status = f"Collision map: {world.solid_count:,} solid voxels"
            finally:
                self._building = False

        threading.Thread(target=work, name="collision-builder", daemon=True).start()
```

`Physics.drop_height(self, x, z, y_from, radius=0.0)`

Dòng 231–242

Mô tả trong mã nguồn. Độ cao mặt đất dưới (x, z) tính từ y_from trở xuống; None nếu không có.

```
    def drop_height(self, x, z, y_from, radius=0.0):
        """Độ cao mặt đất dưới (x, z) tính từ y_from trở xuống; None nếu không có."""
        if self.flat_ground:
            return 0.0
        if not self.has_world:
            return None
        if radius:
            xs, zs = x + radius * _RING[:, 0], z + radius * _RING[:, 1]
        else:
            xs, zs = np.array([x]), np.array([z])
        top = float(self.world.column_top(xs, zs, y_from).max())
        return top if top > -np.inf else None
```

`Physics.visibility(self, eye, targets, skip_end=0.0)`

Dòng 244–249

Mô tả trong mã nguồn. Tỉ lệ các điểm ‘targets‘ nhìn thấy được từ ‘eye‘ (không bị cảnh che); 1.0 nếu chưa có bản đồ.

```
    def visibility(self, eye, targets, skip_end=0.0):
        """Tỉ lệ các điểm `targets` nhìn thấy được từ `eye` (không bị cảnh che); 1.0 nếu chưa có bản đồ."""
        if not self.has_world or len(targets) == 0:
            return 1.0
        seen = sum(not self.world.segment_blocked(eye, t, skip_end=skip_end) for t in targets)
        return seen / len(targets)
```

`Physics.walk(self, forward, right, yaw, jump, sprint, sneak, dt)`

Dòng 252–320

Mô tả trong mã nguồn. Đi bộ: W/S/A/D theo hướng nhìn ‘yaw‘, nhảy, chạy nhanh, đi chậm. Trả về False nếu chưa có bản đồ va chạm (người chơi đứng yên)

Giải thích. Đi bộ kiểu Minecraft: hướng theo `yaw`, trượt tường (thử $x{+}z$, rồi chỉ x, rồi chỉ z), va chạm cột trụ bằng 9 điểm vòng, nhảy $v_y = \sqrt{2gh}$, trọng lực Euler, leo bậc.

Công thức.

$$g = 12H, \quad v_{jump} = \sqrt{2g\,h_{jump}}, \quad h_{jump} = 0.75H$$

$$v_y \leftarrow v_y - g\Delta t, \quad y \leftarrow y + v_y\Delta t, \quad \text{eye}_y \mathrel{+}= (\text{target} - \text{eye}_y)\min(18\Delta t, 1)$$

Va chạm cột trụ: 9 điểm $(0,0)$ và $r\,(\cos\frac{k\pi}{4}, \sin\frac{k\pi}{4})$, $k = 0..7$.

```
    def walk(self, forward, right, yaw, jump, sprint, sneak, dt):
        """Đi bộ: W/S/A/D theo hướng nhìn `yaw`, nhảy, chạy nhanh, đi chậm.
        Trả về False nếu chưa có bản đồ va chạm (người chơi đứng yên)"""
        if not self.ready:
            return False
        player, world, h = self.player, self.world, self.height
        flat = self.flat_ground
        dt = min(dt, 0.05)
        radius, step = self.radius_ratio * h, self.step_ratio * h
        feet = player.feet

```

```
        s, c = math.sin(yaw), math.cos(yaw)
        move = forward * np.array([-s, -c]) + right * np.array([c, -s])
        length = np.linalg.norm(move)
        if length > 0:
            move = move / length * self.walk_speed * (1.7 if sprint else 0.4 if sneak else 1.0) * dt

        def ring(x, z):
            return x + radius * _RING[:, 0], z + radius * _RING[:, 1]

        def body_blocked(x, z):
            if flat:
                return False
            xs, zs = ring(x, z)
            return bool(world.blocked(xs, zs, feet[1] + step, feet[1] + 0.9 * h).any())

        def ground_at(x, z, y_max):
            if flat:
                return 0.0
            xs, zs = ring(x, z)
            top = float(world.column_top(xs, zs, y_max).max())
            return max(top, world.floor_y)              # có sàn an toàn: đi ra khỏi cảnh cũng không rơi mãi

        x, z = feet[0], feet[2]
        if length > 0:
            if not body_blocked(x + move[0], z + move[1]):
                x, z = x + move[0], z + move[1]
            elif not body_blocked(x + move[0], z):      # trượt dọc tường
                x += move[0]
            elif not body_blocked(x, z + move[1]):
                z += move[1]

        y = feet[1]
        ground = ground_at(x, z, y + step)
        if player.on_ground and jump:
            player.vel_y = math.sqrt(2.0 * self.gravity * self.jump_ratio * h)
            player.on_ground = False
        if player.on_ground:
            if ground is not None and y - ground <= step:
                y = ground                              # bám mặt đất khi đi xuống dốc / bậc nhỏ
            else:
                player.on_ground = False
        if not player.on_ground:
            player.vel_y -= self.gravity * dt
            new_y = y + player.vel_y * dt
            if player.vel_y <= 0 and ground is not None and new_y <= ground:
                y, player.vel_y, player.on_ground = ground, 0.0, True
            else:
                if player.vel_y > 0 and not flat:       # đụng trần
                    xs, zs = ring(x, z)
                    if world.blocked(xs, zs, y + 0.9 * h, new_y + h).any():
                        player.vel_y, new_y = 0.0, y
                y = new_y
        feet[:] = (x, y, z)

        if flat:
            y = max(y, 0.0)
            feet[1] = y
        return True
```

1.10. `splat_editor/core/avatar_pose.py`

172 dòng

Vai trò. Tính **góc các khớp** của nhân vật (nhịp đi, cúi, ngồi, cử chỉ) và dựng ma trận model cho 6 khối cơ thể. Không đụng OpenGL; việc vẽ nằm ở `render/avatar_renderer.py`. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§1.10).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Tư thế của nhân vật người chơi (nhìn thấy khi bấm F5): nhịp đi bộ, cúi người, ngồi và các cử chỉ.

Port từ voxel-engine (WalkCycle.java + HumanoidFigure.java). Lớp này chỉ tính GÓC các khớp (độ),
không đụng tới OpenGL nên kiểm thử được; việc dựng ma trận và vẽ nằm ở render/avatar_renderer.py.

Quy ước (giống Java): nhân vật quay mặt về +Z, quay quanh trục X dương là "đưa chi về phía trước"
khi góc ÂM (chân đá ra trước = góc âm); quay quanh Z dương đưa tay ra ngoài về phía +X.
Đơn vị dài là điểm ảnh của skin 64x64 (người cao 32).
"""

"""
pose = AvatarPose()
pose.update(dt, distance, crouching)    # mỗi khung hình; distance = quãng đường ngang vừa đi
pose.toggle_sit()                       # phím C
pose.toggle_gesture("wave")             # phím V; GESTURES = wave, zombie, cheer
joints = pose.joints()                  # dict góc các khớp + độ hạ thân, đưa cho AvatarRenderer
"""

```

Cơ sở toán: Hoạt họa nhân vật

Các công thức dưới đây được cài đặt trong `core/avatar_pose.py`; việc vẽ 6 khối bằng các ma trận này nằm ở `render/avatar_renderer.py`.

- Bộ lọc tiến về mục tiêu: $\text{approach}(v, t, \text{rate}, dt) = v + (t - v)\min(1, dt \cdot \text{rate})$ (cùng dạng quán tính camera bay).
- **Nhịp đi:** walked = distance/height; phase += walked · STRIDE · 1.75 (pha tỉ lệ **quãng đường**, không phải thời gian, nên chân không trượt); swing = sin(phase) · 60° · amount; chân trái +swing, chân phải −swing, tay ngược pha (đối xứng chéo).
- Biên độ mục tiêu = $\min(1, \text{speed}/(4.3/1.75))$.
- **Thở:** idle = $2.2\sin(1.13t) + 0.8\sin(1.91t)$ (hai tần số không công ước nên không lặp cứng).
- **Cử chỉ:** zombie $-85° + 3\sin(2.4t)$; cheer $\pm(158 + 6\sin 7t)$; wave $142 + 20\sin 9t$; pha trộn tuyến tính `_mix_pair`.
- Ngồi: chân −90°, hạ thân 10 đơn vị skin; stance = $1 - 0.48\,\text{sit} - 0.12\,\text{crouch}$.

Cây phân cấp (`limb_matrices`).

$$
\begin{aligned}
\text{root} &= T(\text{feet})\, R_y(\text{facing} + 180°)\, S(\text{scale})\, T(0, -\text{drop}, 0) \\
\text{upper} &= \text{root} \cdot T(0, 12, 0)\, R_x(\text{lean})\, T(0, -12, 0) \quad \text{(xoay quanh hông)} \\
\text{leg} &= \text{root} \cdot T(0, 12, 0)\, R_x(a)\, T(0, -12, 0) \\
\text{arm} &= \text{upper} \cdot T(p_x, 24, 0)\, R_z(a_1)\, R_x(a_0)\, T(-p_x, -24, 0) \quad \text{(quanh vai, } p_x = \pm 6) \\
\text{head} &= \text{upper} \cdot T(0, 24, 0)\, R_y(\text{yaw})\, R_x(\text{pitch})\, T(0, -24, 0) \quad \text{(quanh cổ)}
\end{aligned}
$$

Mẫu chung: $M_{\text{con}} = M_{\text{cha}} \cdot T(p)\, R\, T(-p)$ (xoay quanh điểm p: dời p về gốc, xoay, dời lại). scale = chiều cao/32. Ma trận quay trục `_rot(axis, deg)` điền $c, -s, s, c$ vào cặp chỉ số $(1,2)$ cho X, $(2,0)$ cho Y, $(0,1)$ cho Z.

Khai báo mức module (dòng 19–31)

```
import math

import numpy as np

GESTURES = ("wave", "zombie", "cheer")

SWING_ANGLE = 60.0      # góc vung tối đa của tay / chân khi chạy
STRIDE = 2.0            # radian nhịp bước mỗi đơn vị skin... đã chuẩn hoá theo chiều cao (xem update)
FULL_SPEED = 4.3        # chiều cao người mỗi giây tương ứng bước tối đa (khối/giây trong Java)
BLEND = 8.0             # độ nhạy khi bật / tắt biên độ vung
POSE_BLEND = 7.0        # độ nhạy chuyển giữa các tư thế (ngồi, cúi, cử chỉ)
SIT_DROP = 10.0         # ngồi: hạ cả người xuống (điểm ảnh skin) để đùi chạm đất
CROUCH_LEAN = 28.0      # cúi người về phía trước (độ)
```

`_approach(value, target, rate, dt)`

Dòng 34–35

```
def _approach(value, target, rate, dt):
    return value + (target - value) * min(1.0, dt * rate)
```

`class AvatarPose`

Dòng 38–124

Các phương thức: `__init__`, `toggle_sit`, `stand`, `toggle_gesture`, `update`, `stance`, `joints`.

```
class AvatarPose:
```

`AvatarPose.__init__(self)`

Dòng 39–47

```
    def __init__(self):
        self.phase = 0.0
        self.amount = 0.0                   # biên độ vung tay chân hiện tại, 0..1
        self.time = 0.0
        self.sit = 0.0                      # 0 = đứng, 1 = ngồi hẳn (đã làm mượt)
        self.crouch = 0.0
        self.sitting = False
        self.gesture = None                 # tên cử chỉ đang bật hoặc None
        self.gesture_amount = {name: 0.0 for name in GESTURES}
```

`AvatarPose.toggle_sit(self)`

Dòng 50–53

```
    def toggle_sit(self):
        self.sitting = not self.sitting
        if self.sitting:
            self.gesture = None
```

`AvatarPose.stand(self)`

Dòng 55–56

```
    def stand(self):
        self.sitting = False
```

`AvatarPose.toggle_gesture(self, name)`

Dòng 58–61

```
    def toggle_gesture(self, name):
        self.gesture = None if self.gesture == name else name
        if self.gesture is not None:
            self.sitting = False if name != "wave" else self.sitting     # vẫy tay được cả khi ngồi
```

`AvatarPose.update(self, dt, distance, crouching, height=1.0)`

Dòng 64–79

Mô tả trong mã nguồn. distance: quãng đường ngang đã đi (đơn vị cảnh); height: chiều cao người (cùng đơn vị) để nhịp bước và biên độ không phụ thuộc kích thước cảnh.

Giải thích. Pha đi bộ tỉ lệ với **quãng đường** đi được (không phải thời gian) nên chân không trượt; biên độ tiến dần về mục tiêu bằng bộ lọc mũ `_approach`.

Công thức.

$$\text{approach}(v, t, \text{rate}, dt) = v + (t - v)\min(1,\, dt \cdot \text{rate}), \qquad \text{phase} \mathrel{+}= \frac{\text{dist}}{H} \cdot \text{STRIDE} \cdot 1.75$$

$$\text{swing} = 60^\circ \sin(\text{phase}) \cdot \text{amount}, \qquad \text{idle} = 2.2\sin(1.13t) + 0.8\sin(1.91t)$$

```
    def update(self, dt, distance, crouching, height=1.0):
        """distance: quãng đường ngang đã đi (đơn vị cảnh); height: chiều cao người (cùng đơn vị) để
        nhịp bước và biên độ không phụ thuộc kích thước cảnh."""
        if dt <= 0.0:
            return
        walked = distance / max(height, 1e-9)                # đơn vị: chiều cao người
        speed = walked / dt
        target = min(1.0, speed / (FULL_SPEED / 1.75))
        self.amount = _approach(self.amount, target, BLEND, dt)
        self.phase += walked * STRIDE * 1.75
        self.time += dt
        self.sit = _approach(self.sit, 1.0 if self.sitting else 0.0, POSE_BLEND, dt)
        self.crouch = _approach(self.crouch, 1.0 if crouching and not self.sitting else 0.0, POSE_BLEND, dt)
        for name in GESTURES:
            self.gesture_amount[name] = _approach(self.gesture_amount[name],
                                                  1.0 if self.gesture == name else 0.0, POSE_BLEND, dt)
```

`AvatarPose.stance(self)`

Dòng 81–84

Mô tả trong mã nguồn. Hệ số chiều cao mắt so với lúc đứng (ngồi / cúi thì thấp hơn).

```
    @property
    def stance(self):
        """Hệ số chiều cao mắt so với lúc đứng (ngồi / cúi thì thấp hơn)."""
        return 1.0 - 0.48 * self.sit - 0.12 * self.crouch
```

`AvatarPose.joints(self)`

Dòng 87–124

Mô tả trong mã nguồn. Góc (độ) của từng khớp. Tay: (quay quanh X, quay quanh Z).

Giải thích. Trả về góc (độ) từng khớp: swing chân/tay, thở (hai tần số không công ước), cử chỉ (zombie, cheer, wave), ngồi và cúi.

```
    def joints(self):
        """Góc (độ) của từng khớp. Tay: (quay quanh X, quay quanh Z)."""
        t = self.time
        walk = self.amount * (1.0 - self.sit)
        swing = math.sin(self.phase) * SWING_ANGLE * walk
        idle = math.sin(t * 1.13) * 2.2 + math.sin(t * 1.91) * 0.8     # thở nhẹ khi đứng yên

        leg_l, leg_r = swing, -swing
        arm_l = [-swing + idle, 0.0]
        arm_r = [swing - idle, 0.0]

        g = self.gesture_amount
        if g["zombie"] > 0.001:                       # hai tay đưa thẳng ra trước, hơi rung theo nhịp bước
            raise_x = -85.0 + 3.0 * math.sin(t * 2.4) + 0.15 * swing
            arm_l = _mix_pair(arm_l, [raise_x, 0.0], g["zombie"])
            arm_r = _mix_pair(arm_r, [raise_x, 0.0], g["zombie"])
        if g["cheer"] > 0.001:                        # hai tay giơ cao qua đầu
            bob = 6.0 * math.sin(t * 7.0)
            arm_l = _mix_pair(arm_l, [0.0, -(158.0 + bob)], g["cheer"])
            arm_r = _mix_pair(arm_r, [0.0, 158.0 + bob], g["cheer"])
        if g["wave"] > 0.001:                         # tay phải giơ lên và vẫy
            wave = 142.0 + 20.0 * math.sin(t * 9.0)
            arm_r = _mix_pair(arm_r, [0.0, wave], g["wave"])

        if self.sit > 0.001:                          # ngồi bệt: đùi ngang, tay đặt lên đùi
            leg_l = leg_l * (1 - self.sit) - 90.0 * self.sit
            leg_r = leg_r * (1 - self.sit) - 90.0 * self.sit
            rest = [-35.0 + idle, 0.0]
            keep = 1.0 - 0.8 * max(g["wave"], g["zombie"], g["cheer"])    # cử chỉ vẫn thắng tư thế nghỉ
            arm_l = _mix_pair(arm_l, rest, self.sit * keep)
            arm_r = _mix_pair(arm_r, [-35.0 - idle, 0.0], self.sit * keep * (1.0 - g["wave"]))

        return {
            "leg_l": leg_l, "leg_r": leg_r,
            "arm_l": tuple(arm_l), "arm_r": tuple(arm_r),
            "lean": CROUCH_LEAN * self.crouch,        # thân + tay + đầu nghiêng quanh hông
            "drop": SIT_DROP * self.sit,              # hạ cả người xuống
        }
```

`_mix_pair(a, b, k)`

Dòng 127–128

```
def _mix_pair(a, b, k):
    return [a[0] + (b[0] - a[0]) * k, a[1] + (b[1] - a[1]) * k]
```

Khai báo mức module (dòng 132–135)

```
LIMBS = ("legLeft", "legRight", "body", "armLeft", "armRight", "head")
HIP_Y = 12.0
SHOULDER_Y = 24.0
ARM_X = 6.0
```

`_translate(x, y, z)`

Dòng 138–141

```
def _translate(x, y, z):
    m = np.eye(4)
    m[:3, 3] = (x, y, z)
    return m
```

`_rot(axis, degrees)`

Dòng 144–149

```
def _rot(axis, degrees):
    c, s = math.cos(math.radians(degrees)), math.sin(math.radians(degrees))
    m = np.eye(4)
    i, j = {"x": (1, 2), "y": (2, 0), "z": (0, 1)}[axis]
    m[i, i], m[i, j], m[j, i], m[j, j] = c, -s, s, c
    return m
```

`limb_matrices(feet, facing, head_yaw, head_pitch, joints, scale)`

Dòng 152–172

Mô tả trong mã nguồn. Ma trận model (4x4) của 6 khối theo thứ tự LIMBS.

feet: vị trí bàn chân; facing: yaw camera (rad) của thân; head_yaw: lệch của đầu so với thân (rad); head_pitch: pitch camera (rad, dương = nhìn xuống); scale: kích thước một điểm ảnh skin (= chiều cao / 32).

Giải thích. Dựng ma trận model 4×4 cho 6 khối theo cây phân cấp $M_{con} = M_{cha}\,T(p)\,R\,T(-p)$ (xoay quanh khớp p). Xem §1.10.

Công thức.

$$M_{con} = M_{cha}\,T(p)\,R\,T(-p)$$

$$\text{root} = T(\text{feet})R_y(\text{facing}+180°)S\,T(0,-\text{drop},0), \quad \text{arm} = \text{upper}\,T(\pm 6,24,0)\,R_z(a_1)R_x(a_0)\,T(\mp 6,-24,0)$$

```
def limb_matrices(feet, facing, head_yaw, head_pitch, joints, scale):
    """Ma trận model (4x4) của 6 khối theo thứ tự LIMBS.

    feet: vị trí bàn chân; facing: yaw camera (rad) của thân; head_yaw: lệch của đầu so với thân (rad);
    head_pitch: pitch camera (rad, dương = nhìn xuống); scale: kích thước một điểm ảnh skin (= chiều cao / 32).
    """
    root = (_translate(*feet) @ _rot("y", math.degrees(facing) + 180.0) @ np.diag([scale, scale, scale, 1.0])
            @ _translate(0.0, -joints["drop"], 0.0))
    upper = root @ _translate(0, HIP_Y, 0) @ _rot("x", joints["lean"]) @ _translate(0, -HIP_Y, 0)

    def leg(angle):
        return root @ _translate(0, HIP_Y, 0) @ _rot("x", angle) @ _translate(0, -HIP_Y, 0)

    def arm(px, angles):
        return (upper @ _translate(px, SHOULDER_Y, 0) @ _rot("z", angles[1]) @ _rot("x", angles[0])
                @ _translate(-px, -SHOULDER_Y, 0))

    head = (upper @ _translate(0, SHOULDER_Y, 0) @ _rot("y", math.degrees(head_yaw))
            @ _rot("x", math.degrees(head_pitch)) @ _translate(0, -SHOULDER_Y, 0))
    return [leg(joints["leg_l"]), leg(joints["leg_r"]), upper,
            arm(-ARM_X, joints["arm_l"]), arm(ARM_X, joints["arm_r"]), head]
```

1.11. `splat_editor/core/history.py`

83 dòng

Vai trò. Lịch sử hoàn tác theo mẫu **Command**: mỗi thao tác đã thực hiện là một đối tượng biết tự hoàn tác. `History` giữ tối đa 20 bước. Vì xóa là xóa mềm nên mỗi lệnh chỉ cần giữ danh sách chỉ số đã xóa.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Lịch sử hoàn tác (undo) theo mẫu Command."""

"""
history = History(max_steps=20)

# Sau khi xoá, đẩy lệnh vào lịch sử (snapshots lấy từ Layer.delete_selected)
snapshots = [(layer, layer.delete_selected())]
history.push(DeleteGaussiansCommand(snapshots))

history.can_undo                        # True
command = history.undo()                # hoàn tác lần xoá gần nhất, trả về Command đã hoàn tác
print(command.description)              # "delete 1,234 gaussians"
history.clear()                         # bỏ hết lịch sử (ví dụ khi mở file mới)

# Muốn thêm thao tác hoàn tác được: kế thừa Command, cài description và undo()
"""

```

Cơ sở toán: Undo

Mẫu Command, tối đa 20 bước. Xóa mềm bằng bit `deleted`; hoàn tác là lật lại các chỉ số ($O(k)$). Trạng thái GPU 1 byte/gaussian: bit 0 = chọn, bit 1 = xóa (`STATE_*`). Màu chọn: $\text{mix}(\text{color}, \text{selectColor}, 0.7 \cdot \text{selected})$.

Khai báo mức module (dòng 18–18)

```
from abc import ABC, abstractmethod
```

`class Command(ABC)`

Dòng 21–34

Mô tả trong mã nguồn. Một thao tác đã thực hiện, có thể hoàn tác.

Giải thích. Lớp trừu tượng: mỗi thao tác phải có `description` và `undo()`. Muốn thêm thao tác hoàn tác được chỉ cần kế thừa lớp này.

Các phương thức: `description`, `undo`, `release`.

```
class Command(ABC):
    """Một thao tác đã thực hiện, có thể hoàn tác."""
```

`Command.description(self)`

Dòng 24–27

```
    @property
    @abstractmethod
    def description(self):
        ...
```

`Command.undo(self)`

Dòng 29–31

```
    @abstractmethod
    def undo(self):
        ...
```

`Command.release(self, load)`

Dòng 33–34

Mô tả trong mã nguồn. Thay mọi mảng dữ liệu ‘data‘ đang giữ bằng load(data) (xem Editor._release_file).

```
    def release(self, load):
        """Thay mọi mảng dữ liệu `data` đang giữ bằng load(data) (xem Editor._release_file)."""
```

`class DeleteGaussiansCommand(Command)`

Dòng 37–52

Các phương thức: `__init__`, `count`, `description`, `undo`.

```
class DeleteGaussiansCommand(Command):
```

`DeleteGaussiansCommand.__init__(self, snapshots)`

Dòng 38–39

```
    def __init__(self, snapshots):
        self.snapshots = snapshots     # [(lớp, chỉ số các gaussian đã xoá)] (xoá mềm, xem core/layer.py)
```

`DeleteGaussiansCommand.count(self)`

Dòng 41–44

Mô tả trong mã nguồn. Số gaussian đã xoá.

```
    @property
    def count(self):
        """Số gaussian đã xoá."""
        return sum(len(indices) for _, indices in self.snapshots)
```

`DeleteGaussiansCommand.description(self)`

Dòng 46–48

```
    @property
    def description(self):
        return f"delete {self.count:,} gaussians"
```

`DeleteGaussiansCommand.undo(self)`

Dòng 50–52

```
    def undo(self):
        for layer, indices in self.snapshots:
            layer.restore(indices)
```

`class History`

Dòng 55–83

Giải thích. Ngăn xếp tối đa `max_steps` (mặc định 20). `push` cắt bỏ phần cũ nhất bằng `del self._stack[:-max_steps]`.

Các phương thức: `__init__`, `__len__`, `__iter__`, `can_undo`, `push`, `undo`, `clear`.

```
class History:
```

`History.__init__(self, max_steps=20)`

Dòng 56–58

```
    def __init__(self, max_steps=20):
        self.max_steps = max_steps
        self._stack = []
```

`History.__len__(self)`

Dòng 60–61

```
    def __len__(self):
        return len(self._stack)
```

`History.__iter__(self)`

Dòng 63–64

```
    def __iter__(self):
        return iter(self._stack)
```

`History.can_undo(self)`

Dòng 66–68

```
    @property
    def can_undo(self):
        return bool(self._stack)
```

`History.push(self, command)`

Dòng 70–72

```
    def push(self, command):
        self._stack.append(command)
        del self._stack[:-self.max_steps]
```

`History.undo(self)`

Dòng 74–80

Mô tả trong mã nguồn. Hoàn tác thao tác gần nhất. Trả về Command đã hoàn tác hoặc None.

```
    def undo(self):
        """Hoàn tác thao tác gần nhất. Trả về Command đã hoàn tác hoặc None."""
        if not self._stack:
            return None
        command = self._stack.pop()
        command.undo()
        return command
```

`History.clear(self)`

Dòng 82–83

```
    def clear(self):
        self._stack.clear()
```

1.12. `splat_editor/core/rect.py`

33 dòng

Vai trò. Hình chữ nhật trên màn hình (tọa độ cửa sổ, gốc trên-trái), dùng cho bố cục giao diện, cọ chọn và ma trận chiếu.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Hình chữ nhật trên màn hình (toạ độ cửa sổ, gốc ở góc trên bên trái)."""

"""
rect = Rect(left=330, top=20, width=1170, height=860)   # toạ độ cửa sổ, gốc trên-trái
rect.aspect                             # width / height (dùng cho ma trận chiếu)
rect.center                             # (x, y) tâm hình chữ nhật
rect.contains(400, 300)                 # True nếu điểm nằm trong
rect.is_empty                           # True nếu rộng hoặc cao <= 0
"""

```

Khai báo mức module (dòng 11–11)

```
from typing import NamedTuple
```

`class Rect(NamedTuple)`

Dòng 14–33

Các phương thức: `is_empty`, `aspect`, `center`, `contains`.

```
class Rect(NamedTuple):
    left: float
    top: float
    width: float
    height: float
```

`Rect.is_empty(self)`

Dòng 20–22

```
    @property
    def is_empty(self):
        return self.width <= 0 or self.height <= 0
```

`Rect.aspect(self)`

Dòng 24–26

```
    @property
    def aspect(self):
        return self.width / self.height if self.height > 0 else 1.0
```

`Rect.center(self)`

Dòng 28–30

```
    @property
    def center(self):
        return self.left + self.width / 2, self.top + self.height / 2
```

`Rect.contains(self, x, y)`

Dòng 32–33

```
    def contains(self, x, y):
        return self.left <= x < self.left + self.width and self.top <= y < self.top + self.height
```

Chương 2. Thư mục editor: thao tác chỉnh sửa

Thư mục `editor` là tầng **thao tác chỉnh sửa**. Nó dùng `Scene`, `History` và `LodIndex` của `core` nhưng cũng **không biết gì về OpenGL hay giao diện**. Hai file: `editor.py` (lớp `Editor`: mở/lưu file, chọn, xóa, hoàn tác, phát tín hiệu ”dữ liệu đã đổi” bằng số phiên bản) và `brush.py` (cọ chọn: tìm gaussian nằm trong nét tô trên màn hình).

2.1. `splat_editor/editor/editor.py`

251 dòng

Vai trò. `Editor` là cửa ngõ duy nhất của mọi thao tác chỉnh sửa. Cơ chế quan trọng là **số phiên bản**: mỗi khi dữ liệu đổi, Editor tăng `geometry_version`, `deleted_version`, `selection_version` hoặc `transform_version`; `Viewport.sync` so sánh các số này để biết phần nào cần gửi lại GPU. Nhờ vậy xóa/hoàn tác chỉ gửi mảng trạng thái 1 byte/gaussian, kéo layer chỉ gửi ma trận 4×4.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Editor: giữ cảnh, lịch sử hoàn tác, cọ chọn và thực hiện mọi thao tác chỉnh sửa.

Editor không biết gì về OpenGL hay giao diện: mỗi khi dữ liệu đổi, nó tăng
`geometry_version` / `deleted_version` / `selection_version`; Viewport so sánh số này để biết khi nào gửi lại lên GPU.
"""

"""
editor = Editor(max_undo=20)
editor.open_file("scene.ply")          # thay cảnh bằng một file (lỗi thì giữ nguyên cảnh cũ)
editor.add_file("other.ply")           # thêm thành lớp mới

editor.brush.toggle()                   # bật cọ
editor.brush.begin_stroke(select=True)
editor.paint(x, y, view_proj, rect)    # tô từ vị trí cũ tới (x, y)
editor.brush.end_stroke()

editor.delete_selected()                # xoá gaussian đang chọn, đẩy vào lịch sử
editor.undo()                           # Ctrl+Z
editor.save_file("out.ply")            # gộp các lớp đang hiện rồi ghi

# Mỗi lần dữ liệu đổi, Editor tăng một số phiên bản (geometry/deleted/selection/transform)
# để Viewport.sync() biết phần nào cần gửi lại GPU.
"""

```

Khai báo mức module (dòng 25–34)

```
import os

import numpy as np

from ..core.history import DeleteGaussiansCommand, History
from ..core.layer import Layer
from ..core.lod import LodIndex
from ..core.scene import Scene
from ..io.ply import PlyFile
from .brush import BrushTool
```

`class Editor`

Dòng 37–251

Các phương thức: `__init__`, `splats`, `lod`, `layer_matrices`, `total_count`, `notify_geometry_changed`, `notify_deleted_changed`, `notify_transform_changed`, `notify_selection_changed`, `selection_changes_since`, `set_status`, `_load_layer`, `open_file`, `add_file`, `can_save`, `suggested_save_name`, `save_file`, `_release_file`, `remove_layer`, `select_all`, `select_none`, `invert_selection`, `paint`, `delete_selected`, `delete`, `undo`.

```
class Editor:
```

`Editor.__init__(self, max_undo=20)`

Dòng 38–51

```
    def __init__(self, max_undo=20):
        self.scene = Scene()
        self.history = History(max_undo)
        self.brush = BrushTool()
        self.status = ""
```

```
43          self.geometry_version = 0
44          self.deleted_version = 0       # tăng khi xoá / hoàn tác xoá gaussian (xoá mềm: không đổi geometry)
45          self.selection_version = 0
46          self.transform_version = 0
47          self._splats = None
48          self._splats_version = -1
49          self._lod = None
50          self._lod_version = -1
51          self._selection_log = []     # [(selection_version, chỉ số, giá trị)]: các lần tô gần đây
```

`Editor.splats(self)`

Dòng 54–61

Mô tả trong mã nguồn. Tất cả lớp đang hiện, gộp lại, ở toạ độ file của từng lớp (tính lại khi dữ liệu đổi). Toạ độ thế giới = layer_matrices()[lớp] @ toạ độ file.

Giải thích. Thuộc tính lười: gộp layer đang hiện thành một `Splats` và chỉ tính lại khi `geometry_version` đổi.

```
54      @property
55      def splats(self):
56          """Tất cả lớp đang hiện, gộp lại, ở toạ độ file của từng lớp (tính lại khi dữ liệu đổi).
57          Toạ độ thế giới = layer_matrices()[lớp] @ toạ độ file."""
58          if self._splats_version != self.geometry_version:
59              self._splats = self.scene.render_splats()
60              self._splats_version = self.geometry_version
61          return self._splats
```

`Editor.lod(self)`

Dòng 63–72

Mô tả trong mã nguồn. Chỉ mục không gian (khối + LOD) của self.splats, mỗi lớp một nhóm; dùng chung cho renderer và cọ chọn. Chỉ dựng lại khi dữ liệu đổi, không phải khi kéo / xoay lớp.

Giải thích. Chỉ mục LOD dùng chung cho renderer, bộ sắp xếp và cọ; dựng lại cùng lúc với `splats`, mỗi layer một nhóm.

```
63      @property
64      def lod(self):
65          """Chỉ mục không gian (khối + LOD) của self.splats, mỗi lớp một nhóm; dùng chung cho
66          renderer và cọ chọn. Chỉ dựng lại khi dữ liệu đổi, không phải khi kéo / xoay lớp."""
67          if self._lod_version != self.geometry_version:
68              splats = self.splats
69              ranges = [(start, end) for _, start, end in self.scene.visible_ranges()]
70              self._lod = LodIndex(splats.positions, splats.scales, splats.alphas, ranges=ranges)                if splats is not None else None
71              self._lod_version = self.geometry_version
72          return self._lod
```

`Editor.layer_matrices(self)`

Dòng 74–75

```
74      def layer_matrices(self):
75          return self.scene.layer_matrices()
```

`Editor.total_count(self)`

Dòng 77–80

Mô tả trong mã nguồn. Số gaussian còn lại (chưa xoá) của các lớp đang hiện.

```
77      @property
78      def total_count(self):
79          """Số gaussian còn lại (chưa xoá) của các lớp đang hiện."""
80          return self.scene.count()
```

`Editor.notify_geometry_changed(self)`

Dòng 82–86

Mô tả trong mã nguồn. Dữ liệu gaussian đổi (mở / xoá / ẩn lớp...): phải dựng lại và gửi lại lên GPU.

```
82      def notify_geometry_changed(self):
83          """Dữ liệu gaussian đổi (mở / xoá / ẩn lớp...): phải dựng lại và gửi lại lên GPU."""
84          self.geometry_version += 1
85          self.notify_deleted_changed()
86          self.transform_version += 1
```

`Editor.notify_deleted_changed(self)`

Dòng 88–91

Mô tả trong mã nguồn. Tập gaussian đã xoá đổi (xoá / hoàn tác): renderer chỉ gửi lại cờ, không gửi lại dữ liệu.

```
88      def notify_deleted_changed(self):
89          """Tập gaussian đã xoá đổi (xoá / hoàn tác): renderer chỉ gửi lại cờ, không gửi lại dữ liệu."""
90          self.deleted_version += 1
91          self.notify_selection_changed()
```

`Editor.notify_transform_changed(self)`

Dòng 93–95

Mô tả trong mã nguồn. Chỉ phép biến đổi của lớp (hoặc lật trục) đổi: renderer chỉ cần cập nhật ma trận.

```
93      def notify_transform_changed(self):
94          """Chỉ phép biến đổi của lớp (hoặc lật trục) đổi: renderer chỉ cần cập nhật ma trận."""
95          self.transform_version += 1
```

`Editor.notify_selection_changed(self, indices=None, value=None)`

Dòng 97–104

Mô tả trong mã nguồn. indices / value: thay đổi chỉ gồm ‘vùng chọn[indices] = value‘ (để gửi GPU từng phần).

Giải thích. Ghi lại thay đổi vùng chọn dạng `(chỉ số, giá trị)` để GPU chỉ nhận phần đã tô thay vì cả mảng.

```
97      def notify_selection_changed(self, indices=None, value=None):
98          """indices / value: thay đổi chỉ gồm `vùng chọn[indices] = value` (để gửi GPU từng phần)."""
99          self.selection_version += 1
100         if indices is None:
101             self._selection_log.clear()
102         else:
103             self._selection_log.append((self.selection_version, indices, value))
104             del self._selection_log[:-64]
```

`Editor.selection_changes_since(self, version)`

Dòng 106–111

Mô tả trong mã nguồn. [(chỉ số, giá trị)] đã tô kể từ ‘version‘, hoặc None nếu có thay đổi khác (phải gửi lại cả).

```
106     def selection_changes_since(self, version):
107         """[(chỉ số, giá trị)] đã tô kể từ `version`, hoặc None nếu có thay đổi khác (phải gửi lại cả)."""
108         if version is None:
109             return None
110         changes = [(i, v) for ver, i, v in self._selection_log if ver > version]
111         return changes if len(changes) == self.selection_version - version else None
```

`Editor.set_status(self, text)`

Dòng 113–115

```
113     def set_status(self, text):
114         self.status = text
115         print(text)
```

`Editor._load_layer(self, path)`

Dòng 118–123

```
118     def _load_layer(self, path):
119         try:
120             return Layer.from_file(path)
121         except Exception as e:   # noqa: BLE001
122             self.set_status(f"Failed to open {os.path.basename(path)}: {e}")
123             return None
```

`Editor.open_file(self, path)`

Dòng 125–135

Mô tả trong mã nguồn. Mở file mới, thay thế toàn bộ cảnh. Mở lỗi thì giữ nguyên cảnh cũ.

```
    def open_file(self, path):
        """Mở file mới, thay thế toàn bộ cảnh. Mở lỗi thì giữ nguyên cảnh cũ."""
        layer = self._load_layer(path)
        if layer is None:
            return False
        self.scene.clear()
        self.scene.add(layer)
        self.history.clear()
        self.notify_geometry_changed()
        self.set_status(f"Opened {layer.name}: {layer.count:,} gaussians")
        return True
```

`Editor.add_file(self, path)`

Dòng 137–145

Mô tả trong mã nguồn. Thêm file vào cảnh đang có. Trả về lớp mới hoặc None.

```
    def add_file(self, path):
        """Thêm file vào cảnh đang có. Trả về lớp mới hoặc None."""
        layer = self._load_layer(path)
        if layer is None:
            return None
        self.scene.add(layer)
        self.notify_geometry_changed()
        self.set_status(f"Added {layer.name}: {layer.count:,} gaussians")
        return layer
```

`Editor.can_save(self)`

Dòng 147–148

```
    def can_save(self):
        return bool(self.scene.visible_layers())
```

`Editor.suggested_save_name(self)`

Dòng 150–152

```
    def suggested_save_name(self):
        layers = self.scene.visible_layers()
        return layers[0].name if len(layers) == 1 else "merged.ply"
```

`Editor.save_file(self, path)`

Dòng 154–168

```
    def save_file(self, path):
        dtype = self.scene.export_dtype()
        if dtype is None:
            self.set_status("No visible layers to save")
            return False
        layers = self.scene.visible_layers()
        count = sum(layer.count for layer in layers)
        try:
            self._release_file(path)
            PlyFile.write_blocks(path, dtype, count, self.scene.export_blocks(dtype))
        except Exception as e:   # noqa: BLE001
            self.set_status(f"Failed to save: {e}")
            return False
        self.set_status(f"Saved {path} ({count:,} gaussians, {len(layers)} layer{'s' * (len(layers) != 1)})")
        return True
```

`Editor._release_file(self, path)`

Dòng 170–184

Mô tả trong mã nguồn. File PLY đang mở được ánh xạ thẳng vào bộ nhớ (memmap); Windows không cho ghi đè file đang ánh xạ. Nếu lưu đè lên chính nó thì chép dữ liệu đó vào RAM trước.

Giải thích. Windows không cho ghi đè file đang được memmap, nên trước khi lưu đè phải nạp dữ liệu vào RAM (qua `load`) và thả memmap.

```
    def _release_file(self, path):
        """File PLY đang mở được ánh xạ thẳng vào bộ nhớ (memmap); Windows không cho ghi đè file
        đang ánh xạ. Nếu lưu đè lên chính nó thì chép dữ liệu đó vào RAM trước."""
        if not os.path.exists(path):
            return

        def load(data):
            if isinstance(data, np.memmap) and data.filename and os.path.samefile(data.filename, path):
                return np.array(data)
            return data

        for layer in self.scene.layers:
            layer.data = load(layer.data)
        for command in self.history:
            command.release(load)
```

`Editor.remove_layer(self, index=None)`

Dòng 187–192

```
    def remove_layer(self, index=None):
        index = self.scene.active_index if index is None else index
        if 0 <= index < len(self.scene.layers):
            layer = self.scene.remove(index)
            self.notify_geometry_changed()
            self.set_status(f"Removed layer {layer.name}")
```

`Editor.select_all(self)`

Dòng 195–198

```
    def select_all(self):
        for layer in self.scene.visible_layers():
            layer.select_all()
        self.notify_selection_changed()
```

`Editor.select_none(self)`

Dòng 200–203

```
    def select_none(self):
        for layer in self.scene.visible_layers():
            layer.clear_selection()
        self.notify_selection_changed()
```

`Editor.invert_selection(self)`

Dòng 205–208

```
    def invert_selection(self):
        for layer in self.scene.visible_layers():
            layer.invert_selection()
        self.notify_selection_changed()
```

`Editor.paint(self, x, y, view_proj, rect)`

Dòng 210–221

Mô tả trong mã nguồn. Tô cọ từ vị trí lần trước tới (x, y): chọn hoặc bỏ chọn gaussian.

Giải thích. Tô cọ từ vị trí trước đến vị trí hiện tại: gọi `BrushTool.hit` để lấy chỉ số rồi `Scene.apply_selection_indices`.

```
    def paint(self, x, y, view_proj, rect):
        """Tô cọ từ vị trí lần trước tới (x, y): chọn hoặc bỏ chọn gaussian."""
        splats = self.splats
        if splats is None or rect.is_empty or not self.brush.is_painting:
            return
        indices = self.brush.hit(self.lod, view_proj, rect, self.brush.last or (x, y), (x, y),
                                 self.layer_matrices())
        self.brush.last = (x, y)
        if len(indices):
            indices = self.scene.apply_selection_indices(indices, self.brush.painting)
            if len(indices):
                self.notify_selection_changed(indices, self.brush.painting)
```

`Editor.delete_selected(self)`

Dòng 224–236

Giải thích. Xóa mềm mọi layer đang có vùng chọn, đẩy một `DeleteGaussiansCommand` vào lịch sử và tăng `deleted_version`.

```
224     def delete_selected(self):
225         snapshots = []
226         for layer in self.scene.visible_layers():
227             snapshot = layer.delete_selected()
228             if snapshot is not None:
229                 snapshots.append((layer, snapshot))
230         if not snapshots:
231             self.set_status("Nothing selected (press B for the brush, then paint over what to delete)")
232             return
233         command = DeleteGaussiansCommand(snapshots)
234         self.history.push(command)
235         self.notify_deleted_changed()
236         self.set_status(f"Deleted {command.count:,} gaussians (Ctrl+Z to undo)")
```

`Editor.delete(self)`

Dòng 238–243

Mô tả trong mã nguồn. Phím Delete: xoá gaussian đang chọn nếu có, nếu không thì xoá lớp đang chọn.

```
238     def delete(self):
239         """Phím Delete: xoá gaussian đang chọn nếu có, nếu không thì xoá lớp đang chọn."""
240         if self.scene.selected_count():
241             self.delete_selected()
242         else:
243             self.remove_layer()
```

`Editor.undo(self)`

Dòng 245–251

```
245     def undo(self):
246         command = self.history.undo()
247         if command is None:
248             self.set_status("Nothing to undo")
249             return
250         self.notify_deleted_changed()
251         self.set_status(f"Undid {command.description}")
```

2.2. `splat_editor/editor/brush.py`

140 dòng

Vai trò. Cọ chọn xuyên độ sâu: gaussian nào có tâm chiếu vào nét tô đều được chọn. Để nhanh, cọ tận dụng hộp bao của các khối LOD: chiếu 8 đỉnh hộp ra hình chữ nhật, khối nằm trọn trong nét thì lấy cả khối, khối chỉ chạm thì mới kiểm từng gaussian. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§2.2).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
1  """Cọ chọn gaussian (brush): tô lên màn hình, gaussian nào có tâm nằm trong nét cọ thì được chọn.
2
3  Cọ chọn xuyên qua mọi độ sâu (giống SuperSplat): mọi gaussian có tâm chiếu vào vòng tròn
4  đều bị chọn, kể cả những cái nằm khuất phía sau.
5
6  Để nhanh với hàng chục triệu gaussian, cọ dùng chỉ mục khối của LodIndex: chiếu 8 đỉnh hộp bao
7  của mỗi khối (~15 nghìn khối với 15 triệu gaussian) ra một hình chữ nhật trên màn hình, chỉ
8  những khối chạm nét cọ mới phải chiếu và kiểm tra từng gaussian (chia cho nhiều luồng).
9  """
10
11 """
12 brush = BrushTool()
13 brush.toggle()                          # bật / tắt cọ (phím B)
14 brush.resize(1.2)                       # to lên 20% (phím ]); radius nằm trong [MIN_RADIUS, MAX_RADIUS]
15
16 brush.begin_stroke(select=True)        # bắt đầu tô (False: bỏ chọn)
17 indices = brush.hit(lod, view_proj, viewport_rect, start=(x0, y0), end=(x1, y1), transforms=layer_matrices)
18 brush.end_stroke()                      # nhả chuột
19 """
20
```

Cơ sở toán: biến đổi viewport (`brush._to_screen`)

$$x_{px} = x_0 + \frac{n_x+1}{2}W, \qquad y_{px} = y_0 + \frac{1-n_y}{2}H \text{ (đảo } y\text{: màn hình có } y \text{ hướng xuống).}$$

Cơ sở toán: Cọ chọn

Khoảng cách điểm–đoạn, capsule (`_segment_dist2`). $\tau = \text{clip}\big((p-a)\cdot(b-a)/\|b-a\|^2, 0, 1\big)$, $d^2 = \|p-(a+\tau(b-a))\|^2$; chọn nếu $d^2 \le r^2$. Nếu $a = b$ thì $\tau = 0$. Capsule là tập điểm cách đoạn không quá r (hình viên thuốc, lồi).

Chiếu tâm ra pixel. $\text{clip} = M_{[:3,:3]}p + M_{[:3,3]}$ với $M = VP \cdot M_{layer}$; chỉ xét $w > 10^{-6}$ (trước camera); $x = \text{left} + (c_x/w + 1)/2 \cdot W$, $y = \text{top} + (1 - c_y/w)/2 \cdot H$.

Tăng tốc theo khối. 8 đỉnh hộp bao khối (`_CORNERS`: bit i của chỉ số chọn lo/hi) được chiếu thành hình chữ nhật bao (hộp lồi và phép chiếu bảo toàn tính lồi khi $w > 0$). Khối chạm nét nếu hình chữ nhật giao hộp $[\min(a,b) - r, \max(a,b) + r]$. Khối **nằm trọn trong nét** nếu cả 4 góc hình chữ nhật có $d^2 \le r^2$ (capsule lồi nên cả hình chữ nhật nằm trong), khi đó lấy cả khối mà không kiểm từng gaussian. Khối cắt mặt phẳng camera (dấu w lẫn lộn) luôn được kiểm. Cọ **xuyên sâu** (không test độ sâu) như SuperSplat.

Khai báo mức module (dòng 21–28)

```
21 import os
22 from concurrent.futures import ThreadPoolExecutor
23
24 import numpy as np
25
26 # 8 đỉnh của hộp: chọn lo (0) hoặc hi (1) cho từng trục
27 _WORKERS = min(max((os.cpu_count() or 2) - 1, 1), 6)
28 _CORNERS = np.array([[(i >> axis) & 1 for axis in range(3)] for i in range(8)], bool)
```

`class BrushTool`

Dòng 31–140

Các phương thức: `__init__`, `is_painting`, `toggle`, `resize`, `begin_stroke`, `end_stroke`, `_to_screen`, `_segment_dist2`, `_candidate_chunks`, `hit`.

```
31 class BrushTool:
32     MIN_RADIUS = 2.0
33     MAX_RADIUS = 500.0
```

`BrushTool.__init__(self)`

Dòng 35–39

```
35     def __init__(self):
36         self.active = False       # đang ở chế độ cọ (phím B)
37         self.radius = 40.0        # bán kính (pixel)
38         self.painting = None      # None: không tô; True: đang chọn; False: đang bỏ chọn
39         self.last = None          # vị trí chuột lần tô trước
```

`BrushTool.is_painting(self)`

Dòng 42–44

```
42     @property
43     def is_painting(self):
44         return self.painting is not None
```

`BrushTool.toggle(self)`

Dòng 46–47

```
46     def toggle(self):
47         self.active = not self.active
```

`BrushTool.resize(self, factor)`

Dòng 49–50

```
49     def resize(self, factor):
50         self.radius = min(max(self.radius * factor, self.MIN_RADIUS), self.MAX_RADIUS)
```

`BrushTool.begin_stroke(self, select)`

Dòng 52–55

Mô tả trong mã nguồn. select = True: tô để chọn; False: tô để bỏ chọn.

```
52    def begin_stroke(self, select):
53        """select=True: tô để chọn; False: tô để bỏ chọn."""
54        self.painting = select
55        self.last = None
```

`BrushTool.end_stroke(self)`

Dòng 57–58

```
57    def end_stroke(self):
58        self.painting = None
```

`BrushTool._to_screen(clip, viewport)`

Dòng 61–66

Giải thích. Chiếu tâm gaussian ra pixel bằng $VP \cdot M_{layer}$; chỉ giữ điểm có $w > 10^{-6}$ (trước camera).

Công thức.

$$\text{clip} = M_{[:3,:3]}\, p + M_{[:3,3]}, \quad M = VP \cdot M_{layer}, \qquad x_{px} = x_0 + \frac{c_x/w + 1}{2} W, \quad y_{px} = y_0 + \frac{1 - c_y/w}{2} H$$

```
61    @staticmethod
62    def _to_screen(clip, viewport):
63        left, top, width, height = viewport
64        x = left + (clip[..., 0] / clip[..., 3] + 1) * 0.5 * width
65        y = top + (1 - clip[..., 1] / clip[..., 3]) * 0.5 * height
66        return x, y
```

`BrushTool._segment_dist2(p, a, b)`

Dòng 68–75

Mô tả trong mã nguồn. Bình phương khoảng cách từ các điểm p (...,2) tới đoạn thẳng a -> b.

Giải thích. Bình phương khoảng cách điểm–đoạn (capsule): chiếu điểm lên đường thẳng, kẹp tham số $\tau \in [0, 1]$.

Công thức.

$$\tau = \text{clip}\Big(\frac{(p-a)\cdot(b-a)}{\|b-a\|^2}, 0, 1\Big), \qquad d^2 = \Big\| p - \big(a + \tau(b-a)\big) \Big\|^2 \le r^2$$

```
68    @staticmethod
69    def _segment_dist2(p, a, b):
70        """Bình phương khoảng cách từ các điểm p (...,2) tới đoạn thẳng a -> b."""
71        ab = b - a
72        length2 = float(ab @ ab)
73        t = np.clip((p - a) @ ab / length2, 0, 1) if length2 > 0 else np.zeros(p.shape[:-1])
74        d = p - (a + t[..., None] * ab)
75        return (d * d).sum(axis=-1)
```

`BrushTool._candidate_chunks(self, lod, matrices, viewport, a, b)`

Dòng 77–97

Mô tả trong mã nguồn. (khối phải kiểm tra từng gaussian, khối nằm trọn trong nét cọ) theo hình chữ nhật bao hình chiếu hộp bao của khối. matrices: (số nhóm, 4, 4) chiếu * thế giới của từng nhóm.

Giải thích. Với mỗi khối, chiếu 8 đỉnh hộp bao thành hình chữ nhật; trả về hai nhóm: khối cần kiểm từng gaussian và khối nằm **trọn** trong nét cọ.

Công thức. Hộp bao khối có 8 đỉnh $v_i \in \{lo, hi\}^3$; hình chữ nhật bao $= [\min_i x_i, \max_i x_i] \times [\min_i y_i, \max_i y_i]$. Khối nằm **trọn** trong nét nếu $d^2(\text{4 góc}) \le r^2$ (capsule lồi nên cả hình chữ nhật nằm trong).

```
77    def _candidate_chunks(self, lod, matrices, viewport, a, b):
78        """(khối phải kiểm tra từng gaussian, khối nằm trọn trong nét cọ) theo hình chữ nhật bao
79        hình chiếu hộp bao của khối. matrices: (số nhóm, 4, 4) chiếu * thế giới của từng nhóm."""
80        r = self.radius
81        lo, hi = np.minimum(a, b) - r, np.maximum(a, b) + r
```

```
        corners = np.where(_CORNERS[None], lod.box_hi[:, None], lod.box_lo[:, None])   # (C,8,3)
        m = matrices[lod.chunk_group]                                                    # (C,4,4)
        clip = np.einsum('cij,ckj->cki', m[:, :, :3], corners) + m[:, None, :, 3]       # (C,8,4)
        w = clip[..., 3]
        front = (w > 1e-6).all(axis=1)
        crosses = ~front & (w > 1e-6).any(axis=1)        # hộp cắt mặt phẳng camera: luôn xét
        # hộp lồi -> hình chiếu nằm trong bao của 8 đỉnh chiếu
        x, y = self._to_screen(clip[front], viewport)
        x0, x1, y0, y1 = x.min(axis=1), x.max(axis=1), y.min(axis=1), y.max(axis=1)
        touch = (x1 >= lo[0]) & (x0 <= hi[0]) & (y1 >= lo[1]) & (y0 <= hi[1])
        # nét cọ (hình viên thuốc) là hình lồi: 4 góc hình chữ nhật nằm trong -> cả khối nằm trong
        rect = np.stack([np.stack([x0, y0], -1), np.stack([x1, y0], -1),
                         np.stack([x0, y1], -1), np.stack([x1, y1], -1)], axis=1)[touch]   # (T,4,2)
        inside = (self._segment_dist2(rect, a, b) <= r * r).all(axis=1)
        ids = np.flatnonzero(front)[touch]
        return np.concatenate([ids[~inside], np.flatnonzero(crosses)]), ids[inside]
```

`BrushTool.hit(self, lod, view_proj, viewport, start, end, transforms=None)`

Dòng 99–140

Mô tả trong mã nguồn. Chỉ số các gaussian có tâm cách đoạn thẳng start -> end (pixel) không quá ‘radius‘. transforms: ma trận thế giới 4x4 của từng nhóm trong lod (None: đơn vị).

Giải thích. Hàm chính của cọ: lấy chỉ số gaussian có tâm cách đoạn nét tô không quá bán kính. Khối nằm trọn thì lấy cả khối; khối cắt mặt phẳng camera luôn bị kiểm; phần kiểm chia nhiều luồng.

```
    def hit(self, lod, view_proj, viewport, start, end, transforms=None):
        """Chỉ số các gaussian có tâm cách đoạn thẳng start -> end (pixel) không quá `radius`.
        transforms: ma trận thế giới 4x4 của từng nhóm trong lod (None: đơn vị)."""
        vp = np.asarray(view_proj, np.float64)
        transforms = [np.identity(4)] * lod.group_count if transforms is None else transforms
        matrices = np.array([vp @ np.asarray(t, np.float64) for t in transforms]).reshape(-1, 4, 4)
        a = np.asarray(start, np.float64)
        b = np.asarray(end, np.float64)
        r = self.radius
        chunks, inside = self._candidate_chunks(lod, matrices, viewport, a, b)
        # khối nằm trọn trong nét cọ: lấy cả khối, khỏi chiếu từng gaussian
        sizes = lod.sizes[inside]
        ends = np.cumsum(sizes)
        whole = lod.order[np.arange(int(ends[-1]) if len(ends) else 0) +
                          np.repeat(lod.starts[inside] - (ends - sizes), sizes)].astype(np.int64)
        if len(chunks) == 0:
            return whole
        matrices32 = matrices.astype(np.float32)

        def test(group):
            """Chỉ số gốc của các gaussian trong nhóm khối `group` (cùng một lớp) nằm dưới nét cọ."""
            m32 = matrices32[lod.chunk_group[group[0]]]
            sizes = lod.sizes[group]
            ends = np.cumsum(sizes)
            slots = np.arange(int(ends[-1])) + np.repeat(lod.starts[group] - (ends - sizes), sizes)
            clip = lod.positions[slots] @ m32[:, :3].T + m32[:, 3]
            front = clip[:, 3] > 1e-6
            slots, clip = slots[front], clip[front]
            p = np.stack(self._to_screen(clip, viewport), axis=1)
            return lod.order[slots[self._segment_dist2(p, a, b) <= r * r]]

        # chia việc cho nhiều luồng; mỗi phần chỉ gồm khối của cùng một lớp (cùng ma trận)
        groups = []
        for layer in np.unique(lod.chunk_group[chunks]):
            mine = chunks[lod.chunk_group[chunks] == layer]
            groups += np.array_split(mine, min(_WORKERS, max(len(mine) // 16, 1)))
        if len(groups) == 1:
            parts = [test(groups[0])]
        else:
            with ThreadPoolExecutor(min(len(groups), _WORKERS)) as pool:
                parts = list(pool.map(test, groups))
        return np.concatenate([whole] + [p.astype(np.int64) for p in parts])
```

Chương 3. Thư mục io: đọc ghi và giải mã

Thư mục `io` là tầng **đọc/ghi và biến đổi định dạng** giữa file PLY và cấu trúc `Splats`. `ply.py` lo phần byte (header, kiểu dữ liệu, ánh xạ bộ nhớ); `codec.py` lo phần ý nghĩa (sigmoid, exp, hằng số C_0, chuẩn hóa quaternion). Tách đôi như vậy để thay đổi định dạng lưu mà không đụng tới logic hiển thị.

3.1. `splat_editor/io/ply.py`

134 dòng

Vai trò. Đọc/ghi PLY. File `binary_little_endian` được ánh xạ bộ nhớ (`np.memmap`) nên mở file 4 GB gần như tức thì và không chiếm RAM. Mỗi property là một cột của mảng cấu trúc NumPy (`data['x']`, `data['opacity']`, …). Ghi file theo từng khối để không giữ cả layer trong RAM.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Đọc / ghi file PLY.

PlyFile.read(path)        -> mảng numpy có cấu trúc (mỗi property là một cột: data['x'], data['opacity'], ...)
PlyFile.write(path, data) -> ghi lại mảng đó ra file PLY nhị phân (binary_little_endian)

File binary_little_endian (loại phổ biến nhất) được ánh xạ thẳng vào bộ nhớ (np.memmap):
mở file 15 triệu gaussian (~4 GB) gần như tức thì và không chiếm RAM, hệ điều hành chỉ nạp
phần nào đang được đọc. Lưu ý: trên Windows không ghi đè được file đang được ánh xạ
(xem Editor.save_file).
"""

"""
data = PlyFile.read("scene.ply")        # binary được ánh xạ memmap, mở file 15 triệu gaussian gần như tức thì
print(data.dtype.names)                 # ('x', 'y', 'z', 'f_dc_0', ..., 'rot_3')
xyz = data['x'], data['y'], data['z']

PlyFile.write("copy.ply", data)        # ghi cả mảng ra binary_little_endian

# Ghi từng khối để không giữ cả file trong RAM
PlyFile.write_blocks("big.ply", dtype, count, blocks)   # blocks: iterator các mảng cùng dtype
"""

```

Khai báo mức module (dòng 23–25)

```
import os

import numpy as np
```

`read_block(data, start, end)`

Dòng 28–36

Mô tả trong mã nguồn. Hàng [start, end) của ‘data‘ (có thể là memmap): chép liền một mạch vào RAM rồi trả về (khối, ma trận float32 (n, số cột) nếu mọi cột đều là float32 little endian, ngược lại None). Đọc cột qua ma trận nhanh hơn nhiều so với truy cập từng trường của mảng có cấu trúc.

Giải thích. Chép hàng $[start, end)$ của memmap liền một mạch vào RAM, trả về khối cùng dạng ma trận float32 nếu mọi cột cùng kiểu (nhanh hơn truy cập từng cột).

```
def read_block(data, start, end):
    """Hàng [start, end) của `data` (có thể là memmap): chép liền một mạch vào RAM rồi trả về
    (khối, ma trận float32 (n, số cột) nếu mọi cột đều là float32 little endian, ngược lại None).
    Đọc cột qua ma trận nhanh hơn nhiều so với truy cập từng trường của mảng có cấu trúc."""
    block = np.array(data[start:end])
    dtype = block.dtype
    if all(dtype[name] == np.dtype('<f4') for name in dtype.names) and dtype.itemsize == 4 * len(dtype.names):
        return block, block.view(np.float32).reshape(len(block), len(dtype.names))
    return block, None
```

`class PlyError(ValueError)`

Dòng 39–40

Mô tả trong mã nguồn. File PLY sai định dạng hoặc không được hỗ trợ.

```
class PlyError(ValueError):
    """File PLY sai định dạng hoặc không được hỗ trợ."""
```

`class PlyFile`

Dòng 43–134

Các phương thức: `read`, `_read_header`, `_read_ascii`, `_read_binary`, `write`, `write_blocks`.

```
class PlyFile:
    # Kiểu dữ liệu trong header PLY -> kiểu numpy
    TYPES = {
        'char': 'i1', 'int8': 'i1', 'uchar': 'u1', 'uint8': 'u1',
        'short': 'i2', 'int16': 'i2', 'ushort': 'u2', 'uint16': 'u2',
        'int': 'i4', 'int32': 'i4', 'uint': 'u4', 'uint32': 'u4',
        'float': 'f4', 'float32': 'f4', 'double': 'f8', 'float64': 'f8',
    }
    NUMPY_TO_PLY = {'i1': 'char', 'u1': 'uchar', 'i2': 'short', 'u2': 'ushort',
                    'i4': 'int', 'u4': 'uint', 'f4': 'float', 'f8': 'double'}

    # ---------------------------------------------------------------- đọc
```

`PlyFile.read(cls, path)`

Dòng 55–67

Mô tả trong mã nguồn. Đọc phần ’element vertex’ của file PLY. Hỗ trợ ascii và binary (little/big endian).

```
    @classmethod
    def read(cls, path):
        """Đọc phần 'element vertex' của file PLY. Hỗ trợ ascii và binary (little/big endian)."""
        with open(path, 'rb') as f:
            fmt, elements = cls._read_header(f)
            if not elements or elements[0][0] != 'vertex':
                raise PlyError("The first element in the file must be 'vertex'")
            _, count, props = elements[0]
            if fmt == 'ascii':
                return cls._read_ascii(f, count, props)
            if fmt in ('binary_little_endian', 'binary_big_endian'):
                return cls._read_binary(f, count, props, '<' if fmt == 'binary_little_endian' else '>')
            raise PlyError(f"Unsupported PLY format: {fmt}")
```

`PlyFile._read_header(cls, f)`

Dòng 69–92

Mô tả trong mã nguồn. -> (định dạng, [(tên element, số lượng, [(tên property, kiểu numpy)])])

Giải thích. Phân tích header: định dạng (ascii/binary little/big endian), danh sách element, property và kiểu NumPy tương ứng.

```
    @classmethod
    def _read_header(cls, f):
        """-> (định dạng, [(tên element, số lượng, [(tên property, kiểu numpy)])])"""
        if f.readline().strip() != b'ply':
            raise PlyError("Not a PLY file")
        fmt = None
        elements = []
        while True:
            line = f.readline()
            if not line:
                raise PlyError("PLY header has no 'end_header'")
            parts = line.decode('ascii', errors='replace').split()
            if not parts or parts[0] in ('comment', 'obj_info'):
                continue
            if parts[0] == 'end_header':
                return fmt, elements
            if parts[0] == 'format':
                fmt = parts[1]
            elif parts[0] == 'element':
                elements.append((parts[1], int(parts[2]), []))
            elif parts[0] == 'property':
                if parts[1] == 'list':
                    raise PlyError("List properties are not supported (mesh files)")
                elements[-1][2].append((parts[2], cls.TYPES[parts[1]]))
```

`PlyFile._read_ascii(f, count, props)`

Dòng 94–101

```
    @staticmethod
    def _read_ascii(f, count, props):
        dtype = np.dtype([(name, '<' + t) for name, t in props])
        values = np.loadtxt(f, max_rows=count, ndmin=2)
```

```
        data = np.empty(count, dtype)
        for i, (name, _) in enumerate(props):
            data[name] = values[:, i]
        return data
```

PlyFile._read_binary(f, count, props, endian)

Dòng 103–114

Giải thích. Với little endian dùng `np.memmap` (không đọc cả file). Offset dữ liệu lấy từ độ dài header.

```
    @staticmethod
    def _read_binary(f, count, props, endian):
        dtype = np.dtype([(name, endian + t) for name, t in props])
        offset = f.tell()
        if os.fstat(f.fileno()).st_size < offset + count * dtype.itemsize:
            raise PlyError("PLY file is truncated")
        if endian == '<':
            if count == 0:
                return np.zeros(0, dtype)
            return np.memmap(f.name, dtype=dtype, mode='r', offset=offset, shape=(count,))
        # big endian: đọc vào RAM rồi đổi sang little endian cho thống nhất
        return np.fromfile(f, dtype=dtype, count=count).astype(dtype.newbyteorder('<'))
```

PlyFile.write(cls, path, data)

Dòng 117–120

Mô tả trong mã nguồn. Ghi mảng có cấu trúc (giống kết quả read) ra file PLY binary_little_endian.

```
    @classmethod
    def write(cls, path, data):
        """Ghi mảng có cấu trúc (giống kết quả read) ra file PLY binary_little_endian."""
        cls.write_blocks(path, data.dtype, len(data), [data])
```

PlyFile.write_blocks(cls, path, dtype, count, blocks)

Dòng 122–134

Mô tả trong mã nguồn. Ghi ‘count‘ gaussian kiểu ‘dtype‘ từ các khối (mảng có cấu trúc) lần lượt, để không phải gộp cả cảnh thành một mảng lớn trong RAM.

Giải thích. Ghi từng khối lần lượt để không phải giữ toàn bộ layer trong RAM khi lưu file hàng chục triệu gaussian.

```
    @classmethod
    def write_blocks(cls, path, dtype, count, blocks):
        """Ghi `count` gaussian kiểu `dtype` từ các khối (mảng có cấu trúc) lần lượt, để không
        phải gộp cả cảnh thành một mảng lớn trong RAM."""
        dtype = dtype.newbyteorder('<')
        lines = ['ply', 'format binary_little_endian 1.0', f'element vertex {count}']
        for name in dtype.names:
            lines.append(f'property {cls.NUMPY_TO_PLY[dtype[name].str[1:]]} {name}')
        lines.append('end_header')
        with open(path, 'wb') as f:
            f.write(('\n'.join(lines) + '\n').encode('ascii'))
            for block in blocks:
                np.ascontiguousarray(block.astype(dtype, copy=False)).tofile(f)
```

3.2. splat_editor/io/codec.py

122 dòng

Vai trò. `SplatCodec` chuyển giữa các cột ”thô” của file 3DGS và `Splats` đã giải mã (sigmoid cho α, exp cho scale, C_0 cho màu, chuẩn hóa quaternion), và chiều ngược lại khi lưu. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§3.2).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Chuyển đổi giữa các cột của file 3DGS PLY và Splats (đã giải mã để render)."""

"""
data = PlyFile.read("scene.ply")        # mảng numpy có cấu trúc (62 cột / gaussian)
splats = SplatCodec.decode(data)       # màu = 0.5 + C0*f_dc, alpha = sigmoid(opacity), scale = exp(scale)

# Ghi ngược: áp lại các thay đổi lên bản ghi gốc rồi lưu
out = SplatCodec.encode(data, splats, data.dtype)
PlyFile.write("out.ply", out)
"""

```

Cơ sở toán: Giải mã và mã hóa file PLY

Một gaussian gồm tâm μ, màu c, độ đục α, kích thước s, quay q. File lưu giá trị "thô":

Thuộc tính	Giải mã	Lý do
`x,y,z`	$\mu = (x, y, z)$	
`f_dc_0..2`	$c = 0.5 + C_0 f_{dc},\ C_0 = \frac{1}{2\sqrt{\pi}} = 0.28209479$	$C_0 = Y_{00}$ (SH bậc 0)
`opacity`	$\alpha = 1/(1 + e^{-\text{opacity}})$	tối ưu logit để $\alpha \in (0,1)$
`scale_i`	$s_i = \exp(\text{scale}_i)$	tối ưu log để $s > 0$
`rot_0..3`	$q \leftarrow q/\|q\|$; nếu $\|q\| = 0$ thì $(1,0,0,0)$	quaternion phải đơn vị

Mã hóa ngược: $f_{dc} = (c - 0.5)/C_0$, scale $= \ln \max(s, 10^{-12})$, opacity $= \ln \frac{a}{1-a}$ với $a = \text{clip}(\alpha, 10^{-6}, 1 - 10^{-6})$ để tránh $\ln 0$.

Point cloud (không có `scale_0`): bán kính mỗi điểm $= \frac{1}{2}(V/N)^{1/3}$, với V là tích ba cạnh hộp bao. Đó là ước lượng khoảng cách trung bình giữa các điểm. Màu `uchar` chia 255.

Dung lượng: $3 + 3 + 3 + 45 + 1 + 3 + 4 = 62$ float32 $= 248$ byte/gaussian (SH bậc 3: 15 hệ số $\times$ 3 kênh $= 45$).

Khai báo mức module (dòng 12–25)

```
12 import os
13 from concurrent.futures import ThreadPoolExecutor
14
15 import numpy as np
16
17 from ..core import sh
18 from ..core.splats import Splats
19 from .ply import read_block
20
21 SH_C0 = 0.28209479177387814   # hệ số spherical harmonic bậc 0
22 CHUNK = 1 << 18               # số hàng giải mã mỗi lần (mỗi luồng)
23
24 CANONICAL_FIELDS = ['x', 'y', 'z', 'f_dc_0', 'f_dc_1', 'f_dc_2', 'opacity',
25                     'scale_0', 'scale_1', 'scale_2', 'rot_0', 'rot_1', 'rot_2', 'rot_3']
```

`class SplatCodec`

Dòng 28–122

Các phương thức: `decode`, `_decode_block`, `encode`.

```
28 class SplatCodec:
```

`SplatCodec.decode(cls, data, sh_dtype=np.float16)`

Dòng 29–63

Mô tả trong mã nguồn. Chuyển các cột của file 3DGS PLY thành Splats.

File 3DGS chuẩn có: x y z, f_dc_0..2 (màu), f_rest_* (màu theo hướng nhìn, tuỳ chọn), opacity (logit), scale_0..2 (log), rot_0..3. Nếu chỉ là point cloud (x y z red green blue) thì tự tạo gaussian nhỏ hình cầu.

Giải mã từng đoạn CHUNK hàng (dữ liệu có thể là memmap trên đĩa) để không tạo mảng tạm bằng cả file. Hệ số SH chỉ dùng để hiển thị nên lưu float16 (đỡ một nửa RAM); khi lưu file vẫn đọc lại từ dữ liệu gốc nên không mất độ chính xác.

Giải thích. Giải mã từng đoạn (`_decode_block`) thành các mảng `Splats`; điểm đám mây (không có `scale_0`) được gán bán kính ước lượng.

```
29     @classmethod
30     def decode(cls, data, sh_dtype=np.float16):
31         """Chuyển các cột của file 3DGS PLY thành Splats.
32
33         File 3DGS chuẩn có: x y z, f_dc_0..2 (màu), f_rest_* (màu theo hướng nhìn, tuỳ chọn),
34         opacity (logit), scale_0..2 (log), rot_0..3.
35         Nếu chỉ là point cloud (x y z red green blue) thì tự tạo gaussian nhỏ hình cầu.
36
37         Giải mã từng đoạn CHUNK hàng (dữ liệu có thể là memmap trên đĩa) để không tạo mảng tạm
38         bằng cả file. Hệ số SH chỉ dùng để hiển thị nên lưu float16 (đỡ một nửa RAM); khi lưu file
39         vẫn đọc lại từ dữ liệu gốc nên không mất độ chính xác.
40         """
41         names = data.dtype.names
```

```
        n = len(data)
        k = sh.column_count(names)
        out = Splats(np.empty((n, 3), np.float32), np.empty((n, 3), np.float32), np.empty(n, np.float32),
                     np.empty((n, 3), np.float32), np.empty((n, 4), np.float32), np.empty((n, k, 3), sh_dtype))

        def decode_rows(a):
            block, matrix = read_block(data, a, a + CHUNK)
            rows = slice(a, a + len(block))
            if k:
                out.sh_rest[rows] = sh.block_coeffs(block, matrix, k)
            columns = {name: matrix[:, i] for i, name in enumerate(names)} if matrix is not None else block
            for name, values in zip(Splats.ARRAYS[:5], cls._decode_block(columns, names, data.dtype)):
                getattr(out, name)[rows] = values

        # các đoạn ghi vào vùng riêng của mảng kết quả; numpy nhả GIL nên chạy song song được
        with ThreadPoolExecutor(min(max((os.cpu_count() or 2) - 1, 1), 6)) as pool:
            list(pool.map(decode_rows, range(0, n, CHUNK)))
        if 'scale_0' not in names:
            # point cloud: lấy khoảng cách trung bình giữa các điểm làm bán kính
            extent = np.maximum(out.positions.max(axis=0) - out.positions.min(axis=0), 1e-6) if n else np.ones(3)
            out.scales[:] = 0.5 * (np.prod(extent) / max(n, 1)) ** (1 / 3)
        return out
```

`SplatCodec._decode_block(data, names, dtype)`

Dòng 65–98

Mô tả trong mã nguồn. (tâm, màu, alpha, scale, quaternion) của một đoạn hàng (scale = 1 nếu là point cloud).

Giải thích. Công thức giải mã: $c = 0.5 + C_0 f_{dc}$, α = sigmoid, s = exp, chuẩn hóa quaternion (rỗng $\to (1,0,0,0)$). Bán kính point cloud $= \frac{1}{2}(V/N)^{1/3}$.

Công thức.

$$c = 0.5 + C_0\, f_{dc}, \quad \alpha = \frac{1}{1 + e^{-\text{opacity}}}, \quad s_i = e^{\text{scale}_i}, \quad q \leftarrow \frac{q}{\|q\|}$$

Point cloud: bán kính $r = \frac{1}{2}\sqrt[3]{V/N}$, V là thể tích hộp bao, N số điểm.

```
    @staticmethod
    def _decode_block(data, names, dtype):
        """(tâm, màu, alpha, scale, quaternion) của một đoạn hàng (scale = 1 nếu là point cloud)."""
        col = lambda name: data[name].astype(np.float32)  # noqa: E731
        n = len(data['x'])
        positions = np.stack([col('x'), col('y'), col('z')], axis=1)

        # màu bậc 0; chưa cắt về 0..1 vì còn cộng phần SH bậc cao (shader cắt sau)
        if 'f_dc_0' in names:
            colors = 0.5 + SH_C0 * np.stack([col('f_dc_0'), col('f_dc_1'), col('f_dc_2')], axis=1)
        elif 'red' in names:
            colors = np.stack([col('red'), col('green'), col('blue')], axis=1)
            if dtype['red'].kind in 'ui':   # uchar 0..255
                colors /= 255.0
        else:
            colors = np.full((n, 3), 0.7, np.float32)

        # độ đục: opacity lưu dạng logit -> sigmoid
        alphas = 1.0 / (1.0 + np.exp(-col('opacity'))) if 'opacity' in names else np.ones(n, np.float32)

        # kích thước: scale lưu dạng log -> exp
        if 'scale_0' in names:
            scales = np.exp(np.stack([col('scale_0'), col('scale_1'), col('scale_2')], axis=1))
        else:
            scales = np.ones((n, 3), np.float32)

        # xoay
        if 'rot_0' in names:
            q = np.stack([col('rot_0'), col('rot_1'), col('rot_2'), col('rot_3')], axis=1)
            length = np.linalg.norm(q, axis=1, keepdims=True)
            q = np.where(length > 0, q / np.maximum(length, 1e-12), np.float32([1, 0, 0, 0]))
        else:
            q = np.tile(np.float32([1, 0, 0, 0]), (n, 1))
        return positions, colors, alphas, scales, q
```

`SplatCodec.encode(source, splats, dtype)`

Dòng 100–122

Mô tả trong mã nguồn. Đổi dữ liệu PLY ‘source‘ sang kiểu 3DGS chuẩn ‘dtype‘.

Cột nào có sẵn trong file thì chép nguyên, cột thiếu thì tính từ ‘splats‘ (ví dụ point cloud chỉ có red/green/blue -> tính f_dc, opacity, scale, rot).

Giải thích. Chiều ngược khi lưu: $f_{dc} = (c - 0.5)/C_0$, ln cho scale, logit cho độ đục (kẹp $[10^{-6}, 1 - 10^{-6}]$ tránh $\ln 0$).

Công thức.

$$f_{dc} = \frac{c - 0.5}{C_0}, \quad \text{scale} = \ln \max(s, 10^{-12}), \quad \text{opacity} = \ln \frac{a}{1-a}, \;\; a = \text{clip}(\alpha, 10^{-6}, 1 - 10^{-6})$$

```
    @staticmethod
    def encode(source, splats, dtype):
        """Đổi dữ liệu PLY `source` sang kiểu 3DGS chuẩn `dtype`.

        Cột nào có sẵn trong file thì chép nguyên, cột thiếu thì tính từ `splats`
        (ví dụ point cloud chỉ có red/green/blue -> tính f_dc, opacity, scale, rot).
        """
        derived = {'x': splats.positions[:, 0], 'y': splats.positions[:, 1], 'z': splats.positions[:, 2]}
        for i in range(3):
            derived[f'f_dc_{i}'] = (splats.colors[:, i] - 0.5) / SH_C0
            derived[f'scale_{i}'] = np.log(np.maximum(splats.scales[:, i], 1e-12))
        a = np.clip(splats.alphas, 1e-6, 1 - 1e-6)
        derived['opacity'] = np.log(a / (1 - a))
        for i in range(4):
            derived[f'rot_{i}'] = splats.rotations[:, i]

        out = np.zeros(splats.count, dtype)
        for name in dtype.names:
            if name in source.dtype.names:
                out[name] = source[name]
            elif name in derived:
                out[name] = derived[name]
        return out
```

Chương 4. Thư mục render: OpenGL

Thư mục `render` là nơi duy nhất **nói chuyện trực tiếp với OpenGL 3.3**. Nó nhận dữ liệu từ `Editor/Scene` rồi biến thành hình ảnh. Gồm: bọc shader (`shader.py`), camera (`camera.py`), bộ vẽ gaussian hai pass (`splat_renderer.py`), luồng nền chọn và sắp xếp (`sorter.py`), khung nhìn ghép mọi thứ (`viewport.py`), các bộ vẽ phụ (lưới sàn, đèn, nhân vật) và thư mục `shaders/` chứa mã GLSL. Pipeline hiển thị gaussian gồm 5 bước: (1) đóng gói dữ liệu lên GPU, (2) luồng nền chọn LOD và sắp xếp xa → gần, (3) pass 1 tính ellipse và màu bằng transform feedback, (4) pass 2 vẽ hình vuông và trộn alpha, (5) chỉnh ngân sách theo thời gian GPU.

4.1. `splat_editor/render/shader.py`

110 dòng

Vai trò. Bọc chương trình shader OpenGL: nạp `.vert/.frag` từ thư mục `shaders/`, biên dịch, liên kết, cache vị trí uniform, và các hàm `set_*`. Hỗ trợ chương trình chỉ có vertex shader dùng **transform feedback** (pass 1 của bộ vẽ gaussian).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Chương trình shader OpenGL (vertex + fragment), nạp từ thư mục shaders/."""

"""
# Sau khi đã tạo Window (cần ngữ cảnh GL)
prog = ShaderProgram.load("splat")        # đọc shaders/splat.vert + splat.frag

while not win.should_close:
    Window.poll_events()
    prog.use()                            # kích hoạt trước
    prog.set_mat4("u_view", view_matrix)   # numpy 4x4
    prog.set_mat4("u_proj", proj_matrix)
    prog.set_vec2("u_viewport", w, h)
    prog.set_float("u_scale", 1.0)
    # ... glBindVertexArray, glDrawArraysInstanced ...
    win.swap_buffers()

# Chế độ transform feedback (chỉ vertex shader)
sorter = ShaderProgram.load("depth", fragment=False,
                            feedback_varyings=["out_depth", "out_index"])

"""
```

Khai báo mức module (dòng 25–31)

```
import ctypes
from pathlib import Path

import numpy as np
from OpenGL import GL

SHADER_DIR = Path(__file__).parent / "shaders"
```

`class ShaderError(RuntimeError)`

Dòng 34–35

```
class ShaderError(RuntimeError):
    pass
```

`class ShaderProgram`

Dòng 38–110

Các phương thức: `__init__`, `load`, `_compile`, `use`, `location`, `set_int`, `set_float`, `set_vec2`, `set_vec3`, `set_vec3_array`, `set_float_array`, `set_mat3`, `set_mat4`.

```
class ShaderProgram:
```

`ShaderProgram.__init__(self, vertex_src, fragment_src=None, feedback_varyings=None)`

Dòng 39–58

Mô tả trong mã nguồn. fragment_src = None + feedback_varyings: chương trình chỉ chạy vertex shader và ghi kết quả vào buffer (transform feedback, xen kẽ theo thứ tự ‘feedback_varyings‘).

Giải thích. Biên dịch vertex (và fragment nếu có) shader, liên kết chương trình. Tham số `feedback_varyings` bật transform feedback: chương trình chỉ chạy vertex shader và ghi đầu ra vào buffer.

```
    def __init__(self, vertex_src, fragment_src=None, feedback_varyings=None):
        """fragment_src=None + feedback_varyings: chương trình chỉ chạy vertex shader và ghi
        kết quả vào buffer (transform feedback, xen kẽ theo thứ tự `feedback_varyings`)."""
        shaders = [self._compile(vertex_src, GL.GL_VERTEX_SHADER)]
        if fragment_src is not None:
            shaders.append(self._compile(fragment_src, GL.GL_FRAGMENT_SHADER))
        self.handle = GL.glCreateProgram()
        for shader in shaders:
            GL.glAttachShader(self.handle, shader)
        if feedback_varyings:
            names = (ctypes.c_char_p * len(feedback_varyings))(*(v.encode() for v in feedback_varyings))
            GL.glTransformFeedbackVaryings(self.handle, len(feedback_varyings),
                                           ctypes.cast(names, ctypes.POINTER(ctypes.POINTER(GL.GLchar))),
                                           GL.GL_INTERLEAVED_ATTRIBS)
        GL.glLinkProgram(self.handle)
        if not GL.glGetProgramiv(self.handle, GL.GL_LINK_STATUS):
            raise ShaderError(GL.glGetProgramInfoLog(self.handle).decode())
        for shader in shaders:
            GL.glDeleteShader(shader)
        self._locations = {}
```

`ShaderProgram.load(cls, name, fragment=True, feedback_varyings=None)`

Dòng 60–65

Mô tả trong mã nguồn. Nạp shaders/<name>.vert và shaders/<name>.frag (fragment = False: chỉ .vert).

```
    @classmethod
    def load(cls, name, fragment=True, feedback_varyings=None):
        """Nạp shaders/<name>.vert và shaders/<name>.frag (fragment=False: chỉ .vert)."""
        return cls((SHADER_DIR / f"{name}.vert").read_text(encoding="utf-8"),
                   (SHADER_DIR / f"{name}.frag").read_text(encoding="utf-8") if fragment else None,
                   feedback_varyings)
```

`ShaderProgram._compile(src, kind)`

Dòng 67–74

```
    @staticmethod
    def _compile(src, kind):
        shader = GL.glCreateShader(kind)
        GL.glShaderSource(shader, src)
        GL.glCompileShader(shader)
        if not GL.glGetShaderiv(shader, GL.GL_COMPILE_STATUS):
            raise ShaderError(GL.glGetShaderInfoLog(shader).decode())
        return shader
```

`ShaderProgram.use(self)`

Dòng 76–77

```
    def use(self):
        GL.glUseProgram(self.handle)
```

`ShaderProgram.location(self, name)`

Dòng 79–82

```
    def location(self, name):
        if name not in self._locations:
            self._locations[name] = GL.glGetUniformLocation(self.handle, name)
        return self._locations[name]
```

`ShaderProgram.set_int(self, name, value)`

Dòng 85–86

```
    def set_int(self, name, value):
        GL.glUniform1i(self.location(name), int(value))
```

`ShaderProgram.set_float(self, name, value)`

Dòng 88–89

```
88    def set_float(self, name, value):
89        GL.glUniform1f(self.location(name), float(value))
```

`ShaderProgram.set_vec2(self, name, x, y)`

Dòng 91–92

```
91    def set_vec2(self, name, x, y):
92        GL.glUniform2f(self.location(name), float(x), float(y))
```

`ShaderProgram.set_vec3(self, name, v)`

Dòng 94–95

```
94    def set_vec3(self, name, v):
95        GL.glUniform3f(self.location(name), *(float(c) for c in v))
```

`ShaderProgram.set_vec3_array(self, name, values)`

Dòng 97–99

```
97    def set_vec3_array(self, name, values):
98        values = np.ascontiguousarray(values, np.float32).reshape(-1, 3)
99        GL.glUniform3fv(self.location(name + "[0]"), len(values), values)
```

`ShaderProgram.set_float_array(self, name, values)`

Dòng 101–103

```
101   def set_float_array(self, name, values):
102       values = np.ascontiguousarray(values, np.float32).reshape(-1)
103       GL.glUniform1fv(self.location(name + "[0]"), len(values), values)
```

`ShaderProgram.set_mat3(self, name, m)`

Dòng 105–107

Mô tả trong mã nguồn. m: ma trận numpy theo hàng (row-major), tự chuyển vị khi gửi lên.

```
105   def set_mat3(self, name, m):
106       """m: ma trận numpy theo hàng (row-major), tự chuyển vị khi gửi lên."""
107       GL.glUniformMatrix3fv(self.location(name), 1, GL.GL_TRUE, np.asarray(m, np.float32))
```

`ShaderProgram.set_mat4(self, name, m)`

Dòng 109–110

```
109   def set_mat4(self, name, m):
110       GL.glUniformMatrix4fv(self.location(name), 1, GL.GL_TRUE, np.asarray(m, np.float32))
```

4.2. `splat_editor/render/camera.py`

123 dòng

Vai trò. Camera quỹ đạo (`OrbitCamera`): luôn nhìn vào `target`, quay quanh bằng yaw/pitch. Các thao tác: `orbit`, `pan`, `look`, `fly`, `frame`. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§4.2; ma trận `look_at`/`perspective` ở §1.1).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Camera quỹ đạo (orbit): luôn nhìn vào một điểm `target`, quay quanh nó bằng yaw/pitch."""

"""
cam = OrbitCamera()
view = cam.view_matrix()
proj = cam.projection_matrix(aspect=16 / 9)

cam.orbit(dx, dy)                       # chuột trái kéo: quay quanh target
cam.pan(dx, dy, viewport_height)        # chuột giữa kéo: dịch cả khung nhìn
cam.look(dx, dy)                        # chuột phải kéo: nhìn quanh
cam.fly(forward=1, right=0, up=0, dt=0.016, sprint=False)   # WASD / Space / Shift
cam.frame(points)                       # đưa camera về nhìn toàn bộ các điểm (phím F)
cam.reset()
"""

```

Cơ sở toán: Camera quỹ đạo

Các ma trận `look_at`, `perspective`, `orthographic` được giải thích ở mục 1.1 (file `core/math3d.py`). Dưới đây là toán của `OrbitCamera`.

- **Vị trí mắt** (tọa độ cầu): $e = t + d\,(\cos\varphi\sin\theta,\ \sin\varphi,\ \cos\varphi\cos\theta)$ với θ là yaw, φ là pitch, d là distance (`position`).
- Pitch kẹp ± 1.55 rad ($\approx \pm 88.8°$) để $f \times \text{up}$ không suy biến (`PITCH_LIMIT`).
- `orbit`: yaw $-= dx \cdot 0.005$, pitch $+= dy \cdot 0.005$ (rad/pixel).
- **Pan** (`pan`): chiều cao nhìn thấy tại khoảng cách d là $2d\tan(\text{fov}/2)$, trải trên H pixel, nên units_per_pixel $= 2d\tan(\text{fov}/2)/H$. Khi đó target $+= (-dx \cdot \text{right} + dy \cdot \text{up}) \cdot$ units_per_pixel, với right, up là hàng 0 và 1 của V.
- **Look** (`look`): giữ mắt cố định, đổi yaw/pitch, rồi `target = eye - (position_moi - target)` để mắt không dịch.
- **Bay** (`fly`): hướng ngang $=$ forward $\cdot\,(-\sin\theta, 0, -\cos\theta) +$ right $\cdot\,(\cos\theta, 0, -\sin\theta)$, chuẩn hóa (đi chéo không nhanh hơn), $y =$ up. Vận tốc mong muốn $v^* =$ hướng $\cdot$ tốc độ $\cdot$ (2.5 nếu chạy). **Lọc mũ:** $v \leftarrow v + (v^* - v)\min(10\Delta t, 1)$; target $+= v\Delta t$ (tích phân Euler). Dừng hẳn khi $|v| < 10^{-3} \cdot$ tốc độ.
- **Frame** (`frame`): tâm là median các điểm; bán kính R là phân vị 90% khoảng cách tới tâm (bỏ nhiễu); $d = 1.3R/\sin(\text{fov}/2)$ (hình cầu bán kính R vừa khung dọc: $R = d\sin(\text{fov}/2)$); tốc độ bay $= 0.5R$.
- **Ngôi thứ ba:** mode 1: cam $=$ eye $-$ forward $\cdot D$; mode 2: cam $=$ eye $+$ forward $\cdot D$ nhìn ngược lại. D co lại nếu tia mắt→camera chạm voxel (`segment_blocked`, mục 1.9).
- forward() $=$ (target $-$ position)/distance.

Khai báo mức module (dòng 16–20)

```
import math

import numpy as np

from ..core.math3d import look_at, perspective
```

`class OrbitCamera`

Dòng 23–123

Các phương thức: `__init__`, `reset`, `position`, `view_matrix`, `projection_matrix`, `orbit`, `pan`, `look`, `forward`, `set_eye`, `fly`, `frame`.

```
class OrbitCamera:
    ORBIT_SPEED = 0.005       # radian / pixel
    PITCH_LIMIT = 1.55        # radian, tránh lật camera
    LOOK_SPEED = 0.003        # radian / pixel khi nhìn quanh (chuột phải)
    FLY_ACCEL = 10.0          # độ "bám" của vận tốc bay (lớn hơn: ít trôi hơn)
```

`OrbitCamera.__init__(self)`

Dòng 29–31

```
    def __init__(self):
        self.fov = 60.0   # độ, theo chiều dọc
        self.reset()
```

`OrbitCamera.reset(self)`

Dòng 33–41

```
    def reset(self):
        self.target = np.zeros(3)
        self.yaw = 0.0        # radian, quay quanh trục Y
        self.pitch = 0.3      # radian, ngẩng lên / cúi xuống
        self.distance = 5.0   # khoảng cách tới target
        self.fly_speed = 2.0  # đơn vị / giây khi bay
        self.velocity = np.zeros(3)
        self.view_mode = 0        # 0: góc nhìn thứ nhất, 1: thứ ba phía sau, 2: thứ ba phía trước (F5)
        self.third_distance = 0.0  # khoảng lùi camera khỏi mắt người chơi (đã rút ngắn nếu có tường)
```

`OrbitCamera.position(self)`

Dòng 44–47

```
    def position(self):
        cp = math.cos(self.pitch)
        offset = np.array([cp * math.sin(self.yaw), math.sin(self.pitch), cp * math.cos(self.yaw)])
        return self.target + self.distance * offset
```

`OrbitCamera.view_matrix(self)`

Dòng 49–60

Giải thích. Gọi `look_at(position, target, up)`; vị trí mắt tính từ tọa độ cầu $e = t + d(\cos\varphi\sin\theta, \sin\varphi, \cos\varphi\cos\theta)$.

Công thức.

$$e = t + d\begin{bmatrix}\cos\varphi\sin\theta\\ \sin\varphi\\ \cos\varphi\cos\theta\end{bmatrix}, \qquad V = \text{look_at}(e, t, \hat{y}), \qquad |\varphi| \le 1.55 \text{ rad}$$

```
    def view_matrix(self):
        up = np.array([0.0, 1.0, 0.0])
        if self.view_mode and self.third_distance > 0.0:
            # góc nhìn thứ ba (F5): `position()` vẫn là mắt người chơi, camera lùi ra sau lưng (mode 1)
            # hoặc ra trước mặt nhìn lại (mode 2) một đoạn third_distance
            eye, forward = self.position(), self.forward()
            if self.view_mode == 2:
                cam = eye + forward * self.third_distance
                return look_at(cam, cam - forward, up)
            cam = eye - forward * self.third_distance
            return look_at(cam, cam + forward, up)
        return look_at(self.position(), self.target.astype(np.float64), up)
```

`OrbitCamera.projection_matrix(self, aspect)`

Dòng 62–64

Giải thích. Chiếu phối cảnh với `near = distance*0.001`, `far = near*10^6` để zoom từ rất gần đến rất xa mà không cắt.

Công thức.

$$n = 0.001\,d, \qquad f = 10^6\,n$$

```
    def projection_matrix(self, aspect):
        near = self.distance * 0.001
        return perspective(self.fov, aspect, near, near * 1e6)
```

`OrbitCamera.orbit(self, dx, dy)`

Dòng 67–70

Mô tả trong mã nguồn. dx, dy: số pixel chuột di chuyển.

Giải thích. Công thức cốt lõi của hàm này:

$$\theta \leftarrow \theta - 0.005\,dx, \qquad \varphi \leftarrow \text{clip}(\varphi + 0.005\,dy,\ \pm 1.55)$$

```
    def orbit(self, dx, dy):
        """dx, dy: số pixel chuột di chuyển."""
        self.yaw -= dx * self.ORBIT_SPEED
        self.pitch = min(max(self.pitch + dy * self.ORBIT_SPEED, -self.PITCH_LIMIT), self.PITCH_LIMIT)
```

`OrbitCamera.pan(self, dx, dy, viewport_height)`

Dòng 72–77

Giải thích. Kéo camera: 1 pixel ứng với $2d\tan(fov/2)/H$ đơn vị thế giới tại khoảng cách d, di chuyển theo hai trục right/up của view.

Công thức.

$$u_{px} = \frac{2d\tan(\text{fov}/2)}{H}, \qquad t \leftarrow t + \big(-dx\,\hat{r} + dy\,\hat{u}\big)\,u_{px}$$

```
72      def pan(self, dx, dy, viewport_height):
73          view = self.view_matrix()
74          right, up = view[0, :3], view[1, :3]
75          # 1 pixel tương ứng bao nhiêu đơn vị thế giới tại khoảng cách target
76          units_per_pixel = 2 * self.distance * math.tan(math.radians(self.fov) / 2) / viewport_height
77          self.target += (-dx * right + dy * up) * units_per_pixel
```

`OrbitCamera.look(self, dx, dy)`

Dòng 79–84

Mô tả trong mã nguồn. Nhìn quanh tại chỗ (kiểu góc nhìn thứ nhất): giữ nguyên vị trí, xoay hướng nhìn.

```
79      def look(self, dx, dy):
80          """Nhìn quanh tại chỗ (kiểu góc nhìn thứ nhất): giữ nguyên vị trí, xoay hướng nhìn."""
81          eye = self.position()
82          self.yaw -= dx * self.LOOK_SPEED
83          self.pitch = min(max(self.pitch + dy * self.LOOK_SPEED, -self.PITCH_LIMIT), self.PITCH_LIMIT)
84          self.target = eye - (self.position() - self.target)
```

`OrbitCamera.forward(self)`

Dòng 86–88

Mô tả trong mã nguồn. Vector đơn vị hướng nhìn.

```
86      def forward(self):
87          """Vector đơn vị hướng nhìn."""
88          return (self.target - self.position()) / self.distance
```

`OrbitCamera.set_eye(self, eye)`

Dòng 90–92

Mô tả trong mã nguồn. Đặt mắt camera tại ‘eye‘, giữ nguyên hướng nhìn (góc nhìn thứ nhất của chế độ sinh tồn).

```
90      def set_eye(self, eye):
91          """Đặt mắt camera tại `eye`, giữ nguyên hướng nhìn (góc nhìn thứ nhất của chế độ sinh tồn)."""
92          self.target = np.asarray(eye, np.float64) - (self.position() - self.target)
```

`OrbitCamera.fly(self, forward, right, up, dt, sprint=False)`

Dòng 94–109

Mô tả trong mã nguồn. Bay kiểu Minecraft sáng tạo: W/S A/D đi ngang theo hướng nhìn, lên / xuống theo trục Y.

Vận tốc tiến dần về vận tốc mong muốn nên có quán tính nhẹ khi bắt đầu / dừng.

Giải thích. Bay kiểu Minecraft: hướng ngang chuẩn hóa, lọc mũ vận tốc $v \leftarrow v + (v^{*} - v)\min(10\Delta t, 1)$ rồi tích phân Euler.

Công thức.

$$v^{*} = \hat{w}\cdot\text{speed}\cdot\begin{cases}2.5 & \text{chạy}\\ 1 & \text{đi}\end{cases}, \quad v \leftarrow v + (v^{*} - v)\min(10\Delta t, 1), \quad t \leftarrow t + v\Delta t$$

$$\hat{w} = \text{normalize}\big(f_{in}\,(-\sin\theta, 0, -\cos\theta) + r_{in}\,(\cos\theta, 0, -\sin\theta)\big)$$

```
94      def fly(self, forward, right, up, dt, sprint=False):
95          """Bay kiểu Minecraft sáng tạo: W/S A/D đi ngang theo hướng nhìn, lên / xuống theo trục Y.
96
```

```
        Vận tốc tiến dần về vận tốc mong muốn nên có quán tính nhẹ khi bắt đầu / dừng.
        """
        s, c = math.sin(self.yaw), math.cos(self.yaw)
        direction = forward * np.array([-s, 0.0, -c]) + right * np.array([c, 0.0, -s])
        length = np.linalg.norm(direction)
        if length > 0:
            direction /= length            # đi chéo không nhanh hơn đi thẳng
        direction[1] = up
        wanted = direction * self.fly_speed * (2.5 if sprint else 1.0)
        self.velocity += (wanted - self.velocity) * min(self.FLY_ACCEL * dt, 1.0)
        if not wanted.any() and np.linalg.norm(self.velocity) < 1e-3 * self.fly_speed:
            self.velocity[:] = 0
        self.target += self.velocity * dt
```

`OrbitCamera.frame(self, points)`

Dòng 111–123

Mô tả trong mã nguồn. Đặt camera để nhìn thấy toàn bộ đám điểm.

Giải thích. Đặt camera nhìn toàn bộ đám điểm: tâm là median, bán kính là phân vị 90%, khoảng cách $1.3R/\sin(fov/2)$.

Công thức.

$$t = \text{median}(p_i), \quad R = P_{90}\big(\|p_i - t\|\big), \quad d = \frac{1.3\,R}{\sin(\text{fov}/2)}, \quad \text{speed} = 0.5R$$

```
    def frame(self, points):
        """Đặt camera để nhìn thấy toàn bộ đám điểm."""
        if len(points) == 0:
            self.reset()
            return
        center = np.median(points, axis=0)
        # dùng phân vị 90% để bỏ qua các điểm nằm rất xa (nhiễu)
        radius = float(np.percentile(np.linalg.norm(points - center, axis=1), 90)) or 1.0
        self.target = center.astype(np.float64)
        self.distance = 1.3 * radius / math.sin(math.radians(self.fov) / 2)
        self.yaw, self.pitch = 0.0, 0.3
        self.fly_speed = 0.5 * radius
        self.velocity[:] = 0
```

4.3. `splat_editor/render/sorter.py`

140 dòng

Vai trò. Luồng nền `DepthSorter`: nhận yêu cầu (ma trận camera và ngân sách), chọn gaussian theo LOD rồi sắp xếp **xa → gần** bằng khóa uint16. Luồng chính không bao giờ phải chờ; trong lúc chờ renderer dùng thứ tự cũ. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§4.3).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Chọn gaussian cần vẽ (LOD) và sắp xếp theo độ sâu ở luồng nền, để vòng lặp vẽ không bị chặn.

- Luồng chính gửi yêu cầu (ma trận model-view / projection mới nhất + ngân sách số gaussian),
  luồng nền chọn gaussian theo LOD (xem lod.py) rồi sắp xếp. Trong lúc chờ, renderer vẫn vẽ
  bằng thứ tự cũ (sai lệch nhỏ, mắt gần như không thấy).
- Mọi chỉ số ở đây là "vị trí" trên GPU (thứ tự LodIndex.order), không phải chỉ số trong file.
- Tâm gaussian ở toạ độ file của từng lớp; ma trận thế giới của mỗi lớp đi kèm yêu cầu.
- Kết quả gồm 2 mảng:
    slots: vị trí các gaussian được chọn, tăng dần -> pass 1 đọc bộ nhớ liền mạch;
    ranks: thứ tự vẽ xa -> gần, là chỉ số vào `slots` (bỏ gaussian sau camera).
- Gaussian đã xoá (bit STATE_DELETED trong mảng trạng thái của renderer) bị bỏ khỏi kết quả.
- Độ sâu được lượng tử hoá về uint16 để numpy dùng radix sort (O(N), nhanh hơn argsort
  float ~3 lần). numpy nhả GIL khi nhân ma trận / sắp xếp nên luồng nền chạy song song thật.
"""

"""
sorter = DepthSorter()                  # tạo luồng nền "depth-sorter"
sorter.set_splats(lod, state)           # khi đổi tập gaussian

# Mỗi khung hình
sorter.request(model_view, projection, budget, transforms)   # gửi yêu cầu, không chờ
result = sorter.take_result()           # (slots, ranks) nếu luồng nền đã xong, ngược lại None
if result is not None:
    slots, ranks = result               # cập nhật thứ tự vẽ lên GPU

sorter.close()                          # tắt luồng khi thoát
"""

```

Cơ sở toán: Sắp xếp theo độ sâu

- Độ sâu camera của tâm: $z_{cam} = (MV \cdot M_{layer})[2,:] \cdot (p, 1)$. Camera nhìn $-z$ nên $z_{cam} < 0$ là phía trước, càng âm càng xa. Giữ depth < 0.

- Khóa 16 bit: key = $\lfloor(\text{depth} - lo) \cdot 65535/(hi - lo)\rfloor$ (lượng tử đều trên $[lo, hi]$); `argsort(uint16, stable)` là radix sort $O(N)$. Depth (âm) tăng dần tương ứng xa → gần. Lượng tử 16 bit đủ vì chỉ cần thứ tự gần đúng.
- Chạy ở luồng nền; kết quả cũ bị bỏ nếu `version` đã đổi.

Khai báo mức module (dòng 29–33)

```
import threading

import numpy as np

STATE_DELETED = 2       # = splat_renderer.STATE_DELETED
```

`class DepthSorter`

Dòng 36–140

Các phương thức: `__init__`, `set_splats`, `request`, `take_result`, `close`, `_run`, `_select`, `_depth_rows`, `sort`.

```
class DepthSorter:
```

`DepthSorter.__init__(self)`

Dòng 37–47

```
    def __init__(self):
        self._cond = threading.Condition()
        self._positions = None     # tâm gaussian theo thứ tự vị trí
        self._lod = None
        self._state = None         # trạng thái theo vị trí (uint8), dùng chung với renderer
        self._version = 0          # tăng mỗi khi đổi tập gaussian, để bỏ kết quả cũ
        self._request = None       # (version, model_view, projection, budget, transforms) đang chờ
        self._result = None        # (version, slots, ranks) đã sắp xếp xong, chưa lấy
        self._closed = False
        self._thread = threading.Thread(target=self._run, name="depth-sorter", daemon=True)
        self._thread.start()
```

`DepthSorter.set_splats(self, lod, state=None)`

Dòng 49–58

Mô tả trong mã nguồn. lod: LodIndex (tâm gaussian theo thứ tự vị trí: lod.positions). state: mảng trạng thái (uint8) theo vị trí, renderer sửa tại chỗ; None: không bỏ gì.

```
    def set_splats(self, lod, state=None):
        """lod: LodIndex (tâm gaussian theo thứ tự vị trí: lod.positions).
        state: mảng trạng thái (uint8) theo vị trí, renderer sửa tại chỗ; None: không bỏ gì."""
        with self._cond:
            self._lod = lod
            self._state = state
            self._positions = lod.positions
            self._version += 1
            self._request = None
            self._result = None
```

`DepthSorter.request(self, model_view, projection, budget, transforms=None)`

Dòng 60–66

Mô tả trong mã nguồn. Yêu cầu chọn + sắp xếp. Yêu cầu chưa xử lý bị thay bằng yêu cầu mới nhất. budget: số gaussian tối đa (None: không giới hạn); transforms: ma trận thế giới mỗi lớp.

Giải thích. Gửi yêu cầu cho luồng nền; yêu cầu chưa xử lý bị thay bằng yêu cầu mới nhất (không xếp hàng), nên luồng nền luôn làm việc với camera mới nhất.

```
    def request(self, model_view, projection, budget, transforms=None):
        """Yêu cầu chọn + sắp xếp. Yêu cầu chưa xử lý bị thay bằng yêu cầu mới nhất.
        budget: số gaussian tối đa (None: không giới hạn); transforms: ma trận thế giới mỗi lớp."""
        with self._cond:
            self._request = (self._version, model_view.copy(), projection.copy(), budget,
                             None if transforms is None else np.array(transforms, np.float64))
            self._cond.notify()
```

`DepthSorter.take_result(self)`

Dòng 68–74

Mô tả trong mã nguồn. (slots, ranks) nếu có kết quả mới, ngược lại None.

```
    def take_result(self):
        """(slots, ranks) nếu có kết quả mới, ngược lại None."""
        with self._cond:
            result, self._result = self._result, None
        if result is None or result[0] != self._version:
            return None
        return result[1:]
```

`DepthSorter.close(self)`

Dòng 76–79

```
    def close(self):
        with self._cond:
            self._closed = True
            self._cond.notify()
```

`DepthSorter._run(self)`

Dòng 81–94

```
    def _run(self):
        while True:
            with self._cond:
                while self._request is None and not self._closed:
                    self._cond.wait()
                if self._closed:
                    return
                (version, model_view, projection, budget, transforms), self._request = self._request, None
                positions, lod, state = self._positions, self._lod, self._state
            slots = self._select(lod, state, model_view, projection, budget, transforms)
            ranks = self.sort(positions[slots], self._depth_rows(model_view, transforms, lod, slots))
            with self._cond:
                if version == self._version:
                    self._result = (version, slots.astype(np.uint32), ranks)
```

`DepthSorter._select(lod, state, model_view, projection, budget, transforms)`

Dòng 96–108

Mô tả trong mã nguồn. lod.select rồi bỏ gaussian đã xoá. Nếu bị giới hạn bởi budget mà bỏ đi nhiều, chọn lại với budget nới theo tỉ lệ còn lại để vẫn vẽ gần đủ budget gaussian.

```
    @staticmethod
    def _select(lod, state, model_view, projection, budget, transforms):
        """lod.select rồi bỏ gaussian đã xoá. Nếu bị giới hạn bởi budget mà bỏ đi nhiều, chọn lại
        với budget nới theo tỉ lệ còn lại để vẫn vẽ gần đủ budget gaussian."""
        slots = lod.select(model_view, projection, budget, transforms)
        if state is None:
            return slots
        kept = slots[(state[slots] & STATE_DELETED) == 0]
        if budget is not None and len(slots) >= budget and len(kept) < 0.9 * budget:
            wider = int(budget * len(slots) / max(len(kept), 1))
            slots = lod.select(model_view, projection, wider, transforms)
            kept = slots[(state[slots] & STATE_DELETED) == 0]
        return kept
```

`DepthSorter._depth_rows(model_view, transforms, lod, slots)`

Dòng 110–118

Mô tả trong mã nguồn. Hàng z của ma trận model-view * thế giới cho từng đoạn ‘slots‘ (tăng dần) theo lớp: [(số phần tử, hàng (4,))].

```
    @staticmethod
    def _depth_rows(model_view, transforms, lod, slots):
        """Hàng z của ma trận model-view * thế giới cho từng đoạn `slots` (tăng dần) theo lớp:
        [(số phần tử, hàng (4,))]."""
        mv = np.asarray(model_view, np.float64)
        if transforms is None:
            return [(len(slots), mv[2])]
        bounds = np.searchsorted(slots, lod.group_slots)
        return [(int(bounds[g + 1] - bounds[g]), (mv @ transforms[g])[2]) for g in range(len(transforms))]
```

`DepthSorter.sort(positions, depth_rows)`

Dòng 120–140

Mô tả trong mã nguồn. Chỉ số các tâm nằm trước camera, xếp xa -> gần. depth_rows: ma trận model-view 4x4, hoặc [(số phần tử, hàng z)] cho từng đoạn liên tiếp.

Giải thích. Tính độ sâu camera $z = (MV \cdot M)[2,:] \cdot (p,1)$, giữ điểm trước camera ($z < 0$), lượng tử thành uint16 và `argsort` ổn định (radix) để ra thứ tự xa → gần.

Công thức.

$$z_{cam} = (MV\,M_{layer})_{[2,:]} \cdot (p,1) \;\; (< 0 \text{ là trước camera}), \qquad \text{key} = \left\lfloor \frac{z - lo}{hi - lo} \cdot 65535 \right\rfloor$$

Sắp `key` tăng dần = xa → gần (radix sort $O(N)$).

```
    @staticmethod
    def sort(positions, depth_rows):
        """Chỉ số các tâm nằm trước camera, xếp xa -> gần.
        depth_rows: ma trận model-view 4x4, hoặc [(số phần tử, hàng z)] cho từng đoạn liên tiếp."""
        if not isinstance(depth_rows, list):
            depth_rows = [(len(positions), np.asarray(depth_rows, np.float64)[2])]
        # trong không gian camera, camera nhìn theo -z nên z càng âm càng xa
        depth = np.empty(len(positions), np.float32)
        start = 0
        for count, row in depth_rows:
            row = row.astype(np.float32)
            depth[start:start + count] = positions[start:start + count] @ row[:3] + row[3]
            start += count
        visible = np.flatnonzero(depth < 0)
        depth = depth[visible]
        if len(depth) == 0:
            return np.zeros(0, np.uint32)
        lo, hi = float(depth.min()), float(depth.max())
        scale = 65535.0 / (hi - lo) if hi > lo else 0.0
        key = ((depth - lo) * scale).astype(np.uint16)
        return visible[np.argsort(key, kind='stable')].astype(np.uint32)
```

4.4. `splat_editor/render/splat_renderer.py`

504 dòng

Vai trò. Trái tim của hiển thị: `SplatRenderer` đóng gói gaussian thành 32 byte, đưa lên GPU, chạy pass 1 (transform feedback) và pass 2 (vẽ hình vuông, trộn alpha), quản lý vùng chọn/xóa, bản đồ bóng và tự chỉnh ngân sách. Toán học: xem mục *Cơ sở toán* ngay bên dưới (§6.1, §6.2).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Vẽ Gaussian splat bằng OpenGL 3.3, đủ nhanh cho cảnh 10-15 triệu gaussian trên máy yếu.

Ý tưởng:
1. Mỗi gaussian 3D có tâm, màu, độ đục, scale và quaternion. Dữ liệu được nén còn 32 byte /
   gaussian (2 texel RGBA32UI: tâm float32, còn lại half float / 8 bit) và đưa lên GPU một lần,
   xếp theo thứ tự Morton của LodIndex ("vị trí"), để gaussian gần nhau nằm gần nhau trong bộ nhớ.
   Hệ số SH bậc cao lượng tử 8 bit (1 texel RGBA8 / hệ số); nếu quá ngân sách bộ nhớ
   (SH_VRAM_BUDGET) thì tự hạ bậc SH.
2. LOD (xem lod.py, sorter.py): mỗi khi camera đổi, luồng nền chọn tối đa `budget` gaussian
   (bỏ khối ngoài khung nhìn, khối xa lấy ít) rồi sắp xếp xa -> gần.
3. Pass 1 (transform feedback): mỗi gaussian được chọn chạy vertex shader MỘT lần, theo thứ tự
   bộ nhớ (đọc liền mạch), chiếu Sigma = R S S^T R^T xuống màn hình thành ellipse 2D
   (Sigma' = J W Sigma W^T J^T) và tính màu (kể cả SH, vùng chọn), ghi 32 byte vào buffer.
   Camera đứng yên thì bỏ qua pass này.
4. Pass 2: theo thứ tự xa -> gần, mỗi gaussian vẽ một hình vuông (4 đỉnh, instancing) kéo theo
   2 trục ellipse; fragment shader tô theo hàm gauss exp(-r^2/2) và trộn alpha (premultiplied).
5. Trạng thái mỗi gaussian nằm ở một texture buffer riêng, 1 byte / gaussian, để cập nhật nhanh:
   bit STATE_SELECTED (đang chọn, pha màu vàng) và bit STATE_DELETED (đã xoá mềm: luồng sắp xếp
   bỏ qua, pass 1 loại bỏ). Xoá / hoàn tác chỉ gửi lại mảng này, không gửi lại dữ liệu gaussian.
"""

"""
renderer = SplatRenderer()              # cần ngữ cảnh GL
renderer.set_splats(splats, lod)        # gửi gaussian lên GPU (texture buffer) khi mở file / xoá
renderer.set_transforms(layer_matrices) # phép biến đổi mỗi lớp (đổi là xong, không gửi lại dữ liệu)
renderer.set_selection(mask)            # tô vàng các gaussian đang chọn
renderer.set_deleted(mask)              # ẩn các gaussian đã xoá

# Mỗi khung hình
renderer.draw(model_view, projection, width, height)   # glDrawArraysInstanced: 4 góc x N instance
renderer.adapt_budget(target_ms)        # chế độ Auto: chỉnh số gaussian vẽ theo thời gian GPU
renderer.use_sh = False                 # tắt màu theo hướng nhìn
"""

```

Cơ sở toán: tự chỉnh ngân sách (`SplatRenderer.adapt_budget`)

Đo thời gian GPU bằng query. cost = median(ms/count); wanted = target_ms/cost. Nếu lệch dưới 10% thì giữ nguyên; ngược lại step = clip(wanted/current, 0.75, 1.33)$^{1/2}$ (căn bậc hai nên đổi tối đa khoảng 15%/lần, làm trơn để tránh dao động); budget = clip(current · step, MIN_BUDGET, count).

Cơ sở toán: Lượng tử hóa và đóng gói GPU

Phần này giải thích cách `SplatRenderer.pack` nén dữ liệu gaussian, và cách shader `splat_prep.vert` giải nén (xem thêm mục shader tương ứng).

- **Half-float (IEEE-754 16 bit)**, `halfToFloat`: e là bit 10–14, m là bit 0–9, bit 15 là dấu. Nếu $e = 0$ (subnormal) thì $v = m \cdot 2^{-24}$; nếu $e \neq 0$ thì $v = (m + 1024) \cdot 2^{e-25} = (1 + m/1024) \cdot 2^{e-15}$. Dùng cho màu, α, scale để tiết kiệm bộ nhớ.
- **SH 8 bit:** scale_sh = percentile$_{99.9}$(|coeff|)/127; lưu u = clip(round(coeff/scale_sh) + 128, 1, 255); giải coeff = $(u - 128) \cdot$ scale_sh (shader: `(texel*255 - 128) * uShScale`).
- **Màu RGBA8** (`packColor`): u = clamp$(c, 0, 1) \cdot 255 + 0.5$ (làm tròn).
- 1 gaussian = 8 uint32 (2 texel RGBA32UI): xyz (float), màu rg/b + α, scale xyz (half) + layer id, quaternion 4×8 bit.

Khai báo mức module (dòng 35–54)

```
import ctypes
import os
from collections import deque
from concurrent.futures import ThreadPoolExecutor

import numpy as np
from OpenGL import GL

from ..core import sh
from ..core.lod import LodIndex
from .shader import ShaderProgram
from .sorter import DepthSorter

UPLOAD_CHUNK = 1 << 18              # gửi lên GPU từng đoạn 262144 gaussian (RAM tạm nhỏ)
SH_VRAM_BUDGET = 512 * 1024 * 1024  # bộ nhớ GPU tối đa cho hệ số SH
FEEDBACK_STRIDE = 32                # byte / gaussian trong buffer kết quả pass 1 (2 texel RGBA32UI)
MIN_BUDGET = 200_000                # chế độ tự động không hạ budget thấp hơn mức này
STATE_SELECTED = 1                  # các bit trạng thái (1 byte / gaussian, xem splat_prep.vert)
STATE_DELETED = 2
SHADOW_SIZE = 2048                  # độ phân giải bản đồ bóng của mặt trời
```

`class SplatRenderer`

Dòng 57–504

Các phương thức: `__init__`, `pack`, `_upload_chunked`, `set_splats`, `_upload_sh`, `_upload_state`, `_set_state_bit`, `set_selection`, `set_deleted`, `update_selection`, `set_transforms`, `set_lighting`, `_upload_order`, `_update_order`, `_prepare`, `_set_light_uniforms`, `_create_shadow_map`, `_render_shadow_map`, `_collect_timings`, `adapt_budget`, `draw`, `_draw_splats`.

```
class SplatRenderer:
    SELECT_COLOR = (1.0, 0.8, 0.1)
```

`SplatRenderer.__init__(self)`

Dòng 60–127

```
    def __init__(self):
        self.prep_shader = ShaderProgram.load("splat_prep", fragment=False,
                                              feedback_varyings=["vCenterMajor", "vMinorExtent", "vColor"])
        self.shader = ShaderProgram.load("splat")
        self.shadow_shader = ShaderProgram.load("shadow")
        self.lighting = {'enabled': False, 'signature': (False,)}   # xem core/lighting.py
        self._shadow_fbo = None
        self._shadow_texture = None
        self._shadow_key = None
        self.count = 0
        self.sh_degree = 0          # bậc SH của dữ liệu đang có trên GPU
        self.file_sh_degree = 0     # bậc SH của dữ liệu gốc (có thể cao hơn nếu bị hạ cho đỡ tốn bộ nhớ)
        self.sh_scale = 0.0
        self.use_sh = True          # tắt: chỉ dùng màu bậc 0 (không đổi theo hướng nhìn)
        self.budget = 2_000_000     # số gaussian tối đa vẽ mỗi khung hình (None: không giới hạn)
        self.prep_count = 0         # số gaussian pass 1 xử lý (đã chọn theo LOD)
```

```
        self.draw_count = 0          # số gaussian pass 2 vẽ (bỏ cái sau camera)
        self.last_sort_key = None
        self.sorter = DepthSorter()
        self._slot_order = None      # vị trí trên GPU -> chỉ số gaussian gốc
        self._order_version = 0      # tăng khi danh sách gaussian / vùng chọn đổi -> phải chạy lại pass 1
        self._prep_key = None
        self._capacity = 0
        self._state = None           # trạng thái theo vị trí (uint8, STATE_*), bản sao trên CPU
        self._slot_of = None
        # đo thời gian GPU (GL_TIME_ELAPSED) không chặn: vòng các query, đọc kết quả khi đã sẵn sàng
        self._queries = list(GL.glGenQueries(4))
        self._query_pending = []     # [(query, có chạy pass 1 không)]
        self.gpu_ms = None           # thời gian GPU gần nhất của khung hình có chạy pass 1 (xấu nhất)
        self._samples = []

        self.slot_vbo = GL.glGenBuffers(1)        # vị trí các gaussian được chọn (đầu vào pass 1)
        self.rank_vbo = GL.glGenBuffers(1)        # thứ tự vẽ xa -> gần (đầu vào pass 2)
        self.feedback_buffer = GL.glGenBuffers(1)  # kết quả pass 1
        self.feedback_texture = GL.glGenTextures(1)

        # pass 1: mỗi đỉnh (GL_POINTS) = một vị trí gaussian
        self.prep_vao = GL.glGenVertexArrays(1)
        GL.glBindVertexArray(self.prep_vao)
        GL.glBindBuffer(GL.GL_ARRAY_BUFFER, self.slot_vbo)
        GL.glEnableVertexAttribArray(0)
        GL.glVertexAttribIPointer(0, 1, GL.GL_UNSIGNED_INT, 0, ctypes.c_void_p(0))

        # pass 2: 4 góc hình vuông + mỗi instance một chỉ số vào kết quả pass 1
        corners = np.array([-1, -1, 1, -1, -1, 1, 1, 1], np.float32)   # triangle strip
        self.corner_vbo = GL.glGenBuffers(1)
        self.vao = GL.glGenVertexArrays(1)
        GL.glBindVertexArray(self.vao)
        GL.glBindBuffer(GL.GL_ARRAY_BUFFER, self.corner_vbo)
        GL.glBufferData(GL.GL_ARRAY_BUFFER, corners.nbytes, corners, GL.GL_STATIC_DRAW)
        GL.glEnableVertexAttribArray(0)
        GL.glVertexAttribPointer(0, 2, GL.GL_FLOAT, GL.GL_FALSE, 0, ctypes.c_void_p(0))
        GL.glBindBuffer(GL.GL_ARRAY_BUFFER, self.rank_vbo)
        GL.glEnableVertexAttribArray(1)
        GL.glVertexAttribIPointer(1, 1, GL.GL_UNSIGNED_INT, 0, ctypes.c_void_p(0))
        GL.glVertexAttribDivisor(1, 1)
        GL.glBindVertexArray(0)

        # texture buffer chứa dữ liệu gaussian, vùng chọn và hệ số SH
        self.data_buffer = GL.glGenBuffers(1)
        self.data_texture = GL.glGenTextures(1)
        self.state_buffer = GL.glGenBuffers(1)
        self.state_texture = GL.glGenTextures(1)
        self.sh_buffer = GL.glGenBuffers(1)
        self.sh_texture = GL.glGenTextures(1)
        self.transform_buffer = GL.glGenBuffers(1)     # ma trận thế giới của các lớp (4 texel / lớp)
        self.transform_texture = GL.glGenTextures(1)
        self.transforms = np.identity(4)[None]
```

`SplatRenderer.pack(splats, index=slice(None), layers=0)`

Dòng 130–147

Mô tả trong mã nguồn. Gaussian ‘index‘ -> mảng (n,8) uint32 = 2 texel RGBA32UI / gaussian: texel 0: x y z (float32, toạ độ file của lớp), r g (half) texel 1: b alpha (half), sx sy (half), sz (half) | số thứ tự lớp (16 bit), quaternion w x y z (4 x snorm8).

Giải thích. Đóng gói gaussian thành 8 uint32 (2 texel RGBA32UI): tâm float32; màu, alpha và scale half-float; quaternion 4×8 bit; id layer. Tổng 32 byte/gaussian.

Công thức.

$$32\ \text{byte} = \underbrace{12}_{\text{tâm float32}} + \underbrace{6}_{\text{rgb half}} + \underbrace{2}_{\alpha\ \text{half}} + \underbrace{6}_{\text{scale half}} + \underbrace{2}_{\text{layer}} + \underbrace{4}_{\text{quaternion 4×8bit}}$$

$$u_q = \text{clip}\big(\text{round}(127\,q) + 128,\ 1,\ 255\big), \qquad q = \text{normalize}\Big(\frac{u_q - 128}{127}\Big)$$

```
    @staticmethod
    def pack(splats, index=slice(None), layers=0):
        """Gaussian `index` -> mảng (n,8) uint32 = 2 texel RGBA32UI / gaussian:
        texel 0: x y z (float32, toạ độ file của lớp), r g (half)
        texel 1: b alpha (half), sx sy (half), sz (half) | số thứ tự lớp (16 bit),
                 quaternion w x y z (4 x snorm8)."""
        half = lambda a: np.ascontiguousarray(a, np.float16).view(np.uint16).astype(np.uint32)  # noqa: E731
        pair = lambda lo, hi: half(lo) | (half(hi) << np.uint32(16))                            # noqa: E731
        positions, colors, scales = splats.positions[index], splats.colors[index], splats.scales[index]
        q = np.clip(np.rint(splats.rotations[index] * 127.0) + 128, 1, 255).astype(np.uint32)
        packed = np.empty((len(positions), 8), np.uint32)
        packed[:, 0:3] = np.ascontiguousarray(positions, np.float32).view(np.uint32)
        packed[:, 3] = pair(colors[:, 0], colors[:, 1])
        packed[:, 4] = pair(colors[:, 2], splats.alphas[index])
        packed[:, 5] = pair(scales[:, 0], scales[:, 1])
        packed[:, 6] = half(scales[:, 2]) | (np.asarray(layers, np.uint32) << np.uint32(16))
```

```
        packed[:, 7] = q[:, 0] | (q[:, 1] << 8) | (q[:, 2] << 16) | (q[:, 3] << 24)
        return packed
```

SplatRenderer._upload_chunked(buffer, count, bytes_per_item, make_chunk)

Dòng 149–169

Mô tả trong mã nguồn. Cấp buffer rồi gửi dữ liệu từng đoạn (make_chunk(start, end) -> mảng numpy). Các đoạn được chuẩn bị song song ở nhiều luồng (numpy nhả GIL), luồng chính chỉ gửi lên GPU.

```
    @staticmethod
    def _upload_chunked(buffer, count, bytes_per_item, make_chunk):
        """Cấp buffer rồi gửi dữ liệu từng đoạn (make_chunk(start, end) -> mảng numpy).
        Các đoạn được chuẩn bị song song ở nhiều luồng (numpy nhả GIL), luồng chính chỉ gửi lên GPU."""
        GL.glBindBuffer(GL.GL_TEXTURE_BUFFER, buffer)
        GL.glBufferData(GL.GL_TEXTURE_BUFFER, max(count * bytes_per_item, 16), None, GL.GL_STATIC_DRAW)

        def _upload(start, future):
            chunk = future.result()
            GL.glBufferSubData(GL.GL_TEXTURE_BUFFER, start * bytes_per_item, chunk.nbytes, chunk)

        workers = min(max((os.cpu_count() or 2) - 1, 1), 6)
        make = lambda a: np.ascontiguousarray(make_chunk(a, min(a + UPLOAD_CHUNK, count)))  # noqa: E731
        with ThreadPoolExecutor(workers) as pool:
            pending = deque()
            for start in range(0, count, UPLOAD_CHUNK):
                pending.append((start, pool.submit(make, start)))
                if len(pending) > workers:     # giới hạn số đoạn đang chờ để RAM tạm không phình
                    _upload(*pending.popleft())
            while pending:
                _upload(*pending.popleft())
```

SplatRenderer.set_splats(self, splats, lod=None)

Dòng 172–197

Mô tả trong mã nguồn. lod: LodIndex của splats (None: tự dựng).

Giải thích. Gửi dữ liệu lên GPU theo thứ tự Morton của `LodIndex`, từng đoạn `UPLOAD_CHUNK`; nếu SH vượt `SH_VRAM_BUDGET` thì tự hạ bậc.

```
    def set_splats(self, splats, lod=None):
        """lod: LodIndex của splats (None: tự dựng)."""
        if splats is None or splats.count == 0:
            self.count = 0
            return
        n = splats.count
        if lod is None:
            lod = LodIndex(splats.positions, splats.scales, splats.alphas)
        order = self._slot_order = lod.order
        self._slot_of = np.empty(n, np.uint32)                # chỉ số gaussian gốc -> vị trí trên GPU
        self._slot_of[order] = np.arange(n, dtype=np.uint32)
        # lớp của mỗi vị trí: các lớp chiếm những đoạn vị trí liên tiếp (lod.group_slots)
        layer_of = lambda a, b: (np.searchsorted(lod.group_slots, np.arange(a, b), side='right') - 1)  # noqa: E731
        self._upload_chunked(self.data_buffer, n, 32,
                             lambda a, b: self.pack(splats, order[a:b], layer_of(a, b)))
        GL.glBindTexture(GL.GL_TEXTURE_BUFFER, self.data_texture)
        GL.glTexBuffer(GL.GL_TEXTURE_BUFFER, GL.GL_RGBA32UI, self.data_buffer)
        self.count = n
        self.prep_count = self.draw_count = 0
        self._samples.clear()
        self.last_sort_key = None
        self._state = np.zeros(n, np.uint8)
        self._upload_state(resize=True)
        self.sorter.set_splats(lod, self._state)
        self.set_transforms([np.identity(4)] * lod.group_count)
        self._upload_sh(splats, order)
```

SplatRenderer._upload_sh(self, splats, order)

Dòng 199–223

Mô tả trong mã nguồn. (N,K,3) hệ số SH -> K' texel RGBA8 / gaussian (lượng tử 8 bit, thành phần A bỏ trống). K' <= K: hạ bậc SH nếu không đủ SH_VRAM_BUDGET. Hệ số giữ ở toạ độ file: shader xoay hướng nhìn ngược về toạ độ file thay vì xoay hệ số.

Giải thích. Lượng tử hóa SH về 8 bit: $u = \mathrm{clip}(\mathrm{round}(c/\mathit{scale_sh}) + 128, 1, 255)$ với `scale_sh` là phân vị 99.9% chia 127.

Công thức.

$$\mathrm{scale_sh} = \frac{P_{99.9}(|h|)}{127}, \qquad u = \mathrm{clip}\Big(\mathrm{round}\Big(\frac{h}{\mathrm{scale_sh}}\Big) + 128,\ 1,\ 255\Big), \qquad h \approx (u - 128)\,\mathrm{scale_sh}$$

```
    def _upload_sh(self, splats, order):
        """(N,K,3) hệ số SH -> K' texel RGBA8 / gaussian (lượng tử 8 bit, thành phần A bỏ trống).
        K' <= K: hạ bậc SH nếu không đủ SH_VRAM_BUDGET. Hệ số giữ ở toạ độ file: shader xoay
        hướng nhìn ngược về toạ độ file thay vì xoay hệ số."""
        n, k = splats.sh_rest.shape[:2]
        self.file_sh_degree = sh.degree_for_coeffs(k) or 0
        degree = self.file_sh_degree
        while degree > 0 and n * sh.COEFFS_FOR_DEGREE[degree] * 4 > SH_VRAM_BUDGET:
            degree -= 1
        self.sh_degree = degree
        k = sh.COEFFS_FOR_DEGREE[degree]
        if k:
            sample = splats.sh_rest[::max(n // 100_000, 1), :k].astype(np.float32)
            self.sh_scale = max(float(np.percentile(np.abs(sample), 99.9)), 1e-6) / 127.0

        def chunk(a, b):
            out = np.zeros((b - a, k, 4), np.uint8)
            coeffs = splats.sh_rest[order[a:b], :k].astype(np.float32)
            out[:, :, :3] = np.clip(np.rint(coeffs / self.sh_scale) + 128, 1, 255)
            return out

        # không có SH: buffer tối thiểu để texture vẫn hợp lệ
        self._upload_chunked(self.sh_buffer, n if k else 0, k * 4, chunk)
        GL.glBindTexture(GL.GL_TEXTURE_BUFFER, self.sh_texture)
        GL.glTexBuffer(GL.GL_TEXTURE_BUFFER, GL.GL_RGBA8, self.sh_buffer)
```

`SplatRenderer._upload_state(self, resize=False, lo=0, hi=None)`

Dòng 225–235

Mô tả trong mã nguồn. Gửi self._state[lo:hi] lên GPU (resize: cấp lại cả buffer).

```
    def _upload_state(self, resize=False, lo=0, hi=None):
        """Gửi self._state[lo:hi] lên GPU (resize: cấp lại cả buffer)."""
        GL.glBindBuffer(GL.GL_TEXTURE_BUFFER, self.state_buffer)
        if resize:
            GL.glBufferData(GL.GL_TEXTURE_BUFFER, max(self._state.nbytes, 16), self._state, GL.GL_DYNAMIC_DRAW)
            GL.glBindTexture(GL.GL_TEXTURE_BUFFER, self.state_texture)
            GL.glTexBuffer(GL.GL_TEXTURE_BUFFER, GL.GL_R8UI, self.state_buffer)
        else:
            hi = len(self._state) if hi is None else hi
            GL.glBufferSubData(GL.GL_TEXTURE_BUFFER, lo, hi - lo, self._state[lo:hi])
        self._order_version += 1
```

`SplatRenderer._set_state_bit(self, bit, mask)`

Dòng 237–244

Mô tả trong mã nguồn. Đặt bit ‘bit‘ của mọi gaussian theo mask (N,) bool theo chỉ số gaussian gốc. Sửa tại chỗ: luồng sắp xếp dùng chung mảng này.

Giải thích. Đặt bit trạng thái (chọn/xóa) theo mặt nạ, sửa tại chỗ trên mảng 1 byte/gaussian rồi chỉ gửi đoạn bị đổi.

```
    def _set_state_bit(self, bit, mask):
        """Đặt bit `bit` của mọi gaussian theo mask (N,) bool theo chỉ số gaussian gốc. Sửa tại chỗ:
        luồng sắp xếp dùng chung mảng này."""
        if self.count == 0:
            return
        self._state &= np.uint8(~bit & 255)
        self._state |= mask[self._slot_order].view(np.uint8) * np.uint8(bit)
        self._upload_state()
```

`SplatRenderer.set_selection(self, mask)`

Dòng 246–248

Mô tả trong mã nguồn. mask: (N,) bool theo chỉ số gaussian gốc, gaussian nào đang được chọn.

```
    def set_selection(self, mask):
        """mask: (N,) bool theo chỉ số gaussian gốc, gaussian nào đang được chọn."""
        self._set_state_bit(STATE_SELECTED, mask)
```

`SplatRenderer.set_deleted(self, mask)`

Dòng 250–254

Mô tả trong mã nguồn. mask: (N,) bool theo chỉ số gaussian gốc, gaussian nào đã bị xoá (xoá mềm). Rẻ: chỉ gửi 1 byte / gaussian rồi chọn + sắp xếp lại, không gửi lại dữ liệu gaussian.

```
250     def set_deleted(self, mask):
251         """mask: (N,) bool theo chỉ số gaussian gốc, gaussian nào đã bị xoá (xoá mềm). Rẻ: chỉ gửi
252         1 byte / gaussian rồi chọn + sắp xếp lại, không gửi lại dữ liệu gaussian."""
253         self._set_state_bit(STATE_DELETED, mask)
254         self.last_sort_key = None
```

`SplatRenderer.update_selection(self, indices, value)`

Dòng 256–267

Mô tả trong mã nguồn. Đặt vùng chọn = value cho các gaussian ‘indices‘ (chỉ số gốc). Chỉ gửi lên GPU đoạn bị đổi: nét cọ chạm các gaussian gần nhau, mà gần nhau trong không gian thì cũng gần nhau trên GPU (thứ tự Morton), nên đoạn này thường rất nhỏ.

```
256     def update_selection(self, indices, value):
257         """Đặt vùng chọn = value cho các gaussian `indices` (chỉ số gốc). Chỉ gửi lên GPU đoạn bị
258         đổi: nét cọ chạm các gaussian gần nhau, mà gần nhau trong không gian thì cũng gần nhau
259         trên GPU (thứ tự Morton), nên đoạn này thường rất nhỏ."""
260         if self.count == 0 or len(indices) == 0:
261             return
262         slots = self._slot_of[indices]
263         if value:
264             self._state[slots] |= np.uint8(STATE_SELECTED)
265         else:
266             self._state[slots] &= np.uint8(~STATE_SELECTED & 255)
267         self._upload_state(lo=int(slots.min()), hi=int(slots.max()) + 1)
```

`SplatRenderer.set_transforms(self, matrices)`

Dòng 269–278

Mô tả trong mã nguồn. Ma trận thế giới 4x4 của từng lớp (thứ tự như trong splats). Rất rẻ: kéo / xoay / phóng to lớp chỉ gọi hàm này, không gửi lại dữ liệu gaussian.

Giải thích. Gửi ma trận thế giới 4×4 của mỗi layer làm 4 texel. Rất rẻ: kéo/xoay/phóng layer không phải gửi lại dữ liệu gaussian.

Công thức. Mỗi layer gửi 4 texel = 4 cột của $M = \begin{bmatrix} sA & t \\ 0 & 1 \end{bmatrix}$ (chuyển vị trước khi gửi vì numpy lưu theo hàng).

```
269     def set_transforms(self, matrices):
270         """Ma trận thế giới 4x4 của từng lớp (thứ tự như trong splats). Rất rẻ: kéo / xoay / phóng to
271         lớp chỉ gọi hàm này, không gửi lại dữ liệu gaussian."""
272         self.transforms = np.array(matrices, np.float64).reshape(-1, 4, 4)
273         data = np.ascontiguousarray(self.transforms.transpose(0, 2, 1), np.float32)   # 4 cột / lớp
274         GL.glBindBuffer(GL.GL_TEXTURE_BUFFER, self.transform_buffer)
275         GL.glBufferData(GL.GL_TEXTURE_BUFFER, max(data.nbytes, 64), data, GL.GL_DYNAMIC_DRAW)
276         GL.glBindTexture(GL.GL_TEXTURE_BUFFER, self.transform_texture)
277         GL.glTexBuffer(GL.GL_TEXTURE_BUFFER, GL.GL_RGBA32F, self.transform_buffer)
278         self._order_version += 1
```

`SplatRenderer.set_lighting(self, params)`

Dòng 280–282

Mô tả trong mã nguồn. params: Lighting.params(). Đổi chữ ký -> pass 1 phải tính lại màu của mọi gaussian.

```
280     def set_lighting(self, params):
281         """params: Lighting.params(). Đổi chữ ký -> pass 1 phải tính lại màu của mọi gaussian."""
282         self.lighting = params
```

`SplatRenderer._upload_order(self, slots, ranks)`

Dòng 285–299

```
285     def _upload_order(self, slots, ranks):
286         if len(slots) > self._capacity:
287             # cấp dư một chút để không phải cấp lại mỗi lần số gaussian được chọn nhích lên
288             self._capacity = int(len(slots) * 1.25)
289             for buffer, size in ((self.slot_vbo, 4), (self.rank_vbo, 4), (self.feedback_buffer, FEEDBACK_STRIDE)):
290                 GL.glBindBuffer(GL.GL_ARRAY_BUFFER, buffer)
291                 GL.glBufferData(GL.GL_ARRAY_BUFFER, self._capacity * size, None, GL.GL_DYNAMIC_DRAW)
292             GL.glBindTexture(GL.GL_TEXTURE_BUFFER, self.feedback_texture)
293             GL.glTexBuffer(GL.GL_TEXTURE_BUFFER, GL.GL_RGBA32UI, self.feedback_buffer)
294         for buffer, values in ((self.slot_vbo, slots), (self.rank_vbo, ranks)):
295             if len(values):
296                 GL.glBindBuffer(GL.GL_ARRAY_BUFFER, buffer)
297                 GL.glBufferSubData(GL.GL_ARRAY_BUFFER, 0, values.nbytes, values)
298         self.prep_count, self.draw_count = len(slots), len(ranks)
299         self._order_version += 1
```

`SplatRenderer._update_order(self, model_view, projection)`

Dòng 301–308

```
    def _update_order(self, model_view, projection):
        result = self.sorter.take_result()
        if result is not None:
            self._upload_order(*result)
        key = (model_view.tobytes(), projection.tobytes(), self.budget, self.transforms.tobytes())
        if key != self.last_sort_key:
            self.sorter.request(model_view, projection, self.budget, self.transforms)
            self.last_sort_key = key
```

`SplatRenderer._prepare(self, model_view, projection, width, height)`

Dòng 311–350

Mô tả trong mã nguồn. Pass 1: tính ellipse + màu cho mọi gaussian được chọn, ghi vào feedback_buffer.

Giải thích. Pass 1: bật transform feedback, vẽ `GL_POINTS` cho các gaussian được chọn; `splat_prep.vert` tính ellipse + màu và ghi 32 byte vào `feedback_buffer`. Camera đứng yên thì bỏ qua pass này.

Công thức.

$$\Sigma' = J\,W\,\Sigma\,W^{\top}J^{\top}, \qquad \Sigma = R\,S\,S^{\top}R^{\top}$$

(chi tiết trong shader `splat_prep.vert`, Chương 6).

```
    def _prepare(self, model_view, projection, width, height):
        """Pass 1: tính ellipse + màu cho mọi gaussian được chọn, ghi vào feedback_buffer."""
        degree = self.sh_degree if self.use_sh else 0
        light = self.lighting
        key = (model_view.tobytes(), projection.tobytes(), width, height, degree, self._order_version,
               light['signature'])
        if key == self._prep_key:
            return                      # không có gì đổi: dùng lại kết quả khung hình trước
        self._prep_key = key
        if light['enabled'] and light['shadows']:
            self._render_shadow_map(light)

        s = self.prep_shader
        s.use()
        s.set_mat4('uModelView', model_view)
        s.set_mat4('uProjection', projection)
        s.set_vec2('uViewport', width, height)
        s.set_float('uFocal', height / 2 * projection[1, 1])   # projection[1,1] = 1/tan(fov/2)
        s.set_vec3('uSelectColor', self.SELECT_COLOR)
        rot, t = model_view[:3, :3].astype(np.float64), model_view[:3, 3].astype(np.float64)
        s.set_vec3('uCamPos', -rot.T @ t)                      # vị trí camera trong toạ độ thế giới
        s.set_int('uShDegree', degree)
        s.set_int('uShCount', sh.COEFFS_FOR_DEGREE[self.sh_degree])
        s.set_float('uShScale', self.sh_scale)
        self._set_light_uniforms(s, light)
        for unit, (name, texture) in enumerate((('uData', self.data_texture), ('uState', self.state_texture),
                                                ('uSh', self.sh_texture), ('uLayers', self.transform_texture))):
            GL.glActiveTexture(GL.GL_TEXTURE0 + unit)
            GL.glBindTexture(GL.GL_TEXTURE_BUFFER, texture)
            s.set_int(name, unit)
        GL.glActiveTexture(GL.GL_TEXTURE0)

        GL.glEnable(GL.GL_RASTERIZER_DISCARD)
        GL.glBindBufferBase(GL.GL_TRANSFORM_FEEDBACK_BUFFER, 0, self.feedback_buffer)
        GL.glBindVertexArray(self.prep_vao)
        GL.glBeginTransformFeedback(GL.GL_POINTS)
        GL.glDrawArrays(GL.GL_POINTS, 0, self.prep_count)
        GL.glEndTransformFeedback()
        GL.glBindBufferBase(GL.GL_TRANSFORM_FEEDBACK_BUFFER, 0, 0)
        GL.glDisable(GL.GL_RASTERIZER_DISCARD)
```

`SplatRenderer._set_light_uniforms(self, s, light)`

Dòng 356–387

```
    def _set_light_uniforms(self, s, light):
        s.set_int('uLightingOn', 1 if light['enabled'] else 0)
        if not light['enabled']:
            return
        n = min(len(light['point_range']), self.MAX_POINT_LIGHTS)

        def pad(values):
            values = np.asarray(values, np.float32).reshape(-1, 3)[:n]
            return np.vstack([values, np.zeros((self.MAX_POINT_LIGHTS - n, 3), np.float32)])

        s.set_vec3('uAmbient', light['ambient'])
        s.set_int('uSunOn', 1 if light['sun_on'] else 0)
```

```
        s.set_vec3('uSunDir', light['sun_dir'])
        s.set_vec3('uSunColor', light['sun_color'])
        s.set_float('uSpecular', light['specular'])
        s.set_float('uShininess', light['shininess'])
        s.set_int('uPointCount', n)
        s.set_vec3_array('uPointPos', pad(light['point_pos']))
        s.set_vec3_array('uPointColor', pad(light['point_color']))
        ranges = np.ones(self.MAX_POINT_LIGHTS, np.float32)
        ranges[:n] = light['point_range'][:n]
        s.set_float_array('uPointRange', ranges)
        s.set_int('uShadowOn', 1 if light['shadows'] else 0)
        if light['shadows']:
            s.set_mat4('uLightVP', light['shadow_vp'])
            s.set_float('uShadowTexel', light['shadow_texel'])
            s.set_float('uShadowBias', light['shadow_bias'])
            s.set_float('uShadowInv', 1.0 / SHADOW_SIZE)
            GL.glActiveTexture(GL.GL_TEXTURE0 + self.SHADOW_UNIT)
            GL.glBindTexture(GL.GL_TEXTURE_2D, self._shadow_texture)
            s.set_int('uShadowMap', self.SHADOW_UNIT)
            GL.glActiveTexture(GL.GL_TEXTURE0)
```

`SplatRenderer._create_shadow_map(self)`

Dòng 389–403

```
    def _create_shadow_map(self):
        self._shadow_texture = GL.glGenTextures(1)
        GL.glBindTexture(GL.GL_TEXTURE_2D, self._shadow_texture)
        GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_DEPTH_COMPONENT24, SHADOW_SIZE, SHADOW_SIZE, 0,
                        GL.GL_DEPTH_COMPONENT, GL.GL_FLOAT, None)
        for name, value in ((GL.GL_TEXTURE_MIN_FILTER, GL.GL_NEAREST), (GL.GL_TEXTURE_MAG_FILTER, GL.GL_NEAREST),
                            (GL.GL_TEXTURE_WRAP_S, GL.GL_CLAMP_TO_EDGE), (GL.GL_TEXTURE_WRAP_T, GL.GL_CLAMP_TO_EDGE)):
            GL.glTexParameteri(GL.GL_TEXTURE_2D, name, value)
        GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
        self._shadow_fbo = GL.glGenFramebuffers(1)
        GL.glBindFramebuffer(GL.GL_FRAMEBUFFER, self._shadow_fbo)
        GL.glFramebufferTexture2D(GL.GL_FRAMEBUFFER, GL.GL_DEPTH_ATTACHMENT, GL.GL_TEXTURE_2D,
                                  self._shadow_texture, 0)
        GL.glDrawBuffer(GL.GL_NONE)
        GL.glReadBuffer(GL.GL_NONE)
```

`SplatRenderer._render_shadow_map(self, light)`

Dòng 405–442

Mô tả trong mã nguồn. Shadow mapping: vẽ tâm các gaussian đã chọn vào bản đồ độ sâu nhìn từ mặt trời. Chỉ vẽ lại khi mặt trời hoặc tập gaussian được chọn đổi (không vẽ lại mỗi lần camera nhúc nhích nhẹ).

Giải thích. Bản đồ bóng: vẽ tâm các gaussian ($\alpha \geq 0.3$, chưa xóa) vào bản đồ độ sâu 2048^2 nhìn từ mặt trời; chỉ vẽ lại khi ánh sáng hoặc cảnh đổi.

Công thức.

$$\text{texel} = \frac{2r}{\text{SHADOW_SIZE}}, \qquad \text{SHADOW_SIZE} = 2048$$

```
    def _render_shadow_map(self, light):
        """Shadow mapping: vẽ tâm các gaussian đã chọn vào bản đồ độ sâu nhìn từ mặt trời. Chỉ vẽ lại khi
        mặt trời hoặc tập gaussian được chọn đổi (không vẽ lại mỗi lần camera nhúc nhích nhẹ)."""
        key = (self._order_version, light['signature'])
        if key == self._shadow_key and self._shadow_texture is not None:
            return
        self._shadow_key = key
        previous_fbo = int(GL.glGetIntegerv(GL.GL_DRAW_FRAMEBUFFER_BINDING))
        viewport = [int(v) for v in GL.glGetIntegerv(GL.GL_VIEWPORT)]
        if self._shadow_fbo is None:
            self._create_shadow_map()
        GL.glBindFramebuffer(GL.GL_FRAMEBUFFER, self._shadow_fbo)
        GL.glViewport(0, 0, SHADOW_SIZE, SHADOW_SIZE)
        GL.glEnable(GL.GL_DEPTH_TEST)
        GL.glDepthFunc(GL.GL_LESS)
        GL.glDepthMask(GL.GL_TRUE)
        GL.glDisable(GL.GL_BLEND)
        GL.glEnable(GL.GL_PROGRAM_POINT_SIZE)
        GL.glClear(GL.GL_DEPTH_BUFFER_BIT)

        s = self.shadow_shader
        s.use()
        s.set_mat4('uLightVP', light['shadow_vp'])
        s.set_float('uPointSize', 3.0)
        for unit, name, texture in ((0, 'uData', self.data_texture), (1, 'uState', self.state_texture),
                                    (3, 'uLayers', self.transform_texture)):
            GL.glActiveTexture(GL.GL_TEXTURE0 + unit)
            GL.glBindTexture(GL.GL_TEXTURE_BUFFER, texture)
            s.set_int(name, unit)
        GL.glActiveTexture(GL.GL_TEXTURE0)
```

```
        GL.glBindVertexArray(self.prep_vao)
        GL.glDrawArrays(GL.GL_POINTS, 0, self.prep_count)
        GL.glBindVertexArray(0)

        GL.glDisable(GL.GL_PROGRAM_POINT_SIZE)
        GL.glDisable(GL.GL_DEPTH_TEST)
        GL.glBindFramebuffer(GL.GL_FRAMEBUFFER, previous_fbo)
        GL.glViewport(*viewport)
```

`SplatRenderer._collect_timings(self)`

Dòng 445–455

```
    def _collect_timings(self):
        while self._query_pending:
            query, count = self._query_pending[0]
            if not GL.glGetQueryObjectiv(query, GL.GL_QUERY_RESULT_AVAILABLE):
                break
            self._query_pending.pop(0)
            self._queries.append(query)
            ns = GL.glGetQueryObjectuiv(query, GL.GL_QUERY_RESULT)   # 32 bit: đủ cho < 4 giây
            if count:
                self.gpu_ms = ns / 1e6
                self._samples.append((self.gpu_ms, count))
```

`SplatRenderer.adapt_budget(self, target_ms)`

Dòng 457–470

Mô tả trong mã nguồn. Chỉnh budget để thời gian GPU lúc camera di chuyển (khung hình có chạy pass 1, nặng nhất) gần target_ms. Ước lượng chi phí mỗi gaussian = ms / số gaussian thật sự xử lý ở khung hình đó (không dùng budget: kết quả sắp xếp đến trễ vài khung hình), rồi đổi budget từ từ.

Giải thích. Tự chỉnh ngân sách: wanted = target_ms/cost, bước = clip(wanted/current, 0.75, 1.33)$^{1/2}$ (căn bậc hai làm trơn, đổi tối đa khoảng 15%/lần).

Công thức.

$$\text{cost} = \text{median}\Big(\frac{ms}{count}\Big), \quad \text{wanted} = \frac{\text{target_ms}}{\text{cost}}, \quad \text{step} = \text{clip}\Big(\frac{\text{wanted}}{\text{current}}, 0.75, 1.33\Big)^{1/2}$$

$$\text{budget} \leftarrow \text{clip}(\text{current} \cdot \text{step},\ \text{MIN_BUDGET},\ \text{count})$$

```
    def adapt_budget(self, target_ms):
        """Chỉnh budget để thời gian GPU lúc camera di chuyển (khung hình có chạy pass 1, nặng nhất)
        gần target_ms. Ước lượng chi phí mỗi gaussian = ms / số gaussian thật sự xử lý ở khung hình
        đó (không dùng budget: kết quả sắp xếp đến trễ vài khung hình), rồi đổi budget từ từ."""
        if self.count == 0 or len(self._samples) < 8:
            return
        cost = float(np.median([ms / count for ms, count in self._samples]))
        self._samples.clear()
        current = self.budget or self.count
        wanted = target_ms / max(cost, 1e-9)
        if abs(wanted / current - 1) < 0.1:
            return                                  # đủ gần: giữ nguyên để khỏi dao động
        step = min(max(wanted / current, 0.75), 1.33) ** 0.5   # mỗi lần đổi tối đa ~15%
        self.budget = int(min(max(current * step, MIN_BUDGET), self.count))
```

`SplatRenderer.draw(self, model_view, projection, width, height)`

Dòng 472–490

```
    def draw(self, model_view, projection, width, height):
        if self.count == 0:
            return
        model_view = np.asarray(model_view, np.float32)
        projection = np.asarray(projection, np.float32)
        self._collect_timings()
        self._update_order(model_view, projection)
        if self.draw_count == 0:
            return
        query = self._queries.pop() if self._queries else None
        if query is not None:
            GL.glBeginQuery(GL.GL_TIME_ELAPSED, query)
        prep_key = self._prep_key
        self._prepare(model_view, projection, width, height)
        self._draw_splats()
        if query is not None:
            GL.glEndQuery(GL.GL_TIME_ELAPSED)
            # chỉ khung hình có chạy pass 1 mới dùng để chỉnh budget (0: bỏ qua)
            self._query_pending.append((query, self.prep_count if self._prep_key is not prep_key else 0))
```

`SplatRenderer._draw_splats(self)`

Dòng 492–504

Mô tả trong mã nguồn. Pass 2: vẽ hình vuông xa -> gần.

Giải thích. Pass 2: `glDrawArraysInstanced` vẽ mỗi gaussian một hình vuông (4 đỉnh) theo thứ tự xa → gần; trộn `GL_ONE, GL_ONE_MINUS_SRC_ALPHA` (premultiplied), tắt depth test.

Công thức.

$$C_i = c_i\, a_i + (1 - a_i)\, C_{i-1} \qquad \Longleftrightarrow \qquad C = \text{src.rgb} + (1 - \text{src.a})\, C_{dst}$$

Không giao hoán, nên phải vẽ xa → gần; tắt depth test.

```
492     def _draw_splats(self):
493         """Pass 2: vẽ hình vuông xa -> gần."""
494         s = self.shader
495         s.use()
496         GL.glActiveTexture(GL.GL_TEXTURE0)
497         GL.glBindTexture(GL.GL_TEXTURE_BUFFER, self.feedback_texture)
498         s.set_int('uPrep', 0)
499         GL.glDisable(GL.GL_DEPTH_TEST)
500         GL.glEnable(GL.GL_BLEND)
501         GL.glBlendFunc(GL.GL_ONE, GL.GL_ONE_MINUS_SRC_ALPHA)
502         GL.glBindVertexArray(self.vao)
503         GL.glDrawArraysInstanced(GL.GL_TRIANGLE_STRIP, 0, 4, self.draw_count)
504         GL.glBindVertexArray(0)
```

4.5. `splat_editor/render/viewport.py`

212 dòng

Vai trò. `Viewport` ghép camera, lưới sàn, bộ vẽ gaussian, đèn và nhân vật vào một hình chữ nhật của cửa sổ. Hỗ trợ vẽ ở độ phân giải thấp hơn (`render_scale`) qua framebuffer riêng và các mức chất lượng (`QUALITY_PRESETS`).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
1  """Khung nhìn 3D: camera + lưới sàn + gaussian, vẽ vào một vùng chữ nhật của cửa sổ.
2
3  Để chạy mượt trên máy yếu, cảnh có thể được vẽ ở độ phân giải thấp hơn (render_scale) vào một
4  framebuffer riêng rồi phóng lên khung nhìn. Khi camera đang di chuyển có thể hạ thêm
5  (motion_scale): lúc chuyển động mắt khó thấy mờ, dừng lại thì vẽ lại nét ngay.
6  """
7
8  """
9  viewport = Viewport()
10 viewport.set_quality("Auto")            # Auto / Low / Medium / High / Ultra
11 viewport.frame(positions)               # đưa camera về nhìn toàn cảnh
12
13 # Mỗi khung hình
14 viewport.sync(editor)                   # nếu Editor đổi thì gửi lại GPU
15 viewport.render(rect, window.size, window.framebuffer_size)
16
17 vp = viewport.view_projection(rect)    # dùng cho cọ chọn
18 """
19
```

Khai báo mức module (dòng 20–29)

```
20 from typing import NamedTuple
21
22 import numpy as np
23 from OpenGL import GL
24
25 from .avatar_renderer import AvatarRenderer
26 from .camera import OrbitCamera
27 from .grid_renderer import GridRenderer
28 from .light_renderer import LightRenderer
29 from .splat_renderer import SplatRenderer
```

`class QualityPreset(NamedTuple)`

Dòng 32–36

```
32 class QualityPreset(NamedTuple):
33     budget: int | None       # số gaussian tối đa mỗi khung hình (None: không giới hạn)
34     render_scale: float      # tỉ lệ độ phân giải khi đứng yên
35     motion_scale: float      # tỉ lệ độ phân giải khi camera đang di chuyển
36     auto: bool = False       # tự chỉnh budget theo thời gian GPU đo được để đạt target_fps
```

Khai báo mức module (dòng 39–47)

```
QUALITY_PRESETS = {
    "Auto": QualityPreset(1_500_000, 1.0, 0.6, auto=True),
    "Low": QualityPreset(1_000_000, 0.75, 0.5),
    "Medium": QualityPreset(2_000_000, 1.0, 0.6),
    "High": QualityPreset(4_000_000, 1.0, 0.75),
    "Ultra": QualityPreset(None, 1.0, 1.0),
}
DEFAULT_QUALITY = "Auto"
GPU_SHARE = 0.7    # phần thời gian mỗi khung hình dành cho gaussian (còn lại: giao diện, CPU, ...)
```

`class Viewport`

Dòng 50–212

Các phương thức: `__init__`, `sync`, `set_quality`, `view_projection`, `frame`, `render`, `sky`, `_draw_scene`, `_bind_offscreen`.

```
class Viewport:
    MOTION_HOLD_FRAMES = 6
```

`Viewport.__init__(self)`

Dòng 53–83

```
    def __init__(self):
        self.camera = OrbitCamera()
        self.splat_renderer = SplatRenderer()
        self.grid = GridRenderer()
        self.light_renderer = LightRenderer()
        self.avatar_renderer = AvatarRenderer()
        self.avatar = None            # (6 ma trận model, độ sáng RGB) của nhân vật người chơi, None: không vẽ
        self.occluder = None          # Physics: dùng bản đồ va chạm để cảnh che mặt trời
        self.lighting = None         # core.lighting.Lighting (gắn bởi Application)
        self.unit = 1.0              # chiều cao người chơi: thước đo kích thước vật thể
        self.clock = 0.0             # giây, cho hạt chuyển động
        self.show_grid = True
        self.background = [0.13, 0.13, 0.15]
        self.render_scale = 1.0
        self.motion_scale = 1.0
        self.quality = None
        self.auto_budget = False
        self.target_fps = 60
        self.set_quality(DEFAULT_QUALITY)
        self._fbo = GL.glGenFramebuffers(1)
        self._fbo_texture = GL.glGenTextures(1)
        self._fbo_depth = GL.glGenRenderbuffers(1)     # độ sâu riêng cho nhân vật (gaussian không ghi độ sâu)
        self._fbo_size = None
        self._last_view = None
        self._still_frames = self.MOTION_HOLD_FRAMES
        self.moving = False
        # phiên bản dữ liệu đã gửi lên GPU (so với Editor để biết khi nào cần gửi lại)
        self._geometry_version = None
        self._deleted_version = None
        self._selection_version = None
        self._transform_version = None
```

`Viewport.sync(self, editor)`

Dòng 85–108

Mô tả trong mã nguồn. Gửi lại gaussian / vùng chọn lên GPU nếu Editor đã thay đổi.

Giải thích. So sánh số phiên bản của Editor với số đã gửi: `geometry` → gửi lại dữ liệu; `deleted` → gửi mảng cờ; `selection` → gửi phần thay đổi; `transform` → gửi ma trận.

```
    def sync(self, editor):
        """Gửi lại gaussian / vùng chọn lên GPU nếu Editor đã thay đổi."""
        if self._geometry_version != editor.geometry_version:
            self.splat_renderer.set_splats(editor.splats, editor.lod)
            self._geometry_version = editor.geometry_version
            self._deleted_version = None
            self._selection_version = None
            self._transform_version = None
        if self._deleted_version != editor.deleted_version:
            # xoá / hoàn tác (xoá mềm): chỉ gửi lại cờ đã xoá, không gửi lại dữ liệu gaussian
            self.splat_renderer.set_deleted(editor.scene.deleted_mask())
            self._deleted_version = editor.deleted_version
        if self._transform_version != editor.transform_version:
            # kéo / xoay / phóng to lớp hoặc lật trục: chỉ gửi ma trận, không gửi lại dữ liệu
            self.splat_renderer.set_transforms(editor.layer_matrices())
```

```
            self._transform_version = editor.transform_version
        if self._selection_version != editor.selection_version:
            changes = editor.selection_changes_since(self._selection_version)
            if changes is None:
                self.splat_renderer.set_selection(editor.scene.selection_mask())
            else:
                for indices, value in changes:      # chỉ các nét cọ mới: gửi phần bị đổi
                    self.splat_renderer.update_selection(indices, value)
            self._selection_version = editor.selection_version
```

`Viewport.set_quality(self, name)`

Dòng 110–116

```
    def set_quality(self, name):
        preset = QUALITY_PRESETS[name]
        self.quality = name
        self.splat_renderer.budget = preset.budget
        self.render_scale = preset.render_scale
        self.motion_scale = preset.motion_scale
        self.auto_budget = preset.auto
```

`Viewport.view_projection(self, rect)`

Dòng 118–119

```
    def view_projection(self, rect):
        return self.camera.projection_matrix(rect.aspect) @ self.camera.view_matrix()
```

`Viewport.frame(self, positions)`

Dòng 121–126

Mô tả trong mã nguồn. Đưa camera về nhìn toàn bộ các điểm ‘positions‘ (toạ độ thế giới; None: đặt lại camera).

```
    def frame(self, positions):
        """Đưa camera về nhìn toàn bộ các điểm `positions` (toạ độ thế giới; None: đặt lại camera)."""
        if positions is None:
            self.camera.reset()
        else:
            self.camera.frame(positions)
```

`Viewport.render(self, rect, window_size, framebuffer_size)`

Dòng 128–170

Mô tả trong mã nguồn. rect: vùng 3D theo toạ độ cửa sổ (gốc trên-trái), được đổi sang pixel framebuffer.

Giải thích. Vẽ một khung hình vào `rect`: chọn độ phân giải theo trạng thái chuyển động, bind framebuffer riêng khi `render_scale<1`, vẽ nền, mặt trời, gaussian, lưới, đèn, nhân vật rồi phóng lên khung nhìn.

Công thức.

$$x_{px} = x_0 + \frac{n_x + 1}{2} W, \qquad y_{px} = y_0 + \frac{1 - n_y}{2} H$$

```
    def render(self, rect, window_size, framebuffer_size):
        """rect: vùng 3D theo toạ độ cửa sổ (gốc trên-trái), được đổi sang pixel framebuffer."""
        fb_w, fb_h = framebuffer_size
        GL.glViewport(0, 0, fb_w, fb_h)
        GL.glClearColor(*self.sky(), 1.0)
        GL.glClear(GL.GL_COLOR_BUFFER_BIT)
        if rect.is_empty or window_size[0] <= 0 or window_size[1] <= 0:
            return

        # màn hình HiDPI: framebuffer có thể lớn hơn kích thước cửa sổ
        sx, sy = fb_w / window_size[0], fb_h / window_size[1]
        x, w = int(rect.left * sx), int(rect.width * sx)
        h = int(rect.height * sy)
        y = fb_h - int((rect.top + rect.height) * sy)        # OpenGL tính y từ dưới lên
        if w <= 0 or h <= 0:
            return

        view = self.camera.view_matrix().astype(np.float32)
        projection = self.camera.projection_matrix(w / h)
        # coi là đang di chuyển thêm vài khung hình sau lần đổi cuối (chuột không gửi sự kiện
        # đều mỗi khung hình), tránh độ phân giải nhảy qua lại liên tục
        changed = self._last_view is None or not np.array_equal(view, self._last_view)
```

```
        self._still_frames = 0 if changed else self._still_frames + 1
        self.moving = self._still_frames < self.MOTION_HOLD_FRAMES
        self._last_view = view
        scale = min(self.render_scale, self.motion_scale) if self.moving else self.render_scale

        if scale >= 0.999:
            GL.glViewport(x, y, w, h)
            self._draw_scene(view, projection, w, h)
        else:
            # vẽ vào framebuffer nhỏ hơn rồi phóng lên khung nhìn (lọc tuyến tính)
            sw, sh = max(int(w * scale), 1), max(int(h * scale), 1)
            self._bind_offscreen(sw, sh)
            GL.glViewport(0, 0, sw, sh)
            GL.glClearColor(*self.sky(), 1.0)
            GL.glClear(GL.GL_COLOR_BUFFER_BIT)
            self._draw_scene(view, projection, sw, sh)
            GL.glBindFramebuffer(GL.GL_READ_FRAMEBUFFER, self._fbo)
            GL.glBindFramebuffer(GL.GL_DRAW_FRAMEBUFFER, 0)
            GL.glBlitFramebuffer(0, 0, sw, sh, x, y, x + w, y + h, GL.GL_COLOR_BUFFER_BIT, GL.GL_LINEAR)
            GL.glBindFramebuffer(GL.GL_FRAMEBUFFER, 0)
        GL.glViewport(0, 0, fb_w, fb_h)
```

`Viewport.sky(self)`

Dòng 172–177

Mô tả trong mã nguồn. Màu nền: đổi theo giờ trong ngày khi có mặt trời, ngược lại là màu người dùng chọn.

```
    def sky(self):
        """Màu nền: đổi theo giờ trong ngày khi có mặt trời, ngược lại là màu người dùng chọn."""
        light = self.lighting
        if light is not None and light.enabled and light.sun_on and light.sky_follows:
            return light.sky_color()
        return self.background
```

`Viewport._draw_scene(self, view, projection, w, h)`

Dòng 179–193

```
    def _draw_scene(self, view, projection, w, h):
        if self.show_grid:
            self.grid.draw(view.astype(np.float64), projection, self.camera.distance)
        if self.lighting is not None:
            self.splat_renderer.set_lighting(self.lighting.params())
            self.light_renderer.draw_sun(view, projection, self.lighting)     # xa nhất: vẽ trước gaussian
        self.splat_renderer.draw(view, projection, w, h)
        if self.lighting is not None:
            self.light_renderer.draw(view, projection, self.lighting, self.clock, self.unit,
                                     self.occluder)
        if self.avatar is not None and self.camera.view_mode:
            matrices, brightness = self.avatar
            self.avatar_renderer.draw(view, projection, matrices, brightness)
        if self.auto_budget:
            self.splat_renderer.adapt_budget(GPU_SHARE * 1000.0 / self.target_fps)
```

`Viewport._bind_offscreen(self, w, h)`

Dòng 195–212

```
    def _bind_offscreen(self, w, h):
        GL.glBindFramebuffer(GL.GL_FRAMEBUFFER, self._fbo)
        if self._fbo_size == (w, h):
            return
        # cấp lại texture khi kích thước đổi (thường chỉ vài cỡ: lúc đứng yên / lúc di chuyển)
        GL.glBindTexture(GL.GL_TEXTURE_2D, self._fbo_texture)
        GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA8, w, h, 0, GL.GL_RGBA, GL.GL_UNSIGNED_BYTE, None)
        GL.glTexParameteri(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MIN_FILTER, GL.GL_LINEAR)
        GL.glTexParameteri(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MAG_FILTER, GL.GL_LINEAR)
        GL.glFramebufferTexture2D(GL.GL_FRAMEBUFFER, GL.GL_COLOR_ATTACHMENT0, GL.GL_TEXTURE_2D,
                                  self._fbo_texture, 0)
        GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
        GL.glBindRenderbuffer(GL.GL_RENDERBUFFER, self._fbo_depth)
        GL.glRenderbufferStorage(GL.GL_RENDERBUFFER, GL.GL_DEPTH_COMPONENT24, w, h)
        GL.glFramebufferRenderbuffer(GL.GL_FRAMEBUFFER, GL.GL_DEPTH_ATTACHMENT, GL.GL_RENDERBUFFER,
                                     self._fbo_depth)
        GL.glBindRenderbuffer(GL.GL_RENDERBUFFER, 0)
        self._fbo_size = (w, h)
```

4.6. `splat_editor/render/grid_renderer.py`

51 dòng

Vai trò. Lưới sàn vô hạn: một tam giác phủ toàn màn hình, fragment shader bắn tia từ camera tới mặt phẳng $y = h$ và tô đường lưới.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Lưới sàn vô hạn trên mặt phẳng y = const (giống SuperSplat / Blender).

Vẽ một tam giác phủ kín màn hình. Với mỗi pixel, fragment shader bắn một tia từ camera,
tìm giao điểm với mặt phẳng sàn rồi tô đường lưới tại đó. Kích thước ô lưới tự đổi
theo khoảng cách camera (1, 10, 100 ...), chuyển mượt giữa các mức.
Trục X tô đỏ, trục Z tô xanh dương.
"""

"""
# Sau khi đã có ngữ cảnh GL
grid = GridRenderer()
grid.draw(view, projection, camera_distance=cam.distance)   # gọi trong vòng lặp vẽ, trước khi vẽ gaussian
"""

```

Khai báo mức module (dòng 15–19)

```
import math

from OpenGL import GL

from .shader import ShaderProgram
```

`class GridRenderer`

Dòng 22–51

Các phương thức: `__init__`, `draw`.

```
class GridRenderer:
```

`GridRenderer.__init__(self)`

Dòng 23–26

```
    def __init__(self):
        self.shader = ShaderProgram.load("grid")
        self.vao = GL.glGenVertexArrays(1)
        self.height = 0.0
```

`GridRenderer.draw(self, view, projection, camera_distance)`

Dòng 28–51

```
    def draw(self, view, projection, camera_distance):
        cam_rot = view[:3, :3].T                       # nghịch đảo của ma trận xoay = chuyển vị
        cam_pos = -cam_rot @ view[:3, 3]
        # kích thước ô theo độ cao camera so với sàn và khoảng cách tới target
        size = max(abs(cam_pos[1] - self.height), camera_distance * 0.5, 1e-4)
        level = math.log10(size)
        cell = 10.0 ** (math.floor(level) - 1)

        s = self.shader
        s.use()
        s.set_mat3('uCamRot', cam_rot)
        s.set_vec3('uCamPos', cam_pos)
        s.set_vec2('uProjScale', projection[0, 0], projection[1, 1])
        s.set_float('uHeight', self.height)
        s.set_float('uCell', cell)
        s.set_float('uBlend', level - math.floor(level))
        s.set_float('uFade', size * 4.0)

        GL.glDisable(GL.GL_DEPTH_TEST)
        GL.glEnable(GL.GL_BLEND)
        GL.glBlendFunc(GL.GL_ONE, GL.GL_ONE_MINUS_SRC_ALPHA)
        GL.glBindVertexArray(self.vao)
        GL.glDrawArrays(GL.GL_TRIANGLES, 0, 3)
        GL.glBindVertexArray(0)
```

4.7. `splat_editor/render/light_renderer.py`

114 dòng

Vai trò. Vẽ mặt trời, các khối đèn (quả cầu phát sáng) và đom đóm (hạt) quanh đèn.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Vẽ mặt trời, các khối đèn (quả cầu phát sáng) và đom đóm bay quanh đèn (hạt, T5).

Vẽ sau các gaussian, không thử độ sâu (gaussian không ghi độ sâu): luôn thấy được để dễ đặt / chỉnh đèn.
Đom đóm tính vị trí hoàn toàn trong vertex shader theo thời gian (không có dữ liệu hạt trên CPU).
"""

"""
renderer = LightRenderer()                                  # cần ngữ cảnh GL
renderer.draw(view, projection, lighting, time_seconds)         # sau khi vẽ gaussian
"""

```

Khai báo mức module (dòng 12–17)

```
import numpy as np
from OpenGL import GL

from .shader import ShaderProgram

FIREFLIES_PER_LIGHT = 24
```

`class LightRenderer`

Dòng 20–114

Các phương thức: `__init__`, `draw_sun`, `draw`, `_sphere`.

```
class LightRenderer:
```

`LightRenderer.__init__(self)`

Dòng 21–32

```
    def __init__(self):
        self.gizmo = ShaderProgram.load("light_gizmo")
        self.firefly = ShaderProgram.load("firefly")
        corners = np.array([-1, -1, 1, -1, -1, 1, 1, 1], np.float32)       # triangle strip
        self.vbo = GL.glGenBuffers(1)
        self.vao = GL.glGenVertexArrays(1)
        GL.glBindVertexArray(self.vao)
        GL.glBindBuffer(GL.GL_ARRAY_BUFFER, self.vbo)
        GL.glBufferData(GL.GL_ARRAY_BUFFER, corners.nbytes, corners, GL.GL_STATIC_DRAW)
        GL.glEnableVertexAttribArray(0)
        GL.glVertexAttribPointer(0, 2, GL.GL_FLOAT, GL.GL_FALSE, 0, None)
        GL.glBindVertexArray(0)
```

`LightRenderer.draw_sun(self, view, projection, lighting)`

Dòng 34–52

Mô tả trong mã nguồn. Vẽ mặt trời TRƯỚC các gaussian: nó là phần tử xa nhất nên các gaussian (đã sắp xa -> gần) tự blend đè lên nó, không cần kiểm tra che khuất.

```
    def draw_sun(self, view, projection, lighting):
        """Vẽ mặt trời TRƯỚC các gaussian: nó là phần tử xa nhất nên các gaussian (đã sắp xa -> gần)
        tự blend đè lên nó, không cần kiểm tra che khuất."""
        if not lighting.enabled or not lighting.sun_on:
            return
        direction = lighting.sun_direction()
        above = float(np.clip((direction[1] + 0.1) / 0.2, 0.0, 1.0))
        if above <= 0:
            return
        view = np.asarray(view, np.float32)
        projection = np.asarray(projection, np.float32)
        GL.glDisable(GL.GL_DEPTH_TEST)
        GL.glEnable(GL.GL_BLEND)
        GL.glBlendFunc(GL.GL_ONE, GL.GL_ONE_MINUS_SRC_ALPHA)
        GL.glBindVertexArray(self.vao)
        center = lighting.center + direction * lighting.radius * 6.0
        self._sphere(view, projection, center, lighting.radius * 0.45, 2.6,
                     np.array(lighting.sun_color_for_gizmo()), above)
        GL.glBindVertexArray(0)
```

`LightRenderer.draw(self, view, projection, lighting, time, unit, physics=None)`

Dòng 54–102

Mô tả trong mã nguồn. unit: chiều cao người chơi (đơn vị cảnh) để chọn kích thước quả cầu.

```
    def draw(self, view, projection, lighting, time, unit, physics=None):
        """unit: chiều cao người chơi (đơn vị cảnh) để chọn kích thước quả cầu."""
        if not lighting.enabled:
            return
        view = np.asarray(view, np.float32)
        projection = np.asarray(projection, np.float32)
        cam_pos = -view[:3, :3].T @ view[:3, 3]
        GL.glDisable(GL.GL_DEPTH_TEST)
        GL.glEnable(GL.GL_BLEND)
        GL.glBlendFunc(GL.GL_ONE, GL.GL_ONE_MINUS_SRC_ALPHA)
        GL.glBindVertexArray(self.vao)

        visible = {}
        for light in lighting.lights:
            base = np.array(light.color)
            glow = 1.0 if light.on else 0.0
            distance = float(np.linalg.norm(light.position - cam_pos))
            radius = max(unit * 0.12, distance * 0.012)       # luôn đủ lớn để nhìn thấy
            seen = 1.0
            if physics is not None:       # khối đèn cũng bị cảnh che (bỏ qua vài ô cuối: đèn nằm sát sàn)
                right, up = view[0, :3], view[1, :3]
                targets = [light.position, light.position + right * radius, light.position - right * radius,
                           light.position + up * radius, light.position - up * radius]
                seen = physics.visibility(cam_pos, targets, skip_end=1.5)
            visible[id(light)] = seen > 0
            if seen > 0:
                self._sphere(view, projection, light.position, radius, 3.2, base / base.max(),
                             (0.15 + 0.85 * glow) * seen)
                if lighting.lights.index(light) == lighting.picked:      # viền sáng quanh đèn đang chọn
                    self._sphere(view, projection, light.position, radius * 1.9, 1.6,
                                 np.array([0.4, 0.8, 1.0]), 0.55 * seen)

        if lighting.fireflies:
            GL.glEnable(GL.GL_PROGRAM_POINT_SIZE)
            s = self.firefly
            s.use()
            s.set_mat4('uView', view)
            s.set_mat4('uProj', projection)
            s.set_float('uTime', time)
            s.set_float('uPixel', 4.0)
            for light in lighting.lights:
                if not light.on or not visible.get(id(light), True):
                    continue
                s.set_vec3('uCenter', light.position)
                s.set_float('uRadius', light.range * 0.25)
                s.set_vec3('uColor', np.array(light.color) / max(light.color))
                GL.glDrawArrays(GL.GL_POINTS, 0, FIREFLIES_PER_LIGHT)
            GL.glDisable(GL.GL_PROGRAM_POINT_SIZE)
        GL.glBindVertexArray(0)
```

`LightRenderer._sphere(self, view, projection, center, radius, halo, color, brightness)`

Dòng 104–114

```
    def _sphere(self, view, projection, center, radius, halo, color, brightness):
        s = self.gizmo
        s.use()
        s.set_mat4('uView', view)
        s.set_mat4('uProj', projection)
        s.set_vec3('uCenter', center)
        s.set_float('uRadius', radius)
        s.set_float('uHalo', halo)
        s.set_vec3('uColor', color)
        s.set_float('uBrightness', brightness)
        GL.glDrawArrays(GL.GL_TRIANGLE_STRIP, 0, 4)
```

4.8. `splat_editor/render/avatar_renderer.py`

190 dòng

Vai trò. Vẽ nhân vật người chơi từ 6 hộp (đầu, thân, hai tay, hai chân) với skin 64×64. Có sẵn bộ giải mã PNG tự viết (`load_png`) nên không cần thư viện ảnh.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Vẽ nhân vật người chơi (zombie skin 64x64) khi xem góc nhìn thứ ba (F5).

Port từ voxel-engine (PlayerMesh.java): sáu hộp (đầu, thân, hai tay, hai chân), mỗi hộp là một khối riêng
để xoay quanh khớp, cộng lớp áo ngoài phồng nhẹ (mũ, áo, tay áo, quần) bỏ điểm ảnh trong suốt.
Mesh dựng theo điểm ảnh skin; kích thước thật do ma trận model (có scale) quyết định.
Gaussian không ghi độ sâu nên nhân vật tự có độ sâu riêng: xoá độ sâu rồi mới vẽ.
"""

"""
renderer = AvatarRenderer()                                  # cần ngữ cảnh GL
```

```
renderer.draw(view, projection, limb_matrices, brightness, light_dir)
"""

```

Cơ sở toán: Texture

Các công thức dưới đây giải thích cách dựng UV từ skin, hộp nhân vật, shader avatar (`avatar.vert`, `avatar.frag`) và bộ giải mã PNG `load_png` trong file này.

- **Skin** 64×64 kiểu chữ thập Minecraft, `avatar_renderer._uv_region(face, ox, oy, w, h, d)` (w, h, d là rộng, cao, sâu của hộp). Mặt trước ($ox + d$, $oy + d$); sau ($ox + 2d + w$, $oy + d$); phải (ox, $oy + d$, $d \times h$); trái ($ox + d + w$, $oy + d$); trên ($ox + d$, oy); dưới ($ox + d + w$, oy). UV = pixel/64. Bốn đỉnh mỗi mặt: (r_x, r_y), $(r_x + r_w, r_y)$, $(r_x + r_w, r_y + r_h)$, $(r_x, r_y + r_h)$.
- Hộp: nửa kích thước $h_x = w/2 +$ inflate, …(lớp áo "inflate" phồng ra để tránh z-fighting). Đỉnh = tâm $+(\pm 1)\cdot$nửa kích thước; pháp tuyến là 6 hướng trục.
- **Avatar shader:** shade $=$ bright $\cdot\ (0.62 + 0.38 \max(n \cdot l, 0))$ (ambient 0.62 + Lambert 0.38); $n_{\text{world}} =$ normalize(mat3(M) n); bỏ texel $\alpha < 0.5$ (lớp áo trong suốt).
- **Giải mã PNG tự viết** (`load_png`): zlib + bộ lọc dòng, với a là trái, b là trên, c là trên-trái và mọi phép cộng đều mod 256: 1 Sub: $x + a$; 2 Up: $x + b$; 3 Average: $x + \lfloor (a + b)/2 \rfloor$; 4 **Paeth**: $p = a + b - c$, chọn trong $\{a, b, c\}$ gần p nhất ($p_a = |p - a|$, p_b, p_c; ưu tiên $a \le b \le c$).
- Texel buffer: dữ liệu gaussian đọc bằng `texelFetch` (số nguyên, không nội suy).

Khai báo mức module (dòng 14–49)

```
import ctypes
import struct
import zlib
from pathlib import Path

import numpy as np
from OpenGL import GL

from ..core.avatar_pose import LIMBS
from .shader import ShaderProgram

SKIN_PATH = Path(__file__).parent / "textures" / "skinzom.png"
LIMB_INFLATE = 0.25      # lớp áo của thân / tay / chân phồng hơn lớp nền nửa điểm ảnh (như Minecraft)
HAT_INFLATE = 0.5        # mũ trên đầu phồng gấp đôi

# 4 góc mỗi mặt (dấu +/-1 trên x, y, z), thứ tự: trái-trên, phải-trên, phải-dưới, trái-dưới của ô skin.
# Mặt: trước (+Z), sau (-Z), phải (+X), trái (-X), trên (+Y), dưới (-Y).
_FACES = (
    ((1, 1, 1), (-1, 1, 1), (-1, -1, 1), (1, -1, 1)),
    ((-1, 1, -1), (1, 1, -1), (1, -1, -1), (-1, -1, -1)),
    ((1, 1, 1), (1, 1, -1), (1, -1, -1), (1, -1, 1)),
    ((-1, 1, -1), (-1, 1, 1), (-1, -1, 1), (-1, -1, -1)),
    ((1, 1, -1), (-1, 1, -1), (-1, 1, 1), (1, 1, 1)),
    ((1, -1, 1), (-1, -1, 1), (-1, -1, -1), (1, -1, -1)),
)
_NORMALS = ((0, 0, 1), (0, 0, -1), (1, 0, 0), (-1, 0, 0), (0, 1, 0), (0, -1, 0))

# uv nền, tâm (x, y, z), kích thước (w, h, d), uv lớp áo, độ phồng
_LIMB_SPECS = {
    "legLeft": ((16, 48), (-2, 6, 0), (4, 12, 4), (0, 48), LIMB_INFLATE),
    "legRight": ((0, 16), (2, 6, 0), (4, 12, 4), (0, 32), LIMB_INFLATE),
    "body": ((16, 16), (0, 18, 0), (8, 12, 4), (16, 32), LIMB_INFLATE),
    "armLeft": ((32, 48), (-6, 18, 0), (4, 12, 4), (48, 48), LIMB_INFLATE),
    "armRight": ((40, 16), (6, 18, 0), (4, 12, 4), (40, 32), LIMB_INFLATE),
    "head": ((0, 0), (0, 28, 0), (8, 8, 8), (32, 0), HAT_INFLATE),
}
```

`load_png(path)`

Dòng 52–102

Mô tả trong mã nguồn. Đọc PNG 8 bit RGB / RGBA không xen kẽ -> mảng (h, w, 4) uint8 (không cần thư viện ảnh).

Giải thích. Giải mã PNG 8 bit RGB/RGBA không xen kẽ bằng `zlib` và 5 bộ lọc dòng (None, Sub, Up, Average, Paeth), không cần thư viện ảnh.

Công thức. Bộ lọc dòng PNG (a trái, b trên, c trên-trái, mọi phép cộng mod256):

$$\text{Sub}: x + a, \quad \text{Up}: x + b, \quad \text{Avg}: x + \left\lfloor \frac{a+b}{2} \right\rfloor, \quad \text{Paeth}: x + \arg\min_{u \in \{a,b,c\}} |a + b - c - u|$$

```
def load_png(path):
    """Đọc PNG 8 bit RGB / RGBA không xen kẽ -> mảng (h, w, 4) uint8 (không cần thư viện ảnh)."""
```

```
    data = Path(path).read_bytes()
    if data[:8] != b"\x89PNG\r\n\x1a\n":
        raise ValueError(f"{path}: không phải file PNG")
    pos, idat, header = 8, [], None
    while pos < len(data):
        length, kind = struct.unpack(">I4s", data[pos:pos + 8])
        body = data[pos + 8:pos + 8 + length]
        if kind == b"IHDR":
            header = struct.unpack(">IIBBBBB", body)
        elif kind == b"IDAT":
            idat.append(body)
        pos += 12 + length
    width, height, depth, color, _, _, interlace = header
    if depth != 8 or color not in (2, 6) or interlace:
        raise ValueError(f"{path}: chỉ hỗ trợ PNG 8 bit RGB / RGBA không xen kẽ")
    channels = 4 if color == 6 else 3
    stride = width * channels
    raw = zlib.decompress(b"".join(idat))
    rows = np.zeros((height, stride), np.int32)
    previous = np.zeros(stride, np.int32)
    for y in range(height):
        base = y * (stride + 1)
        kind = raw[base]
        line = np.frombuffer(raw, np.uint8, stride, base + 1).astype(np.int32)
        if kind == 0:
            current = line
        elif kind == 2:
            current = (line + previous) & 255
        else:                                   # 1 (Sub), 3 (Average), 4 (Paeth): phụ thuộc điểm ảnh bên trái
            current = line.copy()
            for i in range(stride):
                left = int(current[i - channels]) if i >= channels else 0
                up = int(previous[i])
                if kind == 1:
                    predictor = left
                elif kind == 3:
                    predictor = (left + up) >> 1
                else:
                    corner = int(previous[i - channels]) if i >= channels else 0
                    p = left + up - corner
                    pa, pb, pc = abs(p - left), abs(p - up), abs(p - corner)
                    predictor = left if pa <= pb and pa <= pc else up if pb <= pc else corner
                current[i] = (int(line[i]) + predictor) & 255
        rows[y] = current
        previous = current
    image = rows.reshape(height, width, channels).astype(np.uint8)
    if channels == 3:
        image = np.concatenate([image, np.full((height, width, 1), 255, np.uint8)], axis=2)
    return image
```

`_uv_region(face, ox, oy, w, h, d)`

Dòng 105–108

Mô tả trong mã nguồn. Ô skin của một mặt hộp theo kiểu trải chữ thập của Minecraft: (x, y, rộng, cao).

Giải thích. Ô skin của một mặt hộp theo kiểu trải chữ thập của Minecraft: $(x, y, \text{rộng}, \text{cao})$.

Công thức.

$$\text{UV} = \frac{\text{pixel}}{64}, \qquad \text{trước } (o_x + d,\, o_y + d),\ \text{sau } (o_x + 2d + w,\, o_y + d),\ \text{trên } (o_x + d,\, o_y)$$

```
def _uv_region(face, ox, oy, w, h, d):
    """Ô skin của một mặt hộp theo kiểu trải chữ thập của Minecraft: (x, y, rộng, cao)."""
    return ((ox + d, oy + d, w, h), (ox + 2 * d + w, oy + d, w, h), (ox, oy + d, d, h),
            (ox + d + w, oy + d, d, h), (ox + d, oy, w, d), (ox + d + w, oy, w, d))[face]
```

`_box(ox, oy, centre, size, inflate)`

Dòng 111–121

Mô tả trong mã nguồn. Đỉnh (x, y, z, nx, ny, nz, u, v) của một hộp: 6 mặt x 4 đỉnh.

```
def _box(ox, oy, centre, size, inflate):
    """Đỉnh (x, y, z, nx, ny, nz, u, v) của một hộp: 6 mặt x 4 đỉnh."""
    (cx, cy, cz), (w, h, d) = centre, size
    hx, hy, hz = w / 2 + inflate, h / 2 + inflate, d / 2 + inflate
    vertices = []
    for face in range(6):
        rx, ry, rw, rh = _uv_region(face, ox, oy, w, h, d)
        uvs = ((rx, ry), (rx + rw, ry), (rx + rw, ry + rh), (rx, ry + rh))
        for (sx, sy, sz), (u, v) in zip(_FACES[face], uvs):
            vertices.append((cx + sx * hx, cy + sy * hy, cz + sz * hz, *_NORMALS[face], u / 64.0, v / 64.0))
    return vertices
```

`build_mesh()`

Dòng 124–137

Mô tả trong mã nguồn. (đỉnh float32 (n, 8), chỉ số uint32, [(offset byte, số chỉ số)] cho từng khối theo LIMBS).

```
def build_mesh():
    """(đỉnh float32 (n, 8), chỉ số uint32, [(offset byte, số chỉ số)] cho từng khối theo LIMBS)."""
    vertices, indices, ranges = [], [], []
    for name in LIMBS:
        base_uv, centre, size, overlay_uv, inflate = _LIMB_SPECS[name]
        start = len(indices)
        for uv, grow in ((base_uv, 0.0), (overlay_uv, inflate)):
            first = len(vertices)
            vertices += _box(uv[0], uv[1], centre, size, grow)
            for quad in range(6):
                q = first + quad * 4
                indices += [q, q + 1, q + 2, q, q + 2, q + 3]
        ranges.append((start * 4, len(indices) - start))
    return np.array(vertices, np.float32), np.array(indices, np.uint32), ranges
```

`class AvatarRenderer`

Dòng 140–190

Các phương thức: `__init__`, `draw`.

```
class AvatarRenderer:
```

`AvatarRenderer.__init__(self)`

Dòng 141–166

```
    def __init__(self):
        self.shader = ShaderProgram.load("avatar")
        vertices, indices, self.ranges = build_mesh()
        self.vao = GL.glGenVertexArrays(1)
        GL.glBindVertexArray(self.vao)
        self.vbo = GL.glGenBuffers(1)
        GL.glBindBuffer(GL.GL_ARRAY_BUFFER, self.vbo)
        GL.glBufferData(GL.GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL.GL_STATIC_DRAW)
        self.ebo = GL.glGenBuffers(1)
        GL.glBindBuffer(GL.GL_ELEMENT_ARRAY_BUFFER, self.ebo)
        GL.glBufferData(GL.GL_ELEMENT_ARRAY_BUFFER, indices.nbytes, indices, GL.GL_STATIC_DRAW)
        for location, size, offset in ((0, 3, 0), (1, 3, 12), (2, 2, 24)):
            GL.glEnableVertexAttribArray(location)
            GL.glVertexAttribPointer(location, size, GL.GL_FLOAT, GL.GL_FALSE, 32, ctypes.c_void_p(offset))
        GL.glBindVertexArray(0)

        image = np.ascontiguousarray(load_png(SKIN_PATH))
        self.texture = GL.glGenTextures(1)
        GL.glBindTexture(GL.GL_TEXTURE_2D, self.texture)
        GL.glPixelStorei(GL.GL_UNPACK_ALIGNMENT, 1)
        GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA8, image.shape[1], image.shape[0], 0,
                        GL.GL_RGBA, GL.GL_UNSIGNED_BYTE, image)
        for name, value in ((GL.GL_TEXTURE_MIN_FILTER, GL.GL_NEAREST), (GL.GL_TEXTURE_MAG_FILTER, GL.GL_NEAREST),
                            (GL.GL_TEXTURE_WRAP_S, GL.GL_CLAMP_TO_EDGE), (GL.GL_TEXTURE_WRAP_T, GL.GL_CLAMP_TO_EDGE)):
            GL.glTexParameteri(GL.GL_TEXTURE_2D, name, value)
        GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
```

`AvatarRenderer.draw(self, view, projection, matrices, brightness=(1.0, 1.0, 1.0), light...)`

Dòng 168–190

Mô tả trong mã nguồn. matrices: 6 ma trận model theo LIMBS (core.avatar_pose.limb_matrices).

```
    def draw(self, view, projection, matrices, brightness=(1.0, 1.0, 1.0), light_dir=(0.4, 0.8, 0.45)):
        """matrices: 6 ma trận model theo LIMBS (core.avatar_pose.limb_matrices)."""
        s = self.shader
        s.use()
        s.set_mat4('uView', view)
        s.set_mat4('uProj', projection)
        s.set_vec3('uLight', light_dir)
        s.set_vec3('uBright', brightness)
        s.set_int('uSkin', 0)
        GL.glActiveTexture(GL.GL_TEXTURE0)
        GL.glBindTexture(GL.GL_TEXTURE_2D, self.texture)
        GL.glClear(GL.GL_DEPTH_BUFFER_BIT)
        GL.glEnable(GL.GL_DEPTH_TEST)
        GL.glDepthFunc(GL.GL_LESS)
        GL.glDisable(GL.GL_BLEND)
        GL.glDisable(GL.GL_CULL_FACE)
```

```
        GL.glBindVertexArray(self.vao)
        for matrix, (offset, count) in zip(matrices, self.ranges):
            s.set_mat4('uModel', matrix)
            GL.glDrawElements(GL.GL_TRIANGLES, count, GL.GL_UNSIGNED_INT, ctypes.c_void_p(offset))
        GL.glBindVertexArray(0)
        GL.glDisable(GL.GL_DEPTH_TEST)
        GL.glEnable(GL.GL_BLEND)
```

4.9. `shaders/`

Mã GLSL được giải thích riêng ở Chương 6.

Chương 5. Thư mục ui: giao diện

Thư mục `ui` là **giao diện**: cửa sổ GLFW, Dear ImGui, menu, bảng bên trái, thanh trạng thái, xử lý chuột/bàn phím. Thư mục này hầu như không có toán; nó chỉ gọi lại `Editor`, `Viewport`, `Lighting`, `Physics`. Ý tưởng chính là mẫu **Command**: mọi hành động (menu, phím tắt, nút) khai báo một lần trong `ActionRegistry` nên không bao giờ lệch nhau.

5.1. `splat_editor/ui/window.py`

66 dòng

Vai trò. Cửa sổ GLFW với ngữ cảnh OpenGL 3.3 core. Bọc các hàm GLFW (kích thước, vị trí con trỏ, phím, chuột, `swap_buffers`).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Cửa sổ GLFW có ngữ cảnh OpenGL 3.3 core."""

"""
win = Window("Splat Editor", 1280, 720)
while not win.should_close:
    Window.poll_events()
    if win.is_key_down(glfw.KEY_ESCAPE):
        win.close()
    # ... vẽ bằng OpenGL ...
    win.swap_buffers()
win.destroy()
"""

```

Khai báo mức module (dòng 14–14)

```
import glfw
```

`class Window`

Dòng 17–66

Các phương thức: `__init__`, `should_close`, `close`, `size`, `framebuffer_size`, `cursor_pos`, `is_key_down`, `is_mouse_down`, `poll_events`, `swap_buffers`, `destroy`.

```
class Window:
```

`Window.__init__(self, title, width, height)`

Dòng 18–30

```
    def __init__(self, title, width, height):
        if not glfw.init():
            raise RuntimeError("Failed to initialize GLFW")
        glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
        glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
        glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFILE)
        glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, True)
        self.handle = glfw.create_window(width, height, title, None, None)
        if not self.handle:
            glfw.terminate()
            raise RuntimeError("Failed to create an OpenGL 3.3 window")
        glfw.make_context_current(self.handle)
        glfw.swap_interval(1)
```

`Window.should_close(self)`

Dòng 32–34

```
    @property
    def should_close(self):
        return glfw.window_should_close(self.handle)
```

`Window.close(self)`

Dòng 36–37

```
    def close(self):
        glfw.set_window_should_close(self.handle, True)
```

Window.size(self)

Dòng 39–41

```
    @property
    def size(self):
        return glfw.get_window_size(self.handle)
```

Window.framebuffer_size(self)

Dòng 43–45

```
    @property
    def framebuffer_size(self):
        return glfw.get_framebuffer_size(self.handle)
```

Window.cursor_pos(self)

Dòng 47–49

```
    @property
    def cursor_pos(self):
        return glfw.get_cursor_pos(self.handle)
```

Window.is_key_down(self, key)

Dòng 51–52

```
    def is_key_down(self, key):
        return glfw.get_key(self.handle, key) == glfw.PRESS
```

Window.is_mouse_down(self, button)

Dòng 54–55

```
    def is_mouse_down(self, button):
        return glfw.get_mouse_button(self.handle, button) == glfw.PRESS
```

Window.poll_events()

Dòng 57–59

```
    @staticmethod
    def poll_events():
        glfw.poll_events()
```

Window.swap_buffers(self)

Dòng 61–62

```
    def swap_buffers(self):
        glfw.swap_buffers(self.handle)
```

Window.destroy(self)

Dòng 64–66

```
    def destroy(self):
        glfw.destroy_window(self.handle)
        glfw.terminate()
```

5.2. splat_editor/ui/gui.py

58 dòng

Vai trò. Ngữ cảnh Dear ImGui gắn với cửa sổ GLFW: `begin_frame`, `end_frame`, `draw`, `shutdown`, và cờ `wants_mouse` / `wants_text` để biết ImGui đang chiếm chuột/bàn phím.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Ngữ cảnh Dear ImGui gắn với cửa sổ GLFW."""

"""
gui = Gui(window)                       # sau khi tạo Window

while not window.should_close:
    Window.poll_events()
    gui.begin_frame()
    # ... gọi các hàm imgui.* dựng giao diện ...
    gui.end_frame()
    # ... vẽ cảnh 3D ...
    gui.draw()                          # vẽ ImGui đè lên cảnh
    window.swap_buffers()

if gui.wants_mouse:                     # chuột đang ở trên ImGui: đừng điều khiển camera
    pass
gui.shutdown()
"""

```

Khai báo mức module (dòng 20–23)

```
from imgui_bundle import imgui
from imgui_bundle.python_backends.glfw_backend import GlfwRenderer

from .theme import Theme
```

class Gui

Dòng 26–58

Các phương thức: `__init__`, `io`, `wants_mouse`, `wants_text`, `begin_frame`, `end_frame`, `draw`, `shutdown`.

```
class Gui:
```

Gui.__init__(self, window)

Dòng 27–32

```
    def __init__(self, window):
        imgui.create_context()
        imgui.get_io().set_ini_filename("")          # không ghi file imgui.ini
        # tự đăng ký callback ở InputController để vừa chuyển cho ImGui vừa xử lý camera / cọ
        self.backend = GlfwRenderer(window.handle, attach_callbacks=False)
        Theme.apply()
```

Gui.io(self)

Dòng 34–36

```
    @property
    def io(self):
        return imgui.get_io()
```

Gui.wants_mouse(self)

Dòng 38–40

```
    @property
    def wants_mouse(self):
        return imgui.get_io().want_capture_mouse
```

Gui.wants_text(self)

Dòng 42–44

```
    @property
    def wants_text(self):
        return imgui.get_io().want_text_input
```

`Gui.begin_frame(self)`

Dòng 46–48

```
    def begin_frame(self):
        self.backend.process_inputs()
        imgui.new_frame()
```

`Gui.end_frame(self)`

Dòng 50–52

Mô tả trong mã nguồn. Kết thúc khung hình ImGui (chưa vẽ ra màn hình).

```
    def end_frame(self):
        """Kết thúc khung hình ImGui (chưa vẽ ra màn hình)."""
        imgui.render()
```

`Gui.draw(self)`

Dòng 54–55

```
    def draw(self):
        self.backend.render(imgui.get_draw_data())
```

`Gui.shutdown(self)`

Dòng 57–58

```
    def shutdown(self):
        self.backend.shutdown()
```

5.3. `splat_editor/ui/theme.py`

74 dòng

Vai trò. Font và màu sắc của giao diện ImGui.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Font và màu sắc của giao diện ImGui."""

"""
Theme.apply()                          # gọi một lần sau khi tạo ngữ cảnh ImGui: nạp font, áp màu
# Đổi màu chủ đạo: sửa Theme.ACCENT hoặc Theme.COLORS rồi gọi lại apply()
"""

```

Khai báo mức module (dòng 8–10)

```
import os

from imgui_bundle import imgui
```

`class Theme`

Dòng 13–74

Các phương thức: `apply`, `_load_font`.

```
class Theme:
    FONT_SIZE = 16.0
    FONT_CANDIDATES = [
        r"C:\Windows\Fonts\segoeui.ttf",
        r"C:\Windows\Fonts\arial.ttf",
```

```
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
        "/System/Library/Fonts/Supplemental/Arial.ttf",
    ]

    ACCENT = (0.28, 0.52, 0.90)
    COLORS = {
        imgui.Col_.window_bg: (0.11, 0.11, 0.13, 1.00),
        imgui.Col_.child_bg: (0.09, 0.09, 0.11, 1.00),
        imgui.Col_.popup_bg: (0.12, 0.12, 0.14, 0.98),
        imgui.Col_.menu_bar_bg: (0.15, 0.15, 0.18, 1.00),
        imgui.Col_.border: (0.22, 0.22, 0.26, 1.00),
        imgui.Col_.frame_bg: (0.18, 0.18, 0.21, 1.00),
        imgui.Col_.frame_bg_hovered: (0.24, 0.24, 0.28, 1.00),
        imgui.Col_.frame_bg_active: (0.28, 0.28, 0.33, 1.00),
        imgui.Col_.button: (0.21, 0.21, 0.25, 1.00),
        imgui.Col_.button_hovered: (0.28, 0.40, 0.62, 1.00),
        imgui.Col_.button_active: (*ACCENT, 1.00),
        imgui.Col_.header: (0.22, 0.30, 0.45, 1.00),
        imgui.Col_.header_hovered: (0.27, 0.38, 0.58, 1.00),
        imgui.Col_.header_active: (*ACCENT, 1.00),
        imgui.Col_.check_mark: (0.45, 0.68, 1.00, 1.00),
        imgui.Col_.slider_grab: (*ACCENT, 1.00),
        imgui.Col_.slider_grab_active: (0.45, 0.68, 1.00, 1.00),
        imgui.Col_.separator: (0.22, 0.22, 0.26, 1.00),
        imgui.Col_.text_disabled: (0.55, 0.55, 0.60, 1.00),
    }

    # màu cho nút nguy hiểm (xoá)
    DANGER = (0.55, 0.20, 0.20, 1.00)
    DANGER_HOVERED = (0.70, 0.25, 0.25, 1.00)
```

`Theme.apply(cls)`

Dòng 49–66

```
    @classmethod
    def apply(cls):
        cls._load_font()
        imgui.style_colors_dark()
        style = imgui.get_style()
        style.window_rounding = 0.0
        style.child_rounding = 4.0
        style.frame_rounding = 4.0
        style.popup_rounding = 4.0
        style.grab_rounding = 4.0
        style.scrollbar_rounding = 4.0
        style.window_border_size = 0.0
        style.window_padding = imgui.ImVec2(10, 10)
        style.frame_padding = imgui.ImVec2(8, 4)
        style.item_spacing = imgui.ImVec2(8, 6)
        style.item_inner_spacing = imgui.ImVec2(6, 4)
        for index, color in cls.COLORS.items():
            style.set_color_(index, imgui.ImVec4(*color))
```

`Theme._load_font(cls)`

Dòng 68–74

```
    @classmethod
    def _load_font(cls):
        io = imgui.get_io()
        for path in cls.FONT_CANDIDATES:
            if os.path.exists(path):
                io.fonts.add_font_from_file_ttf(path, cls.FONT_SIZE)
                return
```

5.4. `splat_editor/ui/actions.py`

70 dòng

Vai trò. Mẫu Command cho giao diện: `ActionRegistry` là nơi duy nhất khai báo nhãn, phím tắt và hàm xử lý. Menu, nút bấm, phím tắt và bảng Keyboard Shortcuts đều đọc cùng danh sách này.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Hành động (mẫu Command): một nơi khai báo nhãn, phím tắt và hàm xử lý.

Menu, nút bấm, phím tắt và bảng "Keyboard Shortcuts" đều dùng chung danh sách này,
nên không bao giờ bị lệch nhau.
"""

"""
```

```
actions = ActionRegistry()
actions.add("open", Action("Open...", open_dialog, "File", "Ctrl+O", glfw.KEY_O, ctrl=True, deferred=True))
actions.add("grid", Action("Show Grid", toggle_grid, "View", "G", glfw.KEY_G,
                           checked=lambda: viewport.show_grid))

actions.trigger("open")                # từ menu hoặc nút bấm
actions.handle_key(glfw.KEY_O, ctrl=True)   # từ bàn phím; True nếu có hành động khớp
actions.run_pending()                  # cuối mỗi khung hình: chạy các hành động deferred (hộp thoại file)
"""

```

Khai báo mức module (dòng 18–19)

```
from dataclasses import dataclass
from typing import Callable, Optional
```

class Action

Dòng 22–33

```
@dataclass
class Action:
    label: str
    callback: Callable[[], None]
    category: str = "General"
    shortcut: str = ""                          # chữ hiển thị, ví dụ "Ctrl+O"
    key: Optional[int] = None                   # phím GLFW
    ctrl: bool = False
    description: str = ""                       # mô tả trong bảng phím tắt (mặc định = label)
    deferred: bool = False                      # chạy sau khung hình (mở hộp thoại hệ thống)
    enabled: Callable[[], bool] = lambda: True
    checked: Optional[Callable[[], bool]] = None  # mục bật / tắt trong menu
```

class ActionRegistry

Dòng 36–70

Các phương thức: `__init__`, `add`, `__getitem__`, `__iter__`, `trigger`, `handle_key`, `run_pending`.

```
class ActionRegistry:
```

ActionRegistry.__init__(self)

Dòng 37–39

```
    def __init__(self):
        self._actions = {}
        self._pending = []
```

ActionRegistry.add(self, name, action)

Dòng 41–42

```
    def add(self, name, action):
        self._actions[name] = action
```

ActionRegistry.__getitem__(self, name)

Dòng 44–45

```
    def __getitem__(self, name):
        return self._actions[name]
```

ActionRegistry.__iter__(self)

Dòng 47–48

```
    def __iter__(self):
        return iter(self._actions.values())
```

`ActionRegistry.trigger(self, name)`

Dòng 50–58

```
    def trigger(self, name):
        action = self._actions[name]
        if not action.enabled():
            return
        if action.deferred:
            # hộp thoại chọn file chạy ngoài khung hình để không làm hỏng trạng thái ImGui
            self._pending.append(action.callback)
        else:
            action.callback()
```

`ActionRegistry.handle_key(self, key, ctrl)`

Dòng 60–66

Mô tả trong mã nguồn. Chạy hành động gắn với phím. Trả về True nếu có hành động khớp.

Giải thích. Tìm hành động có phím tắt khớp và chạy; trả `True` nếu có.

```
    def handle_key(self, key, ctrl):
        """Chạy hành động gắn với phím. Trả về True nếu có hành động khớp."""
        for name, action in self._actions.items():
            if action.key == key and action.ctrl == bool(ctrl):
                self.trigger(name)
                return True
        return False
```

`ActionRegistry.run_pending(self)`

Dòng 68–70

```
    def run_pending(self):
        while self._pending:
            self._pending.pop(0)()
```

5.5. `splat_editor/ui/input.py`

136 dòng

Vai trò. `InputController` nhận sự kiện chuột/bàn phím: chuyển cho ImGui trước, phần còn lại điều khiển camera (xoay, kéo, zoom, bay), cọ chọn và phím tắt.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Chuột / bàn phím: chuyển cho ImGui trước, phần còn lại điều khiển camera, cọ chọn và phím tắt."""

"""
input = InputController(app)          # tự đăng ký callback chuột / phím / kéo thả với GLFW

# Mỗi khung hình
input.update(dt)                       # WASD bay camera khi giữ chuột phải

# Bên trong, mỗi sự kiện đi qua ImGui trước (nếu ImGui cần thì bỏ qua):
#   on_mouse_move  -> orbit / pan / look / tô cọ
#   on_mouse_button, on_scroll (zoom hoặc đổi cỡ cọ)
#   on_key -> actions.handle_key
#   on_drop -> app.add_files
input.reset()                          # xoá trạng thái nút / phím (sau khi hộp thoại file đóng)
"""

```

Khai báo mức module (dòng 17–17)

```
import glfw
```

`class InputController`

Dòng 20–136

Các phương thức: `__init__`, `brush`, `looking`, `_set_drag`, `reset`, `on_mouse_move`, `on_mouse_button`, `on_scroll`, `on_drop`, `on_key`, `update`.

```
class InputController:
```

`InputController.__init__(self, app)`

Dòng 21–34

```
    def __init__(self, app):
        self.app = app
        self.gui = app.gui.backend
        self.last_mouse = None
        self.drag_button = None      # nút chuột đang kéo trong vùng 3D (xoay / dịch / nhìn quanh)
        self._click_start = None     # vị trí nhấn chuột trái, để phân biệt click (chọn đèn) với kéo (xoay)

        handle = app.window.handle
        glfw.set_cursor_pos_callback(handle, self.on_mouse_move)
        glfw.set_mouse_button_callback(handle, self.on_mouse_button)
        glfw.set_scroll_callback(handle, self.on_scroll)
        glfw.set_key_callback(handle, self.on_key)
        glfw.set_char_callback(handle, self.gui.char_callback)
        glfw.set_drop_callback(handle, self.on_drop)
```

`InputController.brush(self)`

Dòng 36–38

```
    @property
    def brush(self):
        return self.app.editor.brush
```

`InputController.looking(self)`

Dòng 40–43

Mô tả trong mã nguồn. Đang giữ chuột phải: nhìn quanh + bay kiểu Minecraft.

```
    @property
    def looking(self):
        """Đang giữ chuột phải: nhìn quanh + bay kiểu Minecraft."""
        return self.drag_button == glfw.MOUSE_BUTTON_RIGHT
```

`InputController._set_drag(self, button)`

Dòng 45–50

```
    def _set_drag(self, button):
        # khi nhìn quanh thì ẩn và khoá con trỏ, để xoay mãi không bị chạm mép màn hình
        mode = glfw.CURSOR_DISABLED if button == glfw.MOUSE_BUTTON_RIGHT else glfw.CURSOR_NORMAL
        glfw.set_input_mode(self.app.window.handle, glfw.CURSOR, mode)
        self.drag_button = button
        self.last_mouse = None
```

`InputController.reset(self)`

Dòng 52–55

Mô tả trong mã nguồn. Gọi sau khi mở hộp thoại hệ thống (cửa sổ mất focus, chuột có thể đã nhả).

```
    def reset(self):
        """Gọi sau khi mở hộp thoại hệ thống (cửa sổ mất focus, chuột có thể đã nhả)."""
        self._set_drag(None)
        self.brush.end_stroke()
```

`InputController.on_mouse_move(self, window, x, y)`

Dòng 58–74

```
    def on_mouse_move(self, window, x, y):
        self.gui.mouse_callback(window, x, y)
        if self.brush.is_painting:
            self.app.paint(x, y)
        elif self.drag_button is not None and self.last_mouse is not None:
            dx, dy = x - self.last_mouse[0], y - self.last_mouse[1]
            # chế độ cọ: chuột trái dùng để tô nên xoay camera bằng chuột giữa
            orbit = self.drag_button == glfw.MOUSE_BUTTON_LEFT or (
                self.drag_button == glfw.MOUSE_BUTTON_MIDDLE and self.brush.active)
            camera = self.app.viewport.camera
            if self.looking:
```

```
                camera.look(dx, dy)
            elif orbit:
                camera.orbit(dx, dy)
            else:
                camera.pan(dx, dy, max(self.app.layout.viewport.height, 1))
        self.last_mouse = (x, y)
```

`InputController.on_mouse_button(self, window, button, action, mods)`

Dòng 76–99

```
    def on_mouse_button(self, window, button, action, mods):
        self.gui.mouse_button_callback(window, button, action, mods)
        if action == glfw.PRESS:
            x, y = glfw.get_cursor_pos(window)
            if self.app.gui.wants_mouse or not self.app.layout.viewport.contains(x, y):
                return
            if button == glfw.MOUSE_BUTTON_LEFT and self.brush.active:
                self.app.lighting.picked = -1            # tô cọ -> Delete xoá gaussian, không phải đèn
                self.brush.begin_stroke(select=not (mods & glfw.MOD_CONTROL))   # giữ Ctrl: bỏ chọn
                self.app.paint(x, y)
            elif self.drag_button is None:
                if button == glfw.MOUSE_BUTTON_LEFT:
                    self._click_start = (x, y)           # có thể là click chọn đèn (nếu không kéo)
                self._set_drag(button)
        elif action == glfw.RELEASE:
            if button == glfw.MOUSE_BUTTON_LEFT:
                self.brush.end_stroke()
                start, self._click_start = self._click_start, None
                if start is not None and not self.brush.active and not self.app.gui.wants_mouse:
                    x, y = glfw.get_cursor_pos(window)
                    if abs(x - start[0]) < 4 and abs(y - start[1]) < 4:
                        self.app.pick_light(x, y)
            if button == self.drag_button:
                self._set_drag(None)
```

`InputController.on_scroll(self, window, dx, dy)`

Dòng 101–102

```
    def on_scroll(self, window, dx, dy):
        self.gui.scroll_callback(window, dx, dy)
```

`InputController.on_drop(self, window, paths)`

Dòng 105–106

```
    def on_drop(self, window, paths):
        self.app.add_files(paths)
```

`InputController.on_key(self, window, key, scancode, action, mods)`

Dòng 109–114

```
    def on_key(self, window, key, scancode, action, mods):
        self.gui.keyboard_callback(window, key, scancode, action, mods)
        # đang bay thì Ctrl là chạy nhanh, không kích hoạt phím tắt (Ctrl+S, Ctrl+D...)
        if action != glfw.PRESS or self.app.gui.wants_text or self.looking:
            return
        self.app.actions.handle_key(key, mods & glfw.MOD_CONTROL)
```

`InputController.update(self, dt)`

Dòng 116–136

Mô tả trong mã nguồn. Bay kiểu Minecraft sáng tạo (giữ phím), gọi mỗi khung hình.

W A S D: đi ngang, Space: lên, Shift: xuống, Ctrl: bay nhanh (khi giữ chuột phải).

Giải thích. Mỗi khung hình khi giữ chuột phải: đọc W/A/S/D/Q/E và gọi `OrbitCamera.fly` (hoặc `Application.walk` ở chế độ sinh tồn).

```
    def update(self, dt):
        """Bay kiểu Minecraft sáng tạo (giữ phím), gọi mỗi khung hình.

        W A S D: đi ngang, Space: lên, Shift: xuống, Ctrl: bay nhanh (khi giữ chuột phải).
```

```
        """
        camera = self.app.viewport.camera
        io = self.app.gui.io
        pressed = self.app.window.is_key_down
        ctrl = pressed(glfw.KEY_LEFT_CONTROL) or pressed(glfw.KEY_RIGHT_CONTROL)
        if io.want_text_input or (ctrl and not self.looking):
            forward = right = up = 0
        else:
            forward = pressed(glfw.KEY_W) - pressed(glfw.KEY_S)
            right = pressed(glfw.KEY_D) - pressed(glfw.KEY_A)
            shift = pressed(glfw.KEY_LEFT_SHIFT) or pressed(glfw.KEY_RIGHT_SHIFT)
            up = pressed(glfw.KEY_SPACE) - shift
        if self.app.mode == "survival":
            # sinh tồn: Space nhảy, Ctrl chạy nhanh, Shift đi chậm; camera là mắt người chơi
            self.app.walk(forward, right, up > 0, ctrl and (forward or right), up < 0, dt)
        else:
            camera.fly(forward, right, up, min(dt, 0.1), sprint=ctrl)
```

5.6. `splat_editor/ui/layout.py`

53 dòng

Vai trò. Bố cục màn hình: menu trên cùng, bảng bên trái, thanh trạng thái dưới cùng, phần còn lại là vùng 3D.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Bố cục màn hình: menu trên cùng, bảng bên trái, thanh trạng thái dưới cùng, phần còn lại là vùng 3D.

    +--------------------------------------+
    | menu bar                             |
    +-----------+--------------------------+
    | side      |                          |
    | panel     |      3D viewport         |
    |           |                          |
    +-----------+--------------------------+
    | status bar                           |
    +--------------------------------------+
"""

"""
layout = Layout()
layout.update(width, height, menu_height, status_height)   # gọi mỗi khung hình
layout.side_panel                       # Rect bảng bên trái
layout.viewport                         # Rect vùng 3D
layout.status_bar                       # Rect thanh trạng thái
"""

```

Khai báo mức module (dòng 22–22)

```
from ..core.rect import Rect
```

`class Layout`

Dòng 25–53

Các phương thức: `__init__`, `update`, `_body_height`, `side_panel`, `viewport`, `status_bar`.

```
class Layout:
    SIDE_PANEL_WIDTH = 330
```

`Layout.__init__(self)`

Dòng 28–32

```
    def __init__(self):
        self.width = 0
        self.height = 0
        self.menu_height = 0
        self.status_height = 0
```

`Layout.update(self, width, height, menu_height, status_height)`

Dòng 34–36

Giải thích. Tính ba hình chữ nhật (bảng trái, thanh trạng thái, vùng 3D) từ kích thước cửa sổ, chiều cao menu và thanh trạng thái.

```
    def update(self, width, height, menu_height, status_height):
        self.width, self.height = width, height
        self.menu_height, self.status_height = menu_height, status_height
```

`Layout._body_height(self)`

Dòng 38–40

```
    @property
    def _body_height(self):
        return max(self.height - self.menu_height - self.status_height, 0)
```

`Layout.side_panel(self)`

Dòng 42–44

```
    @property
    def side_panel(self):
        return Rect(0, self.menu_height, min(self.SIDE_PANEL_WIDTH, self.width), self._body_height)
```

`Layout.viewport(self)`

Dòng 46–49

```
    @property
    def viewport(self):
        left = self.side_panel.width
        return Rect(left, self.menu_height, max(self.width - left, 0), self._body_height)
```

`Layout.status_bar(self)`

Dòng 51–53

```
    @property
    def status_bar(self):
        return Rect(0, self.height - self.status_height, self.width, self.status_height)
```

5.7. `splat_editor/ui/menu_bar.py`

47 dòng

Vai trò. Thanh menu File/Edit/View/Help sinh từ `ActionRegistry`.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Thanh menu trên cùng (File / Edit / View / Help), sinh từ danh sách hành động."""

"""
menu = MainMenuBar(actions)             # actions: ActionRegistry
menu_height = menu.draw()               # gọi trong khung ImGui mỗi khung hình; trả về chiều cao menu
"""

```

Khai báo mức module (dòng 8–8)

```
from imgui_bundle import imgui
```

`class MainMenuBar`

Dòng 11–47

Các phương thức: `__init__`, `draw`, `_draw_item`.

```
class MainMenuBar:
    # None = đường kẻ ngăn cách
    MENUS = [
        ("File", ["open", "add", None, "save", None, "quit"]),
        ("Edit", ["undo", None, "select_all", "select_none", "invert_selection", None,
                  "delete_selected", "remove_layer"]),
        ("View", ["frame_scene", "frame_layer", None, "show_grid", "view_sh", "flip_y", None, "brush"]),
        ("World", ["toggle_mode", "third_person", None, "sit", "wave", "zombie", "cheer", None,
                   "lighting", "shadows", "add_sun", "add_light"]),
        ("Help", ["shortcuts"]),
    ]
```

`MainMenuBar.__init__(self, actions)`

Dòng 23–24

```
    def __init__(self, actions):
        self.actions = actions
```

`MainMenuBar.draw(self)`

Dòng 26–40

Mô tả trong mã nguồn. Vẽ menu, trả về chiều cao (pixel) của thanh menu.

```
    def draw(self):
        """Vẽ menu, trả về chiều cao (pixel) của thanh menu."""
        height = 0
        if imgui.begin_main_menu_bar():
            for title, items in self.MENUS:
                if imgui.begin_menu(title):
                    for name in items:
                        if name is None:
                            imgui.separator()
                        else:
                            self._draw_item(name)
                    imgui.end_menu()
            height = imgui.get_window_size().y
            imgui.end_main_menu_bar()
        return height
```

`MainMenuBar._draw_item(self, name)`

Dòng 42–47

```
    def _draw_item(self, name):
        action = self.actions[name]
        checked = action.checked() if action.checked else False
        clicked, _ = imgui.menu_item(action.label, action.shortcut, checked, action.enabled())
        if clicked:
            self.actions.trigger(name)
```

5.8. `splat_editor/ui/side_panel.py`

39 dòng

Vai trò. Bảng cố định bên trái, gồm các mục thu gọn được.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Bảng cố định bên trái, gồm các mục thu gọn được (Layers, Transform, Selection, View)."""

"""
panel = SidePanel(app)                  # Layers / Transform / Selection / View
panel.draw(layout.side_panel)           # gọi mỗi khung hình với Rect bảng bên trái
"""

```

Khai báo mức module (dòng 8–11)

```
from imgui_bundle import imgui

from .sections import (LayersSection, SelectionSection, TransformSection, ViewSection)
from .world_sections import LightingSection, ModeSection
```

`class SidePanel`

Dòng 14–39

Các phương thức: `__init__`, `draw`.

```
class SidePanel:
    FLAGS = (imgui.WindowFlags_.no_move | imgui.WindowFlags_.no_resize | imgui.WindowFlags_.no_collapse |
             imgui.WindowFlags_.no_title_bar | imgui.WindowFlags_.no_saved_settings |
             imgui.WindowFlags_.no_bring_to_front_on_focus)
```

`SidePanel.__init__(self, app)`

Dòng 19–21

```
    def __init__(self, app):
        self.sections = [LayersSection(app), ModeSection(app), LightingSection(app), TransformSection(app),
                         SelectionSection(app), ViewSection(app)]
```

`SidePanel.draw(self, rect)`

Dòng 23–39

```
    def draw(self, rect):
        imgui.set_next_window_pos(imgui.ImVec2(rect.left, rect.top))
        imgui.set_next_window_size(imgui.ImVec2(rect.width, rect.height))
        imgui.begin("##side_panel", None, self.FLAGS)
        for section in self.sections:
            if not section.is_visible():
                continue
            if imgui.collapsing_header(section.header, imgui.TreeNodeFlags_.default_open):
                imgui.spacing()
                # mỗi mục một phạm vi ID riêng: "Position" của Lighting không trùng "Position" của Transform
                imgui.push_id(type(section).__name__)
                try:
                    section.draw()
                finally:
                    imgui.pop_id()
                imgui.spacing()
        imgui.end()
```

5.9. `splat_editor/ui/sections.py`

248 dòng

Vai trò. Các mục của bảng trái: Layers, Transform, Selection, View. Mỗi mục là một lớp con của `PanelSection` (mẫu Template Method).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Các mục trong bảng bên trái. Mỗi mục là một lớp con của PanelSection."""

"""
# Mỗi mục là một lớp con của PanelSection, SidePanel gọi draw() lần lượt
sections = [LayersSection(app), TransformSection(app), SelectionSection(app),
            ViewSection(app)]
for s in sections:
    if s.is_visible():                  # ví dụ Transform chỉ hiện khi có lớp đang chọn
        s.draw()

# Thêm một mục mới:
class MySection(PanelSection):
    @property
    def header(self):
        return "My Section"

    def draw(self):
        ...                              # dùng self.editor, self.scene và imgui.*
"""

```

Khai báo mức module (dòng 21–27)

```
import math
from abc import ABC, abstractmethod

from imgui_bundle import imgui

from ..core.math3d import axis_angle_matrix
from . import widgets
```

`class PanelSection(ABC)`

Dòng 30–54

Các phương thức: `__init__`, `editor`, `scene`, `header`, `is_visible`, `draw`.

```
class PanelSection(ABC):
    title = ""
```

`PanelSection.__init__(self, app)`

Dòng 33–34

```
    def __init__(self, app):
        self.app = app
```

`PanelSection.editor(self)`

Dòng 36–38

```
    @property
    def editor(self):
        return self.app.editor
```

`PanelSection.scene(self)`

Dòng 40–42

```
    @property
    def scene(self):
        return self.app.editor.scene
```

`PanelSection.header(self)`

Dòng 44–47

Mô tả trong mã nguồn. Nhãn tiêu đề; phần sau '###' giữ ID cố định khi chữ thay đổi.

```
    @property
    def header(self):
        """Nhãn tiêu đề; phần sau '###' giữ ID cố định khi chữ thay đổi."""
        return f"{self.title}###{type(self).__name__}"
```

`PanelSection.is_visible(self)`

Dòng 49–50

```
    def is_visible(self):
        return True
```

`PanelSection.draw(self)`

Dòng 52–54

```
    @abstractmethod
    def draw(self):
        ...
```

`class LayersSection(PanelSection)`

Dòng 59–120

Các phương thức: `header`, `draw`, `_draw_list`.

```
class LayersSection(PanelSection):
    title = "Layers"
    MAX_VISIBLE_ROWS = 6
```

`LayersSection.header(self)`

Dòng 63–65

```
    @property
    def header(self):
        return f"Layers ({len(self.scene.layers)})###layers"
```

`LayersSection.draw(self)`

Dòng 67–83

```
    def draw(self):
        scene = self.scene
        if scene.is_empty:
            imgui.text_disabled("No layers loaded yet.")
            imgui.text_disabled("Open a .ply file, or drag and drop\nfiles into the window.")
            if widgets.full_width_button("Open File..."):
                self.app.actions.trigger("open")
            return

        self._draw_list()
        clicked = widgets.button_row(["Add...", "Frame", "Remove"])
        if clicked == 0:
            self.app.actions.trigger("add")
        elif clicked == 1:
            self.app.actions.trigger("frame_layer")
        elif clicked == 2:
            self.app.actions.trigger("remove_layer")
```

`LayersSection._draw_list(self)`

Dòng 85–120

```
    def _draw_list(self):
        scene = self.scene
        padding = imgui.ImVec2(6, 6)
        # tự co giãn theo số lớp, quá MAX_VISIBLE_ROWS dòng thì cuộn
        max_height = self.MAX_VISIBLE_ROWS * imgui.get_frame_height_with_spacing() + 2 * padding.y
        imgui.set_next_window_size_constraints(imgui.ImVec2(0, 0), imgui.ImVec2(1e9, max_height))

        imgui.push_style_var(imgui.StyleVar_.window_padding, padding)
        imgui.push_style_var(imgui.StyleVar_.selectable_text_align, imgui.ImVec2(0, 0.5))
        imgui.begin_child("##layer_list", imgui.ImVec2(-1, 0),
                          imgui.ChildFlags_.borders | imgui.ChildFlags_.auto_resize_y)
        for i, layer in enumerate(scene.layers):
            imgui.push_id(str(i))
            changed, layer.visible = imgui.checkbox("##visible", layer.visible)
            if changed:
                self.editor.notify_geometry_changed()
            widgets.tooltip("Show / hide this layer")

            imgui.same_line()
            count = f"{layer.count:,}"
            count_width = imgui.calc_text_size(count).x
            name_width = imgui.get_content_region_avail().x - count_width - imgui.get_style().item_spacing.x
            clicked, _ = imgui.selectable(layer.name, i == scene.active_index, 0,
                                          imgui.ImVec2(name_width, imgui.get_frame_height()))
            if clicked:
                scene.active_index = i
            if imgui.is_item_hovered() and imgui.is_mouse_double_clicked(imgui.MouseButton_.left):
                self.app.frame_scene(layer)
            widgets.tooltip(f"{layer.path or layer.name}\nDouble-click to frame")

            imgui.same_line()
            imgui.align_text_to_frame_padding()
            imgui.text_disabled(count)
            imgui.pop_id()
        imgui.end_child()
        imgui.pop_style_var(2)
```

`class TransformSection(PanelSection)`

Dòng 125–192

Các phương thức: `is_visible`, `draw`, `_draw_level`.

```
class TransformSection(PanelSection):
    title = "Transform"
    LABEL_WIDTH = 70
```

`TransformSection.is_visible(self)`

Dòng 129–130

```
    def is_visible(self):
        return self.scene.active_layer is not None
```

`TransformSection.draw(self)`

Dòng 132–154

```
    def draw(self):
        layer = self.scene.active_layer
        t = layer.transform
        imgui.text_disabled(layer.name)

        imgui.push_item_width(-self.LABEL_WIDTH)
        changed = False
        c, t.position = imgui.drag_float3("Position", t.position, self.app.drag_speed(), format="%.3f")
        changed |= c
        c, t.rotation = imgui.drag_float3("Rotation", t.rotation, 0.5, format="%.1f")
        changed |= c
        widgets.tooltip("Degrees around the X, Y and Z axes")
        c, t.scale = imgui.drag_float("Scale", t.scale, 0.005, 0.001, 1000.0, format="%.3f")
        changed |= c
        imgui.pop_item_width()

        changed |= self._draw_level(t)

        if widgets.full_width_button("Reset Transform"):
            t.reset()
            changed = True
        if changed:
            self.editor.notify_transform_changed()
```

`TransformSection._draw_level(self, t)`

Dòng 156–192

Mô tả trong mã nguồn. Chỉnh cho model nằm ngang: xoay theo hướng đang nhìn, quanh điểm camera đang nhìn vào.

Roll: quanh trục nhìn tới (nằm ngang) -> đường chân trời nghiêng trái / phải trên màn hình. Pitch: quanh trục ngang của màn hình -> mặt sàn chúi về phía mình / ngửa ra xa.

```
    def _draw_level(self, t):
        """Chỉnh cho model nằm ngang: xoay theo hướng đang nhìn, quanh điểm camera đang nhìn vào.

        Roll: quanh trục nhìn tới (nằm ngang)  -> đường chân trời nghiêng trái / phải trên màn hình.
        Pitch: quanh trục ngang của màn hình   -> mặt sàn chúi về phía mình / ngửa ra xa.
        """
        camera = self.app.viewport.camera
        s, c = math.sin(camera.yaw), math.cos(camera.yaw)
        axes = {"Roll": ([-s, 0.0, -c], "Tilt left / right as seen on screen.\n"
                                        "Drag right: clockwise."),
                "Pitch": ([-c, 0.0, s], "Tip forward / backward as seen on screen.\n"
                                        "Drag right: top tips away from you.")}

        imgui.separator_text("Level (relative to view)")
        spacing = imgui.get_style().item_spacing.x
        button_width = imgui.calc_text_size("-90").x + 2 * imgui.get_style().frame_padding.x
        changed = False
        for label, (axis, help_text) in axes.items():
            imgui.push_id(label)
            imgui.set_next_item_width(imgui.get_content_region_avail().x - self.LABEL_WIDTH
                                      - 2 * (button_width + spacing))
            # giá trị luôn bắt đầu từ 0 mỗi khung hình, phần kéo được chính là góc cần xoay thêm
            _, degrees = imgui.drag_float("##drag", 0.0, 0.1, format="<  drag  >",
                                          flags=imgui.SliderFlags_.no_input)
            widgets.tooltip(help_text + "\nShift: faster, Alt: finer")
            for step in (-90.0, 90.0):
                imgui.same_line()
                if imgui.button(f"{step:+.0f}", imgui.ImVec2(button_width, 0)):
                    degrees = step
            imgui.same_line()
            imgui.text(label)
            if degrees:
                t.rotate_about(axis_angle_matrix(axis, degrees), camera.target)
                changed = True
            imgui.pop_id()
        imgui.spacing()
        return changed
```

`class SelectionSection(PanelSection)`

Dòng 197–227

Các phương thức: `draw`.

```
class SelectionSection(PanelSection):
    title = "Selection"
    LABEL_WIDTH = 70
```

`SelectionSection.draw(self)`

Dòng 201–227

```
    def draw(self):
        brush = self.editor.brush
        _, brush.active = imgui.checkbox("Brush tool (B)", brush.active)
        imgui.push_item_width(-self.LABEL_WIDTH)
        _, brush.radius = imgui.slider_float("Size", brush.radius, brush.MIN_RADIUS, brush.MAX_RADIUS,
                                             "%.0f px", imgui.SliderFlags_.logarithmic)
        imgui.pop_item_width()
        widgets.tooltip("Brush radius in pixels  ( [ / ] )")
        if brush.active:
            imgui.text_disabled("Left drag: select     Ctrl + drag: deselect\n"
                                "Middle drag: orbit    Right drag: pan")

        imgui.spacing()
        selected = self.scene.selected_count()
        imgui.text(f"{selected:,} gaussians selected")
        clicked = widgets.button_row(["All", "None", "Invert"])
        if clicked is not None:
            self.app.actions.trigger(("select_all", "select_none", "invert_selection")[clicked])

        imgui.begin_disabled(selected == 0)
        if widgets.danger_button("Delete Selected"):
            self.app.actions.trigger("delete_selected")
        imgui.end_disabled()
        imgui.begin_disabled(not self.editor.history.can_undo)
        if widgets.full_width_button("Undo (Ctrl+Z)"):
            self.app.actions.trigger("undo")
        imgui.end_disabled()
```

`class ViewSection(PanelSection)`

Dòng 232–248

Các phương thức: `draw`.

```
class ViewSection(PanelSection):
    title = "View"
    LABEL_WIDTH = 90
```

`ViewSection.draw(self)`

Dòng 236–248

```
    def draw(self):
        viewport = self.app.viewport
        _, viewport.show_grid = imgui.checkbox("Show grid (G)", viewport.show_grid)
        imgui.push_item_width(-self.LABEL_WIDTH)
        imgui.begin_disabled(not viewport.show_grid)
        _, viewport.grid.height = imgui.drag_float("Grid height", viewport.grid.height,
                                                   self.app.drag_speed(), format="%.3f")
        imgui.end_disabled()
        _, viewport.background = imgui.color_edit3("Background", viewport.background)
        imgui.pop_item_width()

        if widgets.full_width_button("Frame Scene (F)"):
            self.app.actions.trigger("frame_scene")
```

5.10. `splat_editor/ui/world_sections.py`

134 dòng

Vai trò. Hai mục của bảng trái cho ánh sáng và sinh tồn: `ModeSection` (chế độ sáng tạo/sinh tồn) và `Lighting-Section` (mặt trời, đèn, bóng).

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Các mục bảng trái cho T4 (ánh sáng + bóng) và T5 (chế độ sinh tồn + vật lý): Mode và Lighting."""

"""
sections = [ModeSection(app), LightingSection(app)]     # SidePanel vẽ chúng cùng các mục khác
"""

```

Khai báo mức module (dòng 7–11)

```
from imgui_bundle import imgui
```

```
from ..core.lighting import MAX_LIGHTS
from . import widgets
from .sections import PanelSection
```

`class ModeSection(PanelSection)`

Dòng 14–52

Các phương thức: `draw`.

```
class ModeSection(PanelSection):
    title = "Mode"
    LABEL_WIDTH = 100
```

`ModeSection.draw(self)`

Dòng 18–52

```
    def draw(self):
        app, physics = self.app, self.app.physics
        active = imgui.ImVec4(0.26, 0.52, 0.96, 1.0)
        spacing = imgui.get_style().item_spacing.x
        width = (imgui.get_content_region_avail().x - spacing) / 2
        for label, mode in (("Fly", "creative"), ("Walk", "survival")):
            is_active = app.mode == mode
            if is_active:
                imgui.push_style_color(imgui.Col_.button, active)
            if imgui.button(label, imgui.ImVec2(width, 0)):
                app.set_mode(mode)
            if is_active:
                imgui.pop_style_color()
            if mode == "creative":
                imgui.same_line()
        widgets.tooltip("M: toggle.\nFly: move freely through everything.\n"
                        "Walk: gravity, jumping and collision against the ply surface.")
        if app.mode != "survival":
            imgui.text_disabled("WASD fly, Space up, Shift down")
            return

        imgui.text_disabled("WASD walk, Space jump, Ctrl sprint,\nShift sneak. Drag to look around.")
        imgui.push_item_width(-self.LABEL_WIDTH)
        changed, height = imgui.input_float("Player height", physics.height, 0.0, 0.0, "%.4f",
                                            imgui.InputTextFlags_.enter_returns_true)
        if changed and height > 0.0:
            physics.set_height(height)
        widgets.tooltip("Splat scenes have no real unit: match the player's height to the scene.\n"
                        "Walk speed, jump and gravity scale with it. Changing it rebuilds the collision map.")
        _, physics.gravity_scale = imgui.slider_float("Gravity", physics.gravity_scale, 0.1, 3.0, "x%.2f")
        imgui.pop_item_width()
        imgui.text_disabled(physics.status)
        if widgets.full_width_button("Respawn Here"):
            app.set_mode("creative")
            app.set_mode("survival")
```

`class LightingSection(PanelSection)`

Dòng 55–134

Các phương thức: `draw`, `_draw_sun`, `_draw_lights`.

```
class LightingSection(PanelSection):
    title = "Lighting"
    LABEL_WIDTH = 90
```

`LightingSection.draw(self)`

Dòng 59–86

```
    def draw(self):
        lighting = self.app.lighting
        _, lighting.enabled = imgui.checkbox("Enable lighting", lighting.enabled)
        widgets.tooltip("Phong shading (ambient + diffuse + specular) for every gaussian.\n"
                        "The normal is the gaussian's shortest axis, facing the camera.")
        imgui.push_item_width(-self.LABEL_WIDTH)
        _, lighting.ambient = imgui.slider_float("Ambient", lighting.ambient, 0.0, 1.0, "%.2f")
        _, lighting.specular = imgui.slider_float("Specular", lighting.specular, 0.0, 1.0, "%.2f")
        _, lighting.shininess = imgui.slider_float("Shininess", lighting.shininess, 2.0, 128.0, "%.0f",
                                                   imgui.SliderFlags_.logarithmic)
        imgui.pop_item_width()
```

```
        _, lighting.shadows = imgui.checkbox("Sun shadows (shadow map)", lighting.shadows)

        clicked = widgets.button_row(["Remove Sun" if lighting.sun_on else "Add Sun", "Add Light Block"])
        if clicked == 0:
            if lighting.sun_on:
                lighting.remove_sun()
            else:
                self.app.actions.trigger("add_sun")
        elif clicked == 1:
            self.app.actions.trigger("add_light")
        if len(lighting.lights) >= MAX_LIGHTS:
            imgui.text_disabled(f"Maximum {MAX_LIGHTS} light blocks")

        if lighting.sun_on:
            self._draw_sun(lighting)
        if lighting.lights:
            self._draw_lights(lighting)
```

`LightingSection._draw_sun(self, lighting)`

Dòng 88–99

```
    def _draw_sun(self, lighting):
        imgui.separator_text("Sun / day-night")
        imgui.push_item_width(-self.LABEL_WIDTH)
        _, lighting.hour = imgui.slider_float("Time", lighting.hour, 0.0, 24.0, "%.1f h")
        _, lighting.sun_intensity = imgui.slider_float("Sun power", lighting.sun_intensity, 0.0, 2.0, "%.2f")
        _, lighting.sun_azimuth = imgui.slider_float("Azimuth", lighting.sun_azimuth, 0.0, 360.0, "%.0f deg")
        _, lighting.day_length = imgui.slider_float("Day length", lighting.day_length, 5.0, 300.0, "%.0f s",
                                                    imgui.SliderFlags_.logarithmic)
        imgui.pop_item_width()
        _, lighting.animate = imgui.checkbox("Animate day-night", lighting.animate)
        _, lighting.sky_follows = imgui.checkbox("Sky follows time", lighting.sky_follows)
        imgui.text_disabled(f"State: {lighting.cycle.state.value}   (night > dawn > day > dusk)")
```

`LightingSection._draw_lights(self, lighting)`

Dòng 101–134

```
    def _draw_lights(self, lighting):
        physics = self.app.physics
        imgui.separator_text(f"Light blocks ({len(lighting.lights)})")
        for i, light in enumerate(lighting.lights):
            imgui.push_id(i)
            _, light.on = imgui.checkbox("##on", light.on)
            widgets.tooltip("Switch this light on / off")
            imgui.same_line()
            clicked, _ = imgui.selectable(light.name, i == lighting.selected)
            if clicked:
                lighting.selected = i
            imgui.pop_id()
        light = lighting.active_light
        if light is None:
            return

        imgui.spacing()
        imgui.push_item_width(-self.LABEL_WIDTH)
        changed, pos = imgui.drag_float3("Position", [float(v) for v in light.position],
                                         self.app.drag_speed(), format="%.3f")
        if changed:
            light.position[:] = pos
            light.resting = False
        _, light.intensity = imgui.slider_float("Intensity", light.intensity, 0.0, 10.0, "%.2f")
        _, light.range = imgui.slider_float("Range", light.range, physics.height * 0.5, physics.height * 40.0,
                                            "%.3f", imgui.SliderFlags_.logarithmic)
        _, color = imgui.color_edit3("Color", list(light.color))
        light.color = tuple(color)
        imgui.pop_item_width()
        _, light.auto_night = imgui.checkbox("On at night, off by day", light.auto_night)
        widgets.tooltip("Switched by the day-night state machine (needs the sun)")
        _, lighting.fireflies = imgui.checkbox("Fireflies around lights", lighting.fireflies)
        if widgets.danger_button("Remove light##remove_light"):
            lighting.remove_light(lighting.selected)
```

5.11. `splat_editor/ui/status_bar.py`

53 dòng

Vai trò. Thanh trạng thái dưới cùng: thông báo gần nhất bên trái, thống kê bên phải.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Thanh trạng thái dưới cùng: thông báo gần nhất bên trái, thống kê bên phải."""
```

```
"""
bar = StatusBar(app)
height = bar.height()                  # chiều cao để Layout chừa chỗ
bar.draw(layout.status_bar)            # trái: thông báo; phải: số gaussian | đã chọn | FPS
"""

```

Khai báo mức module (dòng 9–9)

```
from imgui_bundle import imgui
```

`class StatusBar`

Dòng 12–53

Các phương thức: `__init__`, `height`, `draw`, `_fit`.

```
class StatusBar:
    PADDING = imgui.ImVec2(10, 5)
    FLAGS = (imgui.WindowFlags_.no_decoration | imgui.WindowFlags_.no_move |
             imgui.WindowFlags_.no_saved_settings | imgui.WindowFlags_.no_bring_to_front_on_focus |
             imgui.WindowFlags_.no_nav)
```

`StatusBar.__init__(self, app)`

Dòng 18–19

```
    def __init__(self, app):
        self.app = app
```

`StatusBar.height(self)`

Dòng 21–22

```
    def height(self):
        return imgui.get_text_line_height() + 2 * self.PADDING.y
```

`StatusBar.draw(self, rect)`

Dòng 24–43

```
    def draw(self, rect):
        editor = self.app.editor
        imgui.set_next_window_pos(imgui.ImVec2(rect.left, rect.top))
        imgui.set_next_window_size(imgui.ImVec2(rect.width, rect.height))
        imgui.push_style_var(imgui.StyleVar_.window_padding, self.PADDING)
        imgui.push_style_color(imgui.Col_.window_bg, imgui.get_style().color_(imgui.Col_.menu_bar_bg))
        imgui.begin("##status_bar", None, self.FLAGS)

        stats = (f"{editor.total_count:,} gaussians    |    "
                 f"{editor.scene.selected_count():,} selected    |    "
                 f"{self.app.fps.value:.0f} FPS")
        stats_width = imgui.calc_text_size(stats).x
        message_width = rect.width - stats_width - 4 * self.PADDING.x
        imgui.text(self._fit(editor.status, message_width))
        imgui.same_line(rect.width - stats_width - self.PADDING.x)
        imgui.text_disabled(stats)

        imgui.end()
        imgui.pop_style_color()
        imgui.pop_style_var()
```

`StatusBar._fit(text, width)`

Dòng 45–53

Mô tả trong mã nguồn. Cắt bớt chữ (thêm '...') cho vừa ‘width‘ pixel.

```
    @staticmethod
    def _fit(text, width):
        """Cắt bớt chữ (thêm '...') cho vừa `width` pixel."""
        text = text.replace('%', '%%')
        if imgui.calc_text_size(text).x <= width:
            return text
        while text and imgui.calc_text_size(text + "...").x > width:
            text = text[:-1]
        return text + "..."
```

5.12. splat_editor/ui/viewport_overlay.py

108 dòng

Vai trò. Vẽ chồng lên vùng 3D: gợi ý khi cảnh trống, nhãn và vòng tròn cọ, tọa độ người chơi, đồng hồ.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Vẽ chồng lên vùng 3D: gợi ý khi cảnh trống, nhãn chế độ cọ và vòng tròn cọ quanh chuột."""

"""
overlay = ViewportOverlay(app)
overlay.draw(layout.viewport)           # gọi mỗi khung hình: gợi ý khi cảnh trống, nhãn "Brush", vòng tròn cọ
"""

```

Khai báo mức module (dòng 8–10)

```
from imgui_bundle import imgui

from .widgets import color_u32
```

class ViewportOverlay

Dòng 13–108

Các phương thức: __init__, draw, _draw_empty_hint, _draw_brush_hint, _draw_brush_cursor, _draw_survival, _draw_coords, _draw_clock.

```
class ViewportOverlay:
    SELECT_COLOR = (1.0, 0.8, 0.1, 1.0)
    DESELECT_COLOR = (1.0, 0.35, 0.3, 1.0)
    HINT_COLOR = (0.75, 0.75, 0.8, 1.0)
    EMPTY_HINT = ["No scene loaded",
                  "File > Open (Ctrl+O), or drag and drop .ply files here"]
    BRUSH_HINT = "BRUSH   Left drag: select   Ctrl: deselect   [ ]: size   Esc: exit"
```

ViewportOverlay.__init__(self, app)

Dòng 21–22

```
    def __init__(self, app):
        self.app = app
```

ViewportOverlay.draw(self, rect)

Dòng 24–38

```
    def draw(self, rect):
        if rect.is_empty:
            return
        if self.app.editor.scene.is_empty:
            self._draw_empty_hint(rect)
        if self.app.editor.brush.active:
            self._draw_brush_hint(rect)
            self._draw_brush_cursor(rect)
        if self.app.mode == "survival":
            self._draw_survival(rect)
        if not self.app.editor.scene.is_empty:
            self._draw_coords(rect)
        lighting = self.app.lighting
        if lighting.enabled and lighting.sun_on:
            self._draw_clock(rect, lighting)
```

ViewportOverlay._draw_empty_hint(self, rect)

Dòng 40–49

```
    def _draw_empty_hint(self, rect):
        draw_list = imgui.get_background_draw_list()
        cx, cy = rect.center
        line_height = imgui.get_text_line_height_with_spacing()
        y = cy - line_height * len(self.EMPTY_HINT) / 2
        for i, line in enumerate(self.EMPTY_HINT):
            width = imgui.calc_text_size(line).x
            color = self.HINT_COLOR if i == 0 else (0.5, 0.5, 0.55, 1.0)
            draw_list.add_text(imgui.ImVec2(cx - width / 2, y), color_u32(color), line)
            y += line_height
```

`ViewportOverlay._draw_brush_hint(self, rect)`

Dòng 51–59

```
    def _draw_brush_hint(self, rect):
        draw_list = imgui.get_background_draw_list()
        size = imgui.calc_text_size(self.BRUSH_HINT)
        pos = imgui.ImVec2(rect.left + (rect.width - size.x) / 2, rect.top + 10)
        pad = 6
        draw_list.add_rect_filled(imgui.ImVec2(pos.x - pad * 2, pos.y - pad),
                                  imgui.ImVec2(pos.x + size.x + pad * 2, pos.y + size.y + pad),
                                  color_u32((0.0, 0.0, 0.0, 0.55)), 4.0)
        draw_list.add_text(pos, color_u32(self.SELECT_COLOR), self.BRUSH_HINT)
```

`ViewportOverlay._draw_brush_cursor(self, rect)`

Dòng 61–68

```
    def _draw_brush_cursor(self, rect):
        io = imgui.get_io()
        brush = self.app.editor.brush
        if io.want_capture_mouse or not rect.contains(io.mouse_pos.x, io.mouse_pos.y):
            return
        erase = brush.painting is False or (brush.painting is None and io.key_ctrl)
        color = self.DESELECT_COLOR if erase else self.SELECT_COLOR
        imgui.get_foreground_draw_list().add_circle(io.mouse_pos, brush.radius, color_u32(color), 0, 1.5)
```

`ViewportOverlay._draw_survival(self, rect)`

Dòng 70–85

```
    def _draw_survival(self, rect):
        draw_list = imgui.get_background_draw_list()
        cx, cy = rect.center
        white = color_u32((1.0, 1.0, 1.0, 0.85))
        draw_list.add_line(imgui.ImVec2(cx - 8, cy), imgui.ImVec2(cx + 8, cy), white, 1.5)      # tâm ngắm
        draw_list.add_line(imgui.ImVec2(cx, cy - 8), imgui.ImVec2(cx, cy + 8), white, 1.5)
        if self.app.physics.ready:
            return
        text = "WALK   " + self.app.physics.status
        size = imgui.calc_text_size(text)
        pos = imgui.ImVec2(rect.left + (rect.width - size.x) / 2, rect.top + 10)
        pad = 6
        draw_list.add_rect_filled(imgui.ImVec2(pos.x - pad * 2, pos.y - pad),
                                  imgui.ImVec2(pos.x + size.x + pad * 2, pos.y + size.y + pad),
                                  color_u32((0.0, 0.0, 0.0, 0.55)), 4.0)
        draw_list.add_text(pos, color_u32((0.55, 0.9, 0.55, 1.0)), text)
```

`ViewportOverlay._draw_coords(self, rect)`

Dòng 87–97

Mô tả trong mã nguồn. Tọa độ XYZ của người đang đứng (vị trí camera) ở góc dưới trái vùng 3D.

```
    def _draw_coords(self, rect):
        """Tọa độ XYZ của người đang đứng (vị trí camera) ở góc dưới trái vùng 3D."""
        x, y, z = self.app.viewport.camera.position()
        text = f"X {x:8.2f}   Y {y:8.2f}   Z {z:8.2f}"
        size = imgui.calc_text_size(text)
        pos = imgui.ImVec2(rect.left + 14, rect.top + rect.height - size.y - 14)
        draw_list = imgui.get_background_draw_list()
        draw_list.add_rect_filled(imgui.ImVec2(pos.x - 8, pos.y - 5),
                                  imgui.ImVec2(pos.x + size.x + 8, pos.y + size.y + 5),
                                  color_u32((0.0, 0.0, 0.0, 0.55)), 4.0)
        draw_list.add_text(pos, color_u32((0.9, 0.95, 1.0, 1.0)), text)
```

`ViewportOverlay._draw_clock(self, rect, lighting)`

Dòng 99–108

```
    def _draw_clock(self, rect, lighting):
        hours, minutes = int(lighting.hour), int((lighting.hour % 1) * 60)
        text = f"{hours:02d}:{minutes:02d}  {lighting.cycle.state.value}"
        size = imgui.calc_text_size(text)
        pos = imgui.ImVec2(rect.left + rect.width - size.x - 20, rect.top + 10)
        draw_list = imgui.get_background_draw_list()
        draw_list.add_rect_filled(imgui.ImVec2(pos.x - 10, pos.y - 5),
                                  imgui.ImVec2(pos.x + size.x + 10, pos.y + size.y + 5),
                                  color_u32((0.0, 0.0, 0.0, 0.55)), 4.0)
        draw_list.add_text(pos, color_u32((1.0, 0.9, 0.6, 1.0)), text)
```

5.13. `splat_editor/ui/shortcuts_window.py`

68 dòng

Vai trò. Cửa sổ Keyboard Shortcuts (F1), lấy danh sách phím trực tiếp từ `ActionRegistry`.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Cửa sổ "Keyboard Shortcuts" (F1). Danh sách phím lấy trực tiếp từ ActionRegistry."""

"""
shortcuts = ShortcutsWindow(actions)    # danh sách phím lấy từ ActionRegistry
shortcuts.toggle()                      # F1 bật / tắt
shortcuts.draw()                        # gọi mỗi khung hình
"""

```

Khai báo mức module (dòng 9–9)

```
from imgui_bundle import imgui
```

`class ShortcutsWindow`

Dòng 12–68

Các phương thức: `__init__`, `toggle`, `draw`, `_table`.

```
class ShortcutsWindow:
    MOUSE = [
        ("Right drag", "Look around (fly mode, like Minecraft)"),
        ("W A S D", "Fly forward / left / back / right"),
        ("Space", "Fly up"),
        ("Shift", "Fly down"),
        ("Ctrl (in fly mode)", "Fly faster"),
        ("Left drag", "Orbit around the target"),
        ("Middle drag", "Pan"),
        ("Drop .ply files", "Add files to the scene"),
    ]
    BRUSH_MOUSE = [
        ("Left drag", "Select gaussians"),
        ("Ctrl + left drag", "Deselect gaussians"),
        ("Middle drag", "Orbit (while the brush is on)"),
    ]
    KEY_COLUMN_WIDTH = 150
```

`ShortcutsWindow.__init__(self, actions)`

Dòng 30–32

```
    def __init__(self, actions):
        self.actions = actions
        self.visible = False
```

`ShortcutsWindow.toggle(self)`

Dòng 34–35

```
    def toggle(self):
        self.visible = not self.visible
```

`ShortcutsWindow.draw(self)`

Dòng 37–55

```
    def draw(self):
        if not self.visible:
            return
        center = imgui.get_main_viewport().get_center()
        imgui.set_next_window_pos(center, imgui.Cond_.appearing, imgui.ImVec2(0.5, 0.5))
        imgui.set_next_window_size(imgui.ImVec2(500, 640), imgui.Cond_.appearing)
        expanded, self.visible = imgui.begin("Keyboard Shortcuts", self.visible,
                                             imgui.WindowFlags_.no_collapse | imgui.WindowFlags_.no_saved_settings)
        if expanded:
            self._table("Camera", self.MOUSE)
            self._table("Brush", self.BRUSH_MOUSE)
            categories = {}
```

```
            for action in self.actions:
                if action.key is not None:
                    categories.setdefault(action.category, []).append(
                        (action.shortcut, action.description or action.label))
            for category, rows in categories.items():
                self._table(category, rows)
        imgui.end()
```

`ShortcutsWindow._table(self, title, rows)`

Dòng 57–68

```
    def _table(self, title, rows):
        imgui.separator_text(title)
        if imgui.begin_table(title, 2, imgui.TableFlags_.row_bg):
            imgui.table_setup_column("Key", imgui.TableColumnFlags_.width_fixed, self.KEY_COLUMN_WIDTH)
            imgui.table_setup_column("Action", imgui.TableColumnFlags_.width_stretch)
            for key, description in rows:
                imgui.table_next_row()
                imgui.table_next_column()
                imgui.text(key)
                imgui.table_next_column()
                imgui.text_disabled(description)
            imgui.end_table()
```

5.14. `splat_editor/ui/widgets.py`

50 dòng

Vai trò. Các widget ImGui nhỏ dùng lại: nút toàn chiều ngang, hàng nút, nút nguy hiểm, tooltip.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Các widget ImGui nhỏ dùng lại nhiều nơi."""

"""
if full_width_button("Reset"):          # nút rộng hết bảng
    ...
if danger_button("Delete"):             # nút màu đỏ
    ...
clicked = button_row(["All", "None", "Invert"])   # một hàng nút; trả về chỉ số nút được bấm hoặc None
tooltip("Giải thích hiện khi rê chuột")
color = color_u32((1.0, 0.8, 0.2, 1.0))   # RGBA 0..1 -> số nguyên cho draw list của ImGui
"""

```

Khai báo mức module (dòng 13–15)

```
from imgui_bundle import imgui

from .theme import Theme
```

`full_width_button(label)`

Dòng 18–19

```
def full_width_button(label):
    return imgui.button(label, imgui.ImVec2(-1, 0))
```

`button_row(labels)`

Dòng 22–32

Mô tả trong mã nguồn. Một hàng nút chia đều chiều ngang. Trả về chỉ số nút được bấm hoặc None.

```
def button_row(labels):
    """Một hàng nút chia đều chiều ngang. Trả về chỉ số nút được bấm hoặc None."""
    spacing = imgui.get_style().item_spacing.x
    width = (imgui.get_content_region_avail().x - spacing * (len(labels) - 1)) / len(labels)
    clicked = None
    for i, label in enumerate(labels):
        if i:
            imgui.same_line()
        if imgui.button(label, imgui.ImVec2(width, 0)):
            clicked = i
    return clicked
```

`danger_button(label)`

Dòng 35–41

Mô tả trong mã nguồn. Nút toàn chiều ngang màu đỏ (thao tác xoá).

```
def danger_button(label):
    """Nút toàn chiều ngang màu đỏ (thao tác xoá)."""
    imgui.push_style_color(imgui.Col_.button, imgui.ImVec4(*Theme.DANGER))
    imgui.push_style_color(imgui.Col_.button_hovered, imgui.ImVec4(*Theme.DANGER_HOVERED))
    clicked = full_width_button(label)
    imgui.pop_style_color(2)
    return clicked
```

`tooltip(text)`

Dòng 44–46

Mô tả trong mã nguồn. Hiện tooltip cho widget vừa vẽ khi rê chuột lên.

```
def tooltip(text):
    """Hiện tooltip cho widget vừa vẽ khi rê chuột lên."""
    imgui.set_item_tooltip(text.replace('%', '%%'))
```

`color_u32(rgba)`

Dòng 49–50

```
def color_u32(rgba):
    return imgui.color_convert_float4_to_u32(imgui.ImVec4(*rgba))
```

5.15. `splat_editor/ui/file_dialog.py`

37 dòng

Vai trò. Hộp thoại chọn file của hệ điều hành bằng `tkinter`.

Mô tả đầu file và cách dùng. Phần chú thích đầu file do tác giả viết, nêu ý tưởng và ví dụ sử dụng:

```
"""Hộp thoại chọn file của hệ điều hành (tkinter có sẵn trong Python)."""

"""
path = FileDialog.open_file()           # một file .ply, hoặc None nếu bấm Cancel
paths = FileDialog.open_files()         # nhiều file (danh sách rỗng nếu Cancel)
out = FileDialog.save_file("scene.ply")  # tên gợi ý; None nếu Cancel
"""

```

Khai báo mức module (dòng 9–10)

```
import tkinter
from tkinter import filedialog
```

`class FileDialog`

Dòng 13–37

Các phương thức: `_run`, `open_file`, `open_files`, `save_file`.

```
class FileDialog:
    FILE_TYPES = [("PLY files", "*.ply"), ("All files", "*.*")]
```

`FileDialog._run(cls, ask, **options)`

Dòng 16–24

```
    @classmethod
    def _run(cls, ask, **options):
        root = tkinter.Tk()
        root.withdraw()
        root.attributes('-topmost', True)
        try:
            return ask(filetypes=cls.FILE_TYPES, **options)
        finally:
            root.destroy()
```

`FileDialog.open_file(cls)`

Dòng 26–28

```
    @classmethod
    def open_file(cls):
        return cls._run(filedialog.askopenfilename, title="Open PLY") or None
```

`FileDialog.open_files(cls)`

Dòng 30–32

```
    @classmethod
    def open_files(cls):
        return list(cls._run(filedialog.askopenfilenames, title="Add PLY Files") or [])
```

`FileDialog.save_file(cls, initial='scene.ply')`

Dòng 34–37

```
    @classmethod
    def save_file(cls, initial="scene.ply"):
        return cls._run(filedialog.asksaveasfilename, title="Save PLY",
                        defaultextension=".ply", initialfile=initial) or None
```

Chương 6. Thư mục render/shaders: vertex và fragment shader

Chương này giải thích **toàn bộ mã GLSL** trong `splat_editor/render/shaders/`. Mỗi chương trình shader gồm một cặp **vertex shader** (`.vert`) và **fragment shader** (`.frag`); riêng pass 1 của bộ vẽ gaussian và bản đồ bóng chỉ dùng vertex shader. Các file đều dùng `\#version 330 core` (OpenGL 3.3, tương thích máy yếu).

Vertex shader và fragment shader làm gì

- **Vertex shader** chạy *một lần cho mỗi đỉnh*. Nhiệm vụ chính là ghi `gl_Position` (vị trí đỉnh trong không gian clip) và các biến `out` sẽ được nội suy.
- Giữa hai giai đoạn, GPU làm bước **rasterization**: ghép các đỉnh thành hình (điểm, tam giác), chia chúng thành các pixel (fragment) và **nội suy** biến `out` của vertex shader.
- **Fragment shader** chạy *một lần cho mỗi pixel* của hình. Nhận biến `in` đã nội suy, ghi màu ra `fragColor`. Lệnh `discard` bỏ pixel đó.

Từ khóa GLSL xuất hiện nhiều lần

Từ khóa	Ý nghĩa
`in` / `out`	Đầu vào / đầu ra của giai đoạn. `out` của vertex nối với `in` cùng tên của fragment.
`layout(location=n)`	Số thứ tự thuộc tính đỉnh, khớp với `glVertexAttribPointer` phía Python.
`uniform`	Hằng số cho cả lần vẽ (ma trận, màu, thời gian…), được đặt bằng `ShaderProgram.set_*`.
`flat`	Không nội suy; lấy giá trị của đỉnh đầu (dùng cho số nguyên như màu đóng gói).
`samplerBuffer`, `usamplerBuffer`	Texture buffer: mảng lớn đọc ngẫu nhiên bằng `texelFetch(buf, i)`; bản `u` đọc ra số nguyên không dấu.
`texelFetch`	Đọc đúng một texel theo chỉ số nguyên, **không nội suy**.
`gl_VertexID`, `gl_PointSize`, `gl_PointCoord`	Số thứ tự đỉnh, kích thước điểm (pixel) và tọa độ trong điểm.
`mat3(m)`, `mat4(c0,c1,c2,c3)`	Ma trận GLSL điền theo **cột**; `mat3(model)` lấy khối 3×3 trên trái.
`uintBitsToFloat`	Đọc lại bit của uint như float (dữ liệu đóng gói trong texel `RGBA32UI`).
`fwidth(x)`	Độ biến thiên của x giữa các pixel kề nhau (đạo hàm màn hình), dùng khử răng cưa.

Hai pass của bộ vẽ gaussian

Mỗi khung hình `SplatRenderer` chạy hai chương trình nối tiếp (xem Chương 4):

1. **Pass 1** `splat_prep.vert` (transform feedback, không có fragment shader): mỗi gaussian đã chọn chạy đúng 1 lần, ghi ellipse 2D và màu vào một buffer 32 byte/gaussian.
2. **Pass 2** `splat.vert + splat.frag`: đọc buffer đó theo thứ tự xa → gần, vẽ mỗi gaussian một hình vuông và trộn alpha.

Cách chia này giúp phần tính nặng (ma trận, SH, Phong, bóng) chỉ làm **một lần cho mỗi gaussian**, thay vì 4 lần cho 4 đỉnh của hình vuông.

Bố cục dữ liệu gaussian trên GPU (`uData`)

Mỗi gaussian là 2 texel RGBA32UI (8 uint32 = 32 byte):

Texel	Kênh	Nội dung
0	`xyz`	tâm (float32, đọc bằng `uintBitsToFloat`)
0	`w`	màu đỏ, xanh lá (hai half-float)
1	`x`	màu xanh dương và độ đục α (hai half-float)
1	`y`, `z[0:16]`	scale x, y và scale z (half-float)
1	`z[16:32]`	chỉ số layer
1	`w`	quaternion 4×8 bit

6.1. splat_prep.vert

Giai đoạn: Vertex (transform feedback) · 215 dòng · splat_editor/render/shaders/splat_prep.vert

Vai trò. Shader nặng nhất chương trình. Nó **không vẽ gì**; kết quả được ghi vào buffer rồi pass 2 đọc lại. Đầu vào duy nhất là aSplatIndex, vị trí gaussian trên GPU (thứ tự Morton); mọi dữ liệu khác được tra từ texture buffer. Toán học: §6.1 (EWA), §6.1 (Phong, bóng đổ), §1.8 (mặt trời, bản đồ bóng).

Cơ sở toán: EWA Splatting, chiếu ellipsoid 3D thành ellipse 2D

Mục này giải thích toán của pass 1 (splat_prep.vert); phần chiếu và trộn nằm ở splat.vert, splat.frag (mục 6.2).

Hiệp phương sai 3D. $\mathbf{\Sigma} = RSS^{\mathsf{T}}R^{\mathsf{T}} = MM^{\mathsf{T}}$, với $M = R\,\mathrm{diag}(s)$ (shader: M = R * mat3(scale...), sigma = M * transpose(M)). $\mathbf{\Sigma}$ đối xứng bán xác định dương; trục ellipsoid là cột của R, bán trục là s.

Jacobian của phép chiếu. Điểm ảnh: $x_{px} = f_{px}v_x/(-v_z)$, $y_{px} = f_{px}v_y/(-v_z)$. Đạo hàm theo (v_x, v_y, v_z) tại tâm:

$$J = \begin{bmatrix} \frac{f}{-v_z} & 0 & \frac{fv_x}{v_z^2} \\ 0 & \frac{f}{-v_z} & \frac{fv_y}{v_z^2} \\ 0 & 0 & 0 \end{bmatrix}, \qquad f_{px} = \frac{H}{2}P_{11} = \frac{H/2}{\tan(\mathrm{fov}/2)}$$

(splat_renderer: uFocal = height/2 * projection[1,1]). Phép chia cho c_w là phi tuyến, nên dùng **xấp xỉ tuyến tính bậc nhất** tại tâm (khai triển Taylor) để ellipsoid vẫn cho ra ellipse.

Hiệp phương sai 2D.

$$\mathbf{\Sigma}' = J\,W\,\mathbf{\Sigma}\,W^{\mathsf{T}}J^{\mathsf{T}}.$$

Trong shader gộp thêm ma trận layer sA: T = J * mat3(ModelView) * linear, cov = T Σ Tᵀ ($\mathbf{\Sigma}$ ở tọa độ file; linear = s·A đã gồm co giãn nên không cần s^2 riêng). Lấy khối 2×2: $a = \mathrm{cov}_{00}$, $b = \mathrm{cov}_{01}$, $c = \mathrm{cov}_{11}$.

Bộ lọc thông thấp: $a \mathrel{+}= 0.3$, $c \mathrel{+}= 0.3$ (tương đương cộng $0.3I$), bảo đảm gaussian rộng ít nhất khoảng 1 pixel, chống răng cưa như EWA gốc và 3DGS.

Trị riêng và vector riêng của ma trận 2x2.

$$\mathrm{mid} = \tfrac{a+c}{2}, \qquad \mathrm{radius} = \sqrt{\left(\tfrac{a-c}{2}\right)^2 + b^2},$$
$$\lambda_1 = \mathrm{mid} + \mathrm{radius}, \qquad \lambda_2 = \max(\mathrm{mid} - \mathrm{radius},\, 0.1)\ \text{(kẹp chống suy biến)},$$
$$\mathrm{axis}_1 = \mathrm{normalize}(b, \lambda_1 - a), \qquad \mathrm{axis}_2 = (-\mathrm{axis}_{1,y}, \mathrm{axis}_{1,x}).$$

Nếu $|b| < 10^{-6}$ thì dùng trục x hoặc y. Kiểm tra: $(A - \lambda_1 I)v = 0 \Rightarrow (a-\lambda_1)v_1 + bv_2 = 0 \Rightarrow v \propto (b, \lambda_1 - a)$. Bán trục ellipse là $\sqrt{\lambda_1}, \sqrt{\lambda_2}$ (độ lệch chuẩn theo hai hướng riêng).

Kích thước hình vuông (billboard).

- extent $= \min\left(\sqrt{2\ln(255\alpha)},\, 3\right)$, từ phương trình $\alpha e^{-r^2/2} = 1/255 \Rightarrow r^2 = 2\ln(255\alpha)$. Gaussian mờ cho hình vuông nhỏ; tối đa 3σ.
- major $= \min(\mathrm{extent}\cdot\sqrt{\lambda_1}, 1024)\cdot \mathrm{axis}_1 \cdot (2/\mathrm{viewport})$, minor tương tự với λ_2. Nhân $2/W$, $2/H$ để đổi pixel sang NDC (NDC rộng 2).
- **Loại bỏ (cull):** $\alpha < 1/255$; $c_w \le 0$ (sau camera); $|\mathrm{ndc}_x|$ hoặc $|\mathrm{ndc}_y| > 1.3$ (đệm 30% vì ellipse có thể lấn vào màn hình).

Cơ sở toán: Ánh sáng Phong và bóng đổ

Các hàm shade và shadowFactor trong splat_prep.vert tính ánh sáng cho từng gaussian. Quỹ đạo mặt trời, máy trạng thái ngày–đêm và thiết lập bản đồ bóng (phía CPU, core/lighting.py) nằm ở mục 1.8.

Pháp tuyến gaussian. n là trục ngắn nhất của ellipsoid: $\mathrm{axis} = \arg\min|s|$, $n = \mathrm{normalize}(\mathrm{linear}\cdot R[\mathrm{axis}])$ (cột R tương ứng, đưa sang thế giới), lật nếu $n \cdot \mathrm{toCam} < 0$. Với gaussian tròn (không có mặt phẳng): $\mathrm{roundness} = \mathrm{smoothstep}(0.35, 0.9, s_{\min}/s_{\max})$, $n = \mathrm{normalize}(\mathrm{mix}(n, (0,1,0), \mathrm{roundness}))$. Hàm $\mathrm{smoothstep}(a,b,x)$: $t = \mathrm{clip}((x-a)/(b-a), 0, 1)$, kết quả $t^2(3-2t)$ (đa thức Hermite bậc 3).

Mô hình Phong.

$$w(n,l) = \text{clip}\left(\frac{n\cdot l + 0.35}{1.35}, 0, 1\right) \quad \text{(wrap diffuse: } n\cdot l = -0.35 \mapsto 0,\ 1 \mapsto 1\text{)}$$

$$I_a = c\,L_a$$

$$I_d = c\sum_j L_j\, w(n,l_j)\, S_j$$

$$I_s = k_s \sum_j L_j \max(r_j \cdot v, 0)^m S_j, \quad r = \text{reflect}(-l,n) = 2(n\cdot l)n - l, \text{ chỉ khi } n\cdot l > 0$$

$$c_{\text{out}} = I_a + I_d + I_s \quad \text{(cuối cùng clamp 0..1)}$$

$v = \text{normalize}(\text{camPos} - p)$; m = `shininess` (mặc định 24); k_s = `specular` (0.25).

Đèn điểm: suy giảm. att $= \text{clamp}(1 - d^2/\text{range}^2, 0, 1)^2$ (bình phương nên mượt về 0 tại d = range). Tối đa 8 đèn, không đổ bóng. Màu = màu × cường độ.

Khai báo đầu vào, uniform và biến ra

Dòng 1–42

Giải thích. Ba nhóm: (1) dữ liệu gaussian đọc từ texture buffer (`uData`, `uState`, `uSh`, `uLayers`); (2) uniform camera (`uModelView`, `uProjection`, `uViewport`, `uFocal`, `uCamPos`); (3) uniform ánh sáng và bóng. Ba biến `out` là kết quả pass 1: `vCenterMajor` (tâm và trục lớn, NDC), `vMinorExtent` (trục nhỏ và bán kính hình vuông tính theo σ), `vColor` (màu RGBA8 đóng gói).

```
#version 330 core
// Pass 1 (transform feedback, không vẽ gì): mỗi gaussian chạy đúng MỘT lần, tính sẵn
// tâm + 2 trục ellipse trên màn hình và màu (kể cả SH). Pass 2 (splat.vert) chỉ còn kéo
// hình vuông theo kết quả này -> bớt 4 lần việc so với tính ở từng đỉnh của hình vuông.
layout(location = 0) in uint aSplatIndex;   // vị trí gaussian trên GPU (thứ tự Morton, tăng dần)

uniform usamplerBuffer uData;  // 2 texel RGBA32UI / gaussian (xem SplatRenderer.pack)
uniform usamplerBuffer uState; // 1 texel / gaussian: bit 1 = đang chọn, bit 2 = đã xoá (SplatRenderer.STATE_*)
uniform samplerBuffer uSh;     // uShCount texel RGBA8 / gaussian: hệ số SH bậc cao (rgb, lượng tử 8 bit)
uniform mat4 uModelView;
uniform mat4 uProjection;
uniform vec2 uViewport;        // kích thước khung hình (pixel)
uniform float uFocal;          // tiêu cự (pixel)
uniform vec3 uSelectColor;     // màu pha cho gaussian đang chọn
uniform int uShDegree;         // 0..3, 0 = màu không đổi theo hướng nhìn
uniform int uShCount;          // số hệ số SH bậc cao mỗi gaussian trên GPU (0, 3, 8, 15)
uniform float uShScale;        // hệ số giải lượng tử SH
uniform vec3 uCamPos;          // vị trí camera (toạ độ thế giới)
uniform samplerBuffer uLayers; // 4 texel (4 cột) / lớp: ma trận thế giới của lớp (s * R | t)

// ---- ánh sáng (T4): ambient + mặt trời + đèn điểm, Phong, bóng đổ bằng shadow map ----
uniform int uLightingOn;
uniform vec3 uAmbient;
uniform int uSunOn;
uniform vec3 uSunDir;          // từ cảnh hướng về mặt trời
uniform vec3 uSunColor;
uniform float uSpecular;
uniform float uShininess;
uniform int uPointCount;
uniform vec3 uPointPos[8];
uniform vec3 uPointColor[8];   // màu * cường độ
uniform float uPointRange[8];
uniform int uShadowOn;
uniform sampler2D uShadowMap;  // độ sâu nhìn từ mặt trời
uniform mat4 uLightVP;
uniform float uShadowTexel;    // cạnh một texel bản đồ bóng (đơn vị thế giới)
uniform float uShadowBias;     // độ lệch độ sâu (đơn vị NDC 0..1)
uniform float uShadowInv;      // 1 / kích thước bản đồ bóng

out vec4 vCenterMajor;        // tâm (NDC) xy, trục lớn (NDC) xy
out vec3 vMinorExtent;         // trục nhỏ (NDC) xy, bán kính hình vuông (đơn vị sigma)
flat out uint vColor;          // màu RGBA8 đóng gói
```

Hàm tiện ích: half-float, đóng gói màu, hệ số SH

Dòng 44–60

Giải thích. `halfToFloat` giải half-float IEEE-754 thủ công (GLSL 3.30 chưa có `unpackHalf2x16`): $e = 0$ là số subnormal $m \cdot 2^{-24}$, còn lại $(m + 1024) \cdot 2^{e-25}$. `packColor` nén RGBA vào một `uint` (mỗi kênh 8 bit, cộng 0.5 để làm tròn). `shCoeff` đọc một hệ số SH 8 bit rồi giải lượng tử (texel $\cdot$ 255 − 128) $\cdot$ *uShScale*.

Công thức.

$$v = (-1)^{s} \cdot \begin{cases} m \cdot 2^{-24} & e = 0 \\ (m + 1024) \cdot 2^{e-25} & e \neq 0 \end{cases}, \qquad h \approx (\text{texel} \cdot 255 - 128) \cdot \text{uShScale}$$

```
// half float (16 bit) -> float; GLSL 3.30 chưa có unpackHalf2x16
float halfToFloat(uint h) {
    uint e = (h >> 10u) & 31u;
    uint m = h & 1023u;
    float v = (e == 0u) ? float(m) * exp2(-24.0) : float(m | 1024u) * exp2(float(e) - 25.0);
    return (h & 32768u) != 0u ? -v : v;
}
vec2 unpackHalf(uint v) { return vec2(halfToFloat(v & 65535u), halfToFloat(v >> 16u)); }

uint packColor(vec4 c) {
    uvec4 u = uvec4(clamp(c, 0.0, 1.0) * 255.0 + 0.5);
    return u.r | (u.g << 8u) | (u.b << 16u) | (u.a << 24u);
}

vec3 shCoeff(int splat, int k) {
    return (texelFetch(uSh, splat * uShCount + k).rgb * 255.0 - 128.0) * uShScale;
}
```

Màu theo hướng nhìn: shColor

Dòng 62–84

Giải thích. Cộng phần SH bậc 1, 2, 3 vào màu theo hướng nhìn $d = (x, y, z)$. Mỗi dòng là một hàm cơ sở (hằng số C_1, C_2, C_3) nhân hệ số tương ứng; `if (uShDegree < 2) return c;` dừng sớm khi người dùng chọn bậc thấp, tiết kiệm tính toán.

Công thức.

$$c_{SH}(d) = \sum_{k=1}^{15} Y_k(d)\, h_k, \qquad d = \frac{A^{\top}(p_w - e)}{\|A^{\top}(p_w - e)\|}$$

```
// phần màu do SH bậc 1..3 đóng góp theo hướng nhìn d (cùng công thức với 3DGS gốc, xem core/sh.py)
vec3 shColor(int splat, vec3 d) {
    float x = d.x, y = d.y, z = d.z;
    vec3 c = -0.4886025119029199 * y * shCoeff(splat, 0)
           +  0.4886025119029199 * z * shCoeff(splat, 1)
           -  0.4886025119029199 * x * shCoeff(splat, 2);
    if (uShDegree < 2) return c;
    float xx = x * x, yy = y * y, zz = z * z;
    c +=  1.0925484305920792 * x * y * shCoeff(splat, 3)
       -  1.0925484305920792 * y * z * shCoeff(splat, 4)
       + 0.31539156525252005 * (2.0 * zz - xx - yy) * shCoeff(splat, 5)
       -  1.0925484305920792 * x * z * shCoeff(splat, 6)
       +  0.5462742152960396 * (xx - yy) * shCoeff(splat, 7);
    if (uShDegree < 3) return c;
    c += -0.5900435899266435 * y * (3.0 * xx - yy) * shCoeff(splat, 8)
       +   2.890611442640554 * x * y * z * shCoeff(splat, 9)
       -  0.4570457994644658 * y * (4.0 * zz - xx - yy) * shCoeff(splat, 10)
       +  0.3731763325901154 * z * (2.0 * zz - 3.0 * xx - 3.0 * yy) * shCoeff(splat, 11)
       -  0.4570457994644658 * x * (4.0 * zz - xx - yy) * shCoeff(splat, 12)
       +   1.445305721320277 * z * (xx - yy) * shCoeff(splat, 13)
       -  0.5900435899266435 * x * (xx - 3.0 * yy) * shCoeff(splat, 14);
    return c;
}
```

Bóng đổ: shadowFactor

Dòng 86–98

Giải thích. Dịch điểm cần tô dọc pháp tuyến hai texel (để gaussian không tự đổ bóng lên mình), chiếu vào không gian ánh sáng bằng `uLightVP`, đổi NDC sang $[0,1]$, rồi lấy **9 mẫu** quanh điểm (PCF 3×3) và so độ sâu với `uShadowBias`. Kết quả 1 là sáng, 0 là bóng, giá trị giữa là mép bóng mềm. Ngoài bản đồ thì coi là sáng.

Công thức.

$$s = \frac{\text{VP}_{light}\,(p + 2\,\text{texel}\,n)}{w} \cdot 0.5 + 0.5, \qquad S = \frac{1}{9} \sum_{i,j=-1}^{1} \Big[\, s_z - \text{bias} \le D\Big(s_{xy} + \frac{(i,j)}{2048}\Big)\Big]$$

```
// 1 = sáng, 0 = trong bóng. Dịch điểm dọc pháp tuyến vài texel + bias để gaussian không tự đổ bóng lên mình.
float shadowFactor(vec3 worldPos, vec3 n) {
    vec4 lp = uLightVP * vec4(worldPos + n * uShadowTexel * 2.0, 1.0);
    vec3 s = lp.xyz / lp.w * 0.5 + 0.5;
    if (s.x < 0.0 || s.x > 1.0 || s.y < 0.0 || s.y > 1.0 || s.z > 1.0) return 1.0;
    float lit = 0.0;
    for (int i = -1; i <= 1; i++)
```

```
        for (int j = -1; j <= 1; j++) {
            float d = textureLod(uShadowMap, s.xy + vec2(i, j) * uShadowInv, 0.0).r;
            lit += (s.z - uShadowBias <= d) ? 1.0 : 0.0;
        }
    return lit / 9.0;
}
```

Chiếu sáng Phong: shade

Dòng 100–122

Giải thích. Ba phần: **ambient** (`uAmbient`); **diffuse** dạng *wrap* $\text{clamp}((n\cdot l+0.35)/1.35, 0, 1)$ để làm mềm pháp tuyến ước lượng nhiễu; **specular** $\max(r\cdot v, 0)^m$ với $r = \text{reflect}(-l, n)$. Mặt trời nhân thêm hệ số bóng; đèn điểm nhân hệ số suy giảm $\text{att} = \text{clamp}(1 - d^2/\text{range}^2, 0, 1)^2$. Màu cuối = albedo · diffuse + spec · k_s.

Công thức.

$$w(n,l) = \text{clip}\Big(\frac{n\cdot l + 0.35}{1.35}, 0, 1\Big), \qquad r = 2(n\cdot l)\,n - l$$

$$c_{out} = c\Big(L_a + \sum_j L_j\, w(n,l_j)\, S_j\Big) + k_s \sum_j L_j \max(r_j\cdot v, 0)^m\, S_j, \qquad \text{att} = \text{clamp}\Big(1 - \frac{d^2}{\text{range}^2}, 0, 1\Big)^2$$

```
// Phong: màu * (ambient + sum diffuse) + specular. Pháp tuyến gaussian = trục ngắn nhất của ellipsoid (mặt phẳng
// mỏng ~ mảnh bề mặt), quay về phía camera. "Wrap" diffuse làm mềm vì pháp tuyến ước lượng khá nhiễu.
vec3 shade(vec3 albedo, vec3 worldPos, vec3 n, vec3 v) {
    vec3 diffuse = uAmbient;
    vec3 spec = vec3(0.0);
    if (uSunOn != 0) {
        float ndl = dot(n, uSunDir);
        float shadow = (uShadowOn != 0 && ndl > -0.4) ? shadowFactor(worldPos, n) : 1.0;
        diffuse += uSunColor * clamp((ndl + 0.35) / 1.35, 0.0, 1.0) * shadow;
        if (ndl > 0.0) spec += uSunColor * pow(max(dot(reflect(-uSunDir, n), v), 0.0), uShininess) * shadow;
    }
    for (int i = 0; i < uPointCount; i++) {
        vec3 d = uPointPos[i] - worldPos;
        float dist = length(d);
        vec3 l = d / max(dist, 1e-6);
        float att = clamp(1.0 - (dist * dist) / (uPointRange[i] * uPointRange[i]), 0.0, 1.0);
        att *= att;
        float ndl = dot(n, l);
        diffuse += uPointColor[i] * clamp((ndl + 0.35) / 1.35, 0.0, 1.0) * att;
        if (ndl > 0.0) spec += uPointColor[i] * pow(max(dot(reflect(-l, n), v), 0.0), uShininess) * att;
    }
    return albedo * diffuse + spec * uSpecular;
}
```

Hàm cull

Dòng 124–128

Giải thích. Gán hình vuông suy biến nằm ngoài màn hình (ndc = $(2,2)$, trục bằng 0, màu 0) để pass 2 không tô pixel nào. Cách này loại gaussian mà không phải đổi số lượng đầu ra của transform feedback.

```
void cull() {
    vCenterMajor = vec4(2.0, 2.0, 0.0, 0.0);   // hình vuông suy biến ngoài màn hình -> không tô pixel nào
    vMinorExtent = vec3(0.0);
    vColor = 0u;
}
```

main (1): đọc dữ liệu, loại bỏ, ma trận layer

Dòng 130–156

Giải thích. Đọc cờ trạng thái (đã xóa thì cull). Đọc hai texel gaussian. Lấy chỉ số layer từ nửa cao của `t1.z`, dựng `mat4` từ 4 texel `uLayers`, đưa tâm sang tọa độ thế giới ($M\,p$), giải α. Tính `extent` $= \min(\sqrt{2\ln(255\alpha)}, 3)$ và chiếu tâm bằng `uProjection*uModelView`; loại gaussian sau camera ($w \le 0$) hoặc ngoài màn hình quá 30%.

Công thức.

$$c_{world} = M\begin{bmatrix} p_f \\ 1 \end{bmatrix}, \quad \text{clip} = P\,V\,c_{world}, \quad \text{ndc} = \frac{\text{clip}_{xyz}}{\text{clip}_w}, \quad \text{extent} = \min\big(\sqrt{2\ln(255\alpha)},\, 3\big)$$

Giải từ $\alpha\, e^{-r^2/2} = \frac{1}{255}$. Cull nếu $w \le 0$, $|\text{ndc}_{x,y}| > 1.3$ hoặc $\alpha < \frac{1}{255}$.

```
void main() {
    int splat = int(aSplatIndex);
```

```
    uint state = texelFetch(uState, splat).r;
    // đã xoá: luồng sắp xếp đã bỏ, nhưng kết quả sắp xếp có thể đến trễ vài khung hình
    if ((state & 2u) != 0u) { cull(); return; }
    uvec4 t0 = texelFetch(uData, splat * 2 + 0);   // tâm xyz (float), màu rg (half)
    uvec4 t1 = texelFetch(uData, splat * 2 + 1);   // màu b + alpha, scale xyz (half), quaternion (4 x snorm8)

    // dữ liệu ở toạ độ file của lớp; ma trận lớp đưa sang toạ độ thế giới (đối được tức thì)
    int layer = int(t1.z >> 16u) * 4;
    mat4 model = mat4(texelFetch(uLayers, layer), texelFetch(uLayers, layer + 1),
                      texelFetch(uLayers, layer + 2), texelFetch(uLayers, layer + 3));
    mat3 linear = mat3(model);                     // s * R
    vec3 center = (model * vec4(uintBitsToFloat(t0.xyz), 1.0)).xyz;
    vec2 rg = unpackHalf(t0.w);
    vec2 ba = unpackHalf(t1.x);
    float alpha = ba.y;
    // hình vuông chỉ cần phủ tới chỗ alpha * exp(-r^2/2) = 1/255 (tối đa 3 sigma):
    // gaussian mờ -> hình vuông nhỏ hơn -> ít pixel phải tô
    if (alpha < 1.0 / 255.0) { cull(); return; }
    float extent = min(sqrt(2.0 * log(255.0 * alpha)), 3.0);

    vec4 viewPos = uModelView * vec4(center, 1.0);
    vec4 clipPos = uProjection * viewPos;
    vec3 ndc = clipPos.xyz / clipPos.w;
    // loại gaussian ở sau camera hoặc ngoài màn hình
    if (clipPos.w <= 0.0 || abs(ndc.x) > 1.3 || abs(ndc.y) > 1.3) { cull(); return; }
```

main (2): hiệp phương sai 3D rồi 2D (EWA)

Dòng 158–179

Giải thích. Giải mã quaternion 4×8 bit và chuẩn hóa; dựng R; $M = R\,\mathrm{diag}(s)$; $\Sigma = MM^{\top}$. Dựng Jacobian J của phép chiếu tại tâm; $T = J \cdot W \cdot (sA)$ gộp luôn ma trận layer; $\Sigma' = T\Sigma T^{\top}$. Lấy ba phần tử a, b, c của khối 2×2 và cộng 0.3 vào đường chéo (bộ lọc thông thấp, ellipse không nhỏ hơn khoảng 1 pixel).

Công thức.

$$\Sigma = MM^{\top}, \quad M = R\,\mathrm{diag}(s), \qquad J = \begin{bmatrix} \dfrac{f}{-v_z} & 0 & \dfrac{f\,v_x}{v_z^2} \\ 0 & \dfrac{f}{-v_z} & \dfrac{f\,v_y}{v_z^2} \\ 0 & 0 & 0 \end{bmatrix}, \quad f = \frac{H}{2} P_{11}$$

$$T = J\,W\,(sA), \qquad \Sigma' = T\,\Sigma\,T^{\top} = \begin{bmatrix} a & b \\ b & c \end{bmatrix} + 0.3\,I$$

```
    // Sigma = R S S^T R^T từ scale + quaternion (w, x, y, z)
    vec3 scale = vec3(unpackHalf(t1.y), halfToFloat(t1.z & 65535u));
    vec4 q = normalize((vec4(uvec4(t1.w, t1.w >> 8u, t1.w >> 16u, t1.w >> 24u) & 255u) - 128.0) / 127.0);
    float w = q.x, x = q.y, y = q.z, z = q.w;
    mat3 R = mat3(1.0 - 2.0 * (y * y + z * z), 2.0 * (x * y + w * z), 2.0 * (x * z - w * y),
                  2.0 * (x * y - w * z), 1.0 - 2.0 * (x * x + z * z), 2.0 * (y * z + w * x),
                  2.0 * (x * z + w * y), 2.0 * (y * z - w * x), 1.0 - 2.0 * (x * x + y * y));
    mat3 M = R * mat3(scale.x, 0.0, 0.0, 0.0, scale.y, 0.0, 0.0, 0.0, scale.z);
    mat3 sigma = M * transpose(M);

    // Jacobian của phép chiếu phối cảnh tại tâm gaussian (GLSL: mat3 theo cột)
    float vx = viewPos.x, vy = viewPos.y, vz = viewPos.z;
    mat3 J = mat3(uFocal / -vz, 0.0, 0.0,
                  0.0, uFocal / -vz, 0.0,
                  uFocal * vx / (vz * vz), uFocal * vy / (vz * vz), 0.0);
    mat3 T = J * mat3(uModelView) * linear;        // Sigma (toạ độ file) -> thế giới -> camera -> màn hình
    mat3 cov = T * sigma * transpose(T);

    // ellipse 2D (pixel); cộng 0.3 để gaussian không nhỏ hơn 1 pixel
    float a = cov[0][0] + 0.3;
    float b = cov[0][1];
    float c = cov[1][1] + 0.3;
```

main (3): trị riêng, trục ellipse, màu và đầu ra

Dòng 181–215

Giải thích. Giải trị riêng của $\begin{bmatrix} a & b \\ b & c \end{bmatrix}$ bằng công thức đóng ($\lambda_{1,2} = \mathrm{mid} \pm \mathrm{radius}$), vector riêng $(b, \lambda_1 - a)$, trục nhỏ vuông góc. Đổi pixel sang NDC (nhân 2/viewport). Tính màu: cộng SH (hướng nhìn đưa ngược về tọa độ file bằng linear$^{\top}$), nếu bật ánh sáng thì lấy pháp tuyến là trục ngắn nhất của ellipsoid rồi `shade`. Cuối cùng pha màu chọn (`mix(color, uSelectColor, 0.7*selected)`) và ghi ba biến đầu ra.

Công thức.

$$\mathrm{mid} = \frac{a + c}{2}, \quad \mathrm{radius} = \sqrt{\left(\frac{a - c}{2}\right)^2 + b^2}, \quad \lambda_1 = \mathrm{mid} + \mathrm{radius}, \quad \lambda_2 = \max(\mathrm{mid} - \mathrm{radius}, 0.1)$$

$$\hat{a}_1 = \frac{(b,\ \lambda_1 - a)}{\|(b, \lambda_1 - a)\|}, \quad \hat{a}_2 = (-\hat{a}_{1,y}, \hat{a}_{1,x}), \quad \text{major} = \min(\text{extent}\sqrt{\lambda_1}, 1024)\,\hat{a}_1 \cdot \frac{2}{\text{viewport}}$$

Pháp tuyến: $n = \text{normalize}(\text{linear} \cdot R_{[\arg\min|s|]})$; pha màu chọn $c \leftarrow \text{mix}(c, c_{sel}, 0.7\,\text{selected})$.

```
    // trị riêng / vector riêng của ma trận 2x2 [[a b] [b c]]
    float mid = 0.5 * (a + c);
    float radius = length(vec2(0.5 * (a - c), b));
    float lambda1 = mid + radius;
    float lambda2 = max(mid - radius, 0.1);
    vec2 axis1 = (abs(b) < 1e-6) ? (a >= c ? vec2(1.0, 0.0) : vec2(0.0, 1.0))
                                 : normalize(vec2(b, lambda1 - a));
    vec2 axis2 = vec2(-axis1.y, axis1.x);

    // nửa cạnh hình vuông = extent độ lệch chuẩn, tối đa 1024 pixel; đổi pixel -> NDC
    vec2 toNdc = 2.0 / uViewport;
    vec2 major = min(extent * sqrt(lambda1), 1024.0) * axis1 * toNdc;
    vec2 minor = min(extent * sqrt(lambda2), 1024.0) * axis2 * toNdc;

    vec3 color = vec3(rg, ba.x);
    // hệ số SH ở toạ độ file: xoay hướng nhìn ngược về toạ độ file (R^T d) thay vì xoay hệ số
    if (uShDegree > 0) color += shColor(splat, normalize(transpose(linear) * (center - uCamPos)));
    color = clamp(color, 0.0, 1.0);
    if (uLightingOn != 0) {
        vec3 sAbs = abs(scale);
        int axis = (sAbs.x <= sAbs.y && sAbs.x <= sAbs.z) ? 0 : ((sAbs.y <= sAbs.z) ? 1 : 2);
        vec3 n = normalize(linear * R[axis]);
        vec3 toCam = normalize(uCamPos - center);
        if (dot(n, toCam) < 0.0) n = -n;
        // gaussian tròn (không có mặt phẳng rõ ràng): pháp tuyến không xác định -> hướng lên
        float roundness = smoothstep(0.35, 0.9, min(sAbs.x, min(sAbs.y, sAbs.z)) / max(max(sAbs.x, max(sAbs.y, sAbs.z)), 1e-9));
        n = normalize(mix(n, vec3(0.0, 1.0, 0.0), roundness));
        color = clamp(shade(color, center, n, toCam), 0.0, 1.0);
    }
    float selected = float(state & 1u);

    vCenterMajor = vec4(ndc.xy, major);
    vMinorExtent = vec3(minor, extent);
    vColor = packColor(vec4(mix(color, uSelectColor, 0.7 * selected), alpha));
}
```

6.2. `splat.vert`

Giai đoạn: Vertex · 20 dòng · `splat_editor/render/shaders/splat.vert`

Vai trò. Vẽ mỗi gaussian bằng **instancing**: 4 đỉnh $(\pm1, \pm1)$ lặp lại cho từng instance; `aRank` là chỉ số vào kết quả pass 1, đã xếp xa → gần.

Toàn bộ shader

Dòng 1–20

Giải thích. `aCorner` là góc hình vuông. `uPrep` chứa kết quả pass 1 (2 texel/gaussian). `offset = cornerx*major + cornery*minor` kéo hình vuông theo hai trục ellipse; `gl_Position` là tâm cộng offset. Màu RGBA8 được tách từ uint bằng dịch bit. `vCorner = aCorner*extent` là tọa độ trong hệ trục ellipse theo đơn vị σ; nội suy giữa các đỉnh cho fragment shader biết pixel cách tâm bao nhiêu σ.

Công thức.

$$\text{gl_Position}_{xy} = \text{center} + x_c\,\text{major} + y_c\,\text{minor}, \qquad (x_c, y_c) \in \{\pm1\}^2, \qquad \text{vCorner} = (x_c, y_c) \cdot \text{extent}$$

```
#version 330 core
// Pass 2: kéo hình vuông theo kết quả đã tính sẵn ở pass 1 (splat_prep.vert).
layout(location = 0) in vec2 aCorner;   // góc hình vuông: (-1,-1) .. (1,1)
layout(location = 1) in uint aRank;     // chỉ số vào kết quả pass 1 (instance xếp xa -> gần)

uniform usamplerBuffer uPrep;           // 2 texel / gaussian: kết quả pass 1 (đọc dạng uint vì màu là uint)

out vec4 vColor;
out vec2 vCorner;

void main() {
    int base = int(aRank) * 2;
    vec4 centerMajor = uintBitsToFloat(texelFetch(uPrep, base));   // tâm (NDC) xy, trục lớn (NDC) xy
    uvec4 t1 = texelFetch(uPrep, base + 1);                         // trục nhỏ xy, bán kính (sigma), màu RGBA8
    vec3 minorExtent = uintBitsToFloat(t1.xyz);
    vec2 offset = aCorner.x * centerMajor.zw + aCorner.y * minorExtent.xy;
    gl_Position = vec4(centerMajor.xy + offset, 0.0, 1.0);
    vColor = vec4(uvec4(t1.w, t1.w >> 8u, t1.w >> 16u, t1.w >> 24u) & 255u) / 255.0;
    vCorner = aCorner * minorExtent.z;   // toạ độ tính theo đơn vị sigma
}
```

Cơ sở toán: hiển thị gaussian (pass 2, đỉnh)

Mỗi instance vẽ 4 góc $(\pm 1, \pm 1)$: `pos_ndc = center + cx*major + cy*minor`; `vCorner = (cx,cy)*extent` (đơn vị σ trong hệ trục ellipse). Fragment shader và phép trộn alpha xem tiếp ở `splat.frag` ngay sau đây; kích thước `major`, `minor`, `extent` lấy từ mục 6.1.

6.3. `splat.frag`

Giai đoạn: Fragment · 12 dòng · `splat_editor/render/shaders/splat.frag`

Vai trò. Biến hình vuông thành gaussian mờ dần.

Toàn bộ shader

Dòng 1–12

Giải thích. $r^2 = |\text{vCorner}|^2$ là bình phương khoảng cách tới tâm (đơn vị σ). Bỏ pixel nếu $r^2 > 9$ (ngoài 3σ). $\alpha_{px} = \alpha\, e^{-r^2/2}$ là mật độ Gauss 2D có đỉnh bằng α; bỏ nếu nhỏ hơn 1/255. Xuất `(rgb*alpha, alpha)`, tức **premultiplied alpha**, khớp với `glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_ALPHA)`; ta được công thức trộn "over" $C = c\,a + (1-a)C_{dst}$.

Công thức.

$$r^2 = \|\text{vCorner}\|^2, \qquad a = \alpha\, e^{-r^2/2}, \qquad \text{fragColor} = (a \cdot \text{rgb},\ a) \qquad (\text{bỏ nếu } r^2 > 9 \text{ hoặc } a < \tfrac{1}{255})$$

```
#version 330 core
in vec4 vColor;
in vec2 vCorner;
out vec4 fragColor;

void main() {
    float r2 = dot(vCorner, vCorner);
    if (r2 > 9.0) discard;
    float alpha = vColor.a * exp(-0.5 * r2);
    if (alpha < 1.0 / 255.0) discard;
    fragColor = vec4(vColor.rgb * alpha, alpha);   // premultiplied alpha
}
```

Cơ sở toán: fragment và trộn alpha "over"

Fragment (`splat.frag`).

- $r^2 = |\text{vCorner}|^2$; bỏ nếu $r^2 > 9$ (ngoài 3σ).
- $a = \alpha \exp(-r^2/2)$ (mật độ Gauss 2D chuẩn hóa về đỉnh bằng α). Bỏ nếu $a < 1/255$.
- Xuất $(\text{rgb} \cdot a,\ a)$, tức **premultiplied alpha**.

Trộn "over" (`glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_ALPHA)`).

$$C_i = c_i a_i + (1 - a_i)\, C_{i-1}$$

(với rgb đã nhân a: $C = \text{src.rgb} + (1 - \text{src.a})\,\text{dst}$). Phép này không giao hoán, nên phải vẽ **từ xa đến gần**. Depth test tắt (vật trong suốt không được che nhau).

6.4. `shadow.vert`

Giai đoạn: Vertex · 34 dòng · `splat_editor/render/shaders/shadow.vert`

Vai trò. Vẽ **tâm** mỗi gaussian như một điểm vài pixel vào bản đồ độ sâu nhìn từ mặt trời. Làm thế rất rẻ so với dựng ellipse.

Toàn bộ shader

Dòng 1–34

Giải thích. Giải half-float (`halfToFloat`) để lấy α; gaussian đã xóa hoặc quá mờ ($\alpha < 0.3$) bị đẩy ra ngoài (`gl_Position = (2,2,2,1)`) nên không đổ bóng. Dựng ma trận layer, đưa tâm sang thế giới rồi nhân `uLightVP`. `gl_PointSize = uPointSize` quyết định mỗi điểm phủ bao nhiêu pixel của bản đồ bóng (tránh lỗ hổng).

Công thức.

$$\text{gl_Position} = \text{VP}_{light}\, M \begin{bmatrix} p_f \\ 1 \end{bmatrix} \qquad (\text{bỏ qua nếu } \alpha < 0.3 \text{ hoặc đã xóa})$$

```
#version 330 core
// Bản đồ bóng: vẽ tâm mỗi gaussian đã được chọn thành một điểm (vài pixel) vào bản đồ độ sâu nhìn từ mặt trời.
layout(location = 0) in uint aSplatIndex;

uniform usamplerBuffer uData;
uniform usamplerBuffer uState;
uniform samplerBuffer uLayers;
uniform mat4 uLightVP;
uniform float uPointSize;

float halfToFloat(uint h) {
    uint e = (h >> 10u) & 31u;
    uint m = h & 1023u;
    float v = (e == 0u) ? float(m) * exp2(-24.0) : float(m | 1024u) * exp2(float(e) - 25.0);
    return (h & 32768u) != 0u ? -v : v;
}

void main() {
    int splat = int(aSplatIndex);
    uvec4 t0 = texelFetch(uData, splat * 2 + 0);
    uvec4 t1 = texelFetch(uData, splat * 2 + 1);
    float alpha = halfToFloat(t1.x >> 16u);
    // đã xoá hoặc quá mờ: không đổ bóng
    if ((texelFetch(uState, splat).r & 2u) != 0u || alpha < 0.3) {
        gl_Position = vec4(2.0, 2.0, 2.0, 1.0);
        gl_PointSize = 1.0;
        return;
    }
    int layer = int(t1.z >> 16u) * 4;
    mat4 model = mat4(texelFetch(uLayers, layer), texelFetch(uLayers, layer + 1),
                      texelFetch(uLayers, layer + 2), texelFetch(uLayers, layer + 3));
    gl_Position = uLightVP * (model * vec4(uintBitsToFloat(t0.xyz), 1.0));
    gl_PointSize = uPointSize;
}
```

6.5. `shadow.frag`

Giai đoạn: Fragment · 3 dòng · `splat_editor/render/shaders/shadow.frag`

Vai trò. Fragment shader **rỗng**: bản đồ bóng chỉ cần giá trị độ sâu do phần cứng tự ghi, không cần màu.

Toàn bộ shader

Dòng 1–3

Giải thích. `main` không làm gì. Độ sâu của điểm được GPU ghi vào depth attachment, sau đó `shadowFactor` trong pass 1 so sánh với nó.

```
#version 330 core
void main() {
}
```

6.6. `grid.vert`

Giai đoạn: Vertex · 8 dòng · `splat_editor/render/shaders/grid.vert`

Vai trò. Vẽ lưới mà **không cần vertex buffer**: tạo tam giác lớn phủ toàn màn hình từ `gl_VertexID`.

Toàn bộ shader

Dòng 1–8

Giải thích. $p = ((\mathrm{id} \ll 1) \,\&\, 2,\ \mathrm{id} \,\&\, 2) \cdot 2 - 1$: `id=0,1,2` cho $(-1,-1)$, $(3,-1)$, $(-1,3)$. Tam giác này phủ kín hình vuông $[-1,1]^2$ nên mỗi pixel màn hình chạy fragment shader đúng một lần. `vNdc` chuyển tọa độ NDC xuống fragment.

Công thức.

$$p = \big((\mathrm{id} \ll 1) \,\&\, 2,\ \mathrm{id} \,\&\, 2\big) \cdot 2 - 1 : \quad \mathrm{id} = 0,1,2 \ \mapsto\ (-1,-1),\ (3,-1),\ (-1,3)$$

```
#version 330 core
out vec2 vNdc;
void main() {
    // tam giác lớn phủ kín màn hình, không cần vertex buffer
    vec2 p = vec2((gl_VertexID << 1) & 2, gl_VertexID & 2) * 2.0 - 1.0;
    vNdc = p;
    gl_Position = vec4(p, 0.0, 1.0);
}
```

Cơ sở toán: tam giác phủ toàn màn hình

Chỉ số đỉnh `gl_VertexID` (ký hiệu id) được đổi thành tọa độ $p = ((\text{id} \ll 1)\,\&\,2,\ \text{id}\,\&\,2)\cdot 2 - 1$, nên id $= 0, 1, 2$ cho $(-1,-1)$, $(3,-1)$, $(-1,3)$.

6.7. `grid.frag`

Giai đoạn: Fragment · 44 dòng · `splat_editor/render/shaders/grid.frag`

Vai trò. Với mỗi pixel, **bắn một tia** từ camera, giao với mặt phẳng $y = h$ và tô đường lưới tại điểm giao.

Uniform và hàm gridLine

Dòng 1–18

Giải thích. `uCamRot` (cột right/up/back của camera), `uCamPos`, `uProjScale` (hai phần tử chéo của ma trận chiếu), `uHeight`, `uCell`, `uBlend`, `uFade`. `gridLine`: $c = p/\text{cell}$, `w = fwidth(c)` là độ rộng 1 pixel theo đơn vị ô, $g = |\text{fract}(c - \frac{1}{2}) - \frac{1}{2}|/w$, đường $= 1 - \min(\min(g_x, g_y), 1)$: đường luôn dày khoảng 1 pixel dù zoom thế nào (khử răng cưa theo đạo hàm).

Công thức.

$$c = \frac{p}{\text{cell}}, \quad w = \text{fwidth}(c), \quad g = \frac{|\text{fract}(c - \frac{1}{2}) - \frac{1}{2}|}{w}, \quad \text{line} = 1 - \min\big(\min(g_x, g_y), 1\big)$$

```
#version 330 core
in vec2 vNdc;
out vec4 fragColor;

uniform mat3 uCamRot;      // cột = trục right / up / back của camera trong toạ độ thế giới
uniform vec3 uCamPos;
uniform vec2 uProjScale;   // projection[0][0], projection[1][1]
uniform float uHeight;     // độ cao mặt sàn (y)
uniform float uCell;       // kích thước ô nhỏ
uniform float uBlend;      // 0..1: mức chuyển sang ô lớn hơn
uniform float uFade;       // khoảng cách bắt đầu mờ dần

float gridLine(vec2 p, float cell) {
    vec2 c = p / cell;
    vec2 w = fwidth(c);
    vec2 g = abs(fract(c - 0.5) - 0.5) / w;
    return 1.0 - min(min(g.x, g.y), 1.0);
}
```

Bắn tia và giao với mặt sàn

Dòng 20–25

Giải thích. Hướng tia dir $=$ CamRot $\cdot\,(x/P_{00},\ y/P_{11},\ -1)$ (ngược phép chiếu). Giao điểm $t = (h - e_y)/\text{dir}_y$, $p = e + t\,\text{dir}$. Bỏ pixel nếu tia song song sàn ($|\text{dir}_y| < 10^{-6}$) hoặc sàn nằm sau camera ($t \le 0$).

Công thức.

$$\hat{d} = C_{rot}\Big(\frac{x_{ndc}}{P_{00}},\ \frac{y_{ndc}}{P_{11}},\ -1\Big), \qquad t = \frac{h - e_y}{\hat{d}_y}, \qquad p = e + t\,\hat{d}$$

```
void main() {
    vec3 dir = uCamRot * vec3(vNdc.x / uProjScale.x, vNdc.y / uProjScale.y, -1.0);
    if (abs(dir.y) < 1e-6) discard;
    float t = (uHeight - uCamPos.y) / dir.y;
    if (t <= 0.0) discard;                      // mặt sàn ở sau camera
    vec3 p = uCamPos + t * dir;
```

Đường lưới, trục tọa độ, mờ dần

Dòng 27–44

Giải thích. Ô nhỏ nhân $(1 - \text{blend})$ và ô lớn $10\times$ tạo hiệu ứng chuyển mức chi tiết liên tục khi zoom. Trục X (đỏ, $z = 0$) và trục Z (xanh, $x = 0$) tô đè lên. Mờ theo khoảng cách $1 - \text{smoothstep}(fade, 3\,fade, dist)$ và theo góc nhìn (mờ khi nhìn gần song song mặt sàn). Xuất premultiplied alpha.

Công thức.

$$\alpha = \max(0.35\,\text{minor}\,(1 - \text{blend}),\ 0.7\,\text{major}) \cdot \big(1 - \text{smoothstep}(f, 3f, \text{dist})\big) \cdot \text{smoothstep}(0, 0.15, |\hat{d}_y|)$$

```
    float minor = gridLine(p.xz, uCell) * (1.0 - uBlend);
    float major = gridLine(p.xz, uCell * 10.0);
    float alpha = max(minor * 0.35, major * 0.7);
    vec3 color = vec3(0.55);

    // trục toạ độ
    float ax = 1.0 - min(abs(p.z) / fwidth(p.z), 1.0);   // trục X (z = 0)
    float az = 1.0 - min(abs(p.x) / fwidth(p.x), 1.0);   // trục Z (x = 0)
    if (ax > 0.0) { color = mix(color, vec3(0.9, 0.25, 0.25), ax); alpha = max(alpha, ax); }
    if (az > 0.0) { color = mix(color, vec3(0.25, 0.45, 0.95), az); alpha = max(alpha, az); }

    // mờ dần theo khoảng cách và khi nhìn gần song song mặt sàn
    float dist = length(p.xz - uCamPos.xz);
    alpha *= 1.0 - smoothstep(uFade, uFade * 3.0, dist);
    alpha *= smoothstep(0.0, 0.15, abs(normalize(dir).y));
    if (alpha < 0.004) discard;
    fragColor = vec4(color * alpha, alpha);      // premultiplied alpha
}
```

Cơ sở toán: lưới sàn

Tia từ camera: $\text{dir} = \text{CamRot} \cdot (\text{ndc}_x/P_{00}, \text{ndc}_y/P_{11}, -1)$ (ngược phép chiếu). **Giao tia–mặt phẳng $y = h$:** $t = (h - e_y)/\text{dir}_y$, $p = e + t \cdot \text{dir}$ (bỏ nếu $|\text{dir}_y| < 10^{-6}$ hoặc $t \le 0$).

Đường lưới: $c = p/\text{cell}$, $w = \text{fwidth}(c)$ (đạo hàm màn hình, tức độ rộng 1 pixel), $g = |\text{fract}(c - \frac{1}{2}) - \frac{1}{2}|/w$, $\text{line} = 1 - \min(\min(g_x, g_y), 1)$ (khử răng cưa theo đạo hàm). Mức chi tiết liên tục: $\text{level} = \log_{10}(\text{size})$, $\text{cell} = 10^{\lfloor \text{level} \rfloor - 1}$, $\text{blend} = \text{level} - \lfloor \text{level} \rfloor$; lưới lớn $= 10$ cell. Mờ theo khoảng cách $1 - \text{smoothstep}(\text{fade}, 3\,\text{fade}, \text{dist})$ và theo góc nhìn $\text{smoothstep}(0, .15, |\text{dir}_y|)$.

Vị trí camera được lấy từ ma trận view: $e = -R^\top t$ (dùng trong `splat_renderer.py`, `light_renderer.py` và `grid_renderer.py`).

6.8. `light_gizmo.vert`

Giai đoạn: Vertex · 18 dòng · `splat_editor/render/shaders/light_gizmo.vert`

Vai trò. Billboard luôn hướng về camera.

Toàn bộ shader

Dòng 1–18

Giải thích. Đưa tâm đèn sang không gian view rồi **cộng offset trong mặt phẳng màn hình** (`viewPos.xy += corner*radius*halo`) nên hình vuông luôn vuông góc với tia nhìn. `vPos` là tọa độ tính theo đơn vị bán kính cầu, `uHalo` là tỉ lệ quầng sáng.

Công thức.

$$v = V\,c, \qquad v_{xy} \leftarrow v_{xy} + x_c \cdot R \cdot \text{halo}, \qquad \text{gl_Position} = P\,v$$

```
#version 330 core
// Quả cầu phát sáng (mặt trời / khối đèn): hình vuông luôn quay về camera, fragment shader tô thành hình cầu + quầng sáng.
layout(location = 0) in vec2 aCorner;   // (-1,-1) .. (1,1)

uniform mat4 uView;
uniform mat4 uProj;
uniform vec3 uCenter;
uniform float uRadius;
uniform float uHalo;                    // hình vuông rộng gấp mấy lần bán kính cầu (quầng sáng)

out vec2 vPos;                          // đơn vị bán kính cầu

void main() {
    vec4 viewPos = uView * vec4(uCenter, 1.0);
    viewPos.xy += aCorner * uRadius * uHalo;
    vPos = aCorner * uHalo;
    gl_Position = uProj * viewPos;
}
```

Cơ sở toán: quả cầu đèn

Billboard: `view.xy += corner*R*halo`. Trên hình tròn $r \le 1$: $z = \sqrt{1 - r^2}$ (cao độ mặt cầu), cho độ sáng $0.7 + 0.5z$. Bên ngoài là quầng $h = \left(1 - \frac{r-1}{\text{halo}-1}\right)^2 \cdot 0.65$. Bán kính hiển thị $\max(0.12\,\text{unit}, 0.012\,\text{khoảng cách})$ (đủ lớn trên màn hình). Phần fragment ở `light_gizmo.frag` dùng r, z và quầng này để tô màu.

6.9. `light_gizmo.frag`

Giai đoạn: Fragment · 21 dòng · `splat_editor/render/shaders/light_gizmo.frag`

Vai trò. Biến hình vuông thành quả cầu phát sáng cộng quầng.

Toàn bộ shader

Dòng 1–21

Giải thích. Trong hình tròn $r \le 1$: $z = \sqrt{1-r^2}$ là cao độ mặt cầu, màu = uColor $\cdot\,(0.7 + 0.5z)$ (sáng nhất ở giữa), pha với màu tối 0.12 theo `uBrightness` (đèn tắt thì mờ). Ngoài hình tròn là **quầng**: $h = \left(1 - \frac{r-1}{\text{halo}-1}\right)^2 \cdot 0.65 \cdot \text{brightness}$, xuất với alpha 0 nên được *cộng thêm* vào cảnh (additive).

Công thức.

$$r \le 1:\ z = \sqrt{1-r^2},\ c = c_0\,(0.7 + 0.5z); \qquad r > 1:\ h = \left(1 - \frac{r-1}{\text{halo}-1}\right)^2 \cdot 0.65 \cdot \text{brightness}$$

```
#version 330 core
in vec2 vPos;
out vec4 fragColor;

uniform vec3 uColor;
uniform float uHalo;
uniform float uBrightness;              // 0..1: mờ dần (ví dụ đèn tắt, mặt trời lặn)

void main() {
    float r = length(vPos);
    if (r <= 1.0) {
        float z = sqrt(1.0 - r * r);
        vec3 col = uColor * (0.7 + 0.5 * z);          // sáng nhất ở giữa
        fragColor = vec4(mix(vec3(0.12), col, uBrightness), 1.0);
    } else {
        float h = clamp(1.0 - (r - 1.0) / max(uHalo - 1.0, 1e-3), 0.0, 1.0);
        h = h * h * 0.65 * uBrightness;
        if (h < 0.004) discard;
        fragColor = vec4(uColor * h, 0.0);            // cộng thêm (premultiplied, alpha 0)
    }
}
```

6.10. `firefly.vert`

Giai đoạn: Vertex · 22 dòng · `splat_editor/render/shaders/firefly.vert`

Vai trò. Hạt bay quanh đèn, **toàn bộ chuyển động tính trên GPU** từ `gl_VertexID` và thời gian, không cần buffer vị trí.

Toàn bộ shader

Dòng 1–22

Giải thích. `phase = i*2.399963` (góc vàng) cho các hạt pha lệch đều. Offset là tổ hợp `sin/cos` với tần số hơi khác nhau theo từng hạt (`0.4+0.13 sin(1.7 i)`) nên quỹ đạo không lặp cứng; nhân bán kính $(0.35 + 0.65\,\text{fract}(0.618i))$ để phân bố đều theo bán kính (dãy Weyl, 0.618 là nghịch đảo tỉ lệ vàng). `gl_PointSize = uPixel` đặt kích thước hạt; `vTwinkle` là hệ số nhấp nháy $0.5 + 0.5\sin(2t + 3\varphi)$.

Công thức.

$$\varphi_i = 2.399963\,i\ (= \pi(3-\sqrt{5})\,i), \qquad \text{offset}_i = \begin{bmatrix} \sin(\omega_i t + \varphi_i) \\ 0.6\sin(0.7t + 1.3\varphi_i) + 0.2 \\ \cos(\omega_i' t + 0.7\varphi_i) \end{bmatrix} R\left(0.35 + 0.65\,\text{fract}(0.618\,i)\right)$$

$$\text{twinkle} = 0.5 + 0.5\sin(2t + 3\varphi_i)$$

```
#version 330 core
// Hạt (T5): đom đóm bay quanh khối đèn. Vị trí là hàm của thời gian và gl_VertexID, tính hoàn toàn trên GPU.
uniform mat4 uView;
uniform mat4 uProj;
uniform vec3 uCenter;
uniform float uRadius;
uniform float uTime;
uniform float uPixel;

out float vTwinkle;

void main() {
```

```
    float i = float(gl_VertexID);
    float phase = i * 2.399963;                     // góc vàng: các hạt phân bố đều
    vec3 offset = vec3(sin(uTime * (0.4 + 0.13 * sin(i * 1.7)) + phase),
                       sin(uTime * 0.7 + phase * 1.3) * 0.6 + 0.2,
                       cos(uTime * (0.35 + 0.11 * cos(i * 2.3)) + phase * 0.7));
    offset *= uRadius * (0.35 + 0.65 * fract(i * 0.618));
    gl_Position = uProj * uView * vec4(uCenter + offset, 1.0);
    gl_PointSize = uPixel;
    vTwinkle = 0.5 + 0.5 * sin(uTime * 2.0 + phase * 3.0);
}
```

Cơ sở toán: đom đóm

Góc vàng $i \cdot 2.399963$ ($= \pi(3-\sqrt{5})$) cho pha. Offset $= (\sin(\omega t+\varphi),\ 0.6\sin(0.7t+1.3\varphi)+0.2,\ \cos(\omega' t+0.7\varphi)) \cdot R \cdot (0.35+0.65\,\mathrm{fract}(0.618\,i))$, trong đó 0.618 là nghịch đảo tỉ lệ vàng (dãy Weyl phân bố đều bán kính). Nhấp nháy $0.5+0.5\sin(2t+3\varphi)$. Điểm mờ: $a = \mathrm{clamp}(1-d,0,1)^2 \cdot \mathrm{twinkle}$ (phần mờ này tính ở `firefly.frag`).

6.11. `firefly.frag`

Giai đoạn: Fragment · 12 dòng · `splat_editor/render/shaders/firefly.frag`

Vai trò. Biến điểm vuông thành chấm sáng mờ dần.

Toàn bộ shader

Dòng 1–12

Giải thích. `gl_PointCoord` là tọa độ trong điểm $[0,1]^2$; $d = 2\,|\mathrm{coord}-0.5|$ là khoảng cách tới tâm. $a = \mathrm{clamp}(1-d,0,1)^2 \cdot \mathrm{twinkle}$. Xuất alpha 0 để cộng thêm vào cảnh như quầng đèn.

Công thức.

$$d = 2\,\|\mathrm{coord}-0.5\|, \qquad a = \mathrm{clamp}(1-d,0,1)^2 \cdot \mathrm{twinkle}$$

```
#version 330 core
in float vTwinkle;
out vec4 fragColor;
uniform vec3 uColor;

void main() {
    float d = length(gl_PointCoord - 0.5) * 2.0;
    float a = clamp(1.0 - d, 0.0, 1.0);
    a = a * a * vTwinkle;
    if (a < 0.01) discard;
    fragColor = vec4(uColor * a, 0.0);              // cộng thêm (premultiplied, alpha 0)
}
```

6.12. `avatar.vert`

Giai đoạn: Vertex · 14 dòng · `splat_editor/render/shaders/avatar.vert`

Vai trò. Vẽ 6 hộp cơ thể; mỗi hộp có ma trận `uModel` riêng nên có thể xoay quanh khớp.

Toàn bộ shader

Dòng 1–14

Giải thích. Ba thuộc tính đỉnh: vị trí, pháp tuyến, UV (khớp bố cục `build_mesh`). Pháp tuyến đổi sang thế giới bằng `mat3(uModel)`; `gl_Position = uProj*uView*uModel*pos`. UV và pháp tuyến truyền xuống fragment.

Công thức.

$$n_w = \mathrm{normalize}(M_{3\times3}\,n), \qquad \mathrm{gl_Position} = P\,V\,M\,p$$

```
#version 330 core
layout(location = 0) in vec3 aPos;
layout(location = 1) in vec3 aNormal;
layout(location =2 ) in vec2 aUV;
uniform mat4 uView;
uniform mat4 uProj;
uniform mat4 uModel;
out vec3 vNormal;
out vec2 vUV;
void main() {
    vNormal = normalize(mat3(uModel) * aNormal);
    vUV = aUV;
    gl_Position = uProj * uView * uModel * vec4(aPos, 1.0);
}
```

6.13. avatar.frag

Giai đoạn: Fragment · 14 dòng · splat_editor/render/shaders/avatar.frag

Vai trò. Tô skin và chiếu sáng Lambert đơn giản.

Toàn bộ shader

Dòng 1–14

Giải thích. Lấy mẫu skin 64×64 bằng `texture`; `discard` nếu $\alpha < 0.5$ (lớp áo trong suốt). shade = bright $(0.62 + 0.38 \max(n \cdot l, 0))$: 0.62 là ambient, 0.38 là Lambert.

Công thức.

$$\text{shade} = \text{bright}\big(0.62 + 0.38 \max(n \cdot l, 0)\big), \qquad \text{fragColor} = (\text{texel}_{rgb} \cdot \text{shade},\ 1)$$

```
#version 330 core
in vec3 vNormal;
in vec2 vUV;
uniform sampler2D uSkin;
uniform vec3 uLight;
uniform vec3 uBright;
out vec4 fragColor;
void main() {
    vec4 texel = texture(uSkin, vUV);
    if (texel.a < 0.5) discard;
    float diffuse = max(dot(normalize(vNormal), normalize(uLight)), 0.0);
    vec3 shade = uBright * (0.62 + 0.38 * diffuse);
    fragColor = vec4(texel.rgb * shade, 1.0);
}
```

Chương 7. Bảng tra nhanh: công thức và file

Bảng dưới liệt kê từng nhóm công thức và nơi nó được cài đặt; cột cuối trỏ tới mục *Cơ sở toán* tương ứng nằm ngay trong chương của file đó.

Chủ đề	File:hàm	Mục
sigmoid, exp, C_0, chuẩn hóa quaternion	`io/codec.py: _decode_block, encode`	§3.2
Euler, Rodrigues, quaternion	`core/math3d.py`	§1.1, §1.4
$M = [sA \mid t]$, xoay quanh pivot	`core/layer.py: world_matrix`; `core/transform.py: rotate_about`	§1.4
view, projection, ortho	`core/math3d.py: look_at, perspective, orthographic`	§1.1
orbit, pan, fly, frame	`render/camera.py`	§4.2
EWA $\mathbf{\Sigma}' = JW\mathbf{\Sigma}W^\top J^\top$, eigen, extent	`render/shaders/splat_prep.vert: main`	§6.1
Gauss và trộn over	`render/shaders/splat.frag`; `splat_renderer.py: _draw_splats`	§6.2, §6.3
SH cơ sở và xoay	`core/sh.py`; `splat_prep.vert: shColor`	§1.6
Phong, suy giảm, wrap	`splat_prep.vert: shade`	§6.1
Mặt trời, ngày–đêm, bóng	`core/lighting.py`	§1.8
PCF	`splat_prep.vert: shadowFactor`	§6.1, §1.8
Morton, radix, frustum, budget	`core/lod.py`; `splat_renderer.py: adapt_budget`	§1.7, §4.4
Sắp xếp độ sâu 16 bit	`render/sorter.py`	§4.3
Cọ capsule, viewport	`editor/brush.py`	§2.2, §2.2
Voxel, khóa 63 bit, trọng lực	`core/physics.py`	§1.9
Hoạt họa và ma trận khớp	`core/avatar_pose.py`	§1.10
Grid ray–plane, fwidth	`render/shaders/grid.frag`; `grid_renderer.py`	§6.7, §6.6
Skin UV, Paeth PNG	`render/avatar_renderer.py`	§4.8
Half-float, lượng tử hóa	`splat_renderer.py: pack`; shaders; quaternion 8 bit	§4.4, §1.1
Hoàn tác (Command, 20 bước), bit trạng thái	`core/history.py`	§1.11
Quả cầu đèn (billboard, quầng)	`render/shaders/light_gizmo.*`	§6.8
Đom đóm (góc vàng, Weyl)	`render/shaders/firefly.*`	§6.10